

Programação Web

Fundamentos, Front-End (HTML, CSS e JavaScript) e
Back-End com Spring Boot

Notas de aula organizadas a partir da disciplina de
Programação Web

Compiladas e organizadas por
Jeann Victor

Prefácio

Este livro nasceu a partir das transcrições das aulas da disciplina de *Programação Web*. Diferentemente de outros materiais de consulta mais enxutos, este livro foi pensado para ser um pouco mais completo: além de organizar e explicar o conteúdo apresentado em aula, ele traz exemplos de código em HTML, CSS, JavaScript e Java/Spring, além de diagramas explicativos para os conceitos mais visuais, como o modelo de caixa do CSS, a árvore do DOM e a arquitetura cliente-servidor.

A estrutura do livro segue a mesma sequência lógica adotada durante as aulas: parte-se dos fundamentos da profissão e da web (redes, protocolos, ferramentas, hospedagem), avança-se para o front-end com HTML e CSS, passa-se pelos fundamentos de JavaScript e, por fim, apresenta-se o back-end com Java, Spring Boot, Docker e persistência de dados.

Como esta é uma disciplina que o leitor ainda cursará, com foco especial na parte de front-end — área na qual ele parte do zero —, alguns capítulos foram enriquecidos com conteúdo complementar não apresentado originalmente nas aulas (por exemplo, estruturas de controle e funções em JavaScript, e a manipulação do DOM), de modo a preencher lacunas naturais de um curso introdutório e tornar a leitura mais autossuficiente. Esses complementos estão sempre claramente integrados ao restante do texto, mas a base principal e a linha condutora do conteúdo continuam sendo o material e a fala do professor responsável pela disciplina.

Boa leitura e bons estudos.

Sumário

Prefácio	i
I Fundamentos de Programação Web e a Profissão	1
1 Panorama da Profissão de Desenvolvedor Web	2
1.1 Objetivos desta primeira parte do curso	2
1.2 Tipos de desenvolvedores web e a origem do termo <i>full stack</i>	3
1.3 Satisfação profissional e tendências tecnológicas em 2019	4
1.4 Salários na carreira de desenvolvedor web	4
1.5 Caminhos de educação para se tornar desenvolvedor web	5
1.5.1 Treinamento liderado por instrutor	5
1.5.2 Estudo autodidata	5
1.5.3 Trabalho em projetos reais	6
1.5.4 <i>Bootcamps</i> e treinamentos intensivos	6
1.5.5 Educação universitária tradicional	6
1.5.6 Aprendizado online	7
1.6 Trilhas típicas de aprendizado	7
2 O Mercado de TI Segundo o Stack Overflow Developer Survey de 2021	9
2.1 Sobre a pesquisa	9
2.2 Perfil do desenvolvedor	9
2.3 Tecnologias mais usadas, mais amadas e mais temidas	10
2.4 Salários por tecnologia e por cargo	11
2.5 Emprego, empresas e a comunidade Stack Overflow	12
3 Processos, Métodos e Práticas de Desenvolvimento de Software	14
3.1 O que é um processo de desenvolvimento de software	14
3.2 Métodos ágeis e o ciclo de vida iterativo-incremental	15
3.2.1 Scrum	15
3.2.2 Extreme Programming (XP)	16
3.3 DevOps	16
3.4 Práticas essenciais de desenvolvimento	17
3.4.1 Entrega contínua (Continuous Delivery)	17
3.4.2 Testes automatizados	17
3.4.3 Controle de versão	17
3.4.4 Documentação	18
3.4.5 Trabalho em equipe	18
3.5 O padrão arquitetural MVC	18
3.6 A importância da flexibilidade	19

4	Conceitos Fundamentais de Redes de Computadores	21
4.1	Protocolos: a base da comunicação em rede	21
4.1.1	HTTP e HTTPS	21
4.1.2	A pilha TCP/IP	22
4.2	Identificando recursos: URI, URL e URN	23
4.3	Servidores de nomes (DNS)	24
4.4	Servidores web e servidores de aplicação	24
4.5	A World Wide Web, computação em nuvem e CDNs	25
4.5.1	Computação em nuvem	25
4.5.2	CDN — Content Delivery Network	25
5	As Linguagens Fundamentais da Web: HTML, CSS e JavaScript	27
5.1	Três linguagens, três responsabilidades	27
5.2	HTML: estrutura e semântica	27
5.3	CSS: estilo e apresentação	28
5.4	JavaScript: comportamento e interatividade	29
5.5	Um primeiro exemplo prático	30
6	Escrevendo, Implantando e Acessando Websites	33
6.1	O que é preciso para começar	33
6.2	Boa prática: separar HTML, CSS e JavaScript	33
6.3	Formas de acessar um documento web	34
6.3.1	Abrindo diretamente pelo sistema de arquivos	34
6.3.2	Editores online	34
6.3.3	Servidor web local instalado na máquina	35
6.3.4	Editores de código com servidor embutido (Live Server)	35
6.4	Resumo comparativo das formas de acesso	36
7	Desvendando o DOM (Document Object Model)	38
7.1	O que é o DOM	38
7.2	Da marcação HTML à árvore de nós	39
7.3	A metáfora da caixa: introdução ao Box Model	39
7.4	Como CSS e JavaScript interagem com o DOM	40
7.5	O conceito de eventos	41
8	Ferramentas de Desenvolvimento: DevTools, Frameworks e Bibliotecas Front-End	43
8.1	Ferramentas de desenvolvedor do navegador (DevTools)	43
8.2	Editores de código e ambientes de desenvolvimento	44
8.3	Por que usar frameworks e bibliotecas	45
8.4	Panorama de bibliotecas e frameworks front-end	45
8.5	Transpiladores e o processo de build	46
9	Linguagens e Frameworks para Backend, APIs e Serviços	48
9.1	Persistência de dados e bancos de dados relacionais	48
9.2	Linguagens dinâmicas de back-end	49
9.3	Sistemas de gestão de conteúdo (CMS)	49
9.4	APIs: quando o back-end não gera páginas	50

9.4.1	O estilo arquitetural REST	50
9.5	Autenticação delegada com OAuth	50
10	WordPress, Domínios e Hospedagem com GitHub Pages	52
10.1	WordPress: três nomes, três coisas diferentes	52
10.1.1	Tipos de conteúdo e arquitetura interna	52
10.1.2	Temas e plugins	53
10.2	Registrando um domínio web	53
10.3	Publicando sites estáticos com GitHub Pages	54
10.3.1	Passo a passo para publicar um site	54
II	HTML	57
11	Introdução à Linguagem HTML	58
11.1	O que é HTML	58
11.2	Anatomia de um elemento HTML	59
11.2.1	Elementos aninhados e ordem de fechamento	59
11.2.2	Elementos de bloco e elementos em linha	60
11.3	Atributos	60
11.3.1	Aspas em valores de atributos	61
11.4	Caracteres especiais (entidades HTML)	61
11.5	Comentários	62
11.6	Anatomia de um documento HTML completo	62
11.6.1	Associando CSS e JavaScript externamente	64
11.7	A MDN: a principal referência para aprender HTML	64
12	Marcação Fundamental de Texto em HTML	67
12.1	Elementos de Título	67
12.2	Listas	68
12.2.1	Listas Não Ordenadas	68
12.2.2	Listas Ordenadas	69
12.2.3	Listas de Descrição	69
12.3	Ênfase e Destaque de Texto	70
12.3.1	Os Elementos em e strong	70
12.3.2	Os Elementos i , b e u	71
12.4	Hiperlinks	71
12.4.1	O Elemento de Âncora	71
12.4.2	Caminhos Relativos e Absolutos	72
12.4.3	Links para Seções Específicas e Outros Esquemas de URL	73
13	Elementos Adicionais de Texto e Estrutura Semântica em HTML	75
13.1	Citações em HTML	75
13.1.1	Citação em Bloco: o Elemento blockquote	75
13.1.2	Citação em Linha: o Elemento q	75
13.1.3	O Elemento cite	76
13.2	Abreviações, Endereços e Outras Marcações Textuais	76
13.2.1	Abreviações: o Elemento abbr	76

13.2.2	Endereços: o Elemento <code>address</code>	77
13.2.3	Superescrito e Subescrito	77
13.2.4	Representação de Código-Fonte	77
13.2.5	Datas e Horários: o Elemento <code>time</code>	78
13.3	Estrutura Semântica de Documentos	78
13.3.1	O Elemento <code>header</code>	79
13.3.2	O Elemento <code>nav</code>	79
13.3.3	O Elemento <code>main</code>	80
13.3.4	Os Elementos <code>article</code> e <code>section</code>	80
13.3.5	O Elemento <code>aside</code>	80
13.3.6	O Elemento <code>footer</code>	81
13.4	Um Exemplo Completo e a Questão da Acessibilidade	81
13.4.1	Quando Usar <code>div</code> e <code>span</code>	82
14	Imagens e Vídeo em HTML	84
14.1	Incorporando Imagens	84
14.2	Boas Práticas: os Atributos <code>alt</code> , <code>width</code> e <code>height</code>	84
14.2.1	O Atributo <code>alt</code>	85
14.2.2	Os Atributos <code>width</code> e <code>height</code>	85
14.2.3	O Atributo <code>title</code>	86
14.3	Legendas com <code>figure</code> e <code>figcaption</code>	86
14.4	Vídeo em HTML	87
14.4.1	Conteúdo de <i>Fallback</i>	87
14.4.2	Múltiplas Fontes com o Elemento <code>source</code>	87
14.5	Atributos do Elemento <code>video</code>	88
14.5.1	O Atributo <code>controls</code>	88
14.5.2	Dimensões: <code>width</code> e <code>height</code>	88
14.5.3	Outros Atributos Úteis	88
III	CSS	90
15	Introdução ao CSS e Primeiros Passos	91
15.1	O que é CSS?	91
15.2	A Sintaxe de uma Regra CSS	92
15.3	Associando CSS a um Documento HTML	92
15.4	Seletores Básicos: Tag, Classe e ID	93
15.4.1	Seletor de Tag (Elemento)	93
15.4.2	Seletor de Classe	94
15.4.3	Seletor de ID	94
15.5	Um Exemplo Combinando Tudo	95
16	Primeiros Passos com CSS (Parte 2)	97
16.1	As Três Formas de Aplicar CSS	97
16.1.1	Folha de Estilo Externa (Recomendada)	97
16.1.2	Folha de Estilo Interna	97
16.1.3	Estilo em Linha (<i>Inline</i>)	98
16.2	Seletores Baseados em Localização e Estado	98

16.2.1	Combinador Descendente	98
16.2.2	Combinador Adjacente	99
16.2.3	Combinando Seletores	99
16.2.4	Seletores de Estado (Pseudo-classes)	100
16.3	Introdução ao Conceito de Caixa (<i>Box</i>)	100
16.4	Funções CSS: o Exemplo de <code>calc()</code>	101
16.5	A Regra <code>@media</code> e o Layout Responsivo	101
16.6	Notação Abreviada (<i>Shorthand</i>) e Comentários	102
17	O Modelo de Caixa CSS (Box Model)	105
17.1	Caixas em Bloco e Caixas em Linha	105
17.1.1	A Propriedade <code>display</code>	105
17.2	As Camadas do Modelo de Caixa	106
17.3	As Propriedades <code>padding</code> , <code>border</code> e <code>margin</code>	107
17.4	Colapso de Margens (<i>Margin Collapsing</i>)	108
17.5	A Propriedade <code>box-sizing</code>	108
17.5.1	O Modelo Padrão (<code>content-box</code>)	108
17.5.2	O Modelo Alternativo (<code>border-box</code>)	109
17.6	Ferramentas do Navegador para Inspecionar o Box Model	110
17.7	O Valor Misto <code>inline-block</code>	110
18	Cascata e Herança em CSS	112
18.1	O Efeito Cascata: a Ordem Importa	112
18.2	Especificidade: Quando a Ordem Não é Suficiente	113
18.2.1	Calculando a Especificidade	113
18.3	Herança em CSS	115
18.3.1	Nem Toda Propriedade é Herdada	115
18.3.2	Valores Universais para Controlar a Herança	116
18.4	A Propriedade Abreviada <code>all</code>	116
18.5	A Importância: o Modificador <code>!important</code>	117
18.6	Exemplo Resolvido: Removendo uma Cor de Fundo sem Especificar uma Cor	118
IV	JavaScript	120
19	Introdução ao JavaScript	121
19.1	O que é JavaScript?	121
19.1.1	O caminho de aprendizagem: HTML, CSS e depois JavaScript	122
19.2	Para que serve o JavaScript na prática	122
19.2.1	Um exemplo mínimo para ganhar intuição	123
19.3	APIs disponibilizadas pelos navegadores	123
19.4	Como incluir JavaScript em uma página	124
19.4.1	JavaScript interno	124
19.4.2	JavaScript externo	124
19.4.3	JavaScript inline	125
19.5	Ordem de carregamento: os atributos <code>defer</code> e <code>async</code>	126
19.6	Características importantes da linguagem	126

19.7	Comentários em JavaScript	127
20	Um Primeiro Mergulho em JavaScript	129
20.1	O projeto: jogo de adivinhação de números	129
20.2	Estrutura HTML de partida	130
20.3	Declarando as variáveis do jogo	130
20.4	Escrevendo a primeira função	131
20.5	Operadores básicos	132
20.6	Completando a lógica de verificação	132
20.7	Capturando o evento de clique	133
20.8	Finalizando o jogo	134
21	Depuração de Programas JavaScript	136
21.1	Dois tipos fundamentais de erro	136
21.2	Usando as ferramentas de desenvolvedor do navegador	136
21.3	Depurando erros de sintaxe comuns	137
21.4	Usando <code>console.log()</code> para depurar	138
21.5	Um erro de lógica clássico: confundir atribuição com comparação . . .	139
21.6	Estratégia geral para depurar	139
22	Variáveis em JavaScript	141
22.1	O que é uma variável	141
22.2	Declarando variáveis: <code>var</code> , <code>let</code> e <code>const</code>	142
22.2.1	<code>var</code> : a forma antiga	142
22.2.2	<code>let</code> : a forma recomendada para valores que mudam	142
22.2.3	<code>const</code> : para valores que não mudam	143
22.3	Escopo: a diferença mais importante entre <code>var</code> e <code>let/const</code>	143
22.4	Tipagem dinâmica	144
22.5	Convenções de nomenclatura para variáveis	145
23	Números e Operadores em JavaScript	147
23.1	O tipo <code>Number</code>	147
23.1.1	O objeto <code>Number</code> e seus métodos	147
23.2	A armadilha da concatenação com strings	148
23.3	Operadores aritméticos	149
23.3.1	Precedência de operadores	150
23.4	Operadores de incremento e decremento	150
23.5	Operadores de atribuição	150
23.6	Operadores de comparação	151
23.7	Um exemplo prático: alternando o estado de um botão	152
24	Estruturas de Controle, Funções e Arrays em JavaScript	154
24.1	Estruturas condicionais	154
24.1.1	<code>if</code> , <code>else if</code> e <code>else</code>	154
24.1.2	<code>switch</code>	155
24.2	Estruturas de repetição	156
24.2.1	O laço <code>for</code>	156
24.2.2	O laço <code>while</code>	156

24.2.3	<code>for...of</code> e <code>for...in</code>	157
24.3	Funções	157
24.3.1	Declaração de função (<i>function declaration</i>)	158
24.3.2	Expressão de função (<i>function expression</i>)	158
24.3.3	<i>Arrow functions</i>	158
24.3.4	Parâmetros e retorno	159
24.4	Arrays	159
24.4.1	Métodos comuns de array	159
24.5	Objetos literais: uma breve introdução	160
25	JavaScript e o DOM: Manipulação de Elementos e Eventos	162
25.1	Relembrando o DOM	162
25.2	Selecionando elementos	162
25.2.1	<code>getElementById</code>	162
25.2.2	<code>querySelector</code> e <code>querySelectorAll</code>	163
25.3	Alterando conteúdo	163
25.3.1	<code>textContent</code>	163
25.3.2	<code>innerHTML</code>	164
25.4	Alterando estilo	164
25.4.1	A propriedade <code>style</code>	164
25.4.2	<code>classList</code>	164
25.5	Criando e removendo elementos	165
25.6	Reagindo a eventos com <code>addEventListener</code>	166
25.6.1	O evento <code>click</code>	166
25.6.2	O evento <code>input</code>	166
25.6.3	O evento <code>submit</code>	167
25.7	O fluxo evento, listener e callback	167
25.8	Fechando a Parte IV: front-end interativo	168
V	Desenvolvimento Back-End com Spring Boot	169
26	Frontend e Backend: a Arquitetura Cliente-Servidor	170
26.1	Programação web: muito além do que se vê na tela	170
26.2	Uma analogia do cotidiano: o restaurante	171
26.3	Frontend e backend em termos técnicos	171
26.4	A arquitetura cliente-servidor	172
26.5	A integração entre frontend e backend	173
27	APIs e o Estilo Arquitetural REST	175
27.1	A origem e a problemática das APIs	175
27.2	O que é uma API	175
27.3	Os verbos (ou métodos) HTTP	176
27.3.1	GET: recuperando informações	176
27.3.2	POST: criando novos recursos	176
27.3.3	PUT: substituindo um recurso por completo	177
27.3.4	PATCH: atualizações parciais	177
27.3.5	DELETE: removendo um recurso	177

27.3.6	HEAD e OPTIONS	178
27.4	O estilo arquitetural REST	178
27.4.1	Vantagens de se construir APIs REST	179
27.5	Códigos de status HTTP	180
28	Spring, Jakarta EE e Spring Boot	182
28.1	O que é o Spring	182
28.1.1	Por que utilizar o Spring	183
28.2	Spring versus Jakarta EE	184
28.3	Spring Boot: simplicidade sobre configuração	185
28.3.1	Características principais do Spring Boot	185
28.4	Iniciando um projeto Spring Boot	186
29	Implementando um Controlador REST: Estudo de Caso BlueVelvet Music Store	188
29.1	O projeto BlueVelvet Music Store	188
29.2	Estrutura inicial do projeto	188
29.3	Organizando o projeto: pacotes controller, request e response	190
29.3.1	A classe ProductRequest	190
29.3.2	A classe ProductResponse	192
29.4	O controlador REST: ProductController	193
29.4.1	Buscando um produto por identificador (GET)	193
29.4.2	Buscando todos os produtos (GET)	195
29.4.3	Removendo um produto (DELETE)	195
29.4.4	Criando um novo produto (POST)	196
29.4.5	Atualizando um produto (PUT)	196
29.5	O que realmente caracteriza um controlador REST	197
30	Docker: Contêineres para Aplicações Web	199
30.1	O problema da divergência de ambientes	199
30.2	O que é um contêiner	200
30.3	Trabalhando com imagens e contêineres	200
30.3.1	Imagens: o ponto de partida de um contêiner	200
30.3.2	Executando e gerenciando contêineres	201
30.4	Criando imagens próprias: o Dockerfile	201
30.5	Orquestrando múltiplos contêineres com o Docker Compose	202
30.6	Visão geral da arquitetura do Docker	204
30.7	Vantagens e desvantagens do Docker	204
31	Docker, Spring Data JPA: Entidades e Repositórios	206
31.1	Subindo o banco de dados com Docker Compose	206
31.2	O script de inicialização e o modelo de dados	207
31.3	Conectando o Spring Boot ao banco de dados	207
31.4	Mapeando tabelas como entidades JPA	208
31.5	Relacionamentos entre entidades	210
31.6	O repositório: acesso a dados sem SQL manual	211

32 Camada de Serviço e Tratamento de Exceções	213
32.1 Por que separar regras de negócio do controlador	213
32.2 Criando a camada de serviço	214
32.2.1 Injeção de dependência: por que preferir o construtor	215
32.3 Tratamento de exceções: o problema	215
32.3.1 Exceções customizadas	216
32.4 Tratamento centralizado com @ControllerAdvice	216
32.5 Validação de entrada	219
33 Introdução ao Thymeleaf	220
33.1 O que é o Thymeleaf	220
33.2 Configurando o projeto para usar Thymeleaf	220
33.3 Expressões do Thymeleaf	222
33.3.1 Exibindo texto com th:text	222
33.3.2 Iterando sobre coleções com th:each	222
33.3.3 Condicionais com th:if	223
33.4 Referenciando arquivos estáticos com expressões de link	223
33.5 Passando dados do controlador para a view com Model	224
34 Spring Security: Fundamentos de Autenticação e Autorização	226
34.1 Autenticação e autorização: duas perguntas diferentes	226
34.2 O filtro de segurança: onde a proteção acontece	227
34.3 Configurando o Spring Security	227
34.4 Autenticação básica versus autenticação com token	228
34.5 Protegendo endpoints por papel	229
34.6 HTTPS: a base sobre a qual tudo isso se sustenta	230
Considerações Finais	233

Parte I

Fundamentos de Programação Web e a Profissão

Capítulo 1

Panorama da Profissão de Desenvolvedor Web

1.1 Objetivos desta primeira parte do curso

Este livro cobre os fundamentos necessários para qualquer pessoa que pretenda trabalhar com desenvolvimento web, seja como desenvolvedor front-end, desenvolvedor back-end ou como um profissional *full stack*, capaz de transitar entre as duas pontas de uma aplicação. É importante deixar claro, desde já, qual é o recorte proposto: esta parte introdutória do curso oferece uma visão geral — “a dez mil pés de altura” — da carreira de desenvolvedor web, com foco nos métodos, técnicas, ferramentas, bibliotecas, frameworks e conceitos de interconexão necessários para que o leitor comece a compreender o que significa, na prática, tornar-se um desenvolvedor web.

Outras partes do curso vão se aprofundar em tópicos complementares, como uma investigação detalhada do protocolo HTTP e de seus cabeçalhos — conhecimento fundamental para quem se especializa em back-end. Esta parte inicial, contudo, é dividida em diversos módulos, organizados da seguinte forma:

- **Módulo 1:** a carreira de desenvolvimento web;
- **Módulo 2:** métodos, processos, padrões e práticas de desenvolvimento de software;
- **Módulo 3:** conceitos fundamentais de redes de computadores;
- **Módulo 4:** as principais linguagens da web — HTML, CSS e JavaScript;
- **Módulo 5:** o DOM (*Document Object Model*), a estrutura hierárquica criada pelos navegadores para representar os elementos de um documento web;
- **Módulo 6:** a publicação de sites desenvolvidos com HTML, CSS e JavaScript;
- **Módulo 7:** ferramentas de desenvolvimento, frameworks, bibliotecas e módulos usados no dia a dia;
- **Módulo 8:** uma visão geral das linguagens usadas para desenvolvimento back-end;
- **Módulo 9:** acessibilidade, tópico especialmente importante para quem deseja atuar com desenvolvimento web profissional.

Dica

Ao longo do curso, o professor faz referência a códigos-fonte de exemplo hospedados em um repositório no GitHub. O procedimento recomendado é fazer o *clone* (cópia de um repositório para a máquina local usando Git) ou simplesmente o download do conteúdo como arquivo compactado, evitando assim ter que digitar manualmente cada trecho de código demonstrado nas aulas. Para o desenvolvimento front-end, o editor sugerido é o Visual Studio Code, gratuito e amplamente utilizado no mercado, especialmente em conjunto com a extensão *Live Server*, que permite visualizar em tempo real, no navegador, as alterações feitas no código-fonte assim que o arquivo é salvo.

1.2 Tipos de desenvolvedores web e a origem do termo *full stack*

Para entender a carreira de desenvolvimento web, é útil começar pelos tipos de desenvolvedores encontrados no mercado. Os títulos clássicos mais comuns são:

Front-end, Back-end e Full Stack

- **Desenvolvedor front-end:** foca no que é exibido para o usuário, tipicamente a partir de um navegador web (Chrome, Firefox, Safari, entre outros). Trabalha no design dos documentos web e nas interações entre os usuários e os sites.
- **Desenvolvedor back-end:** foca no que acontece “por trás dos panos” — o servidor de aplicação. Lida com configuração de máquinas servidoras, estruturas de dados e a lógica que gera as informações consumidas pelas aplicações que o usuário vê no dia a dia.
- **Desenvolvedor full stack:** um misto entre front-end e back-end, capaz de trabalhar tanto com tecnologias de interface quanto com tecnologias de servidor.

Segundo o *Stack Overflow Developer Survey* de 2019 — uma pesquisa anual, com respostas de dezenas de milhares de desenvolvedores ao redor do mundo, que servirá de referência estatística ao longo deste capítulo — cerca de 46% dos desenvolvedores se declararam *front-end developers*, enquanto 46% se identificaram como *back-end developers* (a soma ultrapassa 100% porque um mesmo profissional pode marcar mais de uma opção). A maioria, no entanto, se identifica como desenvolvedor *full stack*: uma combinação de habilidades de front-end e back-end.

Vale um comentário importante: teoricamente, não existe o desenvolvedor “perfeito”, excepcional tanto em front-end quanto em back-end — essa noção de desenvolvedor full stack *completo* é, na prática, comparável à noção de um unicórnio, algo raro e difícil de encontrar em sua forma ideal. Por essa razão, o mercado de desenvolvimento de software tem demandado, nos últimos anos, profissionais no formato “T”: indivíduos com uma especialidade vertical (a haste do T) e conhecimentos horizontais mais rasos em outras tecnologias correlatas (o traço superior do T). Um profissional

em formato T tem uma área de especialização, mas também compreende minimamente tecnologias vizinhas que não são diretamente sua especialidade, o que o torna mais valioso e versátil no mercado.

Dica

Se você está começando agora, não se cobre para se tornar um “unicórnio” imediatamente. É mais realista buscar uma especialidade sólida e, a partir dela, ir ampliando gradualmente seu conhecimento em áreas correlatas — essa é a essência do profissional em formato T, e é isso que o mercado efetivamente valoriza.

1.3 Satisfação profissional e tendências tecnológicas em 2019

O levantamento também trouxe dados sobre satisfação no trabalho: cerca de 66% das pessoas que responderam à pesquisa disseram estar satisfeitas com sua posição atual em equipes de desenvolvimento, e um percentual semelhante relatou confiar que seus gestores realmente sabem o que estão fazendo.

Do ponto de vista técnico, alguns destaques de 2019 merecem atenção porque ilustram como o mercado de tecnologia muda constantemente:

- O Visual Studio Code era, pelo sétimo ano seguido, o ambiente de desenvolvimento mais utilizado pelos desenvolvedores;
- JavaScript era considerada a linguagem de programação mais usada, tendo ultrapassado o Java em popularidade;
- O MySQL era o sistema gerenciador de banco de dados relacional mais utilizado, mas o PostgreSQL já ocupava a segunda posição, à frente do SQL Server;
- O npm era o gerenciador de pacotes mais usado por desenvolvedores web.

A mensagem central que fica é que desenvolvedores web precisam aprender a trabalhar com diferentes tecnologias, linguagens e ambientes de desenvolvimento ao longo da carreira — é raro que as tecnologias usadas em um projeto sejam exatamente as mesmas do próximo projeto. Outro dado relevante: as pessoas que participaram da pesquisa relataram que o uso de testes automatizados de unidade aumenta a satisfação com o emprego. Além disso, 68,8% dos respondentes reconheceram valor na prática de revisão de código, e 65% dos profissionais informaram que contribuem para projetos *open source* pelo menos uma vez ao ano.

1.4 Salários na carreira de desenvolvedor web

Uma pergunta recorrente de quem está iniciando a carreira é: quanto vou ganhar? A resposta depende de vários fatores — tamanho da empresa, popularidade das tecnologias que você domina, anos de experiência, entre outros. Nos Estados Unidos, segundo

a pesquisa de 2019, os salários médios mais altos na área de TI eram de gerentes de engenharia, especialistas em contabilidade de sistemas, cientistas e engenheiros de dados. O salário médio de um gerente de engenharia, por exemplo, girava em torno de US\$ 95.000 anuais. Vale notar que, se considerado o mundo todo (e não apenas os Estados Unidos), a faixa salarial fica consideravelmente menor, embora ainda em patamares elevados quando comparada a outras profissões.

O desenvolvimento back-end costuma ser considerado uma atividade mais desafiadora do que o desenvolvimento front-end, o que se reflete em salários geralmente maiores. Outro dado interessante: metade das pessoas que responderam à pesquisa não acredita que precisa necessariamente migrar para uma função de gestão para continuar tendo aumentos salariais — ou seja, é possível crescer financeiramente permanecendo como especialista técnico.

1.5 Caminhos de educação para se tornar desenvolvedor web

A educação faz uma diferença significativa em qualquer carreira, e na área de desenvolvimento web não é diferente. Existem basicamente quatro formas de educação que as pessoas costumam buscar:

1.5.1 Treinamento liderado por instrutor

É o modelo tradicional de ensino: matrícula em uma disciplina, curso ou treinamento presencial conduzido por um professor ou instrutor que segue um currículo com exercícios, plano de aulas e slides. Funciona muito bem para quem precisa esclarecer dúvidas ao longo do processo e para quem se beneficia de trabalhar em grupo. É o modelo típico de escolas, universidades e também de *workshops* e conferências.

1.5.2 Estudo autodidata

Muitas pessoas aprendem sozinhas, e algumas argumentam — com razão — que nenhum treinamento guiado ensinará tudo o que é necessário para se tornar um desenvolvedor bem-sucedido. O que fica evidente é que os desenvolvedores estão sempre em um processo de aprendizado contínuo: 86,8% das pessoas que responderam à pesquisa indicaram ter aprendido sozinhas uma nova linguagem, framework ou ferramenta, mesmo estando matriculadas em um curso formal.

Como a indústria de software muda rapidamente, talvez seja mais importante aprender os fundamentos necessários e se adaptar continuamente à medida que o mercado evolui, em vez de fixar-se em habilidades muito específicas. Práticas úteis para o autodidata incluem ler documentação oficial das tecnologias, analisar código desenvolvido por outras pessoas e explorar novas ideias que tornem o próprio trabalho mais eficiente.

1.5.3 Trabalho em projetos reais

Nenhuma quantidade de treinamento guiado, leitura de livros ou vídeos substitui a experiência de construir projetos reais. Algumas habilidades só são adquiridas trabalhando em uma empresa real, com desafios reais — muito diferente do ambiente de sala de aula ou de projetos pessoais isolados. Uma mensagem central aqui é a importância de aprender a trabalhar em equipes multifuncionais (*cross-functional*), com profissionais de diferentes áreas, superando o estereótipo de que profissionais de TI trabalham isoladamente.

1.5.4 *Bootcamps* e treinamentos intensivos

Os chamados *coding bootcamps* são cursos intensivos, projetados para ensinar habilidades de mercado em um período curto e condensado de tempo, geralmente com carga horária de 40 horas ou mais por semana, em experiência imersiva e presencial. Costumam ensinar habilidades práticas de mercado (controle de versão, integração contínua, ferramentas de ponta) que nem sempre são cobertas por cursos universitários tradicionais.

Dica

Bootcamps tendem a ter processos seletivos rigorosos, porque as escolas têm interesse em alocar seus alunos no mercado de trabalho e, para isso, selecionam previamente quem já demonstra alto potencial de conseguir uma vaga. Além disso, são financeiramente caros — alguns custam mais de US\$ 10.000 por poucas semanas de imersão — o que os torna mais adequados para quem já tem alguma base ou já está no mercado de trabalho e busca uma transição de carreira, e não necessariamente para quem está começando do zero.

1.5.5 Educação universitária tradicional

Cerca de três quartos dos desenvolvedores profissionais possuem formação superior, sendo que 60% deles estudaram Ciência da Computação, Engenharia de Computação ou Engenharia de Software. É interessante notar que, diferente do Brasil — que possui diversas universidades públicas e gratuitas — a maioria das universidades americanas é privada, e uma universidade nos Estados Unidos pode custar, em média, US\$ 50.000 por ano, valor cerca de três vezes maior do que na década de 1990 (aumento muito acima da inflação e da renda média das famílias).

Apesar do custo elevado nos Estados Unidos, estudos indicam que pessoas com formação superior ganham, em média, cerca de um milhão de dólares a mais ao longo da vida profissional do que pessoas sem diploma. Algumas empresas ainda exigem diploma em seus processos seletivos (cerca de 65% das vagas nos Estados Unidos, segundo a pesquisa), embora várias empresas importantes de tecnologia venham dispensando essa exigência nos últimos anos.

Os benefícios de uma formação tradicional incluem o desenvolvimento de raciocínio mais abstrato para resolução de problemas, o contato com uma ampla gama de disciplinas (incluindo matemática e física, muito úteis em Ciência da Computação) e, frequentemente, disciplinas de administração que ensinam a escrever propostas, planos

de negócio e documentos relevantes para tomadores de decisão.

1.5.6 Aprendizado online

O aprendizado online barateou e democratizou o acesso à educação em tecnologia. Suas vantagens incluem flexibilidade de horário e lugar, além do acesso a uma quantidade enorme de conteúdo por meio de ferramentas como Google e YouTube. Um problema comum, porém, é que a abundância de conteúdo disponível pode dificultar encontrar exatamente a resposta simples e direta que se busca.

Recursos de aprendizado online mencionados

- **Stack Overflow:** site de perguntas e respostas sobre programação, onde profissionais renomados ajudam a esclarecer dúvidas da comunidade constantemente;
- **Fork de projetos:** prática de copiar (*fork*) projetos reais desenvolvidos por terceiros para estudo próprio, permitindo analisar e aprender a partir de código já escrito e testado; o GitHub é a plataforma de desenvolvimento colaborativo mais famosa para isso, embora existam alternativas como o GitLab;
- **MOOCs** (*Massive Open Online Courses*): cursos online abertos para milhares de pessoas, como os oferecidos por Coursera, edX e Udacity;
- **Plataformas como a Udemy:** cursos vendidos a valores acessíveis, embora também existam trilhas gratuitas em diversas plataformas.

1.6 Trilhas típicas de aprendizado

Não existe uma única linguagem que unifique todos os tipos de desenvolvedores web, mas o HTML é considerado a linguagem base para qualquer um deles. A partir desse conhecimento comum, existem caminhos típicos para cada perfil:

- **Front-end:** HTML → CSS → JavaScript, seguido de bibliotecas e frameworks como React, Bootstrap e Vue.js. É comum também aprender sistemas de gestão de conteúdo (CMS) como WordPress, Drupal ou Joomla.
- **Back-end com Java:** HTML → Java SE → Jakarta EE (antigo Java EE) → um framework como Spring Boot.
- **Back-end com PHP:** HTML → PHP → um framework como Laravel.
- **Back-end com JavaScript:** HTML → JavaScript → Node.js → um framework como Express.

De modo geral, desenvolvedores back-end também precisam conhecer alguma linguagem estruturada de manipulação de banco de dados, como SQL, e eventualmente

linguagens não estruturadas associadas a bancos de dados NoSQL. Vale destacar também a relevância do WordPress: esse sistema de gestão de conteúdo está presente em cerca de 30 a 40% de todos os sites da web, tornando-o um conhecimento praticamente obrigatório para qualquer desenvolvedor web, a ponto de existirem empresas cujo modelo de negócio é inteiramente construído em torno de tecnologias baseadas em WordPress.

Em resumo, o caminho para se tornar um desenvolvedor full stack é, essencialmente, a união das trilhas de front-end e back-end. A mensagem final é que o desenvolvimento web é uma carreira de estudo contínuo e sem fim — e é justamente essa característica que exige do profissional flexibilidade e disposição para aprender ao longo de toda a sua trajetória.

Síntese do Capítulo

- Os três perfis clássicos da carreira são front-end, back-end e full stack, sendo o perfil “em T” (especialista com conhecimento amplo em áreas correlatas) o mais valorizado no mercado atual.
- Segundo o Stack Overflow Developer Survey de 2019, JavaScript era a linguagem mais usada, o Visual Studio Code o ambiente mais popular e a maioria dos desenvolvedores se identificava como full stack.
- Testes automatizados, revisão de código e contribuição a projetos open source estão associados a maior satisfação profissional.
- Salários variam conforme tamanho da empresa, tecnologias dominadas e anos de experiência; não é necessário migrar para cargos de gestão para crescer financeiramente.
- Existem quatro grandes caminhos de educação: treinamento com instrutor, estudo autodidata, trabalho em projetos reais e bootcamps intensivos, cada um com vantagens e limitações próprias.
- A formação universitária tradicional oferece base teórica sólida e ainda é o caminho mais comum entre desenvolvedores profissionais, mas não é o único caminho válido para a carreira.
- HTML é a linguagem base comum a todas as trilhas; a partir dela, front-end e back-end seguem caminhos distintos, mas complementares, sendo o aprendizado contínuo a característica definidora da profissão.

Capítulo 2

O Mercado de TI Segundo o Stack Overflow Developer Survey de 2021

2.1 Sobre a pesquisa

Para complementar o panorama traçado a partir da pesquisa de 2019, vale a pena revisitar os resultados do *Stack Overflow Developer Survey* de 2021, realizado em maio daquele ano, com a participação de cerca de 83 mil desenvolvedores de 181 países ao redor do mundo (foram descartadas respostas consideradas não confiáveis, por exemplo aquelas cujo tempo de preenchimento foi curto demais para refletir respostas ponderadas). A pesquisa é dividida em várias seções: perfil do desenvolvedor, tecnologia, trabalho, comunidade e metodologia. Este capítulo percorre os principais achados de cada uma delas, permitindo comparar a evolução do mercado entre 2019 e 2021.

De maneira geral, quase 60% das pessoas que responderam à pesquisa aprendem a programar a partir de recursos online. Um padrão interessante que se repete: desenvolvedores mais jovens tendem a aprender a partir de cursos, fóruns e outros recursos online, enquanto desenvolvedores de idade mais avançada tendem a aprender por meios mais tradicionais, como ensino formal em escolas e livros.

2.2 Perfil do desenvolvedor

O Stack Overflow funciona como uma comunidade verdadeiramente global: a pesquisa recebeu respostas de praticamente todos os países do mundo. Os Estados Unidos e a Índia continuam a fornecer o maior volume de respostas, seguidos por Alemanha e Reino Unido. O Brasil correspondeu a 2,7% do total de aproximadamente 49 mil respostas geográficas coletadas.

Alguns dados demográficos e de formação relevantes:

- A grande maioria dos desenvolvedores escreveu sua primeira linha de código entre 11 e 17 anos de idade;
- 59,53% aprenderam a programar a partir de recursos online (blogs, vídeos etc.), 53,59% em escolas e universidades e 51,53% a partir de livros ou meios físicos — ou seja, é comum combinar mais de uma fonte de aprendizado;
- Considerando anos totais de experiência com programação (incluindo tempo de formação), 29,91% das pessoas relataram entre 5 e 9 anos de experiência; no Brasil, especificamente, a média foi de cerca de 11,5 anos;
- Considerando apenas experiência profissional (excluindo educação), 31,26% relataram entre 1 e 4 anos de experiência no mercado de trabalho;

- Quanto à formação: 42,37% possuem nível universitário (bacharelado), 20,99% possuem título de mestre e apenas 3% possuem doutorado. Isso indica que, mesmo em países como o Brasil, onde a profissão não é regulamentada e não exige diploma, o caminho acadêmico continua sendo comum entre os profissionais da área;
- Quanto à idade: 39,52% têm entre 25 e 34 anos e 25,47% têm entre 18 e 24 anos — ou seja, cerca de 65% dos desenvolvedores têm entre 18 e 34 anos, confirmando uma profissão de perfil etário relativamente jovem;
- Quanto a gênero: 91,67% se identificaram como homens, 5,31% como mulheres e cerca de 1% como pessoas transgênero, evidenciando uma desproporção de gênero significativa na área.

Um padrão interessante relacionado à experiência por cargo: executivos e gerentes tendem a ter mais anos de experiência (na casa de 13 a 16 anos, em média), enquanto profissionais de ciência de dados e aprendizado de máquina apresentam, em média, menos anos de experiência do que mesmo pesquisadores acadêmicos. Entre os papéis específicos, DevOps aparece com média de 11,26 anos de experiência, e desenvolvedores mobile e front-end com cerca de 9 anos.

Quanto ao tipo de atuação declarada (a pergunta permitia múltiplas respostas), 49% se identificaram como desenvolvedores full stack, 43% como back-end, 27% como front-end, 16% como desenvolvedores desktop e 14% como desenvolvedores mobile. Vale notar que o papel de *designer* caiu significativamente em relação a 2020, sendo ultrapassado pelo de administrador de sistemas.

2.3 Tecnologias mais usadas, mais amadas e mais temidas

Um dos aspectos mais interessantes da pesquisa é a comparação entre tecnologias mais usadas, mais adoradas (*loved*), mais temidas (*dreaded*) e mais desejadas (*wanted*) para o futuro. Essa distinção é importante: uma tecnologia pode ser muito usada no mercado sem ser, necessariamente, a preferida dos desenvolvedores.

- **Linguagens mais usadas em 2021:** JavaScript continuou na liderança pelo nono ano consecutivo (usada por 64,96% dos profissionais respondentes), seguida por HTML/CSS (56%), Python (48%) e SQL, que ultrapassou o Java e se tornou a terceira linguagem mais popular entre desenvolvedores web;
- **Bancos de dados:** MySQL (50%) permanece como o mais usado, seguido por PostgreSQL (40,42%), Microsoft SQL Server (32,18%) e MongoDB (27,7%) — uma mistura de bancos relacionais tradicionais e bancos NoSQL;
- **Plataformas de nuvem:** AWS lidera com 54,22% de preferência, seguida por Google Cloud Platform (31%) e Microsoft Azure (30,77%);
- **Frameworks web:** Node.js aparece com 40,14%, seguido por React (34,42%), Express (23,02%) e Angular (22%) — o React ultrapassou o jQuery como framework web mais usado;

- **Ferramentas:** cerca de 90% dos desenvolvedores usam Git como sistema de controle de versão, tornando-o praticamente indispensável na profissão; 48,85% relataram uso extensivo do Docker no último ano;
- **IDEs:** o Visual Studio Code lidera disparadamente, usado por 71,06% dos desenvolvedores, seguido por IntelliJ IDEA (28,74%), Eclipse (15,87%) e PyCharm (7,47%).

Quanto às tecnologias mais amadas e mais temidas:

- **Rust** é a linguagem mais adorada (87% de aprovação entre quem a utiliza), mas também considerada difícil por 13,02% dos respondentes;
- **TypeScript** é amado por 72,73% e temido por 27,27%;
- **Java** é amado por 47,15%, mas considerado difícil por 52,82% — uma das linguagens mais temidas apesar de ser extremamente comum em ambientes universitários;
- **Python** é uma das linguagens que as pessoas mais desejam aprender no futuro, ao passo que apenas 6% desejam aprender Java na sequência da carreira, e 15,29% desejam aprender TypeScript;
- Entre bancos de dados, o **Redis** é o mais amado (70,71%), enquanto o MongoDB, apesar de popular (51% de uso), é considerado intimidador por 48,65% dos respondentes;
- Entre frameworks web, **React** tem cerca de 72% de preferência, e o **jQuery**, apesar de tradicional, vem perdendo espaço frente a frameworks mais modernos.

2.4 Salários por tecnologia e por cargo

A pesquisa também traçou um panorama salarial detalhado. Entre linguagens de programação, o **Clojure** pagava, em 2021, o maior salário mediano anual (cerca de US\$ 95.000), seguido por linguagens como Erlang, F#, Scala, Go e Ruby — muitas delas associadas a nichos especializados do mercado, o que explica salários mais altos. PHP, em contraste, aparece entre as linguagens de menor remuneração mediana (cerca de US\$ 38.000 anuais nos dados apresentados), apesar de ser amplamente utilizada.

Quanto a cargos nos Estados Unidos, executivos de nível sênior lideram com média de US\$ 177.000 anuais, seguidos por gerentes de engenharia, engenheiros de confiabilidade de sites (SRE) e especialistas em DevOps (cerca de US\$ 135.000). Desenvolvedores back-end, de jogos, full stack e front-end aparecem em faixas entre US\$ 115.000 e US\$ 133.000 anuais nos Estados Unidos. É importante lembrar que esses valores refletem o mercado americano; a mediana global, considerando todos os países da pesquisa, é substancialmente menor.

Um padrão consistente aparece quando cruzamos salário com anos de experiência: cargos de gestão têm, em média, tanto salários mais altos quanto mais anos de experiência acumulada, e a experiência profissional tende a se correlacionar positivamente com a remuneração em praticamente todos os papéis analisados.

2.5 Emprego, empresas e a comunidade Stack Overflow

Sobre o tipo de vínculo empregatício, 81% dos desenvolvedores profissionais trabalham em regime integral (*full-time*). O percentual de profissionais que trabalham como autônomos (*freelancers*) cresceu de 9,5% em 2020 para 11,2% em 2021, sugerindo uma tendência de crescimento do trabalho independente na área. Quanto ao tamanho das empresas, cerca de 40% dos desenvolvedores trabalham em empresas de 20 a 500 funcionários, e apenas 6,16% em empresas de 500 a 1.000 funcionários.

Quanto ao uso e à participação na comunidade do Stack Overflow:

- Praticamente todos os desenvolvedores visitam o site quando encontram algum problema;
- 23,76% visitam o site várias vezes ao dia e cerca de 30% quase diariamente;
- 82,37% possuem conta no Stack Overflow, mas 45,56% participam (perguntando ou respondendo) menos de uma vez por mês;
- Apenas 28% se consideram parte da comunidade do Stack Overflow de forma efetiva.

Uma seção curiosa da pesquisa trata de como os desenvolvedores resolvem problemas no dia a dia: quase 90% recorrem ao Google, 80% visitam o Stack Overflow, 44% assistem a tutoriais, e uma fração relevante (37%) relata sair para caminhar ou praticar alguma atividade física antes de retomar o problema — um lembrete de que descanso e distância do problema muitas vezes ajudam a encontrar soluções mais rapidamente do que a insistência imediata.

Dica

Vale registrar esse achado da pesquisa como um conselho prático: quando você travar em um problema de código e sentir frustração crescente, uma pausa breve — uma caminhada, uma xícara de café, uma troca de ambiente — é uma estratégia legítima e frequentemente eficaz, não uma forma de procrastinação.

Síntese do Capítulo

- A pesquisa de 2021 reuniu quase 83 mil respostas de 181 países, confirmando o Stack Overflow como referência global para entender o mercado de TI.
- JavaScript segue como a linguagem mais usada pelo nono ano consecutivo, mas Python ultrapassou SQL em popularidade geral e Rust é a linguagem mais amada pelos desenvolvedores.
- O Visual Studio Code e o Git são, respectivamente, o ambiente de desenvolvimento e a ferramenta de controle de versão mais difundidos no mercado.

- Salários variam fortemente por tecnologia (Clojure e Ruby entre os mais bem pagos, PHP entre os menos) e por país, sendo os valores dos EUA muito superiores à mediana global.
- O trabalho autônomo (freelance) cresceu de 9,5% para 11,2% entre 2020 e 2021, indicando uma tendência de maior flexibilidade de vínculo na área.
- A maioria dos desenvolvedores tem entre 18 e 34 anos e formação superior, ainda que a profissão nem sempre exija diploma formal.
- Resolver problemas de programação envolve tanto pesquisa ativa (Google, Stack Overflow, tutoriais) quanto pausas estratégicas, segundo o próprio relato dos desenvolvedores.

Capítulo 3

Processos, Métodos e Práticas de Desenvolvimento de Software

3.1 O que é um processo de desenvolvimento de software

Ao desenvolver software, é preciso seguir uma sequência de atividades e produzir uma série de artefatos intermediários até chegar ao produto final, executado na máquina do cliente ou em um ambiente de produção. Esse caminho percorrido — da ideia inicial até o software em produção — é chamado de **processo de desenvolvimento de software**: uma sequência de atividades desempenhadas para produzir software de valor.

Ciclo de vida em cascata

Em engenharia tradicional, é muito comum usar o chamado **ciclo de vida em cascata** (*waterfall*), no qual as etapas do processo — geralmente requisitos, design, desenvolvimento, teste e implantação — são executadas sequencialmente, uma após a outra. Apenas ao final da última etapa o produto é entregue ao cliente.

O grande problema do modelo em cascata aplicado a software é que o tempo entre a coleta inicial de requisitos e a entrega final costuma ser de meses, período durante o qual o cliente não recebe nenhum código executável para validar se o time está no caminho correto. Embora o ciclo em cascata funcione bem em várias áreas da engenharia, ele não é adequado para a maioria dos projetos de software por um motivo simples: **software é algo intangível**. Diferente de uma ponte ou de um prédio, o cliente não tem um produto físico e palpável para usar como referência enquanto imagina o que está sendo desenvolvido. Isso faz com que os requisitos sejam voláteis e mudem constantemente ao longo do projeto — o que contraria a premissa do modelo em cascata, que assume que 100% dos requisitos podem ser capturados antes de se iniciar o design e o desenvolvimento.

O modelo em cascata só funciona bem quando os requisitos já são muito bem definidos e o grau de variação esperado é pequeno — casos que constituem exceção, e não regra, no desenvolvimento de software.

3.2 Métodos ágeis e o ciclo de vida iterativo-incremental

Os métodos ágeis surgem como alternativa ao modelo em cascata, substituindo-o pelo chamado **ciclo de vida iterativo e incremental**. A ideia central é que o produto de software é construído ao longo de diversas iterações que se repetem no tempo, e cada iteração contém todas as disciplinas de desenvolvimento — requisitos, design, desenvolvimento, teste e implantação.

Cada iteração é um período curto de tempo (por isso “curto ciclo”), ao final do qual o cliente recebe um produto funcional, ainda que pequeno e incompleto. Ao final de uma iteração, uma nova começa, repassando todas as disciplinas de desenvolvimento novamente — e o software vai crescendo e evoluindo ao longo dessas iterações sucessivas. Essa é a razão pela qual o modelo iterativo-incremental é hoje o mais recomendado para desenvolvimento de software: ele fornece pontos de inspeção frequentes, nos quais o cliente pode verificar se o projeto está indo na direção correta e, caso necessário, solicitar ajustes já na iteração seguinte — ao contrário do modelo em cascata, no qual eventuais insatisfações só são percebidas (e corrigidas, a um custo muito maior) depois de meses de trabalho.

3.2.1 Scrum

O Scrum é, provavelmente, o método ágil mais famoso baseado no ciclo de vida iterativo-incremental. É importante entender uma característica central do Scrum: ele é um **framework de gerenciamento** do processo de desenvolvimento — ele não prescreve práticas técnicas de programação, como TDD, testes automatizados, refatoração ou programação em par. O Scrum foca em como organizar o trabalho da equipe ao longo de ciclos curtos chamados **Sprints**.

Sprint

Um Sprint é uma iteração de tempo limitado (tipicamente entre uma e quatro semanas) na qual a equipe se compromete a entregar a maior quantidade possível de valor funcional ao cliente.

O funcionamento básico do Scrum envolve os seguintes artefatos e eventos:

- **Product Backlog:** lista de itens de trabalho (incluindo requisitos) priorizados pelo time e pelo cliente;
- **Sprint Backlog:** subconjunto de tarefas do Product Backlog selecionado para ser desenvolvido dentro de um Sprint específico;
- **Daily Scrum:** reunião diária e curta em que cada membro da equipe responde a três perguntas: o que fiz ontem para hoje? o que farei hoje para amanhã? há algo me impedindo de avançar?

Ao final de um Sprint, a equipe não entrega documentação — entrega **software funcionando**, ainda que incompleto, para que o cliente possa avaliar, aceitar ou solicitar mudanças a serem tratadas no próximo Sprint.

3.2.2 Extreme Programming (XP)

Enquanto o Scrum organiza o *gerenciamento* do projeto, o **Extreme Programming (XP)** é o método ágil mais famoso voltado às *práticas técnicas* de desenvolvimento de software, sendo comum usar os dois métodos em conjunto: Scrum para gestão das entregas, XP para as práticas de engenharia.

Feedback loop

Um **feedback loop** (laço de retroalimentação) é uma interação em que alguém realiza uma ação, observa o resultado dessa ação e, com base nesse resultado, decide qual será a próxima ação. É um conceito central tanto no Scrum quanto no XP.

O XP trabalha explicitamente com múltiplos ciclos de feedback, em diferentes escalas de tempo:

- **Segundos:** ao escrever uma função e compilá-la/executá-la, o desenvolvedor obtém feedback instantâneo;
- **Minutos:** ao rodar um teste automatizado, o resultado aparece como uma “barra verde” (sucesso) ou “barra vermelha” (falha);
- **Diário:** a reunião diária do Scrum (Daily Scrum) fornece um ponto de inspeção diário sobre o andamento do trabalho;
- **Semanas:** ao final de um Sprint, o cliente avalia o que foi entregue e fornece feedback sobre a direção do projeto;
- **Meses:** liberações maiores (*releases*), compostas por vários Sprints, representam pontos de feedback em nível macro do projeto.

3.3 DevOps

O **DevOps** pode ser entendido como uma evolução natural dos métodos ágeis, combinando **Development** (desenvolvimento) e **Operations** (operação). Tradicionalmente, o time que desenvolve o software não é o mesmo que opera esse software em produção no dia a dia: o time de desenvolvimento entrega o software e, a partir daí, uma equipe de operações (muitas vezes do próprio cliente) cuida da execução em produção.

Uma analogia útil: da mesma forma que os métodos ágeis eliminaram a barreira entre o time de desenvolvimento e o cliente (que, no modelo em cascata tradicional, ficava restrita a analistas de sistemas que levantavam requisitos meses antes da entrega), o DevOps busca eliminar a barreira — ou o “silo” — entre o time de desenvolvimento e o time de operação. A ideia é que ambos os times trabalhem de forma integrada, com pessoas representando operação dentro do time de desenvolvimento e vice-versa, compartilhando a mesma sinergia para entregar valor ao cliente continuamente.

O objetivo principal do DevOps é reduzir o tempo entre a escrita do código e sua efetiva disponibilização em produção para os clientes, promovendo entrega contínua e monitoramento constante do software já em operação.

3.4 Práticas essenciais de desenvolvimento

Independentemente do método ou processo escolhido, algumas práticas são consideradas essenciais para qualquer equipe de desenvolvimento de software profissional.

3.4.1 Entrega contínua (Continuous Delivery)

A prática de **entrega contínua** busca eliminar o gargalo de tempo entre o código estar pronto e estar implantado na máquina do cliente. A ideia é que, a partir do momento em que um código é enviado (*commit*) ao repositório, sejam gerados automaticamente testes, verificações e o empacotamento do software para colocação em produção — tudo de forma automatizada. Ferramentas comuns para isso incluem Jenkins, Travis CI e GitHub Actions.

3.4.2 Testes automatizados

Um dos pilares do XP é o uso de testes automatizados, que podem ser escritos após a criação do código (para validá-lo) ou, em uma prática ainda mais rigorosa chamada **Test-Driven Development (TDD)**, escritos *antes* mesmo do código que deverá passar neles.

Dica

Um erro comum de quem está começando é testar o código manualmente: implementar uma funcionalidade, rodá-la, olhar a saída no console e comparar visualmente com o resultado esperado (às vezes usando `print` ou `console.log` espalhados pelo código). Esse tipo de teste manual não é recomendável, porque depende de intervenção humana constante e não escala à medida que a aplicação cresce. Quando uma nova funcionalidade é adicionada, é comum que funcionalidades anteriores quebrem sem que ninguém perceba — daí a importância dos chamados **testes de regressão**: sempre que uma nova funcionalidade é adicionada, todos os testes anteriores também devem ser executados, para garantir que nada foi quebrado.

Com testes automatizados, o próprio computador executa os testes e reporta o resultado (barra verde ou vermelha) em tempo real, assim que uma alteração é feita. Isso permite identificar erros de forma muito mais rápida do que a verificação manual, especialmente à medida que a aplicação cresce em tamanho e complexidade.

3.4.3 Controle de versão

Trabalhar em equipe pressupõe algum sistema de **controle de versão** por trás das operações, permitindo que várias pessoas trabalhem sobre a mesma base de código sem conflitos descontrolados. O XP, inclusive, prega a *propriedade coletiva de código*: não existe a noção de “esse código é meu”; qualquer pessoa da equipe pode alterar qualquer parte do código, desde que passe pelos testes automatizados.

Dica

Mesmo trabalhando sozinho em projetos pessoais, vale adotar um sistema de controle de versão desde o início. Fazendo commits pequenos e frequentes, fica muito mais fácil desfazer (*rollback*) uma alteração equivocada e voltar a uma versão anterior do código — algo bem mais difícil de fazer sem esse histórico.

O **Git** é hoje, disparadamente, o sistema de controle de versão mais usado no mercado. Trata-se de um sistema de controle de versão *distribuído*, criado por Linus Torvalds (o mesmo criador do kernel do Linux). É importante não confundir Git com GitHub: o Git é o software de controle de versão em si, enquanto o GitHub é um site que hospeda repositórios criados com Git, permitindo colaboração online — alternativas populares incluem GitLab e Bitbucket. É perfeitamente possível usar Git localmente, sem depender de nenhum desses serviços, inclusive configurando um servidor Git dentro da própria intranet de uma organização.

3.4.4 Documentação

A prática de documentação é importante tanto do ponto de vista do usuário final (manuais de uso, muitas vezes produzidos por um escritor técnico especializado) quanto do ponto de vista técnico, voltado para os desenvolvedores responsáveis pela manutenção futura do software — que frequentemente não são os mesmos que o criaram originalmente. Essa documentação pode assumir diversas formas: texto corrido, diagramas UML, esquemas de banco de dados ou qualquer outro artefato que sirva de base para a evolução do sistema.

3.4.5 Trabalho em equipe

Vale desfazer um estereótipo comum: a imagem de que profissionais de TI são pessoas isoladas, que trabalham sozinhas com fones de ouvido sem interagir com ninguém. Na prática, o desenvolvimento de software de qualidade depende fortemente do trabalho em equipe, seguindo algum processo ou método compartilhado por todos. É recomendável, inclusive durante a formação universitária, buscar oportunidades de formar equipes para trabalhos práticos, começando a desenvolver essa habilidade desde cedo.

Essa ideia se conecta diretamente ao conceito de profissional em formato T apresentado no capítulo anterior: nada impede que, em uma equipe ágil, um desenvolvedor back-end contribua ocasionalmente com uma página HTML ou uma folha de estilo CSS, ainda que sua especialidade principal seja outra.

3.5 O padrão arquitetural MVC

Um padrão arquitetural que qualquer desenvolvedor web — front-end, back-end ou full stack — certamente vai encontrar é o **MVC** (*Model-View-Controller*).

MVC — Model-View-Controller

O MVC é uma separação explícita entre o conteúdo de uma aplicação, sua apresentação e o componente que faz a ponte entre os dois:

- **Model** (Modelo): os dados e a lógica de negócio da aplicação;
- **View** (Visão): a apresentação desses dados para o usuário, seja em um navegador web, seja em uma tela de aplicativo móvel;
- **Controller** (Controlador): o componente que faz a interação entre o Model e a View.

O MVC é tão relevante para aplicações web que a maioria das linguagens de back-end possui frameworks que já implementam esse padrão de forma nativa: Laravel para PHP, Spring Boot ou JavaServer Faces para Java, Express.js para Node.js, entre outros. É tecnicamente possível implementar o MVC "na mão", em qualquer linguagem, mas na prática quase ninguém faz isso: usar um framework que já aplica o padrão de forma consistente aumenta muito a produtividade.

3.6 A importância da flexibilidade

Para encerrar, vale reforçar uma mensagem central deste capítulo: o desenvolvedor web vai, ao longo da carreira, aprender diversos métodos, processos, padrões e práticas de desenvolvimento. O mais importante não é memorizar essas práticas de forma rígida, mas entender que novas práticas continuarão a surgir e que outras cairão em desuso com o tempo. Quando um projeto termina, é provável que o próximo — talvez com outra equipe, outra abordagem, mesmo dentro da mesma empresa — exija adaptação a novos métodos. Ser flexível o suficiente para incorporar essas mudanças é o que efetivamente impulsiona uma carreira de sucesso em desenvolvimento web: o profissional competente não domina apenas a implementação de código, mas também os padrões, métodos e processos que organizam esse trabalho.

Síntese do Capítulo

- O modelo em cascata (waterfall) entrega valor apenas ao final do projeto e não lida bem com a natureza volátil dos requisitos de software.
- Métodos ágeis usam o ciclo de vida iterativo-incremental, entregando software funcional em iterações curtas e sucessivas, criando pontos frequentes de inspeção e adaptação.
- Scrum é um framework de gerenciamento organizado em Sprints, com artefatos como Product Backlog e Sprint Backlog e eventos como o Daily Scrum.
- Extreme Programming (XP) prescreve práticas técnicas (testes automatizados, TDD, propriedade coletiva de código) organizadas em torno do conceito de feedback loops.

- DevOps combina desenvolvimento e operação em um único time integrado, eliminando o "silo" entre quem escreve o código e quem o mantém em produção.
- Testes automatizados e controle de versão (especialmente com Git) são práticas essenciais e praticamente indispensáveis no desenvolvimento profissional moderno.
- O padrão arquitetural MVC separa dados (Model), apresentação (View) e lógica de controle (Controller), e está presente na maioria dos frameworks de back-end atuais.

Capítulo 4

Conceitos Fundamentais de Redes de Computadores

4.1 Protocolos: a base da comunicação em rede

Um dos conceitos mais importantes em redes de computadores é o de **protocolo**.

Protocolo

Um protocolo é um conjunto de regras que devem ser seguidas por duas máquinas conectadas em rede para que seja possível trocar mensagens entre elas.

Como aplicações web funcionam, por natureza, de forma distribuída, é impossível discutir programação web sem discutir os conceitos de redes de computadores e seus respectivos protocolos. Existem diversos protocolos de comunicação em uso — POP3, SMTP, IMAP, FTP, UDP, entre outros —, mas para o desenvolvedor web os protocolos mais importantes da pilha TCP/IP (ou do modelo de referência ISO-OSI) são o **HTTP** e sua versão segura, o **HTTPS**.

4.1.1 HTTP e HTTPS

O HTTP (*HyperText Transfer Protocol*) foi projetado para transportar documentos escritos em linguagens como HTML. Seu funcionamento básico envolve no mínimo duas partes interagindo: um **cliente** (tipicamente um navegador web) e um **servidor** (um software chamado servidor web). O cliente faz uma requisição a um recurso hospedado no servidor, e o servidor devolve esse recurso como resposta, para ser renderizado no cliente. Esse modelo é chamado de **arquitetura cliente-servidor**.

Toda essa comunicação na *World Wide Web* acontece sobre HTTP ou sobre sua versão mais segura, o HTTPS — representada pelo ícone de cadeado na barra de endereço do navegador. Quando o HTTPS é usado, a comunicação entre cliente e servidor é criptografada: mesmo que alguém intercepte o tráfego, o conteúdo permanece ilegível sem a chave de descryptografia adequada. Sem HTTPS, ferramentas de interceptação de tráfego — usadas tanto por profissionais bem-intencionados quanto por atacantes — conseguem visualizar exatamente as informações trocadas.

Além da mensagem em si, o protocolo HTTP transporta informações adicionais chamadas **cabeçalhos** (*headers*). Os cabeçalhos fornecem metadados sobre a mensagem e sobre o próprio protocolo, e são fundamentais para o desenvolvedor back-end: muitos comportamentos possíveis em nível de programação de servidor só são alcançáveis quando se compreende exatamente como os cabeçalhos HTTP funcionam.

Verbos HTTP

Além dos cabeçalhos, o HTTP define **verbos** (também chamados métodos), sendo os mais usados **GET**, **POST**, **PUT** e **DELETE**. Todo o estilo arquitetural de APIs RESTful se apoia sobre esses verbos para representar operações de leitura, criação, atualização e remoção de recursos.

Hoje já é comum o uso do HTTP/2, embora muitas aplicações ainda rodem sobre HTTP/1.1. Um grande avanço do HTTP/2 é a **multiplexação**: na versão 1.x, se uma página precisasse de vários arquivos diferentes (um script, uma folha de estilo, uma imagem), o navegador precisava abrir uma conexão TCP separada para cada um deles. Com o HTTP/2, uma única conexão é suficiente para obter todos os recursos, reduzindo a sobrecarga de estabelecer múltiplas conexões. Já existem discussões avançadas em torno do HTTP/3, que deverá se tornar, no futuro, o novo padrão para transferência de hipertexto na web.

4.1.2 A pilha TCP/IP

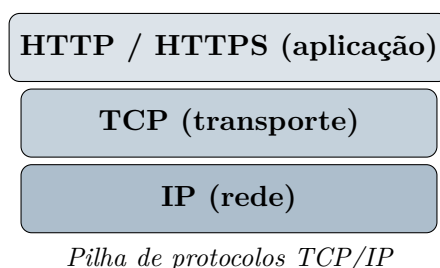
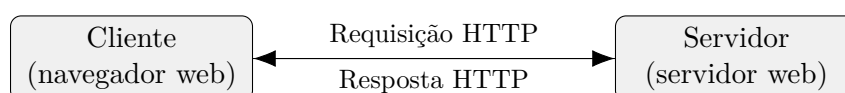
O TCP/IP é o conjunto de protocolos — na verdade, mais do que apenas TCP e IP — que serve de base para a comunicação em redes modernas. Antes da popularização do TCP/IP, nas décadas de 1960 e 1970, redes de computadores tendiam a ser conexões diretas ponto a ponto. O TCP/IP permitiu que diferentes topologias de rede pudessem se comunicar entre si de forma padronizada.

- **TCP** (*Transmission Control Protocol*): protocolo da camada de transporte, responsável por quebrar as mensagens em pacotes menores e garantir a entrega confirmada entre origem e destino;
- **IP** (*Internet Protocol*): protocolo da camada de rede, responsável por endereçar a origem e o destino da transmissão;
- **HTTP**: protocolo da camada de aplicação (ou apresentação), responsável por definir como o conteúdo é transportado entre cliente e servidor.

Quando duas máquinas trocam mensagens em rede, essas mensagens não são enviadas de uma só vez: são quebradas em unidades menores chamadas **pacotes**. Cada pacote pode seguir caminhos diferentes até o destino, graças a algoritmos de roteamento que redirecionam o tráfego caso parte da rede esteja congestionada, aumentando a eficiência da transmissão. O destino final reconstitui a mensagem original a partir dos pacotes recebidos.

Endereços IP identificam unicamente uma máquina dentro de uma rede. Existem endereços **roteáveis** (acessíveis diretamente pela internet) e **não roteáveis** (válidos apenas dentro de uma rede local), sendo estes últimos tipicamente das faixas 192.168.x.x e 10.x.x.x — endereços comuns em redes domésticas e corporativas.

A figura a seguir ilustra como o HTTP se apoia sobre o TCP e o IP para que a comunicação cliente-servidor aconteça na prática.



4.2 Identificando recursos: URI, URL e URN

Para localizar um recurso na web — um arquivo, um documento HTML, um vídeo — usamos o conceito de **URL** (*Uniform Resource Locator*). Formalmente, existem algumas distinções mais refinadas entre URI, URL e URN, mas para fins práticos de desenvolvimento web basta entender:

URL

Segundo a definição da Mozilla Developer Network, uma URL é um localizador que segue um padrão e aponta para um recurso específico na web. Uma URL identifica *o quê* (o recurso) e *onde* (sua localização).

O recurso identificado pode ser um documento HTML, uma imagem, um vídeo, um script ou qualquer outro conteúdo acessível via web. A localização, por sua vez, normalmente corresponde a um endereço IP — mas como decorar sequências de números seria impraticável, usamos o conceito de **servidor de nomes**, que mapeia endereços IP para nomes de domínio legíveis, como `www.google.com`.

Uma URL típica é composta pelas seguintes partes (algumas obrigatórias, outras opcionais):

Estrutura de uma URL

```
http://www.exemplo.com:80/pdf/tubarao.html?param1=valor1&param2=valor2#secao2
\_____/ \_____ / \_____ / \_____ / \_____ /
protocolo      domínio  porta   caminho      query string      âncora
```

- **Protocolo** (`http://` ou `https://`): indica a versão do protocolo de comunicação usada;
- **Nome de domínio** (`www.exemplo.com`): identifica o servidor que hospeda o recurso; poderia ser um endereço IP diretamente, mas o nome de domínio é traduzido para IP por um servidor DNS;
- **Porta** (`:80`): opcional, pois o HTTP usa a porta 80 por padrão (e o HTTPS a porta 443). Uma mesma máquina pode ter várias portas abertas, cada uma

associada a uma aplicação diferente — é comum, por exemplo, servidores de aplicação escutarem na porta 8080;

- **Caminho** (`/pdf/tubarao.html`): indica onde, dentro do servidor, está o recurso desejado. Vale notar que, com a popularização da computação em nuvem, esse “caminho” nem sempre corresponde a um local físico real — pode estar em um servidor virtual;
- **Query string** (`?param1=valor1¶m2=valor2`): parâmetros passados ao recurso, no formato chave-valor separados por `&`. É comum, por exemplo, quando um formulário HTML é enviado usando o método GET;
- **Âncora** (`#secao2`): aponta para um ponto específico dentro do próprio documento, útil para páginas longas onde se deseja direcionar o usuário diretamente a uma seção.

4.3 Servidores de nomes (DNS)

O **DNS** (*Domain Name System*) é o sistema responsável por traduzir nomes de domínio legíveis (como `google.com`) em endereços IP (como `216.239.36.117`). Sem o DNS, seria necessário memorizar sequências numéricas para acessar qualquer site.

O sistema de DNS funciona de forma hierárquica: existem servidores DNS raiz, servidores de domínio de topo (*top-level domain*, como `.com`, `.br`, `.edu`) e servidores autoritativos, responsáveis por uma zona específica da internet. Quando digitamos um endereço como `unifal-mg.edu.br`, uma cadeia de consultas percorre essa hierarquia — do servidor raiz, passando pelo domínio `.br`, até o servidor DNS autoritativo da própria instituição — até que o endereço IP correspondente seja obtido.

Dica

Registrar um domínio próprio significa, na prática, contratar autoridade sobre um *namespace* legível (por exemplo, `meusite.com`), associando esse nome a um endereço IP por meio de um provedor de DNS. Hoje já existem diversas extensões de domínio de topo disponíveis além das tradicionais (`.com`, `.br`, `.edu`), como `.tech` ou `.space`.

4.4 Servidores web e servidores de aplicação

Embora os termos “servidor web” e “servidor de aplicação” sejam às vezes usados como sinônimos, existe uma diferença importante entre eles:

Servidor web vs. servidor de aplicação

- Um **servidor web** tem como objetivo principal servir conteúdo *estático* — arquivos HTML, CSS e JavaScript prontos, sem processamento adicional no momento da requisição;
- Um **servidor de aplicação** estende essa funcionalidade, gerando conteúdo

dinâmico: ao receber uma requisição, ele normalmente consulta um banco de dados, executa alguma lógica sobre esses dados e incorpora o resultado dentro de documentos HTML antes de enviá-los ao navegador.

Desenvolvedores back-end precisam aprender a instalar, configurar, iniciar e parar diversos servidores de aplicação ao longo da carreira. Um desenvolvedor Java, por exemplo, tem à disposição servidores como WildFly, GlassFish, Payara ou Tomcat; cada linguagem de back-end (PHP, Python, Ruby etc.) possui seu próprio ecossistema de servidores de aplicação compatíveis.

4.5 A World Wide Web, computação em nuvem e CDNs

É importante não confundir **internet** com **World Wide Web (WWW)**. A internet é a rede física de computadores, existente desde a década de 1960 (originalmente para fins militares, com a ARPANET). A WWW, por sua vez, é uma *aplicação* que roda sobre essa rede física, utilizando o protocolo HTTP para tornar informações acessíveis, compartilháveis e associadas umas às outras por meio de **hiperlinks**.

Um documento HTML é formado por links que associam esse documento a outros documentos — presentes no mesmo site ou em qualquer outro lugar do mundo. É justamente essa rede de ligações que dá à web o nome de "teia" (*web* significa teia de aranha em inglês): as informações estão interligadas ao redor de todo o planeta, acessíveis a partir de um navegador web instalado nas máquinas dos clientes. Vale destacar que a essência da WWW não depende de estética — CSS não é necessário para que a web, em sua forma mais básica, funcione; o que a torna única é a capacidade de associar documentos entre si.

4.5.1 Computação em nuvem

Não é mais possível falar de desenvolvimento web sem falar em **cloud computing** (computação em nuvem). A nuvem é, essencialmente, uma infraestrutura de servidores físicos, gerenciados por grandes provedores (Google, Amazon, Microsoft, entre outros), que disponibilizam máquinas virtuais e serviços sob demanda, eliminando a necessidade de configurar e manter infraestrutura própria. Isso inclui questões como escalabilidade diante de picos de requisições simultâneas, que passam a ser resolvidas pela plataforma contratada. Os principais provedores de nuvem hoje são Amazon Web Services (AWS), Google Cloud Platform, Microsoft Azure e Alibaba Cloud.

4.5.2 CDN — Content Delivery Network

Outro conceito relevante é o de **CDN** (*Content Delivery Network*, ou rede de entrega de conteúdo). Quanto mais distante geograficamente um servidor estiver do cliente, maior tende a ser o tempo de carregamento de um recurso. Para reduzir esse problema, é comum replicar um mesmo site em vários servidores espalhados pelo mundo, de

forma que cada usuário seja atendido pelo servidor mais próximo geograficamente, melhorando o desempenho de carregamento.

Síntese do Capítulo

- Um protocolo define as regras de comunicação entre máquinas em rede; HTTP e HTTPS são os protocolos mais relevantes da web para o desenvolvedor.
- A pilha TCP/IP organiza a comunicação em camadas: IP endereça, TCP transporta de forma confiável (quebrando dados em pacotes) e HTTP define como o conteúdo é trocado entre cliente e servidor.
- Uma URL identifica e localiza um recurso na web, sendo composta por protocolo, domínio, porta (opcional), caminho, query string e âncora.
- O DNS traduz nomes de domínio legíveis em endereços IP, operando de forma hierárquica através de servidores raiz, de topo e autoritativos.
- Servidores web servem conteúdo estático; servidores de aplicação geram conteúdo dinâmico, normalmente consultando bancos de dados.
- A World Wide Web não é sinônimo de internet: é uma aplicação, baseada em HTTP e hiperlinks, que roda sobre a rede física da internet.
- Computação em nuvem e CDNs são hoje parte indissociável do desenvolvimento e da implantação de aplicações web modernas.

Capítulo 5

As Linguagens Fundamentais da Web: HTML, CSS e JavaScript

5.1 Três linguagens, três responsabilidades

Existem três linguagens fundamentais que qualquer desenvolvedor web precisa conhecer, independentemente de atuar como front-end, back-end ou full stack: **HTML**, **CSS** e **JavaScript**. Cada uma delas tem uma responsabilidade específica e bem definida no desenvolvimento de aplicações para a web:

Responsabilidades de HTML, CSS e JavaScript

- **HTML**: responsável pela *estrutura* e pela *semântica* do documento web;
- **CSS**: responsável pelo *estilo* e pelo *layout* do documento;
- **JavaScript**: linguagem de programação responsável por definir o *comportamento* do documento, permitindo interação com o usuário e, eventualmente, comunicação com um servidor back-end.

Em outras palavras: ao criar um documento HTML, pegamos conteúdo textual e o associamos a imagens, áudio, vídeo e, opcionalmente, folhas de estilo CSS e pequenos programas em JavaScript. O resultado é um documento que não possui apenas estrutura e semântica (fornecidas pelo HTML), mas também estilo (fornecido pelo CSS) e comportamento interativo (fornecido pelo JavaScript).

5.2 HTML: estrutura e semântica

HTML significa *HyperText Markup Language* (linguagem de marcação de hipertexto). É importante destacar: HTML **não é uma linguagem de programação**, mas uma **linguagem de marcação** (*markup language*) de documentos — e é a base de toda a World Wide Web.

A ideia central da web é que documentos estejam associados uns aos outros através de **hiperlinks** (ou simplesmente *links*): elementos de ligação que permitem navegar de um documento para outro, seja dentro do mesmo site, seja para documentos hospedados em servidores completamente diferentes, em qualquer lugar do mundo.

Como linguagem de marcação, HTML define o *papel* de cada trecho de conteúdo dentro de um documento, para que o navegador saiba como renderizá-lo corretamente. Alguns trechos marcados descrevem elementos textuais (um parágrafo, um título, uma lista), enquanto outros descrevem elementos não textuais (uma imagem, um vídeo).

Essas marcações são chamadas de **tags** ou **elementos HTML**, e seguem o formato geral:

Sintaxe de um elemento HTML

```
<nomeDaTag>conteúdo</nomeDaTag>
```

Por exemplo, ao marcar um trecho com `<p>...</p>`, indicamos ao navegador que aquele conteúdo é um parágrafo; ao usar `<ul>...</ul>`, indicamos que se trata de uma lista não ordenada; ao usar `<h1>` até `<h6>`, indicamos títulos e subtítulos com diferentes níveis de importância hierárquica dentro do documento.

Dica

Espaços em branco e indentação dentro de um documento HTML são ignorados durante a renderização — eles existem apenas para tornar o código mais legível para humanos. Um documento HTML tecnicamente funcionaria sem nenhuma formatação (tudo em uma única linha), mas isso prejudicaria muito a leitura e a manutenção do código por outros desenvolvedores. Ou seja: indente e formate seu código sempre, mesmo sabendo que o navegador “não precisa” disso.

5.3 CSS: estilo e apresentação

CSS (*Cascading Style Sheets*, ou folhas de estilo em cascata) também é, originalmente, uma linguagem declarativa, embora tenha incorporado, ao longo dos anos, características cada vez mais próximas de uma linguagem de programação (como variáveis, transições e animações). Enquanto HTML fornece estrutura e semântica, CSS define a **aparência**: cores, fontes, espaçamentos, layout e responsividade.

Nos primórdios da web, era comum misturar diretamente questões de estilo dentro do próprio HTML (por exemplo, usando atributos de cor e fonte diretamente nas tags, ou até criando layouts inteiros com tabelas HTML). Essa prática é hoje considerada inadequada, porque mistura duas responsabilidades que deveriam estar separadas: *conteúdo/estrutura* de um lado e *apresentação* de outro.

Separação de responsabilidades

Separar CSS de HTML traz vantagens práticas importantes: diferentes pessoas podem trabalhar em paralelo (uma cuidando de estilos, outra de estrutura e semântica), e alterar a aparência de uma página não exige mexer diretamente na sua estrutura. Mesmo quando nenhuma folha de estilo é escrita, os navegadores aplicam estilos padrão próprios (por isso uma lista sem CSS ainda aparece com marcadores, e um link aparece sublinhado e azul).

O funcionamento do CSS se baseia em **regras**, formadas por um **seletor** (que identifica quais elementos do HTML devem receber o estilo) e um bloco de **propriedades**:

CSS

```
h1 {  
  color: red;  
  font-size: 5rem;  
}  
  
p {  
  color: black;  
}
```

O nome "cascata" vem justamente do fato de que as regras de estilo são aplicadas em camadas sucessivas: primeiro os estilos padrão dos navegadores, depois os estilos definidos pelo desenvolvedor, sendo que regras definidas posteriormente (ou mais específicas) tomam precedência sobre regras anteriores.

Um dos grandes benefícios do CSS hoje é permitir o chamado **design responsivo**: como a web é acessada a partir de uma enorme variedade de dispositivos — desktops, laptops, tablets, celulares, leitores de tela, até relógios com acesso à internet — é fundamental que um mesmo documento se adapte visualmente a telas de tamanhos muito diferentes. Antigamente, isso exigia criar versões separadas de um mesmo site; hoje, recursos como *media queries* do CSS resolvem esse problema de forma muito mais elegante, permitindo que o mesmo conjunto de HTML e CSS se ajuste automaticamente conforme a largura da tela disponível.

Dica

Uma boa forma de visualizar concretamente o poder do CSS é observar sites como o *CSS Zen Garden*, que apresentam o mesmo conteúdo HTML estilizado com dezenas de folhas de estilo completamente diferentes — a mesma estrutura semântica pode resultar em aparências radicalmente distintas dependendo apenas do CSS aplicado.

5.4 JavaScript: comportamento e interatividade

JavaScript é uma linguagem de programação — diferente de HTML e CSS, que são linguagens de marcação/estilo — responsável por fornecer **comportamento** aos documentos web. Uma boa definição é pensar em JavaScript como uma camada interativa que reside sobre HTML e CSS: HTML dá estrutura e semântica, CSS dá estilo, e JavaScript permite alterar ambos em tempo de execução, respondendo a eventos gerados pela interação do usuário com a página.

JavaScript permite escrever pequenos programas executados diretamente dentro do navegador, capazes de alterar o HTML e o CSS de um documento já carregado, sem que seja necessário fazer uma nova requisição ao servidor. Alguns exemplos típicos de uso: ocultar ou exibir conteúdo dinamicamente, criar animações, responder a cliques de mouse ou teclas pressionadas, validar formulários antes do envio, ou até carregar novo conteúdo de forma assíncrona.

DOM — Document Object Model

JavaScript manipula o documento por meio do **DOM** (*Document Object Model*), a representação em memória, em forma de árvore, dos elementos de um documento HTML carregado pelo navegador. Esse conceito será aprofundado em capítulo posterior, mas vale adiantar que é através do DOM que o código JavaScript "enxerga" e modifica a página.

Vale destacar que o JavaScript hoje não é usado apenas no navegador (também chamado de **Vanilla JavaScript** quando usado sem frameworks adicionais): com o surgimento do **Node.js**, um ambiente de execução JavaScript que roda do lado do servidor, desacoplado do navegador, tornou-se possível construir aplicações back-end inteiras usando JavaScript — um desenvolvedor que já domina JavaScript no front-end pode aprender a usá-lo também no back-end, usando frameworks como o Express.

5.5 Um primeiro exemplo prático

Para consolidar a ideia de que essas três linguagens trabalham juntas, mas com papéis distintos, veja um exemplo mínimo de página web combinando HTML, CSS e JavaScript — um "Hello World" interativo:

HTML + CSS + JavaScript combinados

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Meu primeiro documento web</title>
  <style>
    body {
      background-color: #1c3d5a;
      color: white;
      font-family: sans-serif;
      text-align: center;
      padding-top: 4rem;
    }
    button {
      padding: 0.6rem 1.2rem;
      font-size: 1rem;
      border-radius: 6px;
      border: none;
      cursor: pointer;
    }
  </style>
</head>
<body>
  <h1>Hello, Cruel World!</h1>
  <p>Esta página combina HTML, CSS e JavaScript.</p>
```

```
<button id="botaoSaudacao">Clique aqui</button>

<script>
  const botao = document.getElementById('botaoSaudacao');
  botao.addEventListener('click', function () {
    alert('Bem-vindo ao mundo do desenvolvimento web!');
  });
</script>
</body>
</html>
```

Observe a divisão de responsabilidades mesmo dentro de um único arquivo:

- As tags `<h1>`, `<p>` e `<button>` definem a **estrutura** do conteúdo (HTML);
- O bloco `<style>` define a **aparência** — cor de fundo, cor de texto, alinhamento (CSS);
- O bloco `<script>` define o **comportamento**: quando o botão é clicado, uma janela de alerta é exibida (JavaScript), através da manipulação do DOM via `document.getElementById`.

Dica

Embora seja tecnicamente possível misturar HTML, CSS e JavaScript no mesmo arquivo (como no exemplo acima, usando as tags `<style>` e `<script>`), a **boa prática** recomendada é manter cada linguagem em seu próprio arquivo (`.html`, `.css` e `.js`), conectados entre si por meio de referências no HTML. Isso facilita a manutenção, a divisão de trabalho em equipe e a reutilização de código. O exemplo combinado acima é útil apenas para fins didáticos, para visualizar rapidamente as três linguagens interagindo.

Síntese do Capítulo

- HTML, CSS e JavaScript têm responsabilidades complementares: estrutura/semântica, estilo/layout e comportamento/interatividade, respectivamente.
- HTML é uma linguagem de marcação (não de programação) baseada em tags que descrevem o papel de cada trecho do conteúdo para o navegador.
- CSS separa apresentação de conteúdo através de regras compostas por seletores e propriedades, seguindo o princípio de cascata que dá nome à linguagem.
- O design responsivo em CSS permite que um mesmo documento se adapte a telas de tamanhos muito diferentes, algo essencial na era do acesso multiplataforma à web.

- JavaScript é uma linguagem de programação real, capaz de manipular o DOM e responder a eventos do usuário, alterando a página em tempo de execução sem recarregar o servidor.
- O Node.js estendeu o alcance do JavaScript para o back-end, permitindo que desenvolvedores usem a mesma linguagem em toda a pilha da aplicação.
- A boa prática recomendada é manter HTML, CSS e JavaScript em arquivos separados, ainda que seja tecnicamente possível combiná-los em um único documento.

Capítulo 6

Escrevendo, Implantando e Acessando Websites

6.1 O que é preciso para começar

Depois de conhecer as três linguagens fundamentais da web, é hora de colocar a mão na massa e entender, na prática, as diferentes formas de escrever, implantar e acessar um documento web. Para desenvolver documentos web, o mínimo necessário é:

- um **editor de texto** capaz de salvar arquivos em formato de texto puro (*plain text*), sem formatação rica — exemplos incluem o Bloco de Notas no Windows, o **nano** no Linux ou o TextEdit no macOS;
- um **navegador web** — Edge, Chrome, Firefox, Safari ou qualquer outro, todos funcionam para os fins deste curso e já trazem embutido um conjunto de ferramentas de desenvolvedor úteis para depurar e testar páginas.

Dica

Ao desenvolver páginas web, é comum manter duas janelas abertas simultaneamente: uma com o editor de código e outra com o navegador, exibindo o resultado enquanto o código é alterado. Quem possui mais de um monitor pode dedicar um a cada finalidade; caso contrário, uma boa opção é dividir a tela ao meio ou simplesmente alternar entre as janelas conforme necessário.

Um cuidado importante ao usar editores de texto simples (como o TextEdit no macOS): é preciso garantir explicitamente que o documento seja salvo como **texto não formatado** (*plain text*), e não como um documento com formatação rica (estilo processador de texto). Caso contrário, ao abrir o arquivo no navegador, ele não será interpretado como HTML e aparecerá com o código bruto na tela, ou de forma incorreta.

6.2 Boa prática: separar HTML, CSS e JavaScript

Como já mencionado no capítulo anterior, embora seja tecnicamente possível escrever HTML, CSS e JavaScript misturados dentro de um único arquivo, a boa prática de desenvolvimento web recomenda manter cada linguagem em seu próprio arquivo:

- um arquivo **.html** para a estrutura;
- um arquivo **.css** para os estilos;

- um arquivo `.js` para o comportamento.

Essa separação deve ser buscada mesmo quando se está experimentando código copiado de terceiros ou de editores online: o ideal é sempre reorganizar o conteúdo em suas partes apropriadas antes de considerá-lo pronto.

6.3 Formas de acessar um documento web

Existem diferentes maneiras de testar e visualizar um documento web durante o desenvolvimento, cada uma com vantagens e limitações. Vale a pena conhecer todas elas, já que aparecerão com frequência ao longo da disciplina.

6.3.1 Abrindo diretamente pelo sistema de arquivos

A forma mais simples — e também a menos recomendada — é simplesmente dar um duplo clique no arquivo `.html` para abri-lo diretamente no navegador. Ao fazer isso, repare que a barra de endereço do navegador exibe algo como `file:///Users/nome/Desktop/arquivo.html`: isso indica que o navegador está acessando o arquivo diretamente pelo **sistema de arquivos local**, e não através de um servidor web.

Dica

Testar páginas web abrindo diretamente pelo sistema de arquivos é uma prática ruim porque o navegador perde acesso às características do protocolo HTTP, e alguns links relativos ou requisições podem não funcionar corretamente. Além disso, o endereço exibido é um **caminho absoluto**, que muda conforme o sistema operacional e a estrutura de pastas de cada máquina — ou seja, não é portátil. Ainda assim, essa forma funciona para testes rápidos e simples, e é normal usá-la ocasionalmente.

6.3.2 Editores online

Outra alternativa é usar **editores online**, como CodePen ou JSBin. A grande vantagem desses ambientes é não exigir nenhuma instalação local, além de permitir compartilhar facilmente o trabalho através de uma URL, útil para colaboração e revisão com colegas.

Ao usar um editor como o CodePen, geralmente já existem painéis separados para HTML, CSS e JavaScript — reforçando naturalmente a boa prática de manter as linguagens organizadas. É possível também explorar exemplos públicos feitos por outros desenvolvedores e fazer um **fork** (cópia para a própria conta) de qualquer projeto interessante, desde que se possua uma conta gratuita na plataforma. Sem estar logado, ainda é possível editar e testar o código temporariamente, mas não será possível salvar as alterações.

6.3.3 Servidor web local instalado na máquina

Uma terceira alternativa, mais próxima do que acontece em produção, é instalar um **servidor web** na própria máquina e implantar (*deploy*) o documento dentro do diretório que esse servidor observa. O professor demonstra esse processo usando o **Nginx**, um dos servidores web mais usados atualmente (ao lado do tradicional Apache).

De forma resumida, o processo de implantação local envolve:

1. Instalar o servidor web (por exemplo, via gerenciador de pacotes: `brew install nginx` no macOS);
2. Iniciar o servidor (`brew services start nginx`, ou apenas `nginx` para rodá-lo sem registrá-lo como serviço permanente);
3. Verificar se o servidor está escutando na porta correta (por padrão, o Nginx costuma escutar na porta 8080 em instalações via Homebrew) usando um comando como `netstat -an | grep 8080`;
4. Copiar os arquivos do projeto (desenvolvidos, por exemplo, na área de trabalho) para o diretório raiz do servidor (no caso do Nginx no macOS, tipicamente `/usr/local/var/www`);
5. Acessar o navegador em `http://localhost:8080` para visualizar o resultado.

Dica

Se, ao acessar o endereço do servidor, aparecer uma mensagem de erro **404 Not Found**, isso significa que o recurso solicitado não foi encontrado no caminho esperado — geralmente porque o arquivo ainda não foi copiado para o diretório correto, ou porque há um erro de digitação no caminho. O código **404** é um dos chamados **códigos de status HTTP**, que serão estudados com mais detalhes adiante no curso.

Repare que, quando os arquivos são servidos por um servidor web de verdade, a barra de endereço do navegador passa a exibir algo como `http://localhost:8080/arquivo.html`, em vez do caminho absoluto do sistema de arquivos. Isso reflete o fato de que agora se está de fato usando o protocolo HTTP, com todas as suas características disponíveis — diferente do acesso direto via sistema de arquivos.

Depois de testado o processo de deploy, é possível parar o servidor normalmente (por exemplo, `brew services stop nginx`, ou `nginx -s stop`). É recomendável manter o servidor parado quando não estiver sendo utilizado para desenvolvimento, evitando consumo desnecessário de memória da máquina.

6.3.4 Editores de código com servidor embutido (Live Server)

Por fim, a alternativa mais prática para o dia a dia de desenvolvimento é usar um editor de código mais completo — como o **Visual Studio Code**, que será adotado ao longo deste curso — combinado com uma extensão como o **Live Server**.

O Live Server é, na prática, um servidor web bastante simples (não indicado para produção, apenas para testes locais durante o desenvolvimento), que roda tipicamente na porta 5500. Sua grande vantagem é a integração direta com o editor: basta clicar em “Go Live” na barra inferior do Visual Studio Code para que o navegador abra automaticamente exibindo o documento, e qualquer alteração salva no código é refletida quase instantaneamente no navegador, sem necessidade de atualizar a página manualmente.

Live Server vs. servidor de produção

É importante diferenciar o papel de cada ferramenta: servidores como Nginx ou Apache são projetados para ambientes de **produção**, capazes de lidar com tráfego real e configurações avançadas de segurança e desempenho. Já o Live Server é uma ferramenta **exclusivamente de desenvolvimento**, pensada para agilizar o ciclo de testes locais, sem qualquer pretensão de uso em produção.

Dica

Para instalar o Live Server no Visual Studio Code, acesse a aba de extensões (ícone de blocos na barra lateral), procure por “Live Server” e clique em instalar. Depois disso, basta abrir a pasta do projeto no editor e clicar em “Go Live” no canto inferior direito da janela para iniciar o servidor local automaticamente.

6.4 Resumo comparativo das formas de acesso

A tabela a seguir resume as quatro formas apresentadas neste capítulo para escrever, testar e visualizar um documento web:

Forma de acesso	Vantagens	Limitações
Sistema de arquivos (duplo clique)	Simplicidade máxima, não exige nenhuma instalação	Não usa HTTP de fato; caminhos absolutos não portáteis; links podem falhar
Editor online (CodePen, JSBin)	Sem instalação; fácil de compartilhar; ótimo para experimentação rápida	Requer conexão à internet; salvar exige conta na plataforma
Servidor web local (Nginx, Apache)	Ambiente muito próximo de produção real	Exige instalação e configuração manual; requer redeploy a cada alteração
Live Server (extensão de editor)	Recarregamento automático; integração direta com o editor	Não deve ser usado em produção; é apenas uma ferramenta de desenvolvimento

Ao longo deste curso, a combinação mais usada será o Visual Studio Code com a extensão Live Server, por unir simplicidade de uso com um fluxo de trabalho ágil e próximo do que se espera de um ambiente moderno de desenvolvimento front-end. Ainda assim, é importante ter praticado e compreendido as demais formas de acesso,

já que situações reais de trabalho — e mesmo a implantação final de um projeto em produção — exigirão o uso de servidores web de verdade, como Nginx ou Apache.

Síntese do Capítulo

- Para escrever documentos web básicos, basta um editor de texto simples (capaz de salvar em texto puro) e um navegador web.
- Abrir um arquivo HTML diretamente pelo sistema de arquivos (duplo clique) funciona para testes rápidos, mas não reflete o comportamento real do protocolo HTTP.
- Editores online como CodePen e JSBin permitem testar e compartilhar código sem nenhuma instalação, com painéis separados para HTML, CSS e JavaScript.
- Servidores web locais, como Nginx, exigem instalação e configuração, mas reproduzem fielmente o ambiente de um servidor de produção real.
- O Live Server, extensão do Visual Studio Code, oferece recarregamento automático do navegador a cada alteração salva, sendo ideal apenas para desenvolvimento, nunca para produção.
- Erros como o código de status HTTP 404 indicam que o recurso solicitado não foi encontrado no caminho especificado do servidor.
- Independentemente da ferramenta usada para testar localmente, a boa prática de separar HTML, CSS e JavaScript em arquivos distintos deve ser sempre respeitada.

Capítulo 7

Desvendando o DOM (Document Object Model)

Até este ponto, o leitor já sabe que um navegador web recebe um documento HTML de um servidor e o exibe dentro do seu *viewport*. Essa afirmação, porém, é incompleta. Antes de qualquer coisa aparecer na tela, o navegador realiza um passo intermediário fundamental: ele lê o documento HTML recebido e constrói, em memória, uma representação estrutural e hierárquica desse documento. Essa representação é chamada de DOM — *Document Object Model*, ou Modelo de Objetos do Documento.

Compreender o DOM é uma das etapas mais importantes no início dos estudos de desenvolvimento web, porque é essa estrutura — e não o arquivo HTML em si — que as linguagens CSS e JavaScript efetivamente manipulam. Quando se aplica um estilo ou se adiciona interatividade a uma página, não se está alterando o arquivo de texto que chegou ao navegador; está-se alterando a árvore que representa esse arquivo na memória do navegador.

7.1 O que é o DOM

Document Object Model

O DOM é uma representação estrutural e hierárquica dos elementos presentes em um documento web e do relacionamento entre esses elementos. Ele é criado automaticamente pelo navegador no momento em que este recebe um documento HTML para renderização, e é essa árvore — não o arquivo original — que fica disponível para ser lida e modificada por CSS e JavaScript.

Sempre que o leitor digita um endereço na barra de navegação e o navegador recebe um documento HTML como resposta, ocorre o seguinte processo: o navegador realiza o *parsing* (a leitura e interpretação) do texto HTML e, para cada elemento de marcação encontrado, cria um **nó** correspondente. Esses nós são então conectados uns aos outros de acordo com a relação de aninhamento presente no HTML original, formando uma estrutura em árvore.

É por essa razão que, no dia a dia da programação web, expressões como “percorrer o DOM”, “navegar pelo DOM” ou “buscar um elemento no DOM” são tão comuns. Uma das atividades centrais de qualquer desenvolvedor front-end é localizar um elemento específico (ou um conjunto de elementos) dentro dessa árvore, para então aplicar um estilo CSS ou associar um comportamento em JavaScript a ele.

7.2 Da marcação HTML à árvore de nós

Para entender como o HTML se transforma em DOM, é preciso lembrar que cada elemento HTML é normalmente formado por uma tag de abertura, um conteúdo e uma tag de encerramento (por exemplo, `<h1>Título</h1>`). Quando o navegador realiza o parsing desse elemento, ele cria um nó na árvore do DOM para representá-lo. A relação de aninhamento entre as tags do documento — um elemento dentro de outro — se traduz diretamente em relações de parentesco entre nós: pais, filhos e irmãos, exatamente como em qualquer estrutura de árvore estudada em outras disciplinas de ciência da computação.

Considere um documento HTML mínimo, com apenas um elemento `html`, contendo `head` e `body`, e dentro do `body` um `header`, um `nav` e um `link`. A Figura 7.1 mostra lado a lado o trecho de código HTML e a árvore DOM que o navegador cria a partir dele.

Um HTML simples e sua árvore DOM

HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>Meu Site</title>
  </head>
  <body>
    <header>
      <nav>
        <ul>
          <li><a href="#">Início</a></li>
        </ul>
      </nav>
    </header>
  </body>
</html>
```

Observe a raiz da árvore: o elemento `html` é o único elemento que não possui pai — ele é o ancestral de todos os demais nós do documento. A partir dele, a árvore se ramifica em `head` e `body`, e cada um desses se ramifica novamente, até chegar aos elementos mais internos, que não possuem filhos (as chamadas “folhas” da árvore). O elemento `a` (âncora, usado para links), por exemplo, é filho de `li`, que é filho de `ul`, que é filho de `nav`, que é filho de `header`, que é filho de `body`.

7.3 A metáfora da caixa: introdução ao Box Model

Uma forma extremamente útil de visualizar cada nó do DOM é enxergá-lo como uma **caixa**. Cada elemento HTML, ao ser transformado em nó, passa a ter propriedades associáveis como largura, altura, espaçamento interno e externo. Essa associação entre elemento e caixa é a base conceitual do que, mais adiante no estudo de CSS, será

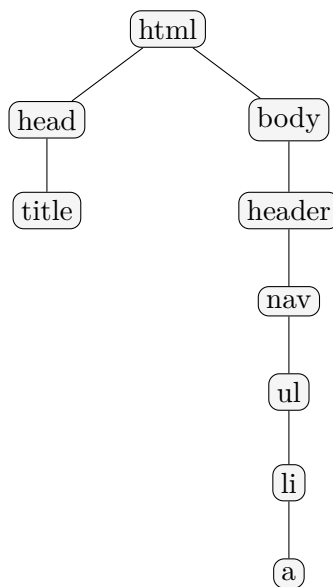


Figura 7.1: Correspondência entre elementos HTML e nós da árvore DOM. Cada elemento de marcação vira um nó; o aninhamento no código vira uma relação de parentesco na árvore.

formalizado como *box model* (modelo de caixas) — um dos conceitos mais importantes para quem deseja estruturar layouts corretamente.

Por ora, basta reter a ideia: pensar em cada nó do DOM como uma caixa retangular, que pode conter outras caixas menores dentro de si, facilita enormemente a compreensão de como CSS e JavaScript atuam sobre a página.

Dica

Ao projetar uma página nova, é uma boa prática — mesmo que seja apenas um rascunho mental ou em papel — esboçar a hierarquia de elementos HTML que você pretende usar antes de escrever o código. Pensar “isto é uma caixa dentro daquela caixa” ajuda a decidir a estrutura de `divs`, `sections` e demais elementos de bloco antes mesmo de abrir o editor.

7.4 Como CSS e JavaScript interagem com o DOM

Uma vez que o DOM existe, ele se torna o alvo de duas linguagens complementares:

- **CSS** define regras que *selecionam* nós da árvore (por exemplo, todos os elementos `p`, ou o elemento com determinada classe) e aplicam propriedades visuais — cor, fonte, espaçamento — a esses nós.
- **JavaScript** localiza nós específicos na árvore e adiciona comportamento a eles, como reagir a um clique, alterar texto dinamicamente ou até criar e inserir novos nós na árvore em tempo de execução.

Um exemplo simples ilustra bem essa segunda interação: um botão HTML pode ter um *event listener* (um “ouvinte de evento”) associado a ele em JavaScript. Quando

o usuário clica no botão, o navegador dispara um evento de clique; o código JavaScript captura esse evento e, em resposta, cria um novo elemento — por exemplo, um parágrafo — e o insere na árvore do DOM. Nesse instante, o DOM literalmente cresce: um novo nó passa a existir na árvore, e o navegador imediatamente re-renderiza a área afetada do *viewport* para refletir essa mudança. É importante entender que esse novo parágrafo nunca existiu no arquivo HTML original — ele foi criado dinamicamente, apenas na árvore em memória.

O CSS, por sua vez, segue um conceito de **herança**: um estilo aplicado a um elemento pai tende a ser herdado por seus elementos filhos, a menos que o filho declare explicitamente uma propriedade diferente, sobrescrevendo o valor herdado. Esse comportamento em cascata — que dá nome à sigla CSS (*Cascading Style Sheets*, Folhas de Estilo em Cascata) — só faz sentido porque existe uma árvore de nós com relações de parentesco bem definidas por trás da página.

7.5 O conceito de eventos

Segundo a definição do MDN (Mozilla Developer Network), **eventos** são “sinalizações de ocorrências que acontecem no sistema que você está desenvolvendo, das quais o sistema o informa para que você possa responder a elas, caso deseje”. Em outras palavras, um evento é qualquer acontecimento dentro do navegador — uma ação do usuário (clicar, passar o mouse, digitar) ou algo disparado pelo próprio navegador (a página terminou de carregar, uma conexão falhou) — que pode ser capturado e tratado por código.

O processo de reagir a um evento é chamado de **tratamento de eventos** (*event handling*) e segue, de forma resumida, três passos:

1. Identificar qual elemento (ou nó do DOM) deve ser observado.
2. Determinar qual evento deve ser escutado nesse elemento (clique, passar o mouse, envio de formulário etc.).
3. Definir a lógica — o algoritmo — que deve ser executada quando o evento ocorrer.

Esse tripé é a base de praticamente toda a interatividade que se constrói em páginas web modernas, e será revisitado repetidamente ao longo dos estudos de JavaScript.

Síntese do Capítulo

- O DOM é a representação em memória, estruturada como árvore, que o navegador cria a partir de um documento HTML recebido do servidor.
- CSS e JavaScript não manipulam o arquivo HTML original: eles leem e alteram a árvore DOM.
- Cada elemento de marcação HTML vira um nó na árvore; o aninhamento das tags define as relações de parentesco (pai, filho, irmão) entre os nós.

- Pensar em cada nó como uma “caixa” é a introdução intuitiva ao futuro conceito de box model do CSS.
- Adicionar ou remover elementos via JavaScript significa, na prática, adicionar ou remover nós da árvore DOM em tempo real.
- O CSS aplica estilo aos nós selecionados e propaga esse estilo por herança aos elementos filhos, salvo sobrescrita explícita.
- Eventos são ocorrências que podem ser capturadas e tratadas por código, formando a base da interatividade em páginas web.

Capítulo 8

Ferramentas de Desenvolvimento: DevTools, Frameworks e Bibliotecas Front-End

Depois de compreender o que é o DOM, o próximo passo natural é conhecer as ferramentas que permitem observar e manipular essa árvore diretamente no navegador, bem como o ecossistema de editores, frameworks e bibliotecas que os desenvolvedores front-end utilizam no dia a dia. Este capítulo reúne dois assuntos complementares: primeiro, as ferramentas de desenvolvedor embutidas nos navegadores (as chamadas *DevTools*); depois, um panorama de editores de código, frameworks e bibliotecas que ampliam as capacidades de HTML, CSS e JavaScript.

8.1 Ferramentas de desenvolvedor do navegador (DevTools)

Todo navegador web moderno — Chrome, Firefox, Safari, Edge, entre outros — já vem com um conjunto integrado de ferramentas voltado especificamente para quem desenvolve páginas web. Esse conjunto é chamado de *Browser Developer Tools*, ou simplesmente DevTools. Ele permite inspecionar o DOM de qualquer página carregada, depurar código JavaScript, testar alterações de CSS em tempo real e investigar requisições de rede, entre muitas outras funções.

Dica

Veja como abrir e usar o DevTools pela primeira vez:

1. **Abrir:** pressione a tecla F12, ou o atalho **Ctrl+Shift+I** (Windows/Linux) ou **Cmd+Option+I** (Mac). Alternativamente, clique com o botão direito em qualquer parte da página e escolha “Inspecionar” ou “Inspecionar elemento”.
2. **Aba Elements (ou Elementos):** mostra o DOM da página em forma de árvore navegável. Ao passar o mouse sobre um nó nessa árvore, o elemento correspondente é destacado visualmente na página. É também aqui que se veem os estilos CSS aplicados a cada elemento selecionado, incluindo estilos padrão do próprio navegador.
3. **Aba Console:** um interpretador de JavaScript interativo. É possível digitar comandos diretamente e executá-los sobre o documento carregado, útil tanto para testes rápidos quanto para depuração de erros reportados pelo próprio navegador.

4. **Aba Network (Rede):** lista todas as requisições feitas pela página (arquivos HTML, CSS, JavaScript, imagens, chamadas a APIs), mostrando tempo de resposta, status HTTP e conteúdo de cada requisição.
5. **Clique com o botão direito em um elemento específico e escolha “Inspecionar”:** em vez de abrir o DevTools genericamente e procurar o elemento na árvore, essa opção já leva diretamente ao nó correspondente na aba Elements — extremamente útil em páginas grandes.

Um detalhe importante para o iniciante: alterações feitas dentro do DevTools — como editar um valor de CSS na aba Elements ou digitar um comando no Console — existem apenas em memória, isto é, apenas na árvore DOM daquela sessão do navegador. Nenhum arquivo do servidor é alterado. Assim que a página é recarregada, o navegador refaz a requisição HTTP, recebe novamente o documento original, reconstrói o DOM do zero e aplica os estilos definidos no CSS real — descartando qualquer experimento feito anteriormente. Isso torna o DevTools um ambiente seguro para testar hipóteses: é possível, por exemplo, abrir o site de uma universidade, mudar a cor de fundo do elemento `html` diretamente no painel de estilos e observar o resultado, sem qualquer risco de afetar o site real.

Embora cada navegador organize sua interface de um jeito ligeiramente diferente — o Firefox, por exemplo, separa estilos “computados” e a lista de regras CSS em painéis distintos, e costuma indicar claramente quando um estilo veio de uma folha embutida (*inline*) no próprio HTML — o conjunto de funcionalidades oferecido é essencialmente o mesmo entre eles. Vale a pena, com a prática, conhecer as particularidades do navegador que se usa no dia a dia, mas também ter familiaridade básica com mais de um, já que certas tarefas de depuração podem ser mais convenientes em uma ferramenta específica.

8.2 Editores de código e ambientes de desenvolvimento

A ferramenta mais utilizada por qualquer desenvolvedor, independentemente da linguagem, é o editor de código. Vale diferenciar dois níveis de ferramentas:

- **Editores de texto puro**, como Bloco de Notas, `nano` ou `vim` em sua forma mais básica: gravam texto simples, sem nenhum auxílio à programação.
- **Editores de código**, um nível acima: oferecem *syntax highlighting* (destaque colorido da sintaxe), indentação automática e outros auxílios, sem impor uma estrutura rígida de projeto. O Visual Studio Code é hoje o mais popular dessa categoria para desenvolvimento web, seguido por opções como Sublime Text, Atom e Brackets.
- **IDEs** (Ambientes de Desenvolvimento Integrados): ferramentas mais completas e pesadas, que tendem a estruturar todo o fluxo de trabalho — criação de projeto, compilação, testes, depuração — dentro de um ambiente único, frequentemente

com assistentes (*wizards*) guiando o desenvolvedor. O Visual Studio (não confundir com o Visual Studio Code) é um exemplo de IDE completa.

Alguns critérios ajudam a avaliar a qualidade de um editor ou IDE: a capacidade de detectar automaticamente a linguagem do arquivo aberto e sugerir extensões apropriadas; o acesso facilitado à documentação da linguagem ou biblioteca em uso; a verificação de erros de sintaxe em tempo real, apontando problemas antes mesmo da execução do código; e recursos de formatação automática e boas práticas de estilo. Ao longo desta disciplina, o Visual Studio Code será a ferramenta de referência para exemplos locais, complementado eventualmente por editores online, como o CodePen, que permitem escrever e compartilhar trechos de HTML, CSS e JavaScript sem qualquer instalação.

8.3 Por que usar frameworks e bibliotecas

Uma dúvida comum de quem está começando é: se HTML, CSS e JavaScript já são as três linguagens centrais da web, por que aprender mais alguma coisa? A resposta tem duas partes. Primeiro, essas três linguagens são as centrais do **front-end**, mas existem inúmeras linguagens e frameworks dedicados ao back-end (Java, PHP, Python, Ruby, C#, entre outros), de modo que o desenvolvedor eventualmente precisará conhecer alguma delas também. Segundo, mesmo dentro do front-end, frameworks e bibliotecas tornam o desenvolvimento mais eficiente.

Framework versus biblioteca

Uma **biblioteca** é um conjunto de funções prontas que o desenvolvedor chama conforme a necessidade, mantendo o controle do fluxo do programa. Um **framework** é mais abrangente: ele impõe uma estrutura e um fluxo de trabalho, e é o framework quem chama o código do desenvolvedor em pontos predefinidos — por isso se diz que, com frameworks, “a inversão de controle” passa a ser do framework, não do programador.

As vantagens de usar essas ferramentas incluem: reuso de código já testado, adoção implícita de boas práticas e padrões de projeto (*design patterns*), maturidade acumulada ao longo de anos de uso pela indústria, e ganho de velocidade de desenvolvimento. A contrapartida costuma ser uma pequena perda de desempenho — por se tratar de uma camada adicional de abstração — e uma curva de aprendizado inicial mais acentuada, que tende a compensar rapidamente conforme o desenvolvedor ganha fluência.

Nesta disciplina, a opção pedagógica é estudar primeiro HTML, CSS e JavaScript em sua forma pura, sem frameworks — muitas vezes chamada de *vanilla* — justamente para que o leitor construa uma base sólida antes de adotar qualquer ferramenta que abstraia esses fundamentos.

8.4 Panorama de bibliotecas e frameworks front-end

Entre as bibliotecas e frameworks mais relevantes do mercado, destacam-se:

- **Lodash**: biblioteca utilitária que facilita a manipulação de arrays, números, objetos e strings em JavaScript, suavizando construções da linguagem consideradas pouco intuitivas por parte da comunidade.
- **D3.js** (*Data-Driven Documents*): biblioteca voltada à criação de gráficos interativos, combinando dados com documentos estruturados.
- **jQuery**: uma das bibliotecas mais influentes da história do JavaScript front-end, criada para facilitar a manipulação do DOM. Embora hoje seja considerada relativamente obsoleta — porque grande parte de suas ideias foi incorporada às especificações modernas do próprio JavaScript — seu impacto histórico e conceitual permanece relevante, e ainda é encontrada em muitos projetos legados.
- **Bootstrap**: framework CSS/JavaScript que fornece componentes visuais prontos (carrosséis, janelas modais, grades responsivas) através de classes que se aplicam diretamente no HTML. É o mais antigo e um dos mais populares frameworks desse tipo, ao lado de alternativas como o Foundation.
- **Angular**: originalmente um framework, hoje já é tratado por muitos como uma plataforma completa, dada sua abrangência. É amplamente adotado em aplicações corporativas de grande porte.
- **React**: atualmente uma das bibliotecas mais populares para construção de interfaces gráficas ricas. Introduziu conceitos hoje centrais no desenvolvimento front-end, como a arquitetura baseada em componentes e o *DOM virtual* — uma técnica que permite gerenciar atualizações do DOM real de forma mais eficiente.
- **Vue.js**: mais recente que Angular e React, é frequentemente descrito como o mais fácil de aprender entre os três, com uma curva de aprendizado mais suave.

Vale notar que, embora sejam tecnologias distintas, Angular, React e Vue.js compartilham conceitos fundamentais de arquitetura de componentes, de modo que aprender um deles com profundidade facilita substancialmente o aprendizado dos demais — muda-se principalmente a sintaxe e certas convenções específicas de cada ferramenta.

8.5 Transpiladores e o processo de build

Um problema recorrente no desenvolvimento web é o suporte desigual dos navegadores a recursos novos das linguagens. Uma funcionalidade recém-lançada do JavaScript pode não estar disponível em navegadores mais antigos ainda em uso por parte dos usuários. Para lidar com isso, os desenvolvedores utilizam **transpiladores**: ferramentas que convertem código escrito com recursos modernos para uma versão equivalente compatível com versões mais antigas dos navegadores.

O **Babel** é o transpilador mais conhecido para JavaScript, enquanto o **Sass** e o **Less** cumprem papel semelhante para CSS, permitindo escrever folhas de estilo com recursos avançados (variáveis, aninhamento de regras, funções) que são então compiladas para CSS puro, compatível com qualquer navegador.

Dica

Um site útil para consultar antes de usar um recurso novo de HTML, CSS ou JavaScript é o *Can I Use* (caniuse.com). Basta digitar o nome do recurso desejado para verificar quais navegadores — e a partir de qual versão — oferecem suporte a ele.

Esse processo de transpilação normalmente não é executado manualmente a cada alteração de código: ele é incorporado a um processo automatizado chamado **build**, frequentemente integrado a um sistema de integração contínua. Nesse fluxo, ao enviar (*commit*) uma alteração para um repositório como o GitHub, um servidor de integração contínua detecta a mudança, executa o build — que inclui a transpilação, a execução de testes automatizados e o empacotamento do projeto — e, em muitos casos, publica automaticamente a nova versão em produção. Essa prática é conhecida pelo conjunto de siglas *CI/CD* (*Continuous Integration, Continuous Delivery e Continuous Deployment*), e ferramentas como o Webpack são comumente usadas para orquestrar esse processo de construção do projeto.

Síntese do Capítulo

- Todo navegador moderno inclui DevTools embutidas, acessíveis via F12 ou clique direito + Inspecionar, com abas como Elements, Console e Network.
- Alterações feitas no DevTools existem apenas em memória e são descartadas ao recarregar a página, tornando-a um ambiente seguro para experimentação.
- Editores de código (como o VS Code) ocupam uma posição intermediária entre editores de texto puro e IDEs completas.
- Frameworks e bibliotecas aceleram o desenvolvimento por meio de reuso, boas práticas consolidadas e maturidade testada pela indústria, ao custo de uma camada extra de abstração.
- Bootstrap, jQuery, Angular, React e Vue.js são exemplos centrais do ecossistema front-end, cada um com um papel e um momento histórico distintos.
- Transpiladores como Babel, Sass e Less convertem código moderno para versões compatíveis com navegadores mais antigos.
- Esse processo é normalmente automatizado dentro de um pipeline de build e integração contínua (CI/CD).

Capítulo 9

Linguagens e Frameworks para Backend, APIs e Serviços

Enquanto o front-end é responsável por tudo aquilo que roda dentro do navegador do usuário, o **back-end** cuida da lógica de negócio, da persistência de dados e da geração das respostas que o front-end consome. Este capítulo apresenta um panorama de como bancos de dados, linguagens de back-end e o conceito de APIs se conectam para formar aplicações web completas.

9.1 Persistência de dados e bancos de dados relacionais

Quem trabalha com desenvolvimento back-end normalmente precisa integrar sua aplicação a um sistema de gestão de banco de dados. Para bancos de dados relacionais, o modelo conceitual amplamente utilizado desde a década de 1970 é o **modelo entidade-relacionamento**: identificam-se as entidades do domínio do problema e as associações entre elas (relacionamentos do tipo um-para-um, um-para-muitos, muitos-para-muitos). Em um sistema de hospedagem, por exemplo, poderíamos ter entidades como *Hóspede*, *Quarto*, *Tipo de Quarto* e *Reserva*, relacionadas entre si.

SQL

SQL (*Structured Query Language*, Linguagem de Consulta Estruturada) é a linguagem padrão para criar esquemas de bancos de dados relacionais e para consultar e manipular as informações neles armazenadas. Pode-se pronunciar tanto soletrando “S-Q-L” quanto como a palavra “sequel” — ambas as formas são de uso corrente.

Uma característica marcante dos bancos relacionais é possuírem um **esquema** bem definido: toda linha de uma tabela segue a mesma estrutura de colunas. Existem diversos sistemas gerenciadores de bancos de dados relacionais no mercado, gratuitos e pagos, como MySQL, MariaDB, PostgreSQL e Oracle, entre outros.

Nos últimos anos, bancos de dados **não relacionais** — os chamados **NoSQL** — têm ganhado espaço tanto na indústria quanto na academia. Sua principal característica é a ausência de um esquema rígido: diferentes registros de uma mesma coleção podem ter formatos distintos entre si. Essa flexibilidade costuma trazer ganhos de escalabilidade e eficiência em determinados cenários, justamente por dispensar a validação estrutural rígida exigida pelos bancos relacionais tradicionais.

9.2 Linguagens dinâmicas de back-end

Além do banco de dados, uma aplicação web precisa de uma linguagem de back-end responsável por consultar os dados, processá-los e uni-los a um *template* de apresentação (HTML/CSS), gerando dinamicamente o documento que será enviado ao navegador. Por isso essas linguagens são chamadas de **linguagens de renderização dinâmica**: o documento web não existe pronto em disco — ele é montado em tempo de execução, no servidor, a partir da combinação entre dados e template.

Existem dezenas de opções de linguagens e frameworks para essa finalidade, e a escolha costuma depender de contexto, experiência da equipe e do ecossistema da empresa. Alguns exemplos amplamente utilizados no mercado:

- **Java**: a plataforma oficial mais conhecida é o Jakarta EE (antigo Java EE), mas frameworks da comunidade como **Spring** e **Spring Boot** são extremamente populares tanto na indústria quanto na academia.
- **PHP**: o framework mais conhecido é o **Laravel**, além do próprio WordPress, que é construído sobre PHP.
- **JavaScript no lado servidor**: viabilizado pelo **Node.js**, com o framework **Express.js** sendo uma das opções mais utilizadas para construir aplicações no padrão MVC.
- **.NET**: plataforma da Microsoft para aplicações distribuídas corporativas, com suporte a múltiplas linguagens.
- **Ruby**: o framework mais associado a essa linguagem é o **Ruby on Rails**.
- **Python**: destacam-se frameworks como **Django** e **Flask**.

Apesar da diversidade de nomes, o conceito subjacente é sempre o mesmo: uma linguagem de back-end que roda no servidor, comunica-se com o banco de dados, aplica a lógica de negócio necessária e une o resultado a um template de apresentação antes de enviar a resposta ao navegador.

9.3 Sistemas de gestão de conteúdo (CMS)

Nem toda aplicação web precisa ser construída do zero. Quando o objetivo principal é publicar e organizar conteúdo — e não implementar algoritmos de negócio muito específicos — é comum recorrer a um **sistema de gestão de conteúdo** (CMS, *Content Management System*). Esses sistemas oferecem uma interface pronta para cadastrar, consultar, atualizar e remover conteúdo na web, sem que o desenvolvedor precise programar essa camada.

Os CMS mais conhecidos do mercado são **WordPress**, **Drupal** e **Joomla**, sendo o WordPress amplamente dominante — estima-se que ele sustente uma fração muito expressiva de todos os sites atualmente publicados na internet. O funcionamento interno de um CMS segue o mesmo princípio dos frameworks MVC de back-end mencionados anteriormente: uma consulta ao banco de dados obtém o conteúdo, que é então combinado com um template HTML/CSS para gerar a página final entregue ao navegador. O WordPress será estudado com mais profundidade em capítulo específico adiante.

9.4 APIs: quando o back-end não gera páginas

Nos últimos anos, uma alternativa ao modelo tradicional de aplicações web centradas em páginas ganhou enorme popularidade: o desenvolvimento de **APIs** (*Application Programming Interfaces*). Em vez de o servidor consultar o banco de dados e gerar uma página HTML completa, ele retorna apenas os **dados**, tipicamente em formato **JSON** ou **XML**, deixando a cargo do cliente — seja ele um navegador, um aplicativo móvel ou outro sistema — decidir como exibir essa informação.

JSON

JSON (*JavaScript Object Notation*) é um formato de texto para representar dados estruturados, facilmente convertido em um objeto de programação e vice-versa. É hoje o formato mais utilizado para comunicação entre APIs por ser mais compacto e ter menos sobrecarga (*overhead*) do que o XML.

Essa separação é o que torna possível que uma mesma lógica de back-end sirva simultaneamente a um site web, a um aplicativo móvel e a uma aplicação de linha de comando, por exemplo — todos consumindo a mesma API, cada um responsável por renderizar a resposta à sua própria maneira. É cada vez mais comum encontrar profissionais que trabalham exclusivamente no desenvolvimento de APIs, sem nunca escrever uma única tela de interface gráfica.

Um exemplo prático e concreto: imagine um serviço dos Correios que recebe como parâmetros um endereço de origem, um endereço de destino e o peso de um produto, e retorna o valor estimado do frete em formato XML ou JSON. Os Correios não precisam desenvolver interface gráfica alguma para esse serviço — quem consome a API é que decide como exibir o resultado, seja em um site de e-commerce, em um aplicativo móvel ou em qualquer outro sistema.

9.4.1 O estilo arquitetural REST

O estilo de arquitetura mais utilizado atualmente para expor esse tipo de serviço é chamado **REST** (*Representational State Transfer*). Antes do REST se popularizar, era comum o uso de protocolos mais verbosos como SOAP, com mensagens baseadas em WSDL sobre HTTP. O estilo REST, em contraste, aproveita diretamente os métodos já existentes do protocolo HTTP — como **GET**, **POST**, **PUT** e **DELETE** — como mecanismo de transporte e semântica de comunicação entre sistemas distribuídos. Uma requisição **GET** a um serviço REST, por exemplo, tipicamente retorna uma resposta em JSON representando o recurso solicitado, e não uma página HTML pronta para visualização.

9.5 Autenticação delegada com OAuth

Outra necessidade comum em aplicações que se comunicam com serviços de terceiros é a autenticação. O protocolo **OAuth** é um padrão da indústria que permite autorizar a comunicação entre dois sistemas distribuídos de forma segura, sem que seja necessário compartilhar senhas diretamente entre eles.

É graças ao OAuth, por exemplo, que um usuário pode autorizar o WordPress a publicar automaticamente em seu nome no Twitter, ou que um site pode oferecer “login com Google” ou “login com Facebook” sem nunca armazenar a senha do usuário — a aplicação apenas delega a autenticação ao serviço terceiro e recebe de volta uma confirmação de aprovação ou negação. Esse mecanismo tornou-se essencial no ecossistema atual de aplicações web integradas, e será revisitado quando o curso tratar de desenvolvimento de aplicações distribuídas com mais profundidade.

Síntese do Capítulo

- O modelo entidade-relacionamento e a linguagem SQL são a base dos bancos de dados relacionais, que possuem esquema rígido e bem definido.
- Bancos de dados NoSQL dispensam esquema fixo, trazendo flexibilidade e, em muitos cenários, ganhos de escalabilidade.
- Linguagens de back-end (Java, PHP, JavaScript/Node.js, .NET, Ruby, Python, entre outras) unem dados do banco a templates HTML/CSS para gerar páginas dinamicamente.
- Sistemas de gestão de conteúdo (CMS) como WordPress, Drupal e Joomla facilitam a publicação de conteúdo sem exigir desenvolvimento de aplicação customizada.
- APIs retornam dados (JSON ou XML) em vez de páginas prontas, permitindo que múltiplos tipos de cliente consumam a mesma lógica de back-end.
- REST é o estilo arquitetural predominante hoje para expor APIs, utilizando os métodos nativos do protocolo HTTP.
- OAuth permite autenticação e autorização seguras entre sistemas distintos, sem compartilhamento direto de senhas.

Capítulo 10

WordPress, Domínios e Hospedagem com GitHub Pages

Depois de entender os conceitos de back-end, CMS e APIs, é hora de colocar a mão na massa em três tarefas bastante práticas que qualquer desenvolvedor web acaba enfrentando: publicar conteúdo usando um sistema de gestão de conteúdo pronto, registrar um domínio próprio e publicar páginas estáticas gratuitamente. Este capítulo percorre essas três frentes, começando por uma introdução ao WordPress.

10.1 WordPress: três nomes, três coisas diferentes

Um ponto que costuma confundir iniciantes é a existência de três termos parecidos, mas com significados distintos dentro do ecossistema WordPress:

- **WordPress** (o software): uma aplicação web escrita em PHP, de código aberto e gratuita, que qualquer pessoa pode baixar, instalar e executar em seu próprio servidor.
- **wordpress.org**: o site oficial do projeto de software WordPress. É lá que se encontra o software para download, além de tutoriais, fóruns de discussão e documentação sobre sua evolução.
- **wordpress.com**: um serviço de hospedagem que permite criar um site em WordPress sem precisar instalar nada localmente — o modelo de computação em nuvem conhecido como *Software as a Service* (SaaS). Nem todos os recursos do wordpress.com são gratuitos; alguns exigem plano pago.

O foco deste capítulo é o *software* WordPress e o site wordpress.org, de onde ele pode ser obtido gratuitamente, inclusive com versão traduzida para português do Brasil.

10.1.1 Tipos de conteúdo e arquitetura interna

O WordPress é descrito como um sistema de gestão de conteúdo, mas sua função central é a **publicação de conteúdo** na web. Esse conteúdo se organiza basicamente em três tipos: **posts** (publicações, geralmente organizadas cronologicamente, como em um blog), **páginas** (conteúdo mais estático, como “Sobre” ou “Contato”) e **itens de mídia** (imagens, vídeos e outros arquivos incorporados aos posts e páginas). Cada um desses tipos possui suas próprias propriedades e características.

Internamente, o WordPress é uma aplicação PHP que depende de conexão com um banco de dados — tipicamente MySQL. A aplicação oferece duas visões distintas:

- Uma visão de **administração** (o *dashboard*), destinada a quem cria e gerencia o conteúdo do site.
- Uma visão para o **visitante**, que acessa o conteúdo já publicado.

Ao cadastrar conteúdo pelo painel administrativo e publicá-lo, esse conteúdo passa a ficar disponível para os visitantes. Todo o conteúdo é armazenado e referenciado a partir do banco de dados: alguns elementos (como o texto dos posts e seus metadados) ficam no banco de dados propriamente dito, enquanto outros (como arquivos de imagem) ficam armazenados no sistema de arquivos do servidor, sendo referenciados a partir de metadados salvos no banco.

Quando um visitante acessa uma URL do site, o que ele recebe não é um arquivo HTML pronto e estático guardado em disco: é uma página gerada *on the fly*, no servidor, unindo o conteúdo vindo do banco de dados com um template visual definido por um **tema**. É por isso que se diz que a página, em certo sentido teórico, “não existe” antes da requisição — ela é montada dinamicamente a cada acesso.

10.1.2 Temas e plugins

A customização e extensão do WordPress ocorrem por meio de dois mecanismos:

- **Temas:** definem a aparência e o layout visual da aplicação.
- **Plugins:** estendem as funcionalidades da aplicação, adicionando recursos que não fazem parte do núcleo do WordPress.

Publicar conteúdo no WordPress, portanto, equivale na prática a cadastrar um novo registro em uma tabela do banco de dados, que futuramente será combinado com o tema ativo para gerar a página exibida ao visitante.

10.2 Registrando um domínio web

Um **domínio** é o endereço legível por humanos (como `meusite.com`) que identifica um site na internet, evitando a necessidade de memorizar endereços IP numéricos. Existem diversas empresas — chamadas **registradoras** — que oferecem serviços de registro de domínio, hospedagem de sites, ou ambos. Algumas oferecem apenas registro, outras apenas hospedagem, e outras ainda, os dois serviços combinados.

O processo típico de registro de um domínio, usando como exemplo uma registradora conhecida mundialmente, segue estes passos:

1. **Verificar disponibilidade:** pesquisar o nome desejado (por exemplo, um nome pessoal) e verificar quais variações de domínio — `.com`, `.com.br`, `.xyz`, entre outras extensões — estão disponíveis para registro.
2. **Escolher a extensão e o período:** diferentes extensões têm preços distintos, e o registro costuma ser cobrado por período (um ano, dois anos etc.).

3. **Adicionar ao carrinho e revisar serviços adicionais:** durante o processo de compra, a registradora tipicamente oferece uma série de serviços extras (proteção de privacidade do domínio, e-mail personalizado, criação de site gratuito), a maioria opcional e paga à parte.
4. **Criar uma conta:** necessária para gerenciar o domínio adquirido.
5. **Efetuar o pagamento:** geralmente disponível via cartão de crédito, boleto bancário ou transferência.
6. **Aguardar confirmação:** após o pagamento (especialmente via boleto, que pode levar algumas horas para compensar), a registradora envia e-mails de confirmação e o domínio passa a estar ativo e disponível para configuração.

Antes da confirmação do pagamento, tentar acessar o domínio recém-solicitado no navegador tipicamente resulta em uma mensagem de erro ou em uma página da própria registradora informando que o domínio ainda não está disponível. Após a confirmação, o domínio pode ser gerenciado a partir do painel de controle da conta, de onde é possível, por exemplo, apontá-lo para um servidor de hospedagem ou configurar registros DNS.

Dica

Ao registrar um domínio, tenha atenção especial aos serviços adicionais oferecidos automaticamente durante o checkout — muitos deles vêm pré-selecionados e são pagos. Se o objetivo é apenas registrar o domínio, revise cada item do carrinho antes de finalizar a compra.

10.3 Publicando sites estáticos com GitHub Pages

Registrar um domínio resolve apenas parte do problema: também é preciso um lugar para hospedar os arquivos do site. Para sites simples, estáticos (compostos apenas por HTML, CSS e JavaScript, sem necessidade de banco de dados ou processamento no servidor), o **GitHub Pages** é uma alternativa gratuita e muito utilizada atualmente.

Git versus GitHub

É importante não confundir os dois termos. **Git** é um sistema de controle de versão distribuído. **GitHub** é uma plataforma web baseada em Git que permite hospedar e gerenciar projetos de desenvolvimento de software de forma colaborativa. Um **repositório** no GitHub é, de forma simplificada, equivalente ao conceito de um projeto.

10.3.1 Passo a passo para publicar um site

O processo básico para publicar um site simples via GitHub Pages é o seguinte:

1. Criar uma conta no GitHub, caso ainda não se tenha uma.

2. Criar um novo repositório. Se o repositório for nomeado exatamente como `nomedeusuario.github.io`, o GitHub Pages o trata como o site principal daquela conta, publicado diretamente em `https://nomedeusuario.github.io`.
3. Adicionar um arquivo `README` (opcional, mas recomendado) com informações sobre o projeto.
4. Enviar (fazer *commit*) os arquivos do site — por exemplo, um arquivo `index.html` — para o repositório.
5. Acessar **Settings** (Configurações) do repositório e, na seção **Pages**, ativar a publicação, indicando a *branch* (ramificação) e a pasta de onde os arquivos devem ser servidos (geralmente a raiz do repositório).
6. Aguardar alguns minutos — tipicamente entre 10 e 15 — para que o GitHub processe e publique o site no endereço configurado.

Estrutura mínima de um repositório para GitHub Pages

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Meu primeiro site</title>
</head>
<body>
  <h1>Olá, mundo!</h1>
</body>
</html>
```

Um detalhe prático relevante: se o repositório contiver um arquivo chamado exatamente `index.html` na raiz, o GitHub Pages o exibe automaticamente ao acessar o endereço raiz do site, sem que seja necessário digitar o nome do arquivo na URL. Arquivos com outros nomes precisam ser acessados explicitamente (por exemplo, `https://usuario.github.io/pagina.html`). Também é possível criar múltiplos repositórios na mesma conta, cada um publicado em um subcaminho próprio, no formato `https://usuario.github.io/nome-do-repositorio/`, permitindo hospedar diversos projetos distintos gratuitamente a partir de uma única conta do GitHub.

Dica

Caso o site não apareça imediatamente após a configuração, verifique em Settings → Pages se a publicação está de fato habilitada e apontando para a branch e pasta corretas antes de assumir que há um erro no código. Muitas vezes o problema é apenas o tempo de propagação da publicação.

Síntese do Capítulo

- WordPress (o software), wordpress.org (o site oficial) e wordpress.com (o serviço de hospedagem SaaS) são três coisas distintas dentro do mesmo ecossistema.
- O WordPress gerencia três tipos de conteúdo — posts, páginas e mídia — armazenados em banco de dados e sistema de arquivos, combinados dinamicamente com um tema no momento da requisição.
- Temas definem aparência; plugins estendem funcionalidades do WordPress.
- Registrar um domínio envolve verificar disponibilidade, escolher extensão e período, revisar serviços adicionais (frequentemente pagos e pré-selecionados) e efetuar o pagamento.
- Git é o sistema de controle de versão; GitHub é a plataforma de hospedagem colaborativa construída sobre o Git.
- Um repositório nomeado `usuario.github.io` é publicado automaticamente como o site principal da conta no GitHub Pages.
- Um arquivo `index.html` na raiz do repositório é servido automaticamente como página padrão, sem necessidade de especificar seu nome na URL.

Parte II

HTML

Capítulo 11

Introdução à Linguagem HTML

Encerrada a parte de fundamentos, o livro entra agora em sua segunda grande parte: o estudo detalhado da linguagem HTML. Este capítulo abre essa jornada apresentando o que é HTML, como seus elementos são construídos, qual a anatomia de um documento HTML completo e, por fim, como usar a documentação da MDN (Mozilla Developer Network) para praticar e resolver exercícios por conta própria.

O objetivo aqui não é esgotar todos os elementos da linguagem — existem ao menos cinquenta elementos e cinquenta atributos relevantes para uma base inicial sólida — mas construir os fundamentos necessários para que o leitor consiga caminhar sozinho a partir daqui, aprofundando-se conforme seu interesse e sua necessidade. O caminho tradicional de aprendizado em desenvolvimento web segue esta ordem: primeiro HTML, para estruturar o conteúdo; depois CSS, para estilizar; e por fim JavaScript, para adicionar interatividade.

11.1 O que é HTML

HTML

HTML (*HyperText Markup Language*, Linguagem de Marcação de Hipertexto) é, como o próprio nome indica, uma linguagem de **marcação** — não uma linguagem de programação. Uma linguagem de marcação parte de um documento textual e insere identificadores (*tags*) nesse texto para fornecer estrutura e semântica ao conteúdo.

Isso significa que, em HTML, não se escrevem instruções lógicas, laços de repetição ou condicionais: **declara-se** que um determinado trecho de texto é um parágrafo, que outro é um título, que outro ainda é uma lista ou uma imagem. Os navegadores web interpretam essas marcações e, com base nelas, decidem como renderizar visualmente cada trecho do documento.

Essa questão de estrutura e semântica está diretamente ligada à **acessibilidade**. Um texto simples, sem qualquer marcação, é difícil de ser interpretado corretamente por softwares leitores de tela, usados por pessoas com deficiência visual ou outras necessidades especiais. Ao marcar um trecho com o elemento semântico correto, o desenvolvedor está informando a esses softwares como aquele conteúdo deve ser interpretado e até mesmo pronunciado — com que ênfase, em que tom. Um elemento como `<strong>`, por exemplo, além de exibir o texto em negrito visualmente, sinaliza a um leitor de tela que aquele trecho deve ser lido com maior ênfase.

11.2 Anatomia de um elemento HTML

Elemento HTML

Um elemento HTML é o conjunto formado por uma **tag de abertura**, um **conteúdo** e uma **tag de encerramento**. A maioria dos elementos segue exatamente essa estrutura, embora existam exceções — os chamados elementos vazios — que serão discutidas adiante.

Considere o texto “Meu gato é muito mal-humorado”. Sem marcação alguma, esse texto é apenas conteúdo cru. Ao envolvê-lo com a tag de abertura `<p>` e a tag de encerramento `</p>`, ele passa a ser um elemento do tipo parágrafo:

Um parágrafo simples

HTML

```
<p>Meu gato é muito mal-humorado.</p>
```

Um ponto importante a destacar logo de início: diferentemente de linguagens como Java, HTML **não diferencia maiúsculas de minúsculas** (é *case-insensitive*). Escrever `<P>`, `<p>` ou qualquer combinação de capitalização produz o mesmo resultado funcional. Ainda assim, é uma boa prática consolidada usar sempre letras minúsculas nos nomes de elementos — assim como em Java existe a convenção de *camelCase* sem que ela seja obrigatória, em HTML a convenção é usar minúsculas por legibilidade e consistência.

11.2.1 Elementos aninhados e ordem de fechamento

É perfeitamente possível — e extremamente comum — colocar um elemento dentro de outro. Por exemplo, para enfatizar apenas a palavra “mal-humorado” dentro do parágrafo, usa-se o elemento `<strong>` aninhado dentro do `<p>`:

Elementos aninhados

HTML

```
<p>Meu gato é muito <strong>mal-humorado</strong>.</p>
```

A regra fundamental aqui é: a ordem de fechamento das tags deve ser sempre o **inverso** da ordem de abertura — exatamente como em qualquer estrutura de pilha estudada em disciplinas de compiladores ou estruturas de dados. Se `<p>` foi aberto primeiro e depois `<strong>`, então `</strong>` deve ser fechado antes de `</p>`. Misturar essa ordem — por exemplo, escrever `<p><strong>texto</p></strong>` — é um erro de marcação e deve ser evitado.

11.2.2 Elementos de bloco e elementos em linha

Antes do HTML5, os elementos eram tradicionalmente categorizados em dois grandes grupos — categorização que o HTML5 expandiu, mas que ainda serve como base conceitual importante:

- **Elementos de bloco** (*block*): ocupam uma linha própria na renderização, independentemente do conteúdo ao redor. O elemento `<p>`, por exemplo, é de bloco — depois dele, o conteúdo seguinte é automaticamente empurrado para uma nova linha visual.
- **Elementos em linha** (*inline*): não forçam quebra de linha; eles fluem dentro do texto ao seu redor, ocupando apenas o espaço necessário para seu próprio conteúdo. O elemento `<em>`, por exemplo, é em linha.

Um detalhe sutil, mas essencial: o espaço em branco entre elementos HTML no código-fonte (quebras de linha, indentação) é, para o navegador, irrelevante — ele é interpretado como um único espaço, independentemente de quantas linhas ou espaços existam no arquivo original. Isso ocorre porque transmitir espaços em branco desnecessários pela rede consome tempo e banda; por isso, o navegador os normaliza. Para o ser humano, entretanto, esse espaçamento é extremamente importante: indentar o código de forma consistente é o que torna um documento HTML legível para quem o escreve e mantém.

11.3 Atributos

Atributo

Um atributo é uma informação adicional associada a um elemento, que não é visível diretamente na renderização da página, mas que fornece metadados ou configurações importantes para o funcionamento do documento, do CSS ou do JavaScript.

Alguns atributos são específicos de determinados elementos — como o atributo `src` (*source*, origem) do elemento `<img>`, que indica onde está localizado o arquivo de imagem — enquanto outros são **globais**, podendo ser usados em praticamente qualquer elemento HTML, como é o caso do atributo `class`.

Um elemento vazio com atributos

HTML

```

```

O elemento `<img>` ilustra também o conceito de **elemento vazio**: nem todo elemento HTML segue a estrutura de tag de abertura mais conteúdo mais tag de encerramento. Elementos vazios, como `<img>`, não possuem conteúdo textual interno nem tag de fechamento — toda a informação relevante é passada por meio de atributos.

Um atributo pode também ser **booleano**, isto é, sua simples presença já indica um valor verdadeiro, sem necessidade de atribuir um valor explícito. O atributo `disabled`, usado em campos de formulário para desabilitar a interação do usuário, é um exemplo: escrever apenas `disabled` já é equivalente a `disabled="disabled"`.

11.3.1 Aspas em valores de atributos

Embora tecnicamente não seja sempre obrigatório colocar o valor de um atributo entre aspas, essa é uma prática fortemente recomendada, pois evita ambiguidades. Observe o problema que pode ocorrer sem o uso de aspas:

Por que usar aspas nos atributos

HTML

```
<!-- Forma recomendada e sem ambiguidade -->
<a href="https://www.mozilla.org" title="Site Favorito">Website Favorito</a>

<!-- Forma arriscada: sem aspas, o navegador pode interpretar
     o espaço em branco como o início de um novo atributo -->
<a href=https://www.mozilla.org title=Site Favorito>Website Favorito</a>
```

No segundo caso, o navegador tenta interpretar tudo após o sinal de igual até o próximo espaço em branco como o valor do atributo. Isso significa que a palavra “Favorito”, separada por um espaço da palavra “Site”, seria interpretada como um atributo booleano à parte, e não como parte do valor de `title`. É possível usar tanto aspas duplas quanto aspas simples, mas nunca misturar os dois tipos ao abrir e fechar o mesmo valor.

Quando o próprio valor do atributo contém um apóstrofo — como em contrações do inglês, por exemplo “it’s” — e o valor está delimitado por aspas simples, é necessário usar uma entidade especial de caractere para representar o apóstrofo sem causar ambiguidade, como será visto a seguir.

11.4 Caracteres especiais (entidades HTML)

Como os sinais de menor (<) e maior (>) têm significado reservado em HTML — delimitam tags —, não é possível inseri-los diretamente no conteúdo visível de uma página sem que o navegador tente interpretá-los como parte de uma marcação. Para contornar isso, existem **entidades de caractere**: códigos especiais que representam esses caracteres como texto literal.

Principais entidades de caractere

HTML

```
&lt;    <!-- sinal de menor (<) -->
&gt;    <!-- sinal de maior (>) -->
&quot;  <!-- aspas duplas (") -->
&#39;  <!-- aspas simples / apóstrofo (') -->
&amp;  <!-- E comercial (&) -->
```

Existem centenas de outras entidades definidas no padrão HTML, mas, para o uso cotidiano, essas cinco cobrem a grande maioria dos casos práticos. Vale notar que, ao declarar corretamente o conjunto de caracteres do documento como UTF-8 (assunto tratado na próxima seção), a maioria dos demais caracteres especiais — como acentos, letras de outros idiomas ou o símbolo de copyright — pode ser digitada diretamente, sem necessidade de codificação especial.

11.5 Comentários

Assim como em qualquer linguagem de programação, é possível inserir comentários em HTML — trechos de texto que não são renderizados pelo navegador, mas que servem como anotação para quem escreve ou mantém o código. Enquanto em Java se usa `//` ou `/* */`, em HTML a sintaxe de comentário é a seguinte:

Comentário em HTML

HTML

```
<!-- Este trecho não aparece na página renderizada -->
<p>Este parágrafo aparece normalmente.</p>
```

11.6 Anatomia de um documento HTML completo

Todo documento HTML segue uma estrutura mínima obrigatória, formada por um preâmbulo (a declaração `DOCTYPE`), o elemento raiz `<html>`, e dentro dele os elementos `<head>` e `<body>`. Essa estrutura evoluiu ao longo dos anos — versões anteriores ao HTML5 exigiam uma declaração de `DOCTYPE` bem mais longa e complexa — e hoje está bastante simplificada.

Esqueleto básico de um documento HTML5

HTML

```
<!DOCTYPE html>
<html lang="pt-br">
  <head>
    <!-- O elemento head contém metadados: nada aqui é
         visível diretamente na página renderizada -->
    <meta charset="utf-8">
    <meta name="description" content="Uma breve descrição da página.">
    <title>Título exibido na aba do navegador</title>

    <!-- Associação de uma folha de estilos CSS externa -->
    <link rel="stylesheet" href="estilos.css">
  </head>
  <body>
    <!-- Tudo dentro do body é visível no navegador -->
    <h1>Bem-vindo à minha página</h1>
    <p>Este é o conteúdo principal, visível para o usuário.</p>

    <!-- Boa prática: script no final do body, para garantir
         que os elementos HTML já foram carregados antes de
         o JavaScript tentar acessá-los -->
    <script src="script.js"></script>
  </body>
</html>
```

Vale destacar alguns pontos importantes dessa estrutura:

- **<!DOCTYPE html>**: informa ao navegador que o documento segue o padrão HTML5. É a forma simplificada introduzida por essa versão da linguagem.
- **<html>**: o elemento raiz de todo documento HTML — todos os demais elementos ficam aninhados dentro dele. O atributo **lang** (também global, mas especialmente importante aqui) define o idioma principal do documento, o que é relevante tanto para acessibilidade (leitores de tela pronunciam palavras de forma diferente conforme o idioma configurado) quanto para mecanismos de busca.
- **<head>**: contém informações não visíveis — metadados — como o conjunto de caracteres do documento, palavras-chave para mecanismos de busca, o título exibido na aba do navegador e a associação com folhas de estilo CSS externas.
- **<meta charset="utf-8">**: declara que o documento usa a codificação de caracteres UTF-8, hoje praticamente universal, cobrindo os caracteres de qualquer idioma do mundo. É fortemente recomendado sempre declarar essa codificação explicitamente: em navegadores mais antigos, a ausência dessa declaração — ou o uso de uma codificação diferente, como ISO-8859-1 — pode causar falhas na exibição de caracteres não latinos, como japonês ou coreano.

- **<body>**: contém todo o conteúdo que efetivamente aparece no navegador — é, por assim dizer, irmão do **<head>** dentro do **<html>**.

11.6.1 Associando CSS e JavaScript externamente

Embora seja tecnicamente possível escrever CSS e JavaScript diretamente dentro do arquivo HTML, essa não é uma boa prática, pois compromete a separação de interesses: HTML deve cuidar de estrutura e semântica, CSS de apresentação visual, e JavaScript de comportamento e interatividade. Manter esses três arquivos separados também facilita o trabalho em equipe, permitindo que diferentes pessoas trabalhem em cada camada de forma independente.

Para associar uma folha de estilo externa, usa-se o elemento **<link>** dentro do **<head>**, com os atributos **rel** (que indica o tipo de relação — no caso, **stylesheet**, folha de estilo) e **href** (que indica o caminho até o arquivo). Para associar um arquivo JavaScript, usa-se o elemento **<script>** com o atributo **src**, apontando para o arquivo **.js** correspondente.

Dica

Como o navegador processa o HTML de cima para baixo, colocar a tag **<script>** logo no início do documento pode causar erros: o JavaScript pode tentar acessar um elemento que ainda não foi lido pelo navegador e, portanto, ainda não existe na árvore DOM. Por isso, a prática recomendada é posicionar o **<script>** no final do **<body>**, garantindo que todo o conteúdo HTML já tenha sido processado antes de o script ser executado.

11.7 A MDN: a principal referência para aprender HTML

MDN Web Docs

A MDN (*Mozilla Developer Network*, hoje oficialmente chamada MDN Web Docs) é a documentação de referência mantida pela Mozilla — a organização por trás do navegador Firefox — sobre tecnologias web: HTML, CSS, JavaScript e APIs relacionadas. É considerada a fonte mais confiável e completa disponível gratuitamente na internet para aprender e consultar detalhes dessas linguagens.

A documentação da MDN organiza os exercícios práticos em diferentes níveis de complexidade. Os exercícios simples e rápidos, chamados *Learn*, servem apenas para testar conhecimentos pontuais logo após a leitura de uma seção teórica. Já os exercícios *Test your skills* (“Teste suas habilidades”) são um pouco mais elaborados, com maior grau de dificuldade, funcionando como uma avaliação intermediária do que foi aprendido.

Há três formas possíveis de resolver esses exercícios: diretamente na própria página da MDN, que frequentemente já embute uma área de edição de código com saída imediata; localmente, em um editor como o Visual Studio Code; ou em uma ferramenta

online como o CodePen, que dispensa qualquer instalação e permite compartilhar o código facilmente.

Dica

Um roteiro prático para resolver os exercícios da MDN:

1. Leia a seção teórica correspondente ao exercício antes de tentar resolvê-lo — os exercícios pressupõem o conteúdo apresentado imediatamente antes deles.
2. Localize a área de edição de código na própria página (geralmente com um campo de entrada e uma área de saída ao lado ou abaixo).
3. Edite o código diretamente na área indicada, seguindo a instrução do exercício (por exemplo, abrir uma tag específica em determinado ponto do texto).
4. Observe o resultado na área de saída. Caso tenha cometido algum erro, é possível reiniciar o exercício usando o botão de reset, sem qualquer penalidade.
5. Caso não consiga resolver sozinho, a própria página costuma oferecer um link para a solução — não é falha usar esse recurso enquanto o material ainda está sendo internalizado.

Boa parte da documentação da MDN já está traduzida para diversos idiomas, incluindo português do Brasil, mas nem todo o conteúdo possui tradução completa. Quando isso ocorrer, vale lembrar que, para quem pretende seguir carreira em ciência da computação, desenvolver a habilidade de leitura em inglês técnico é praticamente indispensável — não é necessário falar ou compreender inglês oral fluentemente para isso, apenas praticar a leitura progressivamente. Recorrer a um tradutor automático em casos pontuais não é um problema, mas vale tentar ler o texto original primeiro, construindo vocabulário técnico aos poucos.

Síntese do Capítulo

- HTML é uma linguagem de marcação, não uma linguagem de programação: ela declara estrutura e semântica, não lógica.
- Um elemento HTML é composto por tag de abertura, conteúdo e tag de encerramento; elementos aninhados devem fechar na ordem inversa da abertura.
- Elementos se dividem (na categorização clássica) em elementos de bloco e elementos em linha, com comportamentos de renderização distintos.
- Atributos fornecem metadados não visíveis, usados por CSS e JavaScript para selecionar e configurar elementos; valores de atributo devem sempre vir entre aspas.

- Todo documento HTML segue uma anatomia padrão: `DOCTYPE`, elemento raiz `html`, com `head` (metadados) e `body` (conteúdo visível).
- Declarar `charset="utf-8"` no `head` é essencial para garantir a correta exibição de caracteres de qualquer idioma.
- A MDN é a principal referência para aprender e praticar HTML, CSS e JavaScript, com exercícios organizados por nível de dificuldade e disponíveis em múltiplos formatos de resolução.

Capítulo 12

Marcação Fundamental de Texto em HTML

O objetivo central do HTML é fornecer aos textos planos que compõem um documento web tanto **estrutura** quanto **significado**. Quando falamos em significado, estamos falando de *semântica* — um conceito que será revisitado constantemente ao longo deste livro. Semântica, em poucas palavras, é a capacidade de uma marcação expressar exatamente o que um determinado trecho de texto representa para quem está lendo (ou ouvindo, no caso de leitores de tela) o documento.

Este capítulo apresenta os elementos fundamentais de marcação de texto em HTML: títulos, parágrafos, listas, elementos de ênfase e os hiperlinks — provavelmente a característica mais icônica da própria web. O objetivo é que o futuro desenvolvedor saia deste capítulo sabendo estruturar qualquer texto corrido de forma correta e acessível.

12.1 Elementos de Título

Assim como um livro é organizado em capítulos, seções e subseções para facilitar a leitura, um documento HTML também precisa de uma estrutura hierárquica de títulos. Todo texto bem escrito — seja um artigo científico, uma reportagem ou um romance — costuma ser organizado dessa maneira, pois isso torna a experiência de leitura muito mais agradável. Imagine um texto de duzentas páginas sem nenhum parágrafo, sem capítulos e sem títulos: a leitura seria exaustiva.

Em HTML, essa estruturação é feita por meio dos elementos de título, que vão de **h1** até **h6**. Cada um desses elementos representa um nível diferente dentro do documento: o **h1** é o título principal (mais genérico), o **h2** funciona como um subtítulo, e assim sucessivamente até o **h6**, o nível mais específico.

Elementos de título não definem tamanho de fonte

No início da web, era comum usar os elementos **h1** a **h6** apenas para deixar um texto maior ou menor na tela — já que o estilo padrão dos navegadores renderiza o **h1** com uma fonte maior que o **h2**, e assim por diante. Essa prática não faz mais sentido: os elementos de título servem para indicar a **estrutura semântica** do documento, não o tamanho visual do texto. Para controlar a aparência (tamanho, cor, espaçamento), a ferramenta correta é o CSS, que será estudado na Parte III deste livro.

HTML

```
<h1>Crime e Castigo</h1>
<p>Fiódor Dostoiévski</p>

<h2>Capítulo 1</h2>
<p>Em uma tarde excepcionalmente quente de começo de julho...</p>

<h2>Capítulo 2</h2>
<h3>A visita ao penhorista</h3>
<p>Ele desceu a escada com cautela...</p>
```

Observe que, no exemplo acima, existe apenas um **h1** — reservado para o título mais genérico do documento (no caso, o título do livro). Os capítulos usam **h2**, e uma subseção dentro de um capítulo usa **h3**. Essa é uma boa prática fundamental: os níveis de título devem seguir uma ordem lógica, do mais genérico para o mais específico, sem “pular” níveis (por exemplo, usar um **h4** logo depois de um **h1**, sem passar por **h2** e **h3**).

Dica

Por convenção, evite usar mais de três níveis de título (**h1**, **h2** e **h3**) em uma única página. Da mesma forma que um documento científico bem escrito raramente ultrapassa a numeração 1.1.1, uma página web com muitos níveis de título aninhados costuma ser sinal de que o conteúdo deveria ser dividido em mais de uma página. Além disso, a boa prática mais comum é usar apenas um único **h1** por página, reservado para o assunto principal daquele documento — lembrando que um site é composto de várias páginas, e cada uma delas pode (e deve) ter o seu próprio **h1**.

12.2 Listas

Listas são extremamente comuns no dia a dia: uma lista de ingredientes de uma receita, o passo a passo de um manual, os tópicos de uma apresentação de slides. O HTML fornece três tipos de listas: **não ordenadas**, **ordenadas** e **de descrição**.

12.2.1 Listas Não Ordenadas

Uma lista não ordenada é usada quando a ordem dos itens não importa — por exemplo, uma lista de ingredientes. Ela é criada com o elemento **ul** (*unordered list*), e cada item da lista é delimitado pelo elemento **li** (*list item*).

HTML

```
<ul>
  <li>Leite</li>
  <li>Ovos</li>
  <li>Pão</li>
  <li>Presunto</li>
</ul>
```


Repare que os navegadores web já possuem estilos padrão implementados para as listas: ao abrir esse código em qualquer navegador (Chrome, Firefox, Safari etc.), os itens já aparecem automaticamente marcados com marcadores (bullets). Isso é apenas o estilo de fábrica do navegador — mais adiante, com CSS, será possível personalizar completamente essa aparência, inclusive trocando os marcadores por outros símbolos ou removendo-os por completo.

12.2.2 Listas Ordenadas

Quando a ordem dos itens importa — como em um passo a passo de instruções — utiliza-se o elemento `ol` (*ordered list*), cujo estilo padrão dos navegadores é a numeração sequencial (1, 2, 3...).

HTML

```
<ol>
  <li>Dirija até o final da estrada.</li>
  <li>Vire à direita.</li>
  <li>Siga em frente nas próximas duas rotatórias.</li>
  <li>Vire à esquerda na terceira rotatória.</li>
  <li>A escola estará à sua direita, 300 metros à frente.</li>
</ol>
```

É possível, ainda, aninhar uma lista dentro de outra — por exemplo, quando um dos passos de uma receita se desdobra em subpassos:

HTML

```
<ol>
  <li>Separe os ingredientes.</li>
  <li>Pré-aqueça o forno a 180°C.</li>
  <li>Bata os ovos com o açúcar.</li>
  <li>Adicione a farinha aos poucos.</li>
  <li>Processe os temperos:
    <ul>
      <li>Descasque o alho.</li>
      <li>Pique as ervas finas.</li>
    </ul>
  </li>
  <li>Leve ao forno por 40 minutos.</li>
</ol>
```

12.2.3 Listas de Descrição

Um terceiro tipo de lista, menos conhecido mas bastante útil, é a lista de descrição. Ela é usada quando existe um termo e, associado a ele, uma definição — como em um glossário. Os elementos envolvidos são `dl` (*description list*), `dt` (*description term*) e `dd` (*description definition*).

HTML

```
<dl>
  <dt>Solilóquio</dt>
  <dd>Tipo de fala em uma peça de teatro na qual um personagem
    expressa seus pensamentos em voz alta, sozinho no palco.</dd>

  <dt>Monólogo</dt>
  <dd>Longo discurso proferido por um único personagem,
    geralmente dirigido a outros personagens presentes em cena.</dd>

  <dt>Aparte</dt>
  <dd>Comentário breve feito por um personagem diretamente
    ao público, sem que os demais personagens o percebam.</dd>
</dl>
```

Ao renderizar esse código, o estilo padrão dos navegadores exibe o termo (**dt**) e, logo abaixo, um pouco recuada, a sua definição (**dd**). O ganho em relação a um texto sem marcação é enorme: o significado de cada trecho fica evidente tanto para o navegador quanto para leitores de tela.

12.3 Ênfase e Destaque de Texto

É muito comum, ao falar ou escrever, querer enfatizar determinadas palavras — alterando a entonação da fala ou destacando visualmente um trecho do texto. O HTML oferece elementos semânticos específicos para isso.

12.3.1 Os Elementos `em` e `strong`

O elemento `em` (*emphasis*) indica ênfase: o texto é normalmente exibido em itálico, mas, mais importante, ele carrega um significado semântico — leitores de tela pronunciam esse trecho com uma entonação diferenciada.

HTML

```
<p>Estou <em>muito</em> feliz que você não tenha se atrasado.</p>
```

Já o elemento `strong` indica importância ou urgência forte, sendo exibido em negrito e, da mesma forma, pronunciado com ênfase por leitores de tela.

HTML

```
<p><strong>Aviso:</strong> este líquido é altamente tóxico.</p>
<p><strong>Não</strong> use este produto perto de crianças.</p>
```

É possível, inclusive, combinar os dois elementos, aninhando um dentro do outro:

HTML

```
<p>Se você comer isso, <strong>você <em>pode</em> morrer</strong>.</p>
```

12.3.2 Os Elementos **i**, **b** e **u**

O HTML também possui elementos mais antigos, herdados dos primórdios da linguagem: **i** (itálico), **b** (negrito, de *bold*) e **u** (sublinhado, de *underline*). A diferença fundamental é que esses três elementos são puramente **apresentacionais** — eles alteram apenas a aparência visual, sem carregar nenhum significado semântico.

HTML

```
<p>O termo <i>habeas corpus</i> vem do latim.</p>
<p>Esta é uma <b>palavra-chave</b> do glossário.</p>
<p>A palavra <u>spell</u> está incorreta.</p>
```

Dica

Tenha muito cuidado ao usar o elemento **u** para sublinhar texto. Os usuários da web estão culturalmente acostumados a associar sublinhado a **link**. Se você sublinhar um texto que não é um link, pode confundir o leitor. Se realmente precisar de um sublinhado decorativo (por exemplo, para indicar um erro ortográfico, como em corretores automáticos), prefira usar CSS para estilizar esse sublinhado de uma forma visualmente distinta de um link — por exemplo, com uma linha ondulada em vermelho.

A recomendação geral é: sempre que possível, prefira **em** e **strong** a **i**, **b** e **u**, pois os primeiros comunicam significado tanto para navegadores quanto para leitores de tela, enquanto os últimos comunicam apenas aparência.

12.4 Hiperlinks

Se há um conceito que definiu a própria existência da web, esse conceito é o **hiperlink**. É a capacidade de, a partir de um texto (ou imagem), clicar e ser redirecionado para outro documento — seja ele hospedado no mesmo site ou em qualquer lugar do mundo.

12.4.1 O Elemento de Âncora

Um hiperlink é criado com o elemento **a** (*anchor*, âncora), utilizando o atributo **href** (*hypertext reference*) para indicar o destino do link.

HTML

```
<p>
  <a href="https://www.mozilla.org">Link para a página principal da Mozilla</a>
</p>
```

No estilo padrão dos navegadores, links são exibidos sublinhados e em uma cor diferenciada. Ao clicar, o navegador realiza uma requisição HTTP para o endereço indicado no atributo **href**. É possível, inclusive, transformar uma imagem em um link, envolvendo-a com o elemento âncora:

HTML

```
<a href="https://www.mozilla.org">
  
</a>
```

O elemento `a` também aceita o atributo `title`, global a vários elementos HTML, que exibe uma dica (*tooltip*) quando o usuário passa o mouse sobre o link e aguarda alguns segundos.

HTML

```
<a href="https://www.mozilla.org" title="Visite o site oficial da Mozilla">
  Mozilla home page
</a>
```

Dica

Nem todo usuário navega com o mouse — algumas pessoas usam apenas o teclado. Como o atributo `title` só é revelado ao passar o mouse, ele não é acessível a esses usuários. Se a informação for realmente importante, prefira colocá-la como texto visível na página em vez de escondê-la em um atributo `title`.

12.4.2 Caminhos Relativos e Absolutos

Ao criar um link, é preciso indicar o endereço (URL) de destino, que pode ser um **caminho relativo** ou um **caminho absoluto**. Uma URL absoluta contém todas as informações necessárias, desde o protocolo até o domínio: `https://www.mozilla.org/pt-BR/`. Já um caminho relativo localiza o recurso a partir da posição atual dentro do próprio sistema de arquivos do servidor web — o mesmo conceito usado para navegar entre pastas no Windows, macOS ou Linux.

Considere a seguinte estrutura de arquivos de um site:

Estrutura de diretórios

```
/ (raiz do site)
  index.html
  curriculo.pdf
  projetos/
    index.html
```

Se estivermos dentro de `index.html` (na raiz) e quisermos criar um link para `projetos/index.html`, que está em um subdiretório, o caminho relativo seria:

HTML

```
<a href="projetos/index.html">Veja meus projetos</a>
```

Se, ao contrário, estivermos dentro de `projetos/index.html` e quisermos voltar para a raiz e acessar o `curriculo.pdf`, usamos `..` para "subir" um nível no sistema de arquivos:

HTML

```
<a href="../../../curriculo.pdf">Baixe meu currículo</a>
```

Dica

Sempre que o link apontar para um recurso dentro do seu próprio site, prefira caminhos relativos: eles são mais curtos e não dependem do domínio completo. Reserve caminhos absolutos para links que apontam para sites externos.

12.4.3 Links para Seções Específicas e Outros Esquemas de URL

Também é possível criar um link para uma seção específica de um documento (ao invés do documento inteiro), usando o atributo `id` para identificar de forma única um elemento, e referenciando-o com `#` no `href`:

HTML

```
<!-- Em contato.html -->
<h2 id="endereco">Endereço de correspondência</h2>
<p>Rua das Flores, 123 - Centro</p>

<!-- Em outra página -->
<a href="contato.html#endereco">Ver nosso endereço</a>
```

O HTML também permite criar links que abrem o cliente de e-mail padrão do usuário, usando o esquema `mailto:`, e links de download, usando o atributo `download`:

HTML

```
<a href="mailto:contato@exemplo.com?subject=Duvida&body=Ola">
  Envie um e-mail
</a>

<a href="instalador.exe" download="instalador-programa.exe">
  Baixar instalador
</a>
```

Dica

Evite textos de link genéricos como "clique aqui". Muitos usuários (e leitores de tela) fazem uma leitura rápida (*skimming*) de uma página, prestando atenção apenas nos links e títulos. Um texto de link como "Baixe o Firefox" é muito mais informativo do que "clique aqui para baixar o Firefox" com o link apenas na palavra "aqui". Da mesma forma, informe sempre que um link aponta para um recurso pesado (como um vídeo em alta definição ou um arquivo grande), para que o usuário decida conscientemente se quer prosseguir.

Síntese do Capítulo

- Os elementos **h1** a **h6** estruturam hierarquicamente o documento e não devem ser usados apenas para alterar o tamanho da fonte — essa é uma tarefa do CSS.
- Existem três tipos de listas em HTML: não ordenadas (**ul**), ordenadas (**ol**) e de descrição (**dl/dt/dd**), e listas podem ser aninhadas dentro de outras listas.
- Os elementos **em** e **strong** carregam significado semântico (afetando inclusive a entonação de leitores de tela), enquanto **i**, **b** e **u** são puramente apresentacionais.
- Hiperlinks são criados com o elemento **a** e o atributo **href**, podendo apontar para outros documentos, seções específicas (com **#id**), endereços de e-mail (**mailto:**) ou arquivos para download.
- URLs podem ser relativas (baseadas na posição atual dentro do sistema de arquivos do site) ou absolutas (endereço completo); prefira relativas para recursos internos ao próprio site.
- Boas práticas de acessibilidade recomendam textos de link descritivos, evitando frases genéricas como "clique aqui".

Capítulo 13

Elementos Adicionais de Texto e Estrutura Semântica em HTML

O capítulo anterior apresentou os elementos fundamentais de marcação de texto. Este capítulo avança em duas frentes complementares: primeiro, um conjunto de elementos textuais menos conhecidos, mas ainda importantes — citações, abreviações, endereços, superescrito/subescrito e representação de código-fonte; segundo, e mais importante para o restante da sua trajetória como desenvolvedor, os elementos estruturais que organizam as grandes áreas de uma página web, como cabeçalho, navegação, conteúdo principal e rodapé.

13.1 Citações em HTML

Quando você utiliza uma ideia que não é sua, é fundamental citar a fonte — tanto por uma questão ética quanto, em muitos contextos, por uma questão legal. O HTML oferece elementos específicos para marcar citações.

13.1.1 Citação em Bloco: o Elemento `blockquote`

O elemento `blockquote` é usado quando você cita, na íntegra, um conteúdo mais extenso — um trecho completo de outra fonte, e não apenas uma palavra ou frase isolada.

HTML

```
<p>Segundo a documentação da Mozilla:</p>
<blockquote cite="https://developer.mozilla.org/pt-BR/">
  <p>A MDN Web Docs é um recurso aberto e colaborativo para
    desenvolvedores, por desenvolvedores, documentando tecnologias
    web, incluindo CSS, HTML e JavaScript.</p>
</blockquote>
```

O atributo `cite` indica de onde a citação foi extraída. É importante notar que esse atributo **não é visível** na tela — ele existe apenas como metadado, útil caso você (ou algum script) queira acessá-lo programaticamente.

13.1.2 Citação em Linha: o Elemento `q`

Quando a citação é apenas um trecho dentro de um parágrafo maior — e não o parágrafo inteiro — utiliza-se o elemento `q` (*quotation*), que é um elemento em linha (*inline*).

HTML

```
<p>
  De acordo com a documentação da MDN, um dos objetivos do projeto é
  <q cite="https://developer.mozilla.org/">fornecer um recurso completo
  e confiável para desenvolvedores web de todos os níveis</q>.
</p>
```

Muitos navegadores adicionam automaticamente aspas ao redor do conteúdo de um elemento `q`, mesmo sem que você as digite manualmente — outra vantagem de usar marcação semântica ao invés de digitar as aspas você mesmo.

13.1.3 O Elemento `cite`

Existe ainda um terceiro elemento relacionado a citações, o `cite`, que não deve ser confundido com o atributo `cite` de `blockquote` e `q`. O elemento `cite` é usado para indicar o **título de uma obra** — um livro, um artigo, uma música.

Dica

Nenhum desses três recursos (elemento `blockquote`, elemento `q` ou atributo `cite`) torna a referência clicável automaticamente. Uma boa prática recomendada é envolver o elemento `cite` (ou o texto da fonte) dentro de um elemento de âncora (`a`), permitindo que o leitor acesse a fonte original com um clique:

HTML

```
<blockquote cite="https://developer.mozilla.org/">
  <p>Este é um exemplo de citação em bloco.</p>
</blockquote>
<p>
  - <a href="https://developer.mozilla.org/">
    <cite>MDN Web Docs</cite>
  </a>
</p>
```

13.2 Abreviações, Endereços e Outras Marcações Textuais

13.2.1 Abreviações: o Elemento `abbr`

O elemento `abbr` marca abreviações e siglas, utilizando o atributo `title` para fornecer a forma expandida do termo. A vantagem prática é que, ao passar o mouse sobre a sigla, o navegador exibe automaticamente um *tooltip* com o significado completo.

HTML

```
<p>
  Nós usamos <abbr title="HyperText Markup Language">HTML</abbr>
  para estruturar nossos documentos web.
</p>
```


</p>

Dica

O HTML já teve um elemento **acronym**, específico para acrônimos (siglas que se pronunciam como palavras). Esse elemento se tornou obsoleto e não é mais suportado de forma consistente pelos navegadores modernos — utilize sempre **abbr**, tanto para siglas quanto para acrônimos.

13.2.2 Endereços: o Elemento address

O elemento **address** marca um trecho de texto que representa informações de contato — endereço residencial, comercial ou de correspondência de uma pessoa ou organização.

HTML

```
<address>
  Maria Oliveira<br>
  Rua das Palmeiras, 456<br>
  Manchester, Reino Unido
</address>
```

Repare no uso do elemento **br** (*line break*), que insere uma quebra de linha simples — lembre-se de que o HTML ignora espaços em branco e quebras de linha do código-fonte, então, sem o **br**, todo esse texto apareceria em uma única linha corrida.

Dica

O elemento **address** deve conter apenas informações de contato relacionadas ao autor do documento (ou da seção em que está inserido) — não o utilize para qualquer endereço genérico mencionado no texto, como o endereço de uma empresa citada em uma notícia.

13.2.3 Superescrito e Subescrito

Os elementos **sup** (superescrito) e **sub** (subescrito) posicionam o conteúdo ligeiramente acima ou abaixo da linha do texto, respectivamente — úteis para datas por extenso, fórmulas químicas e expressões matemáticas.

HTML

```
<p>Meu aniversário é no dia 25<sup>th</sup> de maio.</p>
<p>A fórmula química da cafeína é C<sub>8</sub>H<sub>10</sub>N<sub>4</sub>O<sub>2</sub>.</p>
<p>Considere a equação  $x^2 + 2x + 1 = 0$ .</p>
```

13.2.4 Representação de Código-Fonte

Ao escrever documentação técnica, é comum precisar representar código-fonte, comandos de teclado e saídas de terminal. O HTML oferece elementos específicos para cada um desses casos: **code** (código genérico), **var** (variáveis), **kbd** (*keyboard*, entradas de teclado) e **samp** (*sample output*, saída de um programa).

HTML

```

<pre><code>
let x = 42;
console.log(x);
</code></pre>

<p>
  A variável <var>x</var> armazena o valor calculado.
</p>

<p>
  Para copiar, pressione <kbd>Ctrl</kbd> + <kbd>C</kbd>.
</p>

<p>
  A saída do comando foi: <samp>Build concluído com sucesso.</samp>
</p>

```

Note o uso do elemento `pre` (*preformatted text*) envolvendo o `code`: diferentemente do restante do HTML, o `pre` preserva espaços em branco e quebras de linha exatamente como foram digitados no código-fonte — essencial para exibir trechos de código com indentação correta.

13.2.5 Datas e Horários: o Elemento `time`

Como sites e aplicações web são utilizados no mundo inteiro, com formatos de data distintos entre culturas, o elemento `time` permite indicar, de forma padronizada e não ambígua, o valor de uma data ou horário através do atributo `datetime`, enquanto o conteúdo visível do elemento pode ser escrito da forma mais amigável para o leitor humano.

HTML

```

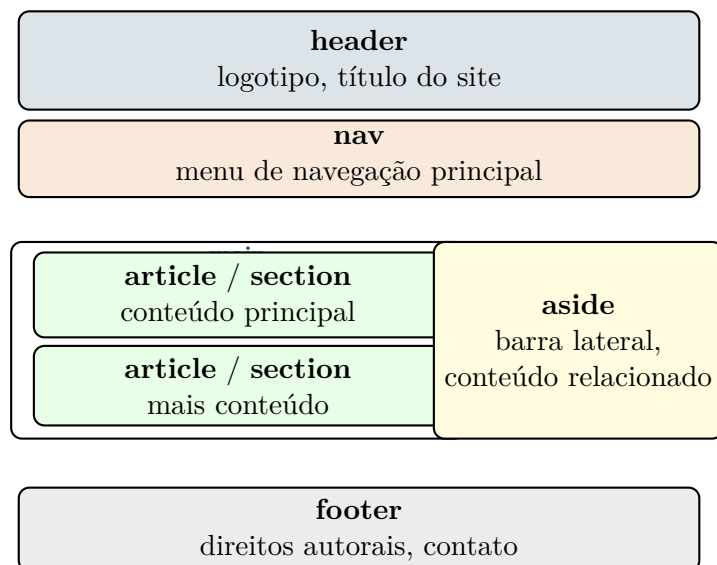
<p>
  Publicado em <time datetime="2016-01-20">20 de janeiro de 2016</time>.
</p>
<p>
  O evento começa às <time datetime="2016-01-20T19:30">19h30</time>.
</p>

```

13.3 Estrutura Semântica de Documentos

Até aqui, vimos elementos que marcam trechos específicos de texto dentro de um parágrafo. Agora avançamos para um nível mais amplo: os elementos que estruturam as **grandes áreas** de uma página web — cabeçalho, navegação, conteúdo principal, barra lateral e rodapé. Todo site, independentemente do seu propósito, costuma compartilhar essas mesmas grandes divisões.

A figura a seguir ilustra visualmente como essas tags se organizam em uma página típica.



13.3.1 O Elemento `header`

O elemento `header` representa um cabeçalho introdutório, que normalmente contém o logotipo, o título do site ou da seção, e eventualmente um menu de navegação ou um campo de busca. É importante entender que `header` pode aparecer mais de uma vez no documento: um `header` diretamente dentro do `body` representa o cabeçalho do site inteiro, mas um `header` dentro de um `article` ou `section` representa o cabeçalho específico daquela seção.

HTML

```
<header>
  <h1>Minha Empresa</h1>
  
  <nav>
    <ul>
      <li><a href="/">Início</a></li>
      <li><a href="/sobre">Sobre</a></li>
      <li><a href="/contato">Contato</a></li>
    </ul>
  </nav>
</header>
```

13.3.2 O Elemento `nav`

O elemento `nav` agrupa os links de **navegação principal** do site. É importante reservá-lo apenas para os menus mais importantes — links secundários, presentes apenas em rodapés ou em contextos específicos, não precisam necessariamente estar dentro de um `nav`.

13.3.3 O Elemento `main`

O elemento `main` envolve o conteúdo **principal** e único daquela página específica — ou seja, o conteúdo que não se repete em outras páginas do site (diferentemente do cabeçalho e do rodapé, que normalmente são os mesmos em todas as páginas). Deve haver, no máximo, um elemento `main` por página.

13.3.4 Os Elementos `article` e `section`

Dentro do `main`, o conteúdo costuma ser subdividido em `article` e/ou `section`. A distinção entre eles não é rígida e depende do contexto:

- `article` representa um conteúdo autocontido e independente, que faria sentido mesmo se distribuído isoladamente (por exemplo, uma postagem de blog, uma notícia, um comentário de usuário).
- `section` agrupa um conjunto de conteúdo tematicamente relacionado, mas que não necessariamente é independente — por exemplo, uma seção sobre "música" e, mais abaixo na mesma página, outra seção sobre "culinária".

HTML

```
<main>
  <article>
    <header>
      <h2>Como aprender CSS do zero</h2>
    </header>
    <p>Neste artigo, vamos abordar os fundamentos do CSS...</p>
  </article>

  <section>
    <h2>Comentários dos leitores</h2>
    <article>
      <p>Ótimo artigo, me ajudou muito!</p>
    </article>
  </section>
</main>
```

Dica

Uma boa prática é sempre iniciar cada `section` (ou `article`) com um elemento de título (`h1` a `h6`), da mesma forma que fizemos no capítulo anterior ao estruturar capítulos e subseções de um texto.

13.3.5 O Elemento `aside`

O elemento `aside` representa um conteúdo que está relacionado ao conteúdo principal, mas que não é o foco central da página — uma barra lateral, uma nota explicativa, um bloco de anúncios ou links relacionados.

HTML

```
<aside>
  <h2>Artigos relacionados</h2>
  <ul>
    <li><a href="/artigo-1">Introdução ao HTML</a></li>
    <li><a href="/artigo-2">Primeiros passos com CSS</a></li>
  </ul>
</aside>
```

13.3.6 O Elemento footer

O elemento `footer` representa o rodapé — normalmente contendo informações de contato, direitos autorais, links de política de acessibilidade ou de privacidade. Assim como o `header`, ele pode aparecer tanto em nível global (rodapé do site inteiro) quanto dentro de uma `section` ou `article` específico.

HTML

```
<footer>
  <p>&copy; 2026 Minha Empresa. Todos os direitos reservados.</p>
  <p><a href="/politica-privacidade">Política de Privacidade</a></p>
</footer>
```

13.4 Um Exemplo Completo e a Questão da Acessibilidade

Juntando todos esses elementos, uma página típica pode ser esboçada da seguinte forma:

HTML

```
<body>
  <header>
    <h1>Blog de Tecnologia</h1>
    <nav>
      <ul>
        <li><a href="/">Início</a></li>
        <li><a href="/artigos">Artigos</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <article>
      <h2>Entendendo o Box Model do CSS</h2>
      <p>O modelo de caixa é um dos conceitos mais importantes...</p>
    </article>

    <aside>
```

```

    <h2>Sobre o autor</h2>
    <p>Desenvolvedor front-end apaixonado por acessibilidade.</p>
  </aside>
</main>

<footer>
  <p>© 2026 - Todos os direitos reservados.</p>
</footer>
</body>

```

Vale destacar por que essa estruturação semântica importa tanto: estima-se que entre 4% e 5% da população mundial tenha algum tipo de deficiência visual — e, considerando outras formas de deficiência, esse número pode chegar a cerca de 15% da população global. Ao usar elementos genéricos como `div` e `span` em vez dos elementos semânticos apropriados, o desenvolvedor perde a oportunidade de comunicar corretamente o significado de cada parte do documento para leitores de tela e outras tecnologias assistivas.

13.4.1 Quando Usar `div` e `span`

Nem sempre existe um elemento HTML semântico adequado para o que se deseja representar — afinal, o HTML precisa ser genérico o suficiente para servir a qualquer tipo de site, e não conhece conceitos de nicho como “carrinho de compras”. Nesses casos, é aceitável usar os elementos genéricos e não semânticos `div` (para blocos) e `span` (para conteúdo em linha), em conjunto com o atributo `class` para que o CSS ou o JavaScript possam localizá-los.

HTML

```

<div class="carrinho-compras">
  <ul>
    <li>Produto A - R$ 29,90</li>
    <li>Produto B - R$ 59,90</li>
  </ul>
  <p>Total: R$ 89,80</p>
</div>

```

Dica

O problema não é usar `div` e `span` em si — é usá-los **sem perceber** que eles não carregam nenhum poder semântico, quando existiria um elemento mais apropriado disponível. Sempre pergunte primeiro: “existe um elemento HTML semântico que descreve melhor este conteúdo?” Só recorra a `div`/`span` quando a resposta for não.

Síntese do Capítulo

- Citações usam `blockquote` (em bloco) ou `q` (em linha), com o atributo `cite` indicando a fonte; o elemento `cite` é reservado para títulos de obras.

- Elementos como `abbr`, `address`, `sup/sub`, `code/var/kbd/samp` e `time` cobrem casos textuais específicos com forte valor semântico e de acessibilidade.
- A estrutura de uma página web é organizada nos elementos `header`, `nav`, `main`, `article`, `section`, `aside` e `footer`.
- `header` e `footer` podem existir tanto em nível global (o cabeçalho/rodapé do site) quanto em nível local (dentro de uma `section` ou `article` específico).
- A diferença entre `article` e `section` não é rígida: `article` é conteúdo autocon-tido e independente; `section` agrupa conteúdo tematicamente relacionado.
- `div` e `span` são elementos não semânticos, úteis apenas quando não existe uma alternativa semântica adequada disponível em HTML.
- A escolha correta de elementos semânticos impacta diretamente a acessibilidade do site para leitores de tela e outras tecnologias assistivas.

Capítulo 14

Imagens e Vídeo em HTML

Até agora, este livro tratou exclusivamente de marcação de texto. Este capítulo introduz os elementos que permitem incorporar conteúdo de mídia — imagens e vídeos — diretamente em documentos HTML, sempre com atenção especial à acessibilidade e às boas práticas de marcação.

14.1 Incorporando Imagens

Para incorporar uma imagem em um documento HTML, utiliza-se o elemento `img`, combinado com o atributo `src` (*source*), que indica o caminho até o arquivo de imagem.

HTML

```

```

O elemento `img` é um elemento vazio

Diferentemente da maioria dos elementos HTML vistos até aqui, `img` é um **elemento vazio** (*void element*): ele não possui conteúdo textual nem tag de encerramento. Toda a informação relevante é fornecida através de seus atributos.

O caminho informado em `src` pode ser relativo (quando a imagem está no mesmo projeto) ou absoluto (quando a imagem é hospedada em outro servidor):

HTML

```
<!-- Caminho relativo: a imagem está na raiz do projeto -->


<!-- Caminho relativo: a imagem está dentro de uma subpasta "images" -->


<!-- Caminho absoluto: a imagem está hospedada em outro domínio -->

```

14.2 Boas Práticas: os Atributos `alt`, `width` e `height`

Embora o exemplo acima já seja suficiente para exibir uma imagem, ele está incompleto do ponto de vista de boas práticas de marcação. Uma imagem bem marcada em HTML deve sempre incluir, além de `src`, os atributos `alt`, `width` e `height`.

14.2.1 O Atributo `alt`

O atributo `alt` (*alternative text*) fornece um texto alternativo que é exibido caso a imagem, por algum motivo, não possa ser renderizada — por exemplo, se o arquivo foi removido do servidor, se o nome do arquivo foi digitado incorretamente, ou se o usuário está navegando com um leitor de tela.

HTML

```

```

Dica

O atributo `alt` é essencial para acessibilidade. Usuários com deficiência visual, que navegam com leitores de tela, dependem inteiramente desse texto para entender o que a imagem representa. Além disso, se o arquivo de imagem for renomeado ou removido do servidor por engano, o texto alternativo garante que o usuário ainda receba alguma informação útil no lugar da imagem quebrada. Diferentes navegadores tratam esse cenário de forma um pouco distinta — alguns exibem o texto de `alt` diretamente no espaço da imagem quebrada, outros não — mas o atributo continua sendo indispensável.

14.2.2 Os Atributos `width` e `height`

Os atributos `width` e `height` indicam, em pixels, as dimensões da imagem.

HTML

```

```

Esses valores são importantes porque permitem que o navegador reserve, antecipadamente, o espaço necessário para a imagem no layout da página — mesmo antes que a imagem termine de carregar. Isso evita que o restante do conteúdo “pule” de posição repentinamente assim que a imagem é finalmente carregada, um problema conhecido como *layout shift*, que prejudica a experiência do usuário.

Dica

Os atributos `width` e `height` do HTML **não devem ser usados para redimensionar imagens de forma expressiva**. Se o objetivo é alterar visualmente o tamanho de exibição de uma imagem, a ferramenta correta é o CSS. Redimensionar drasticamente uma imagem apenas com esses atributos HTML pode causar distorção e perda de qualidade (pixelização), pois é fácil esquecer de manter a proporção original entre largura e altura. Para descobrir as dimensões originais de uma imagem, basta consultar as propriedades do arquivo no sistema operacional (por exemplo, clicando com o botão direito e verificando os detalhes da imagem).

14.2.3 O Atributo `title`

Assim como no elemento de âncora, o atributo `title` também pode ser aplicado a uma imagem, exibindo um texto ao passar o mouse sobre ela:

HTML

```

```

Dica

Nem todos os dispositivos possuem a capacidade de "passar o mouse" sobre um elemento (por exemplo, celulares e tablets com tela sensível ao toque). Por isso, o atributo `title` não deve ser usado como única fonte de uma informação importante — considere-o um complemento opcional, nunca essencial.

14.3 Legendas com `figure` e `figcaption`

É muito comum, em livros e artigos, associar uma imagem a uma legenda numerada — "Figura 1", "Figura 2" e assim por diante. Antes do HTML5, essa associação era feita de forma não semântica, geralmente com uma `div` e um parágrafo separado:

HTML — forma antiga, não recomendada

```
<div>
  
  <p>Fig. 1: Um T-Rex em exposição no museu.</p>
</div>
```

O problema dessa abordagem é que não existe nenhuma relação semântica explícita entre a imagem e o parágrafo que a legenda. Em uma página com dez imagens e dez legendas, seria impossível para um navegador (ou leitor de tela) determinar, com certeza, qual legenda pertence a qual imagem.

O HTML5 resolveu esse problema introduzindo os elementos `figure` e `figcaption`:

HTML

```
<figure>
  
  <figcaption>
    Fig. 1: T-Rex em exposição no Museu da Universidade de Manchester.
  </figcaption>
</figure>
```

O elemento `figure` envolve tanto a imagem quanto sua legenda, enquanto `figcaption` — sempre um filho direto de `figure` — marca explicitamente qual trecho de texto é a legenda daquela imagem específica.

Dica

Embora `figure` seja frequentemente usado com imagens, ele não se limita a elas — pode envolver, por exemplo, um trecho de código, um gráfico ou uma tabela que precise de uma legenda associada.

14.4 Vídeo em HTML

Assim como as imagens, os vídeos podem ser incorporados diretamente em um documento HTML, utilizando o elemento `video` e o atributo `src`.

HTML

```
<video src="coelho320.webm"></video>
```

14.4.1 Conteúdo de *Fallback*

Nem todos os navegadores suportam todos os formatos de vídeo. É uma boa prática fornecer um conteúdo alternativo (*fallback*) — um texto, geralmente com um link para download — que será exibido apenas se o navegador não conseguir renderizar o vídeo.

HTML

```
<video src="coelho320.webm">
  <p>
    Seu navegador não suporta vídeo em HTML.
    <a href="coelho320.webm">Clique aqui para baixar o vídeo.</a>
  </p>
</video>
```

14.4.2 Múltiplas Fontes com o Elemento `source`

Como diferentes navegadores suportam diferentes formatos de vídeo (por exemplo, alguns navegadores suportam o formato aberto WebM, enquanto outros dependem do MP4), é recomendável oferecer mais de uma versão do mesmo vídeo, permitindo que o próprio navegador escolha qual formato consegue reproduzir. Isso é feito com o elemento `source`, colocando um ou mais elementos dentro de `video` (sem mais usar o atributo `src` diretamente na tag `video`):

HTML

```
<video controls>
  <source src="coelho320.mp4" type="video/mp4">
  <source src="coelho320.webm" type="video/webm">
  <p>
    Seu navegador não suporta vídeo em HTML.
    <a href="coelho320.mp4">Clique aqui para baixar o vídeo.</a>
  </p>
</video>
```

O navegador percorre a lista de elementos `source` na ordem em que aparecem, e utiliza o primeiro formato que conseguir reproduzir. O atributo `type` (embora não obrigatório) informa antecipadamente ao navegador qual é o tipo MIME de cada fonte, evitando que ele precise baixar parte do arquivo apenas para descobrir se é compatível.

14.5 Atributos do Elemento `video`

14.5.1 O Atributo `controls`

Por padrão, um vídeo incorporado com o elemento `video` não exibe nenhum controle de reprodução — nenhum botão de *play*, pausa ou volume. Para adicionar esses controles, utiliza-se o atributo booleano `controls`.

HTML

```
<video src="coelho320.mp4" controls></video>
```

Com `controls` presente, o navegador insere automaticamente uma barra de controles padrão, permitindo reproduzir, pausar, ajustar volume, exibir em tela cheia e até baixar o vídeo.

14.5.2 Dimensões: `width` e `height`

Assim como em imagens, é possível definir as dimensões de exibição do vídeo:

HTML

```
<video src="coelho320.mp4" controls width="400" height="400"></video>
```

14.5.3 Outros Atributos Úteis

HTML

```
<video
  controls
  width="400"
  height="400"
  loop
  muted
  poster="capa-video.png"
  preload="metadata"
>
  <source src="coelho320.mp4" type="video/mp4">
  <source src="coelho320.webm" type="video/webm">
</video>
```

- `loop`: atributo booleano que faz o vídeo reiniciar automaticamente assim que termina.
- `muted`: atributo booleano que inicia o vídeo sem áudio (o usuário pode reativar o som manualmente).

- **poster**: define uma imagem de capa exibida antes que o usuário inicie a reprodução do vídeo.
- **preload**: indica ao navegador como se comportar antes da reprodução — os valores possíveis são **auto** (carregar o vídeo inteiro antecipadamente), **metadata** (carregar apenas metadados, como duração) ou **none** (não carregar nada até que o usuário inicie a reprodução). Essa opção é especialmente útil para economizar dados em vídeos longos.

Dica

Assim como fizemos para o **width** e **height** de imagens, evite usar esses atributos HTML para controlar minuciosamente a aparência do player de vídeo — para personalizações visuais mais avançadas, o CSS (e, eventualmente, JavaScript) continua sendo a ferramenta apropriada. Os atributos vistos aqui cobrem apenas o comportamento básico de carregamento e reprodução.

Síntese do Capítulo

- O elemento **img** é vazio e usa o atributo **src** para indicar o caminho da imagem (relativo ou absoluto).
- Toda imagem bem marcada deve incluir os atributos **alt** (texto alternativo, essencial para acessibilidade), **width** e **height** (que evitam saltos de layout durante o carregamento).
- **width** e **height** não devem ser usados para redimensionamento visual expressivo — essa é uma responsabilidade do CSS.
- Os elementos **figure** e **figcaption** associam semanticamente uma imagem (ou outro conteúdo) à sua legenda, substituindo a antiga prática de usar **div** e parágrafos soltos.
- O elemento **video** incorpora vídeos, sendo recomendável usar múltiplos elementos **source** para oferecer diferentes formatos e maximizar a compatibilidade entre navegadores.
- O atributo **controls** é indispensável para que o usuário consiga interagir com o vídeo (reproduzir, pausar, ajustar volume).
- Atributos como **loop**, **muted**, **poster** e **preload** controlam comportamentos adicionais de reprodução do vídeo.

Parte III

CSS

Capítulo 15

Introdução ao CSS e Primeiros Passos

Com os fundamentos de HTML consolidados, chegou o momento de iniciar a Parte III deste livro: o **CSS** (*Cascading Style Sheets*, ou Folhas de Estilo em Cascata). Se o HTML é responsável por definir a estrutura e a semântica dos nossos documentos, o CSS é a tecnologia que define sua **apresentação visual** — cores, fontes, espaçamentos, layout de colunas, posicionamento de elementos e muito mais.

Este é, tradicionalmente, o caminho natural de aprendizado de qualquer desenvolvedor front-end: primeiro o HTML (estrutura e semântica), depois o CSS (estilo e layout) e, por fim, o JavaScript (comportamento e interatividade) — que será o assunto da Parte IV deste livro.

15.1 O que é CSS?

A primeira observação importante é que, mesmo que você nunca escreva uma única linha de CSS, os navegadores web ainda assim conseguem renderizar visualmente qualquer documento HTML. Isso é possível porque os navegadores já implementam, internamente, um conjunto de estilos padrão para os elementos definidos na especificação HTML. É por isso que uma lista não ordenada (`ul`) já aparece automaticamente com marcadores (bullets) na tela, mesmo sem nenhuma folha de estilo associada: esse é o estilo padrão do navegador. Quando escrevemos nossas próprias regras CSS, na prática estamos **sobrescrevendo** esses estilos padrão recebidos do navegador.

O CSS, portanto, é a linguagem que especifica como os elementos definidos estruturalmente e semanticamente pelo HTML devem aparecer para o usuário final. Com CSS, é possível alterar tipo e tamanho de fonte, cores, espaçamentos, disposição em colunas, posicionamento de elementos na tela e, nas versões mais recentes da linguagem, até criar animações — tarefas que antigamente exigiam linguagens de programação completas.

CSS não é exclusivo do HTML

Embora, na prática, usemos CSS quase sempre para estilizar documentos HTML, essa não é a única linguagem de marcação que o CSS pode estilizar — existem outras, como SVG e XML. Esse detalhe se tornará relevante mais adiante, quando você trabalhar com programação back-end e entender melhor o conceito de *user agent*, um termo mais amplo do que “navegador web”, que se refere a qualquer software capaz de interpretar e representar um documento em nome do usuário.

15.2 A Sintaxe de uma Regra CSS

O CSS é uma linguagem baseada em **regras**. Cada regra é composta por um **seletor** — que indica qual elemento (ou elementos) do documento HTML deve receber o estilo — e um **bloco de declarações**, delimitado por chaves, contendo pares de propriedade e valor.

CSS

```
h1 {  
  color: red;  
  font-size: 5rem;  
}
```

Nesse exemplo, **h1** é o seletor: ele indica que a regra se aplica a todos os elementos **h1** do documento. Dentro do bloco de declarações, temos duas propriedades: **color**, cujo valor é **red** (vermelho), e **font-size**, cujo valor é **5rem** (uma unidade relativa de tamanho que estudaremos com mais detalhe adiante). Se essa regra for aplicada a um documento HTML contendo um elemento **h1**, o título ficará vermelho e com tamanho de fonte maior.

Dica

Uma folha de estilo é composta de várias regras, escritas uma após a outra. Sempre que tiver dúvidas sobre quais valores são aceitos por uma determinada propriedade CSS, uma estratégia muito eficaz é pesquisar no Google o nome da propriedade seguido de "MDN" (por exemplo, "color MDN"). O primeiro resultado normalmente será a página de referência da Mozilla Developer Network para aquela propriedade, com diversos exemplos práticos de uso.

15.3 Associando CSS a um Documento HTML

Existem três formas de aplicar CSS a um documento HTML: folha de estilo externa, folha de estilo interna e estilo em linha (*inline*). Este capítulo apresenta a forma recomendada — a folha de estilo externa — que será aprofundada, junto das outras duas, no próximo capítulo.

Para associar uma folha de estilo externa, utiliza-se o elemento **link** dentro do **head** do documento HTML, com dois atributos: **rel** (indicando o *relationship*, ou seja, que o documento associado é uma folha de estilo) e **href** (indicando o caminho, relativo ou absoluto, até o arquivo **.css**).

HTML — index.html

```
<!DOCTYPE html>  
<html lang="pt-BR">  
<head>  
  <meta charset="UTF-8">  
  <title>Meu primeiro site com CSS</title>  
  <link rel="stylesheet" href="estilos.css">
```



```
</head>
<body>
  <h1>Bem-vindo ao meu site</h1>
  <p class="destaque">Este parágrafo tem uma classe especial.</p>
  <p id="rodape-info">Informações de rodapé.</p>
</body>
</html>
```

CSS — estilos.css

```
h1 {
  color: navy;
}

p {
  color: black;
  font-size: 1.1rem;
}
```

Repare que o arquivo `estilos.css` está localizado no mesmo diretório do arquivo `index.html` — por isso, o caminho informado em `href` é apenas o nome do arquivo. Se a folha de estilo estivesse dentro de uma subpasta chamada `styles`, o caminho seria `styles/estilos.css`, seguindo exatamente a mesma lógica de caminhos relativos já estudada no capítulo sobre hiperlinks.

15.4 Seletores Básicos: Tag, Classe e ID

Um dos aspectos mais poderosos do CSS é a capacidade de selecionar exatamente quais elementos do documento devem receber um determinado estilo. Os três seletores mais fundamentais são o seletor de **tag** (elemento), o seletor de **classe** e o seletor de **ID**.

15.4.1 Seletor de Tag (Elemento)

O seletor de tag seleciona *todos* os elementos daquele tipo no documento, simplesmente escrevendo o nome da tag:

CSS

```
p {
  color: green;
}
```

Essa regra transforma a cor de todos os parágrafos da página para verde, sem exceção. É possível, ainda, aplicar a mesma regra a múltiplos seletores de uma vez, separando-os por vírgula:

CSS

```
p, li {
  color: green;
}
```

```
}
```

Essa regra diz: "tanto os parágrafos quanto os itens de lista devem ter a fonte na cor verde".

15.4.2 Seletor de Classe

Quando queremos selecionar apenas um **subconjunto** de elementos — e não todos os elementos de um determinado tipo — usamos o atributo `class` no HTML, e o seletor de classe no CSS, precedido de um ponto (`.`).

HTML

```
<ul>
  <li>Item comum</li>
  <li class="especial">Item especial</li>
  <li>Item comum</li>
</ul>
```

CSS

```
.especial {
  color: orange;
  font-style: italic;
}
```

Essa regra diz: "todo elemento que possuir um atributo `class` com valor **especial** deve ter a fonte na cor laranja e em itálico". Note que a mesma classe pode ser reaproveitada em quantos elementos forem necessários, inclusive de tags diferentes — por exemplo, tanto um `li` quanto um `span` poderiam compartilhar a classe **especial**:

CSS

```
.especial, span.especial {
  color: orange;
}
```

15.4.3 Seletor de ID

O atributo `id` identifica um elemento de forma **única** dentro do documento — ou seja, teoricamente, apenas um único elemento no documento inteiro deve possuir um determinado valor de `id`. No CSS, o seletor de ID é precedido de uma cerquilha (`#`).

HTML

```
<p id="rodape-info">Informações de rodapé.</p>
```

CSS

```
#rodape-info {
  color: gray;
  font-size: 0.9rem;
}
```

Dica

Um mesmo elemento pode ter apenas um valor de `id`, mas pode ter **vários** valores de `class` simultaneamente, separados por espaço no HTML (por exemplo, `class="destaque grande"`). Além disso, como veremos com mais profundidade no capítulo sobre cascata e especificidade, um seletor de ID tem precedência sobre um seletor de classe, que por sua vez tem precedência sobre um seletor de tag — essa hierarquia de "peso" entre seletores será fundamental para resolver conflitos entre regras CSS.

15.5 Um Exemplo Combinando Tudo

Vamos consolidar os conceitos apresentados com um exemplo completo, combinando um documento HTML com uma folha de estilo externa que utiliza os três tipos de seletores estudados:

HTML — index.html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Exemplo combinado</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Blog de Programação</h1>
  <p>Este é um parágrafo comum.</p>
  <p class="destaque">Este parágrafo recebe destaque visual.</p>
  <p id="assinatura">Escrito por um futuro desenvolvedor front-end.</p>
</body>
</html>
```

CSS — estilos.css

```
h1 {
  color: navy;
}

p {
  color: #333333;
  font-family: Georgia, serif;
}

.destaque {
  background-color: yellow;
  font-weight: bold;
}

#assinatura {
```

```
font-style: italic;  
color: gray;  
}
```

Nesse exemplo, o título recebe a cor azul-marinho por meio do seletor de tag; todos os parágrafos recebem uma cor de texto e uma família de fontes específica, também por seletor de tag; o parágrafo com `class="destaque"` ganha um fundo amarelo e negrito adicional, sem afetar os demais parágrafos; e o parágrafo com `id="assinatura"` recebe um estilo próprio, exclusivo daquele único elemento.

Dica

Ao testar seus próprios exemplos, você pode trabalhar tanto localmente (com um editor como o Visual Studio Code, por exemplo usando a extensão "Live Server" para servir a página) quanto em editores online, como o CodePen ou diretamente nos exemplos interativos disponibilizados pela própria documentação da MDN. Ambas as abordagens são válidas para praticar os conceitos apresentados neste capítulo.

Síntese do Capítulo

- CSS é a linguagem responsável por definir a apresentação visual e o layout de documentos estruturados com HTML, complementando a estrutura e a semântica fornecidas por esta última.
- Mesmo sem CSS, navegadores já aplicam estilos padrão internos aos elementos HTML; escrever CSS é, na prática, sobrescrever esses estilos.
- Uma regra CSS é composta por um seletor e um bloco de declarações entre chaves, com pares de propriedade e valor separados por dois-pontos.
- A forma recomendada de associar CSS a HTML é a folha de estilo externa, vinculada com `<link rel="stylesheet" href="...">` dentro do `head`.
- O seletor de tag afeta todos os elementos daquele tipo; o seletor de classe (precedido de ponto) afeta um subconjunto de elementos marcados com `class`; o seletor de ID (precedido de cerquilha) afeta um único elemento identificado por `id`.
- Seletores podem ser combinados em uma mesma regra separando-os por vírgula, aplicando o mesmo bloco de declarações a múltiplos alvos.

Capítulo 16

Primeiros Passos com CSS (Parte 2)

No capítulo anterior, vimos os conceitos fundamentais do CSS: o que é a linguagem, como funciona uma regra, e os três seletores básicos (tag, classe e ID). Este capítulo aprofunda a discussão, revisando as três formas de aplicar CSS a um documento, explorando seletores mais avançados baseados na posição e no estado dos elementos, e introduzindo conceitos que serão centrais no restante da Parte III: funções CSS, o conceito de *box* (caixa) e a regra `@media`.

16.1 As Três Formas de Aplicar CSS

Existem três maneiras de associar estilos CSS a um documento HTML: folha de estilo **externa**, folha de estilo **interna** e estilo **em linha** (*inline*).

16.1.1 Folha de Estilo Externa (Recomendada)

Como já vimos, a forma recomendada é a folha de estilo externa, associada via elemento `link` dentro do `head`:

HTML

```
<head>
  <link rel="stylesheet" href="styles/estilo.css">
</head>
```

A grande vantagem dessa abordagem é o **reúso**: quando um site possui várias páginas, todas podem compartilhar a mesma folha de estilo externa, garantindo uma aparência consistente. Se, no futuro, for necessário alterar uma cor em todo o site, basta modificar um único arquivo CSS, em vez de precisar editar manualmente cada página individualmente.

16.1.2 Folha de Estilo Interna

É possível, também, declarar estilos dentro do próprio documento HTML, utilizando o elemento `style` dentro do `head`:

HTML

```
<head>
  <style>
    h1 {
      color: blue;
      background-color: yellow;
    }
  </style>
</head>
```

```
    }  
    p {  
        color: red;  
    }  
    </style>  
</head>
```

16.1.3 Estilo em Linha (*Inline*)

Por fim, é possível aplicar estilo diretamente em um elemento específico, usando o atributo `style`:

HTML

```
<h1 style="color: blue; background-color: yellow;">Hello, world!</h1>  
<p style="color: red;">Texto de exemplo.</p>
```

Dica

Estilos internos e, principalmente, estilos em linha, **não são recomendados** como prática padrão. Se você quiser replicar o mesmo estilo em várias páginas de um site, precisará copiar e colar o mesmo trecho repetidamente — e, se um dia precisar alterar uma cor, terá que editar cada cópia manualmente, um verdadeiro pesadelo de manutenção. Além disso, misturar HTML (estrutura) e CSS (apresentação) no mesmo arquivo dificulta o trabalho em equipe: se um desenvolvedor é especialista em HTML e outro em CSS, separar os arquivos permite que ambos trabalhem em paralelo sem conflitos.

Existem, porém, exceções legítimas. Um exemplo clássico é o desenvolvimento de e-mails de marketing (mala direta): muitos clientes de e-mail removem folhas de estilo externas e internas por segurança, então, nesse contexto específico, o estilo em linha se torna necessário para garantir que o e-mail seja exibido corretamente. Outro exemplo é quando você trabalha com um sistema de gestão de conteúdo (CMS) como WordPress ou Drupal, e não possui permissão de acesso para editar a folha de estilo principal do site — nesses casos, uma solução de contorno pode ser aplicar estilos internos ou em linha nas páginas que você efetivamente controla.

16.2 Seletores Baseados em Localização e Estado

Além dos seletores de tag, classe e ID, o CSS permite selecionar elementos com base em sua **posição** dentro da árvore do documento (o DOM) e em seu **estado** atual.

16.2.1 Combinador Descendente

O combinador descendente, representado por um espaço em branco entre dois seletores, seleciona um elemento que esteja *dentro* de outro, em qualquer nível de profundidade.

CSS

```
li em {  
  color: rebeccapurple;  
}
```

Essa regra seleciona qualquer elemento **em** que esteja dentro de um elemento **li**, em qualquer profundidade — e apenas esses elementos **em** específicos, não todos os **em** do documento.

16.2.2 Combinador Adjacente

O combinador de irmão adjacente, representado pelo sinal de mais (+), seleciona um elemento que esteja imediatamente após outro, no mesmo nível hierárquico (e não aninhado dentro dele).

CSS

```
h1 + p {  
  font-size: 200%;  
}
```

Essa regra seleciona o primeiro parágrafo que aparece imediatamente após um **h1**, no mesmo nível do DOM — e apenas esse primeiro parágrafo, não os demais parágrafos da página.

16.2.3 Combinando Seletores

É possível combinar múltiplos seletores em uma única regra, criando combinações mais específicas:

CSS

```
article p span {  
  color: red;  
}  
  
h1 + ul li {  
  color: green;  
}  
  
.especial p + h1 {  
  background-color: black;  
  color: yellow;  
}
```

A primeira regra seleciona qualquer **span** que esteja dentro de um **p**, que por sua vez esteja dentro de um **article**. A segunda seleciona qualquer **li** dentro de uma **ul** que venha logo após um **h1**. A terceira combina um seletor de classe com o combinador adjacente, selecionando um **h1** que venha logo após um **p** que esteja dentro de um elemento com a classe **especial**.

16.2.4 Seletores de Estado (Pseudo-classes)

O CSS também permite estilizar elementos com base em seu **estado** — por exemplo, um link que já foi visitado ou um elemento sobre o qual o mouse está passando no momento. O exemplo mais clássico é o elemento de âncora, que possui diferentes estados possíveis.

CSS

```
a:visited {
  color: pink;
}

a:hover {
  text-decoration: none;
}
```

A primeira regra estiliza um link que já foi visitado pelo menos uma vez pelo usuário. A segunda remove o sublinhado de um link no momento em que o mouse passa sobre ele.

Dica

Tenha cuidado ao alterar comportamentos que os usuários já esperam por convenção — como o sublinhado dos links. Embora o CSS permita remover completamente essa decoração, pense com calma se essa mudança realmente compensa do ponto de vista de acessibilidade e usabilidade. Lembre-se, também, de que o CSS nem sempre é percebido por todos os usuários da mesma forma: alguém navegando por leitor de tela está ouvindo o conteúdo, e não vendo as cores e tamanhos que você definiu — use estilos com moderação e sempre pensando na experiência de todos os públicos.

16.3 Introdução ao Conceito de Caixa (*Box*)

Todo elemento HTML, ao ser renderizado, é representado internamente como uma **caixa** (*box*) — um retângulo (ou quadrado) que envolve o conteúdo daquele elemento. Esse conceito será aprofundado com muito mais detalhe no próximo capítulo, dedicado inteiramente ao modelo de caixa do CSS (*box model*), mas já vale introduzi-lo aqui.

HTML

```
<h1>Hello <span class="especial">world</span></h1>
```

CSS

```
.especial {
  background-color: black;
  color: yellow;
  padding: 10px;
}
```


Ao renderizar esse exemplo, fica visualmente claro que o elemento `span` com a classe `especial` ocupa uma "caixa" retangular ao seu redor — com fundo preto, texto amarelo e um espaçamento interno de 10 pixels entre o texto e a borda imaginária dessa caixa. Essa distância entre o conteúdo e a borda da caixa é chamada de `padding`, uma das propriedades centrais do modelo de caixa que exploraremos em profundidade no próximo capítulo.

16.4 Funções CSS: o Exemplo de `calc()`

Os valores atribuídos a uma propriedade CSS nem sempre são literais simples como `10px` ou `red`. Muitas vezes, esses valores podem ser o resultado de **funções** CSS. Um exemplo útil é a função `calc()`, que permite realizar operações matemáticas diretamente no valor de uma propriedade.

HTML

```
<div class="externo">
  <div class="interno">Texto de exemplo</div>
</div>
```

CSS

```
.externo {
  border: 5px solid black;
}

.interno {
  width: calc(90% - 30px);
  padding: 10px;
  background-color: purple;
  color: white;
}
```

Nesse exemplo, a largura da `div` interna é calculada como noventa por cento da largura do elemento pai, subtraído de 30 pixels — permitindo, por exemplo, reservar um espaço visual fixo entre os dois elementos, independentemente do tamanho da tela do usuário.

16.5 A Regra `@media` e o Layout Responsivo

Uma das características mais importantes do CSS moderno é a capacidade de adaptar o layout de um site de acordo com as dimensões da tela do usuário — seja um computador, um tablet ou um celular. Essa adaptação é feita através de *at-rules* (regras precedidas de `@`), sendo a mais utilizada a regra `@media`.

CSS

```
body {
  background-color: pink;
}
```

```
@media (min-width: 30em) {  
  body {  
    background-color: blue;  
  }  
}
```

Essa folha de estilo diz: "a cor de fundo do corpo do documento é rosa por padrão; porém, quando a largura da tela (*viewport*) for maior ou igual a 30em, a cor de fundo passa a ser azul". Esse mecanismo é a base do que chamamos de **design responsivo**: definir regras de estilo distintas para diferentes larguras de tela, permitindo que o mesmo site se adapte graticamente a celulares, tablets e monitores grandes.

Dica

Uma abordagem muito comum no desenvolvimento moderno é o chamado *mobile first*: projetar o layout inicialmente pensando em telas pequenas (dispositivos móveis) e, em seguida, usar `@media` para adicionar ajustes de layout à medida que a tela cresce. Essa estratégia tende a produzir sites mais robustos e mais fáceis de adaptar a qualquer tamanho de dispositivo.

16.6 Notação Abreviada (*Shorthand*) e Comentários

Muitas propriedades CSS possuem uma forma abreviada de escrita, chamada *shorthand*, que permite configurar várias propriedades relacionadas com uma única linha.

CSS

```
/* Forma longa (uma propriedade por lado) */  
.caixa-longa {  
  padding-top: 10px;  
  padding-right: 15px;  
  padding-bottom: 10px;  
  padding-left: 15px;  
}  
  
/* Forma abreviada equivalente (sentido horário: topo, direita, baixo, esquerda) */  
.caixa-curta {  
  padding: 10px 15px;  
}
```

Dica

Ao usar a notação abreviada, tenha cuidado: quando você fornece menos valores do que os quatro lados possíveis, o CSS aplica regras específicas de repetição (por exemplo, dois valores se aplicam a topo/baixo e direita/esquerda, respectivamente). Além disso, algumas propriedades *shorthand* redefinem **todos** os valores relacionados para seus padrões, mesmo os que você não mencionou explicitamente

— o que pode gerar resultados inesperados se você não conhecer bem o comportamento da propriedade abreviada em questão.

Por fim, o CSS ignora espaços em branco (da mesma forma que o HTML), mas isso não significa que você deva abrir mão de uma boa formatação. Comentários em CSS são delimitados por `/*` e `*/`, e podem se estender por múltiplas linhas — são fundamentais para documentar o código e facilitar a manutenção futura, tanto para você mesmo quanto para outros desenvolvedores.

CSS

```
/*
  Seção: estilos do cabeçalho principal
  Autor: Equipe de Front-End
*/
header {
  background-color: navy;
  color: white;
}
```

Dica

Se você aplicar uma regra CSS e ela parecer não estar funcionando, uma técnica útil de depuração é comentar temporariamente uma das regras conflitantes e recarregar a página. Isso ajuda a identificar se o problema é um conflito de especificidade entre regras — assunto que será tratado em detalhe no capítulo sobre cascata e herança.

Síntese do Capítulo

- Existem três formas de aplicar CSS: folha externa (recomendada, melhor para reúso e manutenção), folha interna (dentro de `<style>`) e estilo em linha (atributo `style`, geralmente desaconselhado).
- Combinadores como o descendente (espaço) e o adjacente (+) permitem selecionar elementos com base em sua posição na árvore do documento.
- Pseudo-classes como `:visited` e `:hover` permitem estilizar elementos com base em seu estado atual.
- Todo elemento HTML é renderizado como uma caixa (*box*), conceito que será aprofundado no próximo capítulo sobre o box model.
- Funções CSS, como `calc()`, permitem calcular valores de propriedades dinamicamente, combinando unidades diferentes.
- A regra `@media` permite aplicar estilos condicionalmente, de acordo com as dimensões da tela do usuário, sendo a base do design responsivo.

- A notação abreviada (*shorthand*), como em `padding: 10px 15px;`, agiliza a escrita de CSS, mas exige atenção às regras de repetição de valores.

Capítulo 17

O Modelo de Caixa CSS (Box Model)

Chegamos a um dos conceitos mais importantes de todo o CSS — e, sem exagero, um dos conceitos-chave para qualquer pessoa que deseje criar layouts de páginas web. Como já adiantamos rapidamente no capítulo anterior, tudo em HTML e CSS pode ser visualizado como se estivesse dentro de uma **caixa** (*box*), seja ela quadrada ou retangular. Entender profundamente esse conceito é essencial para alinhar elementos, controlar espaçamentos e construir qualquer layout mais sofisticado.

17.1 Caixas em Bloco e Caixas em Linha

Antes de entrar no modelo de caixa propriamente dito, é importante relembrar uma distinção já mencionada nos capítulos de HTML: alguns elementos são de **bloco** (*block*) e outros são **em linha** (*inline*). Essa classificação também se aplica diretamente ao CSS, determinando como cada caixa se comporta em relação ao fluxo de disposição dos elementos na tela.

- Uma caixa em **bloco** ocupa uma linha inteira por si só, dentro do espaço disponível (*viewport*), e força um salto para a próxima linha no elemento seguinte. Exemplos de elementos HTML que são caixas em bloco por padrão: `div`, `p`, `ul`, `h1` a `h6`.
- Uma caixa **em linha** não quebra para uma nova linha — ela ocupa apenas o espaço necessário para seu conteúdo, na mesma linha em que está posicionada, e normalmente está inserida dentro de uma caixa em bloco. Exemplos: `a`, `span`, `em`, `strong`.

Uma diferença fundamental entre os dois tipos: propriedades de largura (`width`) e altura (`height`) não têm efeito sobre uma caixa em linha — quem determina essas dimensões é a caixa em bloco que a envolve. Já propriedades como `padding`, `margin` e `border` podem ser aplicadas a ambos os tipos, embora com efeitos distintos, conforme veremos mais adiante neste capítulo.

17.1.1 A Propriedade `display`

É possível alterar esse comportamento padrão usando a propriedade `display`, transformando um elemento originalmente em bloco para se comportar como em linha, e vice-versa:

CSS

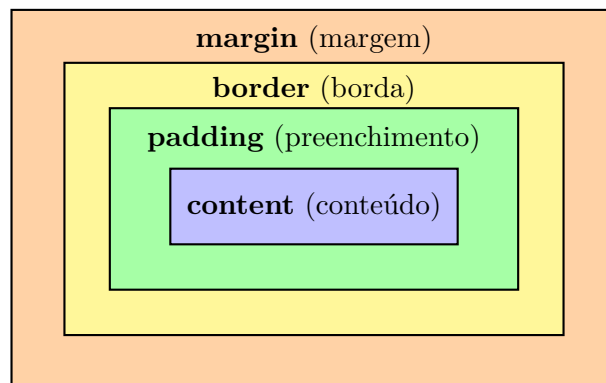
```
/* Transforma um elemento normalmente em linha (span) em bloco */
span.destaque {
  display: block;
}

/* Transforma um elemento normalmente em bloco (li) em elemento
   com layout flexível interno */
ul.menu {
  display: flex;
}
```

A propriedade **display** controla, na verdade, dois aspectos distintos e complementares: o **display externo** (se a caixa se comporta como bloco ou como linha, em relação aos elementos ao redor) e o **display interno** (como os elementos *dentro* daquela caixa são organizados — o padrão é chamado de fluxo normal, mas valores como **flex** alteram completamente essa disposição interna).

17.2 As Camadas do Modelo de Caixa

O modelo de caixa CSS (*CSS Box Model*) descreve como cada elemento HTML é representado como uma caixa composta por quatro camadas concêntricas: o **conteúdo** (*content*), o **preenchimento interno** (*padding*), a **borda** (*border*) e a **margem** (*margin*).



O conteúdo é o texto ou elemento em si; o **padding** é a distância entre o conteúdo e a borda; a **border** é a linha visível (ou invisível) ao redor do preenchimento; e a **margin** é o espaço externo à borda, responsável por afastar essa caixa de outras caixas vizinhas no layout.

Margem não é "parte" da caixa visível

Um detalhe frequentemente confundido por iniciantes: a margem é um espaço **externo** à caixa, usado apenas para separar elementos entre si — ela não recebe cor de fundo (**background-color**) nem borda. Já o **padding** é um espaço **interno**, e por isso herda a cor de fundo do próprio elemento.

17.3 As Propriedades `padding`, `border` e `margin`

Vamos consolidar essas três propriedades com um exemplo prático completo:

HTML

```
<div class="caixa">Conteúdo de exemplo</div>
```

CSS

```
.caixa {  
  width: 300px;  
  height: 100px;  
  padding: 20px;  
  border: 5px solid black;  
  margin: 30px;  
  background-color: lightgray;  
}
```

Cada uma dessas três propriedades — `padding`, `border` e `margin` — também aceita valores individuais para cada um dos quatro lados de uma caixa (topo, direita, baixo e esquerda), tanto na forma longa quanto na forma abreviada (*shorthand*):

CSS

```
/* Forma longa: um lado por vez */  
.caixa-detalhada {  
  padding-top: 10px;  
  padding-right: 15px;  
  padding-bottom: 10px;  
  padding-left: 15px;  
  
  margin-top: 20px;  
  margin-right: 0;  
  margin-bottom: 20px;  
  margin-left: 0;  
  
  border-top: 1px solid black;  
  border-right: 2px dashed gray;  
  border-bottom: 1px solid black;  
  border-left: 2px dashed gray;  
}  
  
/* Forma abreviada: sentido horário, começando pelo topo */  
.caixa-resumida {  
  padding: 10px 15px 10px 15px;  
  margin: 20px 0;  
  border: 2px dotted blue;  
}
```

Dica

A margem, diferentemente do **padding**, pode receber valores **negativos**. Uma margem negativa aproxima a caixa de seus vizinhos, podendo até causar sobreposição entre elementos — um efeito às vezes desejado, mas que exige cuidado ao ser utilizado.

17.4 Colapso de Margens (*Margin Collapsing*)

Um comportamento importante — e por vezes surpreendente para iniciantes — é o chamado **colapso de margens**. Quando duas margens verticais de elementos adjacentes se encontram, elas não são simplesmente somadas; em vez disso, o CSS aplica um algoritmo específico:

- Se ambas as margens forem positivas, o valor resultante é o **maior** dos dois valores (e não a soma).
- Se uma margem for positiva e a outra negativa, o valor resultante é a soma algébrica (o valor positivo menos o valor absoluto do negativo).
- Se ambas as margens forem negativas, o valor resultante é o **mais negativo** dos dois (o de maior valor absoluto).

CSS

```
.paragrafo-um {  
  margin-bottom: 50px;  
}  
  
.paragrafo-dois {  
  margin-top: 30px;  
}
```

Nesse exemplo, a margem final entre os dois parágrafos **não** será de 80px (a soma). Como ambos os valores são positivos, o resultado será o maior valor entre eles: 50px.

17.5 A Propriedade **box-sizing**

Agora chegamos a um ponto crucial, que costuma gerar bastante confusão entre quem está começando: como o navegador calcula a largura e a altura **totais** de uma caixa, quando existem **padding** e **border** envolvidos.

17.5.1 O Modelo Padrão (**content-box**)

No modelo de caixa padrão do CSS (chamado **content-box**), as propriedades **width** e **height** definem apenas as dimensões da área de **conteúdo** — sem contar o **padding** nem a **border**. Isso significa que, para saber a largura total ocupada pela caixa na tela, é preciso somar manualmente o **padding** (dos dois lados) e a **border** (dos dois lados) ao valor de **width**.

CSS

```
.caixa-padrao {  
  width: 350px;  
  height: 150px;  
  padding: 25px;  
  border: 5px solid black;  
  box-sizing: content-box; /* valor padrão dos navegadores */  
}
```

Nesse exemplo, embora `width` esteja definido como `350px`, a largura total da caixa, visível na tela, será:

$$350 + (25 \times 2) + (5 \times 2) = 350 + 50 + 10 = 410\text{px}$$

E a altura total será:

$$150 + (25 \times 2) + (5 \times 2) = 150 + 50 + 10 = 210\text{px}$$

Dica

A margem **não** entra nesse cálculo de dimensões da caixa — ela fica, por definição, do lado de fora da caixa (`border-box`), servindo apenas para afastar a caixa de seus vizinhos no layout.

17.5.2 O Modelo Alternativo (`border-box`)

Para simplificar esses cálculos — e evitar ter que somar manualmente `padding` e `border` toda vez que se define uma largura — o CSS oferece um modelo alternativo, ativado com a propriedade `box-sizing: border-box`. Nesse modelo, o valor definido em `width` e `height` já representa a largura e altura **totais** da caixa, incluindo `padding` e `border`.

CSS

```
.caixa-alternativa {  
  width: 350px;  
  height: 150px;  
  padding: 25px;  
  border: 5px solid black;  
  box-sizing: border-box;  
}
```

Nesse segundo caso, a caixa ocupará exatamente `350px` de largura e `150px` de altura na tela — o próprio navegador se encarrega de reduzir proporcionalmente a área de conteúdo interna para acomodar o `padding` e a `border` dentro desses limites.

Dica

É extremamente comum que desenvolvedores optem por aplicar `box-sizing: border-box` a **todos** os elementos do documento, de uma só vez, logo no início da folha de estilo, justamente para evitar ter que ficar calculando manualmente

as dimensões totais de cada caixa:

CSS

```
*, *::before, *::after {  
    box-sizing: border-box;  
}
```

17.6 Ferramentas do Navegador para Inspeccionar o Box Model

Os navegadores modernos — especialmente Chrome e Firefox — oferecem ferramentas de desenvolvedor (*DevTools*) excelentes para inspecionar visualmente o modelo de caixa de qualquer elemento na página. Ao clicar com o botão direito sobre um elemento e escolher “Inspecionar”, é possível visualizar um diagrama exatamente igual ao apresentado neste capítulo, com os valores reais de **margin**, **border**, **padding** e da área de conteúdo daquele elemento específico.

Dica

Essa ferramenta é indispensável para depuração de layout. Sempre que um elemento parecer estar com um tamanho ou posicionamento inesperado, inspecione seu box model diretamente pelo navegador — muitas vezes o problema está relacionado a uma margem ou **padding** não considerado, ou a uma confusão entre os modelos **content-box** e **border-box**.

17.7 O Valor Misto **inline-block**

Além de **block** e **inline**, a propriedade **display** aceita um terceiro valor bastante útil: **inline-block**. Esse valor combina características dos dois mundos: o elemento não quebra para uma nova linha (como um elemento em linha), mas respeita integralmente as propriedades **width**, **height**, **padding** e **margin** (como um elemento em bloco).

HTML

```
<nav class="menu">  
  <a href="/" class="link-menu">Início</a>  
  <a href="/sobre" class="link-menu">Sobre</a>  
  <a href="/contato" class="link-menu">Contato</a>  
</nav>
```

CSS

```
.link-menu {  
    display: inline-block;  
    padding: 10px 20px;  
    background-color: navy;  
    color: white;
```

```
text-decoration: none;
}

.link-menu:hover {
  background-color: darkblue;
}
```

Nesse exemplo, o elemento `a` — que por padrão é um elemento em linha e, portanto, ignora `padding` vertical de forma consistente — passa a respeitar plenamente o `padding` definido, aumentando a área clicável do link muito além do texto original, sem forçar uma quebra de linha entre um link e o próximo. Essa técnica é extremamente comum em barras de navegação e botões estilizados a partir de links.

Síntese do Capítulo

- Toda caixa CSS é composta por quatro camadas concêntricas: conteúdo, `padding` (preenchimento interno), `border` (borda) e `margin` (margem externa).
- Elementos podem ser de bloco (ocupam uma linha inteira) ou em linha (ocupam apenas o espaço do conteúdo), e esse comportamento pode ser alterado com a propriedade `display`.
- No modelo padrão (`content-box`), `width/height` definem apenas a área de conteúdo; `padding` e `border` são somados à parte para obter as dimensões totais da caixa.
- No modelo alternativo (`box-sizing: border-box`), `width/height` já representam a dimensão total da caixa, incluindo `padding` e `border` — simplificando bastante os cálculos de layout.
- Margens verticais adjacentes sofrem colapso: o valor final não é a soma das margens, mas segue regras específicas dependendo dos sinais dos valores envolvidos.
- As ferramentas de desenvolvedor do navegador (DevTools) permitem inspecionar visualmente o box model de qualquer elemento, sendo essenciais para depuração de layout.
- O valor `inline-block` combina características de bloco e de linha, sendo muito usado para aumentar a área clicável de links em menus de navegação.

Capítulo 18

Cascata e Herança em CSS

O próprio nome da tecnologia já entrega uma pista de sua importância: CSS significa *Cascading Style Sheets*, ou "Folhas de Estilo em **Cascata**". Entender como esse mecanismo de cascata funciona é uma habilidade essencial para qualquer desenvolvedor front-end. Cedo ou tarde, você criará duas regras CSS que, sem perceber, entram em conflito ao serem aplicadas ao mesmo elemento — e é justamente a cascata (junto de um conceito intimamente relacionado, a **especificidade**) que determina qual regra efetivamente prevalece.

Além da cascata e da especificidade, este capítulo também aborda a **herança** — o mecanismo pelo qual algumas propriedades CSS definidas em um elemento pai são automaticamente repassadas para seus elementos filhos, enquanto outras propriedades não seguem essa mesma lógica.

18.1 O Efeito Cascata: a Ordem Importa

Vamos começar com o exemplo mais simples possível: um documento HTML com um único elemento de título.

HTML

```
<h1>Este é o elemento de título</h1>
```

Suponha que a folha de estilo associada a esse documento contenha duas regras que, ambas, têm como alvo o mesmo elemento **h1**:

CSS

```
h1 {  
  color: red;  
}  
  
h1 {  
  color: blue;  
}
```

Qual cor vencerá: vermelho ou azul? A resposta é **azul**. Isso ocorre porque ambas as regras possuem exatamente a mesma especificidade (as duas usam um simples seletor de elemento **h1**), e, nesse caso de empate, a regra definida **por último** na folha de estilo é a que prevalece. Esse é precisamente o efeito cascata: as regras "escorrem" de cima para baixo, e quando duas regras de igual peso competem pelo mesmo elemento, a que vem mais abaixo no arquivo vence.

Cascata

O termo *cascata* descreve a ordem de prioridade das regras CSS aplicadas a um elemento. Quando duas regras têm a mesma especificidade, a posição da regra na folha de estilo (qual delas vem depois) decide o resultado final.

18.2 Especificidade: Quando a Ordem Não é Suficiente

A cascata, sozinha, não resolve todos os conflitos. Considere agora um cenário em que as duas regras conflitantes têm seletores **diferentes**:

HTML

```
<h1 class="marren">Este é o elemento de título</h1>
```

CSS

```
.marren {  
  color: red;  
}  
  
h1 {  
  color: blue;  
}
```

Mesmo o seletor de classe `.marren` tendo sido escrito **antes** do seletor de elemento `h1` na folha de estilo, o resultado final será a cor **vermelha** — e não azul, como a simples ordem de cascata poderia sugerir. Isso acontece porque, aqui, entra em jogo um segundo fator: a **especificidade**.

Especificidade

Especificidade é o mecanismo pelo qual o navegador decide qual regra aplicar quando várias regras, com seletores diferentes, têm como alvo o mesmo elemento. De forma geral, um seletor de elemento (como `h1`) é considerado menos específico — ele seleciona todos os elementos daquele tipo na página. Já um seletor de classe (como `.marren`) é mais específico, pois seleciona apenas os elementos que possuem aquele valor exato de atributo `class`. Por isso, o seletor de classe obtém uma pontuação de especificidade mais alta e vence o conflito, independentemente de sua posição na folha de estilo.

18.2.1 Calculando a Especificidade

O cálculo de especificidade pode ser pensado como quatro colunas de dígitos, na ordem de importância crescente: **unidade** (seletores de elemento e pseudo-elementos), **dezena** (seletores de classe e pseudo-classes), **centena** (seletores de ID) e **milhar** (estilos em linha, aplicados diretamente com o atributo `style`).

CSS

```
/* especificidade: 0-0-1 (um elemento) */
a {
  background-color: pink;
}

/* especificidade: 0-1-1 (um ID + um elemento) */
#alvo a {
  background-color: purple;
}

/* especificidade: 0-1-4 (um ID + quatro elementos) */
#alvo div ul li a {
  color: yellow;
}
```

Cada tipo de seletor conta apenas para a sua própria coluna. Um número muito grande de seletores de elemento (a coluna de unidades) **nunca** supera um único seletor de classe (a coluna de dezenas), pois as colunas são comparadas da mais significativa para a menos significativa, uma de cada vez — assim como comparamos números decimais.

CSS

```
/* Especificidade 0-0-4: quatro elementos (div, ul, li, a) */
div ul li a {
  color: white;
}

/* Especificidade 0-1-0: apenas uma classe */
.link-especial {
  color: black;
}

/* .link-especial VENCE, mesmo com muito menos seletores,
   porque uma classe pesa mais do que qualquer quantidade
   de seletores de elemento */
```

Dica

Existem calculadoras de especificidade CSS disponíveis online, muito úteis quando você estiver lidando com folhas de estilo grandes e complexas, nas quais o cálculo manual se torna trabalhoso. Se um estilo que você aplicou parece não estar “pegando” quando você recarrega a página, é bastante provável que exista um conflito de especificidade em jogo — considere comentar temporariamente uma das regras conflitantes para isolar o problema.

18.3 Herança em CSS

O terceiro conceito fundamental deste capítulo é a **herança**. Algumas propriedades CSS, quando definidas em um elemento pai, são automaticamente repassadas para todos os seus elementos filhos — a menos que um valor diferente seja explicitamente definido em algum filho específico.

HTML

```
<body>
  <p>Primeiro parágrafo.</p>
  <p>Segundo parágrafo com <span>uma palavra especial</span>.</p>
</body>
```

CSS

```
body {
  color: blue;
}
```

Como **color** é uma propriedade **herdada**, ao definir a cor azul no elemento **body** (o pai de todo o conteúdo visível da página), tanto os dois parágrafos quanto o **span** dentro do segundo parágrafo também ficarão azuis, por herança — mesmo sem que nenhuma regra tenha sido escrita diretamente para **p** ou **span**.

É possível, é claro, sobrescrever esse valor herdado em um elemento filho específico:

CSS

```
body {
  color: blue;
}

span {
  color: black;
}
```

Agora, apenas o conteúdo dentro do **span** ficará preto, enquanto o restante do texto permanece azul, herdado do **body**.

18.3.1 Nem Toda Propriedade é Herdada

É fundamental entender que **nem todas** as propriedades CSS seguem esse comportamento de herança. Um exemplo clássico é a propriedade **width**: se você definir **width: 30%** em um elemento pai, seus elementos filhos **não** herdarão automaticamente essa largura de 30% relativa ao pai. Da mesma forma, as propriedades do modelo de caixa — **margin**, **padding** e **border** — também não são herdadas.

Dica

Para descobrir se uma determinada propriedade CSS é herdada por padrão, consulte sua página de referência na MDN (pesquisando "nome-da-propriedade CSS

MDN”no Google). A documentação sempre indica explicitamente, em uma seção de definição formal, se aquela propriedade é herdada ou não.

18.3.2 Valores Universais para Controlar a Herança

O CSS oferece quatro valores especiais, aceitos por **qualquer** propriedade, que permitem controlar explicitamente o comportamento de herança: `inherit`, `initial`, `revert` e `unset`.

CSS

```
/* inherit: força a propriedade a herdar o valor do elemento pai,
   mesmo que ela normalmente não seja uma propriedade herdada */
.forçar-heranca a {
  color: inherit;
}

/* initial: usa o valor inicial padrão da especificação CSS
   para aquela propriedade (por exemplo, preto para "color") */
.resetar-cor a {
  color: initial;
}

/* unset: comporta-se como "inherit" se a propriedade for
   normalmente herdada, ou como "initial" caso contrário */
.comportamento-natural a {
  color: unset;
}
```

Dica

Na prática, `unset` costuma ser o valor mais útil e mais utilizado no dia a dia, pois ele “faz a coisa certa” automaticamente: se a propriedade normalmente herda do pai, ele herda; caso contrário, ele reverte ao valor inicial da especificação. O valor `revert`, mais recente, é semelhante a `unset`, mas retorna ao estilo padrão do próprio navegador (em vez do valor inicial da especificação) quando a propriedade não é herdada.

18.4 A Propriedade Abreviada `all`

Existe ainda uma propriedade CSS especial chamada `all`, que permite desfazer, de uma única vez, **todas** as propriedades aplicadas a um elemento por uma regra específica, retornando-as a um dos valores universais discutidos acima.

CSS

```
blockquote {
  background-color: red;
  border: 2px solid green;
```



```
}  
  
.blockquote-limpo {  
  all: unset;  
}
```

Nesse exemplo, um elemento `blockquote` que também possua a classe `blockquote-limpo` terá **todas** as suas propriedades (incluindo `background-color` e `border`, definidas pela primeira regra) desfeitas de uma só vez, como se a regra do seletor de elemento nunca tivesse sido aplicada.

18.5 A Importância: o Modificador `!important`

Além da posição na folha de estilo e da especificidade, existe um terceiro fator que pode alterar completamente o resultado de um conflito: a **importância**, especificada com o modificador `!important`.

```
CSS  
  
#vencedor {  
  background-color: red;  
  border: 1px solid black;  
}  
  
.forçar {  
  background-color: red !important;  
  border: none !important;  
}
```

O modificador `!important`, adicionado logo após o valor de uma propriedade, faz com que essa declaração específica sobrescreva **qualquer** outra regra conflitante, mesmo que a regra concorrente possua especificidade maior (como um seletor de ID) e mesmo que venha depois na folha de estilo.

Dica

Recomenda-se fortemente **nunca** usar `!important`, a menos que você tenha absoluta certeza de que é realmente necessário. Esse modificador quebra o funcionamento normal da cascata, tornando problemas de depuração muito mais difíceis de resolver — especialmente em folhas de estilo grandes, mantidas por várias pessoas. A única forma de sobrescrever uma declaração marcada com `!important` é usando outra declaração também marcada com `!important`, com especificidade igual ou maior, definida posteriormente na cascata — o que rapidamente se torna insustentável. Uma situação legítima para seu uso é quando você trabalha com um sistema de gestão de conteúdo (CMS) e não tem acesso para editar a folha de estilo principal, precisando sobrescrever um estilo que não pode ser alterado de nenhuma outra forma.

18.6 Exemplo Resolvido: Removendo uma Cor de Fundo sem Especificar uma Cor

Para consolidar os conceitos deste capítulo, vamos resolver um exercício prático que ilustra bem o uso combinado de cascata, especificidade e dos valores universais de herança.

Enunciado: considere o seguinte documento HTML, contendo dois links, e a seguinte folha de estilo, que aplica uma cor de fundo azul a esses links:

HTML

```
<ul>
  <li><a href="#" id="link-um">Link 1</a></li>
  <li><a href="#" id="link-dois">Link 2</a></li>
</ul>
```

CSS — estado inicial

```
#link-um,
#link-dois {
  background-color: blue;
}
```

O objetivo do exercício é criar uma nova regra CSS capaz de **eliminar** a cor de fundo azul desses links — porém, com uma restrição importante: a solução **não pode especificar explicitamente nenhuma cor** (não é permitido, por exemplo, simplesmente escrever `background-color: transparent` ou `background-color: white`).

Solução: a resposta está exatamente nos valores universais de propriedade estudados neste capítulo. Como `background-color` é uma propriedade que **não é herdada**, podemos usar tanto `initial` quanto `unset` para resetá-la ao seu comportamento padrão (que é, precisamente, a ausência de cor de fundo — tecnicamente, um valor transparente):

CSS — solução

```
#link-um,
#link-dois {
  background-color: blue;
}

a {
  background-color: unset;
}
```

Repare que essa regra final usa apenas um seletor de elemento (`a`), com especificidade 0-0-1 — muito menor do que a especificidade 0-1-0 dos dois seletores de ID definidos anteriormente. Ainda assim, o resultado funciona como esperado: isso ocorre porque `unset` não está competindo pela especificidade da propriedade em si, mas sim resetando o valor daquele elemento específico para seu comportamento natural, aproveitando o fato de que `background-color` não é herdada.

Dica

Também seria possível resolver esse mesmo exercício utilizando `initial` no lugar de `unset`, com o mesmo resultado prático nesse caso específico — já que, como `background-color` não é uma propriedade herdada, `unset` se comporta exatamente como `initial`. Na prática, `unset` costuma ser preferido por ser um valor universal mais recente e mais previsível quando aplicado a diferentes tipos de propriedades ao longo de um projeto.

Síntese do Capítulo

- A cascata determina que, entre regras de mesma especificidade, a que aparece por último na folha de estilo prevalece.
- A especificidade é calculada em quatro níveis (do menos ao mais específico): seletores de elemento, seletores de classe/pseudo-classe, seletores de ID e estilo em linha; níveis mais altos sempre vencem, independentemente da quantidade de seletores do nível inferior.
- Herança é o mecanismo pelo qual algumas propriedades (como `color` e `font-family`) são repassadas automaticamente de um elemento pai para seus filhos; outras (como `width`, `margin`, `padding` e `border`) não seguem essa regra.
- Os valores universais `inherit`, `initial`, `revert` e `unset` permitem controlar explicitamente o comportamento de herança de qualquer propriedade CSS.
- A propriedade abreviada `all` permite resetar de uma só vez todas as propriedades aplicadas a um elemento.
- O modificador `!important` sobrescreve qualquer outra regra conflitante, mas deve ser evitado ao máximo, pois compromete a previsibilidade da cascata.
- Propriedades não herdadas podem ser "removidas" de um elemento usando `initial` ou `unset`, sem a necessidade de especificar um novo valor concreto.

Parte IV

JavaScript

Capítulo 19

Introdução ao JavaScript

Até este ponto do livro, o leitor trabalhou com duas linguagens declarativas: o HTML, responsável pela estrutura e pela semântica do conteúdo, e o CSS, responsável pela apresentação visual desse conteúdo. Ambas são fundamentais, mas têm uma limitação em comum: sozinhas, elas produzem páginas essencialmente estáticas. Uma vez que o navegador termina de renderizar o documento, HTML e CSS não têm mecanismos próprios para reagir a cliques, validar formulários em tempo real, atualizar apenas uma parte da tela sem recarregar a página inteira, ou implementar jogos e animações interativas. É exatamente nesse ponto que entra o JavaScript, a terceira e última linguagem fundamental do desenvolvimento front-end, e o assunto que ocupará toda esta parte do livro.

19.1 O que é JavaScript?

Em uma definição de alto nível, JavaScript é uma linguagem de programação que permite implementar comportamento dinâmico e interatividade em aplicações web. Enquanto HTML fornece a estrutura e o CSS cuida da estética, o JavaScript é responsável por capturar eventos — cliques, digitação, envio de formulários, passagem do cursor sobre um elemento, entre tantos outros — e, a partir desses eventos, executar lógica de negócio que pode, por exemplo, atualizar a interface em tempo real, sem que seja necessária uma nova requisição ao servidor.

JavaScript em poucas palavras

JavaScript é uma linguagem de programação interpretada, originalmente criada para rodar dentro de navegadores web, que permite capturar eventos do usuário e manipular dinamicamente o conteúdo, o estilo e a estrutura de uma página, sem depender de uma nova requisição ao servidor a cada mudança.

Vale destacar uma diferença importante em relação a HTML e CSS: essas duas são linguagens declarativas e de marcação/estilo, ao passo que o JavaScript é uma linguagem de programação completa, no sentido usual da Ciência da Computação. Isso significa que o leitor encontrará ali os elementos que já conhece de outras linguagens: variáveis, tipos de dados primitivos e objetos, arrays, laços de repetição, estruturas condicionais, funções e tratamento de eventos. Tudo o que for aprendido sobre lógica de programação em JavaScript é, em grande medida, reaproveitável em qualquer outra linguagem de programação — o que muda são as particularidades de sintaxe e a forma como a linguagem se integra ao navegador.

É comum classificar o JavaScript apenas como "linguagem de script", em oposição a uma "linguagem de programação completa". Essa distinção, hoje, é discutível. Desde

o surgimento do Node.js, há mais de uma década, o JavaScript deixou de ser uma linguagem restrita ao front-end rodando dentro do navegador e passou a poder ser executado também no lado do servidor, sem depender de um navegador. Além disso, a linguagem incorporou ao longo do tempo conceitos de orientação a objetos e de programação funcional, tornando-se uma linguagem de propósito mais geral. Por essas razões, é mais correto pensar em JavaScript como uma linguagem de programação multiparadigma, que pode ser usada tanto no front-end quanto no back-end (o que forma o chamado desenvolvedor *full stack*).

19.1.1 O caminho de aprendizagem: HTML, CSS e depois JavaScript

O caminho mais comum trilhado por quem está aprendendo desenvolvimento front-end é começar por HTML, seguir para CSS e, por fim, chegar ao JavaScript. Essa ordem faz sentido pedagógico: primeiro se aprende a estruturar o conteúdo (HTML), depois a estilizá-lo (CSS) e, só então, a torná-lo interativo (JavaScript). Este livro, mais adiante, também abordará o desenvolvimento back-end, formando o quadro completo de um desenvolvedor full stack.

19.2 Para que serve o JavaScript na prática

Historicamente, um dos grandes problemas da programação para web era a necessidade de recarregar a página inteira a cada pequena atualização de conteúdo. Imagine um portal de notícias que precisa atualizar uma manchete a cada poucos minutos: se toda atualização exigisse reescrever o HTML no servidor e recarregar a página inteira no navegador do usuário, a experiência seria lenta e pouco prática. O JavaScript resolve esse tipo de problema ao permitir capturar atualizações — muitas vezes de forma assíncrona, buscando dados no servidor sem bloquear a interface — e atualizar apenas a parte específica da página onde a mudança deve ocorrer, sem a necessidade de um recarregamento completo (o chamado *reload* da página).

Além da atualização dinâmica de conteúdo, o JavaScript é amplamente utilizado para:

- capturar e responder a interações do usuário (cliques, digitação, rolagem, etc.);
- validar formulários no lado do cliente antes de enviá-los ao servidor;
- criar animações e efeitos visuais dinâmicos;
- desenvolver jogos, inclusive com desenho 2D e 3D usando a API de *canvas*;
- manipular áudio, vídeo e outros recursos de multimídia;
- se comunicar de forma assíncrona com um servidor (por exemplo, via requisições Ajax ou `fetch`), buscando ou enviando dados sem recarregar a página.

19.2.1 Um exemplo mínimo para ganhar intuição

Antes de entrar em detalhes de sintaxe — que serão vistos com calma nos próximos capítulos —, vale observar um exemplo simples apenas para ganhar intuição sobre como o JavaScript se conecta ao HTML. Considere o seguinte parágrafo em HTML:

HTML

```
<p id="jogador">Jogador 1: Cris</p>
```

Com JavaScript, é possível selecionar esse elemento, associar um evento de clique a ele e, quando o evento ocorrer, alterar dinamicamente o seu conteúdo:

JavaScript

```
const paragrafo = document.querySelector("#jogador");

paragrafo.addEventListener("click", function () {
  const nome = prompt("Digite o novo nome:");
  paragrafo.textContent = "Jogador 1: " + nome;
});
```

Repare que, na primeira linha, é criada uma constante que aponta para o parágrafo dentro do DOM (a árvore de elementos que o navegador cria em memória a partir do HTML). Em seguida, é registrado um "ouvinte" de evento (*event listener*) nesse parágrafo: sempre que o usuário clicar nele, uma função é executada. Essa função abre uma caixa de diálogo pedindo um novo nome e, assim que o usuário confirma, atualiza o conteúdo textual do parágrafo usando a propriedade `textContent`. Note que, em nenhum momento, houve uma nova requisição ao servidor: toda a atualização aconteceu inteiramente no navegador, no lado do cliente. Esse é, em essência, o tipo de comportamento dinâmico que o JavaScript viabiliza, e que será estudado em profundidade ao longo desta parte do livro.

19.3 APIs disponibilizadas pelos navegadores

Quando se trabalha com JavaScript associado a um navegador web, além da linguagem em si, o desenvolvedor tem acesso a diversas APIs (*Application Programming Interfaces*) fornecidas pelo próprio navegador. Essas APIs permitem, entre outras coisas, manipular o DOM — pesquisar elementos, criar novos elementos, remover elementos existentes, atualizar conteúdo e estilo. É justamente essa API de manipulação do DOM que permite construir aplicações web front-end cada vez mais ricas e interativas.

Além das APIs padrão dos navegadores, existem também APIs de terceiros, desenvolvidas por empresas ou comunidades, que podem ser incorporadas a um projeto. Exemplos incluem APIs de mapas (como o Google Maps ou o OpenStreetMap) ou de redes sociais (como incorporar publicações do Twitter em um site). Este livro concentra-se nas APIs padrão, disponíveis nativamente nos navegadores, documentadas oficialmente na Mozilla Developer Network (MDN), uma das referências mais completas e confiáveis para consulta sobre JavaScript e APIs web.

Dica

A documentação da Mozilla Developer Network (MDN) é, hoje, a referência mais respeitada da comunidade para consultar detalhes sobre JavaScript, HTML, CSS e as APIs dos navegadores. Sempre que surgir uma dúvida sobre um método, uma propriedade ou uma mensagem de erro, vale a pena buscar diretamente na documentação do MDN antes de recorrer a fontes de terceiros.

19.4 Como incluir JavaScript em uma página

Assim como ocorre com o CSS, existem diferentes formas de associar código JavaScript a um documento HTML: de forma interna, de forma externa e, ainda, de forma *inline* em atributos de elementos HTML.

19.4.1 JavaScript interno

É possível escrever código JavaScript diretamente dentro do documento HTML, utilizando o elemento `script`:

HTML

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Exemplo JavaScript interno</title>
</head>
<body>
  <button id="botao">Clique aqui</button>

  <script>
    function criarParagrafo() {
      const paragrafo = document.createElement("p");
      paragrafo.textContent = "Você clicou no botão!";
      document.body.appendChild(paragrafo);
    }

    const botao = document.querySelector("#botao");
    botao.addEventListener("click", criarParagrafo);
  </script>
</body>
</html>
```

19.4.2 JavaScript externo

A abordagem preferida na prática, e recomendada como boa prática de desenvolvimento, é manter o código JavaScript em um arquivo separado, com extensão `.js`, e referenciá-lo a partir do HTML usando o atributo `src` do elemento `script`:

HTML

```
<body>
  <button id="botao">Clique aqui</button>

  <script src="script.js" defer></script>
</body>
</html>
```

script.js

```
function criarParagrafo() {
  const paragrafo = document.createElement("p");
  paragrafo.textContent = "Você clicou no botão!";
  document.body.appendChild(paragrafo);
}

const botao = document.querySelector("#botao");
botao.addEventListener("click", criarParagrafo);
```

Essa separação torna o código mais modular e mais legível, evitando misturar estrutura (HTML), apresentação (CSS) e comportamento (JavaScript) em um único arquivo — o que é uma boa prática amplamente adotada no mercado.

19.4.3 JavaScript inline

Também é possível associar código JavaScript diretamente a atributos de eventos de um elemento HTML, como `onclick`:

HTML

```
<button onclick="criarParagrafo()">Clique aqui</button>
```

Essa forma, no entanto, não é recomendada. Além de misturar HTML e JavaScript, ela se torna pouco eficiente quando há muitos elementos que precisam do mesmo comportamento: seria necessário repetir o atributo em cada botão. Uma alternativa muito mais elegante é selecionar todos os elementos de uma vez e associar o mesmo tratador de evento a todos eles em um laço:

JavaScript

```
const botoes = document.querySelectorAll("button");

botoes.forEach(function (botao) {
  botao.addEventListener("click", criarParagrafo);
});
```

Dessa forma, não importa se a página tem 2 ou 200 botões: o mesmo trecho de código associa o comportamento a todos eles, sem necessidade de alterar o JavaScript cada vez que um novo botão for adicionado.

19.5 Ordem de carregamento: os atributos `defer` e `async`

Como o JavaScript frequentemente precisa manipular elementos do DOM, a ordem em que os arquivos são carregados importa. Se um script tentar acessar um elemento que ainda não foi criado pelo navegador (porque o HTML correspondente ainda não foi processado), ocorrerá um erro. Existem algumas estratégias para lidar com isso:

- **Colocar o script no final do body:** uma solução mais antiga, que garante que todo o HTML já foi processado antes do script ser executado, mas que pode prejudicar a performance, pois o carregamento do script só começa depois de todo o HTML ter sido baixado.
- **Atributo `defer`:** instrui o navegador a baixar o script em paralelo à renderização da página, mas só executá-lo depois que o documento HTML tiver sido completamente processado. Os scripts com `defer` são executados na ordem em que aparecem no documento.
- **Atributo `async`:** também faz o download do script em paralelo, mas o executa assim que o download terminar, sem esperar o restante do HTML nem garantir qualquer ordem entre múltiplos scripts.
- **Evento `DOMContentLoaded`:** permite registrar um bloco de código que só será executado depois que o DOM tiver sido completamente construído, ainda que outros recursos (como imagens) possam continuar carregando.

JavaScript

```
document.addEventListener("DOMContentLoaded", function () {  
    // este código só executa depois que o DOM estiver pronto  
    const botao = document.querySelector("#botao");  
    botao.addEventListener("click", criarParagrafo);  
});
```

Como regra prática: use `async` quando os scripts carregados forem independentes entre si (sem que um dependa de código definido em outro); use `defer` quando existir dependência entre os scripts e a ordem de execução precisar ser respeitada.

19.6 Características importantes da linguagem

Algumas características do JavaScript merecem destaque neste momento inicial, pois vão acompanhar o leitor ao longo de toda esta parte do livro:

- **Linguagem interpretada:** tradicionalmente, o código JavaScript é interpretado pelo navegador, executado na ordem em que aparece no documento (de cima para baixo), e não compilado previamente como ocorre em outras linguagens. Isso não impede, porém, que motores modernos de JavaScript utilizem compilação *just-in-time* (JIT) internamente para ganhar desempenho durante a execução.

- **Execução síncrona por padrão, com suporte a assincronismo:** por padrão, o JavaScript executa uma instrução após a outra, na ordem em que aparecem. No entanto, é extremamente comum, e cada vez mais central à linguagem, o uso de programação assíncrona, que permite executar operações (como requisições de rede) sem bloquear o restante do código.
- **Sandbox de segurança:** cada aba do navegador executa seu próprio código JavaScript em um ambiente isolado (*sandbox*), por questões de segurança. Isso não impede totalmente a comunicação entre abas, mas tal comunicação exige mecanismos específicos que fogem do escopo deste livro introdutório.
- **Frontend e backend com a mesma linguagem:** com o Node.js, tornou-se possível escrever tanto o código do lado do cliente quanto o código do lado do servidor em JavaScript, o que representa uma vantagem de produtividade para equipes e desenvolvedores individuais.

19.7 Comentários em JavaScript

Assim como em outras linguagens de programação, o JavaScript oferece duas formas de comentário: comentários de uma linha, com `//`, e comentários de múltiplas linhas, delimitados por `/*` e `*/`.

JavaScript

```
// Isso é um comentário de uma linha

/*
  Isso é um comentário
  que ocupa várias linhas
*/

let idade = 25; // comentário ao final da linha
```

Comentários devem agregar valor semântico ao código, explicando decisões, lógicas não triviais ou avisos importantes para quem for dar manutenção no futuro. Comentários óbvios ou redundantes — como comentar que uma variável chamada `idade` “guarda a idade” — apenas poluem o código sem trazer benefício real. A melhor forma de tornar um código autoexplicativo é escolher bons nomes para variáveis, funções e parâmetros, reduzindo a necessidade de comentários excessivos.

Síntese do Capítulo

- JavaScript é a terceira linguagem fundamental do desenvolvimento web, responsável por adicionar comportamento dinâmico e interatividade a HTML e CSS.
- Diferentemente de HTML e CSS, que são declarativas, o JavaScript é uma linguagem de programação completa, com variáveis, tipos, estruturas de controle, funções e orientação a objetos.

- Desde o surgimento do Node.js, o JavaScript deixou de ser exclusivo do navegador e passou a poder rodar também no lado do servidor, formando a base do desenvolvimento full stack.
- O JavaScript pode ser incluído em uma página de três formas: interno (dentro de um elemento `script`), externo (em um arquivo `.js` separado, a forma recomendada) e inline (em atributos como `onclick`, não recomendado).
- Os atributos `defer` e `async`, além do evento `DOMContentLoaded`, controlam a ordem e o momento em que scripts são executados em relação à construção do DOM.
- O navegador expõe APIs padrão para manipulação do DOM, geolocalização, canvas, áudio e vídeo, entre outras, documentadas oficialmente pela Mozilla Developer Network.
- Comentários devem agregar valor ao código, evitando tanto a ausência de documentação necessária quanto o excesso de comentários óbvios.

Capítulo 20

Um Primeiro Mergulho em JavaScript

Depois de entender o que é o JavaScript e para que ele serve, é hora de colocar a mão na massa com um exemplo completo, ainda que simples. O objetivo deste capítulo é dar ao leitor uma visão prática de tudo aquilo que um programador ou programadora JavaScript utiliza no dia a dia — variáveis, funções, eventos, laços e manipulação do DOM — através da construção passo a passo de um pequeno jogo: um jogo de adivinhação de números.

20.1 O projeto: jogo de adivinhação de números

A ideia do jogo é simples de descrever, mas rica o suficiente para ilustrar praticamente todos os conceitos fundamentais da linguagem. Quando a página carrega, o programa sorteia um número aleatório entre 1 e 100. O usuário tem 10 tentativas para descobrir esse número, digitando um valor em um campo de texto e confirmando com um botão. A cada tentativa, o sistema informa se o palpite foi maior ou menor que o número sorteado. Se o usuário acertar dentro das 10 tentativas, uma mensagem de sucesso é exibida; caso contrário, o jogo termina e é exibido um botão para reiniciar.

Requisitos funcionais do jogo

- Sortear um número aleatório entre 1 e 100 no início do jogo.
- Permitir até 10 tentativas de adivinhação.
- A cada tentativa, informar se o valor digitado foi muito alto ou muito baixo.
- Exibir o histórico de tentativas anteriores.
- Ao acertar, exibir mensagem de parabéns e encerrar o jogo.
- Ao esgotar as tentativas sem acertar, exibir mensagem de fim de jogo.
- Oferecer um botão para reiniciar o jogo a qualquer momento após o término.

Esse exemplo é interessante justamente por ser pequeno o bastante para caber neste capítulo, mas completo o bastante para exercitar variáveis, constantes, funções, eventos, laços, manipulação do DOM e um pouco de orientação a objetos. Cada um desses conceitos será aprofundado nos capítulos seguintes; aqui, o objetivo é apenas ganhar familiaridade prática com a "sensação" de programar em JavaScript.

20.2 Estrutura HTML de partida

Antes de escrever qualquer JavaScript, é preciso ter uma estrutura HTML mínima para trabalhar. O esqueleto do jogo é formado por um título, um parágrafo de instruções, um formulário simples (com um campo de texto e um botão) e uma área destinada às mensagens de status:

HTML

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Jogo de Adivinhação</title>
</head>
<body>
  <h1>Jogo de Adivinhação de Números</h1>
  <p>Selecionamos um número aleatório entre 1 e 100.
    Você consegue adivinhar em 10 tentativas ou menos?</p>

  <form>
    <label for="guessField">Digite uma tentativa: </label>
    <input type="text" id="guessField" class="guessField">
    <input type="submit" value="Enviar tentativa" class="guessSubmit">
  </form>

  <div class="resultParas">
    <p class="guesses"></p>
    <p class="lastResult"></p>
    <p class="lowHigh"></p>
  </div>

  <script src="jogo.js" defer></script>
</body>
</html>
```

Note que os elementos que precisarão ser manipulados por JavaScript recebem uma classe (ou poderiam receber um `id`) para que seja possível localizá-los depois a partir do código.

20.3 Declarando as variáveis do jogo

O primeiro passo dentro do arquivo JavaScript é declarar as variáveis que o programa vai utilizar. Uma variável é, essencialmente, um espaço reservado na memória do computador para armazenar um valor.

JavaScript

```
let randomNumber = Math.floor(Math.random() * 100) + 1;
let guessCount = 1;
let resetButton;

const guesses = document.querySelector(".guesses");
const lastResult = document.querySelector(".lastResult");
const lowOrHigh = document.querySelector(".lowHigh");

const guessSubmit = document.querySelector(".guessSubmit");
const guessField = document.querySelector(".guessField");

guessField.focus();
```

Vale destacar o funcionamento de `Math.random()`, um método da classe `Math` que gera um número decimal aleatório entre 0 (inclusive) e 1 (exclusive). Multiplicando esse valor por 100, obtém-se um número entre 0 e 99,999...; aplicando `Math.floor()`, o valor é arredondado para baixo, resultando em um número inteiro entre 0 e 99. Somando 1, finalmente, chega-se a um número inteiro entre 1 e 100, exatamente como desejado.

As constantes `guesses`, `lastResult` e `lowOrHigh` são ponteiros (referências) para os três parágrafos definidos dentro da `div` de resultado. Da mesma forma, `guessSubmit` e `guessField` apontam, respectivamente, para o botão e para o campo de texto do formulário. Uma vez obtidas essas referências, é possível manipular esses elementos livremente a partir do código.

Dica

Repare que `querySelector` recebe como argumento um seletor CSS — os mesmos seletores usados nas folhas de estilo. Um ponto na frente do nome (`.guesses`) indica que se está buscando um elemento pelo valor do atributo `class`; já um sustenido (`#jogador`) indica busca por `id`. Essa é uma das grandes vantagens de já conhecer CSS antes de aprender JavaScript.

20.4 Escrevendo a primeira função

Uma função em JavaScript agrupa um bloco de código que pode ser executado (chamado) quando necessário. Para um primeiro teste, é possível escrever uma função simples que apenas exibe um alerta:

JavaScript

```
function checkGuess() {
    alert("Eu sou um placeholder!");
}
```

A palavra reservada `function` inicia a definição, seguida do nome escolhido para a função, parênteses (que conteriam os parâmetros, se houvesse algum) e um bloco de código delimitado por chaves. A função `alert()` é uma função global do navegador

que exibe uma caixa de diálogo simples com uma mensagem informativa — útil para testes rápidos, embora pouco usada em interfaces reais e profissionais.

Dica

Uma forma prática de testar pequenos trechos de JavaScript, sem precisar editar arquivos, é abrir o console do navegador (parte das ferramentas de desenvolvedor, acessíveis geralmente pela tecla F12 ou pelo menu de inspeção). É possível digitar expressões diretamente ali e ver o resultado na hora — inclusive chamar funções já definidas no script carregado pela página.

20.5 Operadores básicos

Antes de completar a lógica do jogo, vale revisar rapidamente os operadores mais comuns, todos testáveis diretamente no console do navegador:

JavaScript

```
6 + 9;    // 15   (adição)
20 - 15;  // 5    (subtração)
3 * 7;    // 21   (multiplicação)
10 / 5;   // 2    (divisão)

// concatenação de strings com o operador +
let name = "Cris";
"Olá, " + name;    // "Olá, Cris"

// operadores de comparação
5 === 2 + 3;    // true   (estritamente igual: mesmo valor e mesmo tipo)
"2" === 2;      // false  (tipos diferentes: string x number)
"2" !== 2;      // true   (estritamente diferente)
```

Os operadores de comparação `===` (estritamente igual) e `!==` (estritamente diferente) levam em conta tanto o valor quanto o tipo dos operandos, e serão discutidos com mais detalhes no capítulo sobre números e operadores.

20.6 Completando a lógica de verificação

Com as variáveis e o entendimento básico de funções e operadores em mãos, é possível escrever a função que de fato verifica a tentativa do usuário:

JavaScript

```
function checkGuess() {
  const guess = Number(guessField.value);

  if (guessCount === 1) {
    guesses.textContent = "Tentativas anteriores: ";
  }
}
```



```
guesses.textContent += guess + " ";

if (guess === randomNumber) {
  lastResult.textContent = "Parabéns! Você acertou o número!";
  lastResult.style.backgroundColor = "green";
  lowOrHigh.textContent = "";
  setGameOver();
} else if (guessCount === 10) {
  lastResult.textContent = "!!! GAME OVER !!!";
  setGameOver();
} else {
  lastResult.textContent = "Errado!";
  lastResult.style.backgroundColor = "red";

  if (guess < randomNumber) {
    lowOrHigh.textContent = "Tentativa muito baixa!";
  } else if (guess > randomNumber) {
    lowOrHigh.textContent = "Tentativa muito alta!";
  }
}

guessCount++;
guessField.value = "";
guessField.focus();
}
```

Repare em vários detalhes importantes deste trecho. Primeiro, `guessField.value` retorna o valor digitado pelo usuário no campo de texto — sempre como uma *string* — e por isso ele é convertido explicitamente para número com o construtor `Number()`. Segundo, a propriedade `textContent` permite ler e escrever o conteúdo textual de um elemento; já a propriedade `style` dá acesso às propriedades CSS daquele elemento diretamente pelo JavaScript, permitindo, por exemplo, mudar a cor de fundo dinamicamente. Por fim, a lógica condicional compara o palpite do usuário com o número sorteado e decide qual mensagem exibir.

20.7 Capturando o evento de clique

Até aqui, a função `checkGuess` existe, mas nada a executa automaticamente: é preciso registrar um "ouvinte" do evento de clique no botão de envio:

JavaScript

```
guessSubmit.addEventListener("click", checkGuess);
```

O método `addEventListener` recebe dois argumentos: o tipo do evento de interesse (aqui, `"click"`, o clique do mouse) e a função que deve ser chamada quando o evento ocorrer. Note que o nome da função é passado sem parênteses — apenas uma referência à função, e não uma chamada imediata dela.

20.8 Finalizando o jogo

Falta ainda escrever a função que trata o fim do jogo, seja por vitória, seja por esgotamento das tentativas:

JavaScript

```
function setGameOver() {
  guessField.disabled = true;
  guessSubmit.disabled = true;

  resetButton = document.createElement("button");
  resetButton.textContent = "Iniciar novo jogo";
  document.body.appendChild(resetButton);

  resetButton.addEventListener("click", resetGame);
}
```

Esta função desabilita o campo de texto e o botão de envio (para impedir novas tentativas) e cria dinamicamente, via `document.createElement`, um novo botão de reinício, que é inserido no documento com `appendChild`. Em seguida, um evento de clique é associado a esse novo botão, apontando para uma função `resetGame`, responsável por reiniciar o estado do jogo:

JavaScript

```
function resetGame() {
  guessCount = 1;

  const resetParas = document.querySelectorAll(".resultParas p");
  for (let i = 0; i < resetParas.length; i++) {
    resetParas[i].textContent = "";
  }

  resetButton.parentNode.removeChild(resetButton);

  guessField.disabled = false;
  guessSubmit.disabled = false;
  guessField.value = "";
  guessField.focus();

  lastResult.style.backgroundColor = "white";

  randomNumber = Math.floor(Math.random() * 100) + 1;
}
```

Observe o laço `for`, que percorre todos os parágrafos de resultado limpando seu conteúdo, e a remoção do botão de reinício por meio de `parentNode.removeChild()` — já que um elemento não pode remover a si mesmo do DOM, mas seu elemento pai pode.

Síntese do Capítulo

- O jogo de adivinhação de números serve como exemplo prático que reúne variáveis, funções, eventos, laços e manipulação do DOM em um único programa coeso.
- `Math.random()` combinado com `Math.floor()` é a forma padrão de gerar números inteiros aleatórios dentro de um intervalo em JavaScript.
- `querySelector` e `querySelectorAll` localizam elementos do DOM usando a mesma sintaxe de seletores do CSS.
- `textContent` lê e altera o texto de um elemento; a propriedade `style` permite alterar CSS diretamente via JavaScript.
- `addEventListener` associa uma função a um evento de um elemento, sendo a forma recomendada de tratar eventos (em vez de atributos como `onclick`).
- Elementos podem ser criados dinamicamente com `createElement` e inseridos no DOM com `appendChild`, assim como removidos com `removeChild` a partir do elemento pai.
- O console do navegador é uma ferramenta valiosa para testar pequenos trechos de código e funções isoladamente, antes de integrá-los ao programa completo.

Capítulo 21

Depuração de Programas JavaScript

Escrever código é apenas metade do trabalho de um programador; a outra metade, tão importante quanto, é descobrir por que o código não está funcionando como esperado. Este capítulo tem como objetivo fornecer as habilidades necessárias para começar a identificar e corrigir os problemas mais comuns encontrados em programas JavaScript, usando como pano de fundo o próprio jogo de adivinhação de números apresentado no capítulo anterior, agora propositalmente "quebrado" com alguns defeitos.

21.1 Dois tipos fundamentais de erro

De forma geral, ao se trabalhar com qualquer linguagem de programação, os erros podem ser classificados em duas grandes categorias: erros de sintaxe e erros de lógica. Entender a diferença entre eles é o primeiro passo para depurar com eficiência.

Erros de sintaxe

São erros que ocorrem quando o código não está de acordo com a gramática da linguagem — por exemplo, uma palavra reservada digitada incorretamente, um parêntese ou uma chave que não foi fechada, ou uma aspa esquecida. Quando há um erro de sintaxe, o interpretador normalmente recusa-se a executar o programa (ou para de executá-lo no ponto do erro) e emite uma mensagem de erro indicando, ainda que de forma aproximada, o que está errado e em qual linha.

Erros de lógica

São erros nos quais a sintaxe está perfeitamente correta — não há nenhuma mensagem de erro emitida pelo interpretador —, mas o programa produz um resultado diferente do esperado. Esses erros costumam ser mais difíceis de encontrar justamente porque não vêm acompanhados de nenhuma indicação automática de onde está o problema: cabe ao programador raciocinar sobre o comportamento do código e localizar a falha.

21.2 Usando as ferramentas de desenvolvedor do navegador

Todo navegador moderno oferece um conjunto de ferramentas voltadas a desenvolvedores (*Developer Tools*), geralmente acessível pela tecla F12, pelo menu de "Inspecionar elemento" (clique com o botão direito sobre a página) ou por um atalho específico de cada navegador. Dentro dessas ferramentas, a aba de **Console** é o ponto de partida

mais importante para depuração de JavaScript: é ali que aparecem as mensagens de erro emitidas pelo interpretador, além de qualquer saída gerada explicitamente pelo próprio código através de `console.log()`.

Dica

Para abrir o console em praticamente qualquer navegador (Chrome, Firefox, Edge, Safari), a tecla F12 costuma funcionar, assim como o atalho Ctrl+Shift+J (Chrome/Windows) ou Cmd+Option+J (Chrome/Mac). Alternativamente, clique com o botão direito em qualquer parte da página, escolha "Inspecionar" e navegue até a aba "Console". É nesse painel que aparecerão erros de sintaxe, erros em tempo de execução e qualquer mensagem produzida por `console.log()`.

Ao encontrar um erro no console, normalmente é exibido um ícone de alerta (frequentemente vermelho), seguido do tipo do erro, uma breve descrição e uma referência ao arquivo e à linha aproximada onde o problema ocorreu. Um ponto de atenção: quando se utiliza uma ferramenta como o *Live Server* (comum em editores como o Visual Studio Code), o navegador pode injetar código adicional na página para viabilizar o recarregamento automático, o que faz com que o número de linha indicado no console não corresponda exatamente ao número de linha no editor de texto. É preciso ter esse detalhe em mente e, se necessário, contar algumas linhas de diferença.

21.3 Depurando erros de sintaxe comuns

Considere o seguinte trecho, semelhante ao jogo de adivinhação, mas com um pequeno erro de digitação:

JavaScript

```
guessSubmit.addEventListener("click", checkGuess);
```

Ao executar esse código, o console exibiria algo como:

Mensagem de erro

```
Uncaught TypeError: guessSubmit.addEventListener is not a function
```

A mensagem indica que `addEventListener` não é reconhecido como uma função — porque, de fato, o nome correto do método é `addEventListener`, e faltou a letra "t" na palavra "Event". Esse tipo de erro é extremamente comum, sobretudo porque o JavaScript diferencia letras maiúsculas de minúsculas e é sensível a qualquer caractere digitado incorretamente em identificadores e nomes de métodos.

Dica

Sempre que o console indicar que "X não é uma função" ("*X is not a function*"), a primeira suspeita deve recair sobre um erro de digitação no nome do método ou sobre o fato de o objeto em questão realmente não possuir aquele método — talvez porque a variável aponte para outro tipo de objeto do que se imagina.

Outro erro de sintaxe comum ocorre quando se tenta acessar uma propriedade ou método a partir de uma referência que não foi corretamente inicializada — por exemplo, um seletor CSS escrito incorretamente:

JavaScript

```
const lowOrHigh = document.querySelector("lowHigh"); // faltou o ponto!  
lowOrHigh.textContent = "Tentativa muito baixa!";
```

Aqui, o seletor deveria ser `".lowHigh"` (buscando um elemento pela classe), mas, sem o ponto, o navegador interpreta que se está buscando um elemento chamado literalmente `lowHigh` — o que não existe. Como resultado, `querySelector` retorna `null`, e a linha seguinte tenta acessar uma propriedade de `null`, gerando um erro do tipo:

Mensagem de erro

```
Uncaught TypeError: lowOrHigh is null
```

Esse tipo de mensagem — “algo é `null`” ou “não é um objeto” — costuma indicar que uma referência esperada não foi encontrada, seja por um seletor incorreto, seja porque o elemento ainda não existia no DOM no momento em que o script tentou acessá-lo.

Outros erros de sintaxe frequentes incluem: parênteses não fechados após a lista de parâmetros de uma função, chaves não fechadas ao final do corpo de uma função ou bloco condicional, e aspas de abertura de uma string sem a correspondente aspa de fechamento. Todos eles geram mensagens específicas no console, e a documentação da Mozilla Developer Network mantém, inclusive, páginas de referência dedicadas a explicar cada tipo comum de mensagem de erro em detalhes.

21.4 Usando `console.log()` para depurar

Uma das técnicas de depuração mais simples e eficazes é inserir chamadas a `console.log()` em pontos estratégicos do código, para inspecionar o valor de variáveis durante a execução:

JavaScript

```
function checkGuess() {  
  const guess = Number(guessField.value);  
  console.log("valor digitado:", guess);  
  console.log("número sorteado:", randomNumber);  
  
  if (guess === randomNumber) {  
    // ...  
  }  
}
```

Ao executar o jogo com esses logs inseridos, é possível observar no console, a cada tentativa, exatamente quais valores estão sendo comparados — o que ajuda a confirmar (ou refutar) hipóteses sobre onde está o problema.

Dica

`console.log()` é o equivalente, em JavaScript, ao clássico `System.out.println` do Java ou ao `print()` de outras linguagens. É uma ferramenta simples, mas frequentemente suficiente para depurar a maior parte dos problemas do dia a dia — especialmente quando combinada com breakpoints, descritos a seguir, para investigações mais profundas.

Além de `console.log()`, as ferramentas de desenvolvedor permitem também trabalhar com **breakpoints**: pontos marcados no código-fonte (na aba "Sources" ou equivalente do navegador) que pausam a execução do programa exatamente naquela linha, permitindo inspecionar, passo a passo, o valor de cada variável no momento em que a execução foi interrompida. Essa técnica é particularmente útil quando o problema não é óbvio a partir de alguns `console.log()` isolados, pois permite "andar" pela execução do código linha a linha.

21.5 Um erro de lógica clássico: confundir atribuição com comparação

Um dos erros de lógica mais comuns entre iniciantes é confundir o operador de atribuição `=` com os operadores de comparação `==` ou `===`. Considere o trecho a seguir:

JavaScript

```
if (guess = randomNumber) {  
    lastResult.textContent = "Parabéns! Você acertou o número!";  
}
```

Esse código não gera nenhum erro de sintaxe — e é justamente por isso que ele é perigoso. Em vez de *comparar* `guess` com `randomNumber`, a expressão `guess = randomNumber` *atribui* o valor de `randomNumber` a `guess`, e o resultado dessa atribuição (o próprio valor atribuído) é avaliado como verdadeiro dentro da condição do `if`. Na prática, o jogador "ganharia" em toda tentativa, independentemente do número digitado — um comportamento claramente incorreto, mas que não produz nenhuma mensagem de erro no console.

Dica

Para evitar esse tipo de deslize, vale o hábito de sempre usar `===` (ou, no mínimo, `==`) dentro de condições, e nunca um único `=`. Alguns editores de código e ferramentas de análise estática (linters) já alertam automaticamente quando detectam uma atribuição dentro de uma condição, ajudando a prevenir esse erro antes mesmo de rodar o programa.

21.6 Estratégia geral para depurar

Diante de um erro, um roteiro simples de investigação costuma ser eficaz:

- Leia atentamente a mensagem de erro no console: qual é o tipo do erro e em qual linha (aproximada) ele ocorreu?
- Se a linha indicada não parecer fazer sentido, procure algumas linhas acima — muitas vezes o erro de fato se origina antes da linha reportada.
- Insira `console.log()` para inspecionar os valores das variáveis envolvidas naquele trecho.
- Se o problema persistir, use breakpoints para executar o código passo a passo e observar exatamente onde o comportamento diverge do esperado.
- Verifique se não há confusão entre operadores de atribuição e de comparação, um dos erros de lógica mais comuns e mais difíceis de perceber a olho nu.

À medida que a experiência com a linguagem aumenta, a leitura das mensagens de erro se torna cada vez mais rápida e intuitiva, e o programador passa a reconhecer padrões recorrentes de falha quase instantaneamente.

Síntese do Capítulo

- Erros de sintaxe violam a gramática da linguagem e normalmente impedem a execução, gerando uma mensagem de erro explícita no console.
- Erros de lógica não geram mensagens de erro, mas produzem resultados diferentes do esperado, sendo mais difíceis de localizar.
- O console do navegador (acessível via F12 ou pelo menu de inspeção) é a ferramenta central de depuração, exibindo mensagens de erro e saídas de `console.log()`.
- Mensagens como "X is not a function" ou "X is null" geralmente indicam erro de digitação em nomes de métodos ou seletores incorretos.
- Breakpoints permitem pausar a execução do código em uma linha específica e inspecionar variáveis passo a passo.
- Confundir o operador de atribuição (`=`) com os operadores de comparação (`==` ou `===`) é um erro de lógica clássico, silencioso e traiçoeiro.
- Uma boa estratégia de depuração combina leitura cuidadosa das mensagens de erro, `console.log()` estratégico e, quando necessário, execução passo a passo com breakpoints.

Capítulo 22

Variáveis em JavaScript

Tendo já visto variáveis em ação nos exemplos dos capítulos anteriores, é hora de detalhar esse conceito com mais profundidade. Variáveis estão entre os blocos de construção mais básicos de qualquer linguagem de programação, e o JavaScript não é exceção: entender bem como declará-las, como atribuir valores a elas e quais são as diferenças entre as formas de declaração disponíveis é essencial para escrever código correto e livre de armadilhas comuns.

22.1 O que é uma variável

Uma variável é uma forma de reservar um espaço na memória do computador para armazenar um valor. Esse espaço recebe um nome (o identificador da variável), que passa a ser usado no código para ler ou alterar o valor armazenado.

Variável

Uma variável é um espaço nomeado na memória, capaz de armazenar um valor (chamado *literal*) que pode, em geral, ser alterado ao longo da execução do programa. O valor guardado em uma variável pode ser de diferentes tipos: texto (*string*), número, booleano, objeto, entre outros.

Para tornar isso concreto, considere um botão HTML simples:

HTML

```
<button id="botao">Preencha o formulário</button>
```

Com JavaScript, é possível capturar uma referência a esse botão, registrar um evento de clique nele e, a cada clique, pedir ao usuário um nome, guardá-lo em uma variável e exibi-lo na tela:

JavaScript

```
const botao = document.querySelector("#botao");

botao.addEventListener("click", function () {
  let nome = prompt("Qual é o seu nome?");
  alert("Olá, " + nome + "! Bom te ver.");
});
```

Note que, a cada clique no botão, o usuário pode digitar um nome diferente, e a variável `nome` passa a conter aquele novo valor — daí o termo “variável”: o conteúdo armazenado pode variar ao longo da execução do programa.

22.2 Declarando variáveis: `var`, `let` e `const`

O JavaScript oferece três palavras reservadas para declarar variáveis: `var`, `let` e `const`. Embora as três sirvam, em essência, para o mesmo propósito, elas se comportam de maneiras bastante diferentes, e entender essas diferenças é fundamental para evitar erros sutis.

22.2.1 `var`: a forma antiga

Historicamente, `var` foi a única forma de declarar variáveis em JavaScript. Hoje, seu uso não é mais recomendado, principalmente por causa de seu comportamento de escopo, que pode gerar código confuso e propenso a erros.

JavaScript

```
var idade = 25;  
console.log(idade); // 25
```

Uma peculiaridade de `var` é um mecanismo chamado *hoisting* (algo como " içamento" ou "elevação"), que promove a **declaração** da variável para o início do escopo em que ela foi definida, embora a atribuição do valor continue ocorrendo apenas na linha original. Isso significa que o código a seguir não gera erro, mesmo referenciando a variável antes da linha onde ela aparentemente é declarada:

JavaScript

```
console.log(minhaVariavel); // undefined (não dá erro!)  
var minhaVariavel = "Olá";
```

Esse comportamento pode causar confusão, já que o valor exibido é `undefined` em vez de um erro claro informando que a variável ainda não foi definida. Outra característica pouco desejável de `var` é que ela permite redeclarar a mesma variável múltiplas vezes no mesmo escopo sem gerar erro algum — o que não faz sentido do ponto de vista de organização do código e pode facilmente introduzir bugs:

JavaScript

```
var cidade = "Alfenas";  
var cidade = "Poços de Caldas"; // nenhum erro, apenas sobrescreve  
console.log(cidade); // "Poços de Caldas"
```

22.2.2 `let`: a forma recomendada para valores que mudam

Introduzida no ECMAScript 2015 (também chamado ES6), a palavra reservada `let` corrige os principais problemas de `var`. Diferente de `var`, uma variável declarada com `let` não pode ser redeclarada no mesmo escopo:

JavaScript

```
let cidade = "Alfenas";  
let cidade = "Poços de Caldas"; // Erro! Identifier 'cidade' has already been declared
```

O valor de uma variável declarada com `let` pode, no entanto, ser alterado normalmente por meio de uma nova atribuição (sem repetir a palavra `let`):

JavaScript

```
let cidade = "Alfenas";  
cidade = "Poços de Caldas"; // permitido, é apenas uma nova atribuição  
console.log(cidade); // "Poços de Caldas"
```

22.2.3 `const`: para valores que não mudam

A palavra reservada `const` (também introduzida no ES2015) é usada para declarar constantes: valores que, uma vez atribuídos, não podem ser reatribuídos.

JavaScript

```
const diasDaSemana = 7;  
diasDaSemana = 8; // Erro! Assignment to constant variable.
```

O uso de `const` sempre que o valor não precisar mudar é uma boa prática amplamente recomendada, pois deixa explícita a intenção do programador e evita efeitos colaterais indesejados no código — alguém não poderá, por engano, alterar aquele valor mais adiante. É especialmente comum usar `const` para guardar referências a elementos do DOM, já que a referência em si normalmente não muda, mesmo que o conteúdo do elemento apontado seja atualizado:

JavaScript

```
const paragrafo = document.querySelector("#mensagem");  
paragrafo.textContent = "Isto pode mudar"; // permitido: o conteúdo mudou  
// paragrafo = document.querySelector("#outro"); // Erro: a referência em si é constante
```

Dica

Uma boa regra prática: comece sempre declarando suas variáveis com `const`. Se, ao longo do código, você perceber que realmente precisa reatribuir aquele valor, troque para `let`. Evite `var` em código novo — ele existe hoje principalmente por razões de compatibilidade com códigos antigos.

22.3 Escopo: a diferença mais importante entre `var` e `let/const`

Um dos motivos mais importantes para preferir `let` e `const` a `var` é a diferença de escopo entre elas. Variáveis declaradas com `var` têm escopo de **função**, enquanto variáveis declaradas com `let` e `const` têm escopo de **bloco** (ou seja, respeitam os limites de qualquer par de chaves `{ }`, como um `if`, um `for` ou um `while`).

JavaScript

```
function testarEscopoVar() {
  if (true) {
    var mensagem = "Definida dentro do if";
  }
  console.log(mensagem); // "Definida dentro do if" -- var "vaza" do bloco
}

function testarEscopoLet() {
  if (true) {
    let mensagem = "Definida dentro do if";
  }
  console.log(mensagem); // Erro! mensagem não está definida aqui fora
}
```

No primeiro exemplo, embora `mensagem` tenha sido declarada dentro do bloco do `if`, ela continua acessível fora dele, pois `var` ignora os limites de blocos internos e respeita apenas os limites da função. Já no segundo exemplo, `mensagem` declarada com `let` só existe dentro do bloco do `if` em que foi criada; fora dali, tentar acessá-la gera um erro de referência.

Esse comportamento de `var` é uma fonte clássica de bugs, especialmente em laços de repetição. Considere o exemplo a seguir, um erro muito comum entre iniciantes:

JavaScript

```
for (var i = 0; i < 3; i++) {
  setTimeout(function () {
    console.log(i);
  }, 100);
}
// imprime: 3, 3, 3 (e não 0, 1, 2, como se poderia esperar)
```

Como `var` não cria um novo escopo a cada iteração do laço, todas as funções registradas em `setTimeout` compartilham a mesma variável `i`, e por isso todas exibem seu valor final. Trocando `var` por `let`, cada iteração do laço passa a ter sua própria cópia da variável `i`, respeitando a intuição natural do programador:

JavaScript

```
for (let i = 0; i < 3; i++) {
  setTimeout(function () {
    console.log(i);
  }, 100);
}
// imprime: 0, 1, 2
```

22.4 Tipagem dinâmica

Diferentemente de linguagens fortemente tipadas, como Java, em que é preciso declarar explicitamente o tipo de uma variável (por exemplo, `String nome;`), o JavaScript é

uma linguagem de **tipagem dinâmica**. Isso significa que o tipo de uma variável não é declarado explicitamente pelo programador, mas sim inferido automaticamente pelo ambiente de execução, de acordo com o valor atribuído a ela.

JavaScript

```
let valor = "Alfenas";  
console.log(typeof valor); // "string"  
  
valor = 42;  
console.log(typeof valor); // "number" -- a mesma variável mudou de tipo!
```

O operador `typeof` permite consultar, em tempo de execução, qual é o tipo atualmente armazenado em uma variável. Essa flexibilidade tem vantagens (menos código repetitivo, maior agilidade para prototipar) e desvantagens (menos verificações em tempo de compilação, o que pode levar a erros que só aparecem quando o programa já está em execução). Para quem prefere um meio-termo entre a flexibilidade do JavaScript e a segurança de tipos de linguagens fortemente tipadas, existe o TypeScript, um superconjunto de JavaScript que adiciona tipos estáticos opcionais e é convertido (*transpilado*) para JavaScript puro antes de ser executado — um assunto que foge do escopo deste livro introdutório, mas que vale a pena conhecer no futuro.

22.5 Convenções de nomenclatura para variáveis

O JavaScript segue regras específicas — e algumas convenções amplamente adotadas pela comunidade — para nomear variáveis:

- nomes podem conter letras, dígitos, o sinal de underline (`_`) e o cifrão (`$`);
- nomes não podem começar com um dígito;
- nomes diferenciam maiúsculas de minúsculas (`idade` e `Idade` são variáveis distintas);
- não é permitido usar palavras reservadas da linguagem como nome de variável (por exemplo, `let`, `var`, `function`, `for`).

Por convenção — e não por exigência da linguagem —, o JavaScript adota o estilo *camelCase* para nomear variáveis: a primeira palavra em letras minúsculas e cada palavra subsequente iniciando com letra maiúscula, sem espaços ou underlines entre elas.

JavaScript

```
let nomeCompleto = "Cris Fernandes";  
let quantidadeDeTentativas = 10;  
let estaLogado = false;
```

Além de seguir as convenções de estilo, escolher nomes de variáveis descritivos é uma das práticas mais valiosas para a legibilidade do código. Uma variável chamada `x` ou `dado1` conta muito pouco sobre seu propósito; uma variável chamada

`quantidadeDeTentativas` comunica sua intenção de forma imediata, tornando comentários explicativos frequentemente desnecessários.

Síntese do Capítulo

- Uma variável é um espaço nomeado na memória usado para armazenar um valor que pode mudar ao longo da execução do programa.
- `var` é a forma antiga de declarar variáveis, com escopo de função e comportamento de *hoisting*; seu uso não é mais recomendado em código novo.
- `let` é a forma recomendada para variáveis cujo valor precisa mudar, com escopo de bloco e sem permitir redeclaração no mesmo escopo.
- `const` é usada para valores que não devem ser reatribuídos após a declaração, sendo a opção preferencial sempre que possível.
- A diferença de escopo entre `var` (função) e `let/const` (bloco) é a principal razão para preferir as formas modernas de declaração.
- O JavaScript é uma linguagem de tipagem dinâmica: o tipo de uma variável é inferido a partir do valor atribuído, podendo mudar ao longo da execução.
- Boas convenções de nomenclatura, como o uso de *camelCase* e nomes descritivos, tornam o código mais legível e reduzem a necessidade de comentários.

Capítulo 23

Números e Operadores em JavaScript

Este capítulo tem como objetivo aprofundar o conhecimento sobre dois assuntos intimamente relacionados: como o JavaScript representa números e como a linguagem disponibiliza operadores para manipulá-los, compará-los e combiná-los com outros tipos de dados, como strings.

23.1 O tipo Number

Diferentemente de linguagens como Java, que possuem diversos tipos numéricos (`byte`, `short`, `int`, `long`, `float`, `double`), o JavaScript utiliza um único tipo, chamado `Number`, tanto para números inteiros quanto para números decimais (de ponto flutuante).

JavaScript

```
let meuInteiro = 5;
let meuFlutuante = 6.667;

console.log(typeof meuInteiro); // "number"
console.log(typeof meuFlutuante); // "number"
```

Como se pode ver, `typeof` retorna `"number"` para os dois casos, evidenciando que `Number` é um tipo "guarda-chuva" que engloba qualquer valor numérico em JavaScript, independentemente de ser inteiro ou decimal.

BigInt

Para valores inteiros extremamente grandes, que ultrapassam a precisão segura do tipo `Number` (aproximadamente 2^{53}), o JavaScript oferece um segundo tipo numérico chamado `BigInt`. Um valor `BigInt` é criado adicionando a letra `n` ao final do literal numérico, ou passando o valor para o construtor `BigInt()`:

```
const numeroEnorme = 9007199254740993n;
const outroEnorme = BigInt(9007199254740993);
```

Esse tipo é utilizado apenas em situações específicas que exigem precisão além do padrão, sendo pouco comum no dia a dia de quem está começando.

23.1.1 O objeto Number e seus métodos

Como praticamente tudo em JavaScript, um valor numérico pode ser tratado como uma instância de um objeto, o que permite chamar métodos e acessar propriedades

sobre ele usando o operador de ponto. Um exemplo útil é o método `toFixed()`, que arredonda um número para uma quantidade específica de casas decimais, retornando o resultado como uma string:

JavaScript

```
let valor = 6.667321;
console.log(valor); // 6.667321

valor = valor.toFixed(2);
console.log(valor); // "6.67"
```

Outro uso comum do objeto `Number` é a conversão de uma string numérica em um valor numérico de fato. Isso é especialmente relevante ao capturar dados de formulários HTML: mesmo quando o usuário digita um número em um campo do tipo `text` ou `number`, o valor retornado pela propriedade `value` do campo é sempre uma *string*.

JavaScript

```
const campoTexto = document.querySelector("#idade");
const valorDigitado = campoTexto.value;           // sempre uma string, ex: "25"
const idade = Number(valorDigitado);              // convertido para número: 25

console.log(typeof valorDigitado); // "string"
console.log(typeof idade);         // "number"
```

23.2 A armadilha da concatenação com strings

Um dos pontos que mais confundem quem está começando é a interação entre números e strings através do operador `+`. Em JavaScript, quando pelo menos um dos operandos do operador `+` é uma string, o outro operando é convertido para string e ocorre uma **concatenação**, e não uma soma:

JavaScript

```
let myNumber = "74"; // string, não número!
myNumber = myNumber + 3;

console.log(myNumber);           // "743" (concatenação, não soma!)
console.log(typeof myNumber);    // "string"
```

Isso ocorre porque `"74"` é uma string, e o operador `+` concatena essa string com a representação textual de `3`, resultando em `"743"`, e não em `77` (que seria o resultado de uma soma numérica).

Dica

Um lembrete importante aqui: *strings* em JavaScript são **imutáveis**. Isso significa que uma string, uma vez criada, nunca é alterada “no lugar”—qualquer operação que pareça modificá-la (como concatenação) na verdade cria e retorna uma nova

string. Se o resultado dessa operação não for capturado em uma variável (ou reatribuído à mesma variável), o valor original permanece inalterado.

Esse detalhe fica mais claro no exemplo a seguir:

JavaScript

```
let myNumber = "74";
myNumber + 3;           // resultado "743" é calculado, mas descartado

console.log(myNumber);  // ainda "74" -- nada mudou!

const newNumber = myNumber + 3; // agora capturamos o resultado
console.log(newNumber);      // "743"
console.log(myNumber);      // "74" -- continua o mesmo
```

E se o objetivo for, de fato, somar um valor a um número que veio como string? É necessário convertê-la explicitamente antes de somar:

JavaScript

```
let myNumber = "74";
myNumber = Number(myNumber) + 3;

console.log(myNumber);      // 77 (soma correta!)
console.log(typeof myNumber); // "number"
```

23.3 Operadores aritméticos

Os operadores aritméticos do JavaScript são, em sua maioria, os mesmos já conhecidos da matemática básica e de outras linguagens de programação:

JavaScript

```
10 + 7;    // 17  (adição)
20 - 15;   // 5   (subtração)
3 * 7;     // 21  (multiplicação)
60 / 3;    // 20  (divisão)
60 % 9;    // 6   (módulo: resto da divisão inteira)
7 ** 3;    // 343 (exponenciação: 7 elevado ao cubo)
```

O operador de módulo (%) retorna o resto de uma divisão entre dois números inteiros — um operador extremamente útil na Ciência da Computação, por exemplo, para verificar se um número é par ($n \% 2 === 0$). Já o operador de exponenciação (**), introduzido no ECMAScript 2016, é equivalente a chamar o método estático `Math.pow()`:

JavaScript

```
7 ** 3;           // 343
Math.pow(7, 3);  // 343 (equivalente)
```

23.3.1 Precedência de operadores

Assim como na matemática, o JavaScript segue regras de precedência entre operadores: multiplicação e divisão têm precedência sobre adição e subtração, e operações de mesma precedência são avaliadas da esquerda para a direita.

JavaScript

```
5 + 5 * 2;  // 15, e não 20 -- a multiplicação ocorre primeiro
(5 + 5) * 2; // 20 -- parênteses alteram a precedência
```

Sempre que houver dúvida sobre a ordem de avaliação, o uso de parênteses é a forma mais clara e segura de deixar explícita a intenção do código, exatamente como se faz na matemática.

23.4 Operadores de incremento e decremento

O JavaScript oferece operadores de incremento (**++**) e decremento (**--**), que somam ou subtraem 1 do valor de uma variável. É importante notar que eles só podem ser aplicados sobre variáveis, e não sobre valores literais diretamente:

JavaScript

```
let numero = 4;
numero++;
console.log(numero); // 5
```

Há uma diferença sutil entre colocar o operador antes ou depois da variável. Quando colocado depois (**numero++**), o valor *atual* da variável é retornado antes de o incremento acontecer; quando colocado antes (**++numero**), o incremento acontece primeiro, e o novo valor já incrementado é retornado:

JavaScript

```
let a = 4;
console.log(a++); // 4 -- retorna o valor antes de incrementar
console.log(a);   // 5 -- agora já incrementado

let b = 4;
console.log(++b); // 5 -- incrementa primeiro, depois retorna
```

23.5 Operadores de atribuição

Além do operador básico de atribuição (**=**), o JavaScript oferece operadores de atribuição "atalho", que combinam uma operação aritmética com a atribuição em

um único passo:

JavaScript

```
let x = 3;
let y = 4;

x = y;      // x agora vale 4

x += 4;     // equivalente a: x = x + 4
x -= 2;     // equivalente a: x = x - 2
x *= 3;     // equivalente a: x = x * 3
x /= 5;     // equivalente a: x = x / 5
```

23.6 Operadores de comparação

Operadores de comparação retornam um valor booleano (**true** ou **false**), representando o resultado de uma comparação entre dois valores. O JavaScript possui, na verdade, duas variantes de igualdade e desigualdade, e a diferença entre elas é um dos pontos mais importantes deste capítulo:

JavaScript

```
5 === 2 + 3;    // true   (estritamente igual: mesmo valor E mesmo tipo)
"2" === 2;      // false  (tipos diferentes: string x number)

5 !== 2 + 3;    // false  (estritamente diferente)
"2" !== 2;      // true   (são diferentes, pois os tipos diferem)

"2" == 2;       // true   (igualdade "solta": converte o tipo antes de comparar)
"2" != 2;       // false  (diferença "solta")
```

Igualdade estrita x igualdade solta

Os operadores `==` e `!=` (com dois caracteres) realizam uma comparação "solta", convertendo os tipos dos operandos antes de compará-los quando eles são diferentes. Já os operadores `===` e `!==` (com três caracteres) realizam uma comparação estrita, que só é verdadeira quando o valor e o tipo dos dois operandos coincidem. A recomendação amplamente adotada pela comunidade é sempre preferir `===` e `!==`, pois eles resultam em código mais previsível e menos sujeito a erros sutis decorrentes de conversões automáticas de tipo.

Outros operadores de comparação seguem a intuição matemática usual: `>` (maior que), `<` (menor que), `>=` (maior ou igual) e `<=` (menor ou igual).

JavaScript

```
10 > 5;    // true
10 < 5;    // false
10 >= 10;  // true
```

```
9 <= 10; // true
```

23.7 Um exemplo prático: alternando o estado de um botão

Para fechar o capítulo com um exemplo aplicado, considere uma pequena "máquina de estados" controlada por um botão: ao clicar, o rótulo do botão e a mensagem exibida alternam entre dois estados, usando o operador de igualdade estrita para verificar o estado atual:

HTML

```
<button id="btn">Iniciar máquina</button>
<p id="txt">A máquina está parada.</p>
```

JavaScript

```
const btn = document.querySelector("#btn");
const txt = document.querySelector("#txt");

btn.addEventListener("click", updateBtn);

function updateBtn() {
  if (btn.textContent === "Iniciar máquina") {
    btn.textContent = "Parar máquina";
    txt.textContent = "A máquina foi iniciada.";
  } else {
    btn.textContent = "Iniciar máquina";
    txt.textContent = "A máquina está parada.";
  }
}
```

Note como o operador `===` é usado para testar, de forma estrita, se o rótulo atual do botão corresponde a um determinado texto, e como esse teste direciona qual ramo do `if/else` será executado.

Síntese do Capítulo

- O JavaScript utiliza um único tipo, `Number`, para representar tanto números inteiros quanto decimais; `BigInt` existe apenas para valores inteiros excepcionalmente grandes.
- O operador `+` realiza concatenação (e não soma) sempre que pelo menos um dos operandos é uma string.
- Strings são imutáveis: operações sobre elas sempre retornam um novo valor, que precisa ser capturado em uma variável para não ser perdido.

- Os operadores aritméticos básicos (+, -, *, /, %, **) seguem as mesmas regras de precedência da matemática tradicional.
- Operadores de atribuição "atalho" (+=, -=, *=, /=) combinam uma operação aritmética com a atribuição.
- Os operadores === e !== (comparação estrita) são preferíveis a == e != (comparação solta), pois evitam conversões implícitas de tipo que podem gerar comportamentos inesperados.
- Dados capturados de campos de formulário chegam sempre como string, sendo necessário convertê-los explicitamente com `Number()` antes de realizar operações aritméticas.

Capítulo 24

Estruturas de Controle, Funções e Arrays em JavaScript

Os capítulos anteriores apresentaram variáveis, números e operadores — os blocos de construção mais elementares de qualquer programa. Este capítulo dá um passo além e apresenta as ferramentas que permitem controlar o *fluxo* de execução de um programa: estruturas condicionais, estruturas de repetição, funções e arrays. Esses são, sem exagero, os elementos mais usados no dia a dia de qualquer pessoa que programa em JavaScript, e dominá-los bem é o que torna possível escrever os primeiros programas verdadeiramente úteis.

24.1 Estruturas condicionais

Estruturas condicionais permitem que um programa tome decisões, executando diferentes blocos de código dependendo se uma determinada condição é verdadeira ou falsa.

24.1.1 `if`, `else if` e `else`

A estrutura condicional mais fundamental é o `if`, que executa um bloco de código apenas quando a condição entre parênteses é avaliada como verdadeira (`true`):

JavaScript

```
let idade = 20;

if (idade >= 18) {
  console.log("Maior de idade.");
}
```

É possível encadear alternativas com `else if`, e definir um bloco padrão, executado quando nenhuma das condições anteriores for verdadeira, com `else`:

JavaScript

```
let nota = 7.5;

if (nota >= 9) {
  console.log("Conceito A");
} else if (nota >= 7) {
  console.log("Conceito B");
} else if (nota >= 5) {
  console.log("Conceito C");
}
```

```
} else {  
  console.log("Reprovado");  
}  
// imprime: "Conceito B"
```

As condições podem ser combinadas usando operadores lógicos: `&&` (E lógico, ambas as condições precisam ser verdadeiras), `||` (OU lógico, pelo menos uma condição precisa ser verdadeira) e `!` (negação, inverte o valor lógico de uma expressão).

JavaScript

```
let idade = 25;  
let temCarteira = true;  
  
if (idade >= 18 && temCarteira) {  
  console.log("Pode dirigir.");  
}  
  
let diaDeChuva = false;  
let diaDeSemana = true;  
  
if (diaDeChuva || !diaDeSemana) {  
  console.log("Melhor ficar em casa.");  
} else {  
  console.log("Bom dia para sair.");  
}
```

24.1.2 switch

Quando é preciso comparar uma mesma variável com vários valores possíveis, o `switch` costuma tornar o código mais legível do que uma longa cadeia de `else if`:

JavaScript

```
let diaDaSemana = 3;  
let nomeDoDia;  
  
switch (diaDaSemana) {  
  case 1:  
    nomeDoDia = "Domingo";  
    break;  
  case 2:  
    nomeDoDia = "Segunda-feira";  
    break;  
  case 3:  
    nomeDoDia = "Terça-feira";  
    break;  
  default:  
    nomeDoDia = "Dia inválido";  
}
```

```
console.log(nomeDoDia); // "Terça-feira"
```

Dica

Não esqueça do **break** ao final de cada **case**. Sem ele, a execução "cai" para o próximo **case** (comportamento chamado *fall-through*), executando também o bloco seguinte, o que raramente é o comportamento desejado. O bloco **default**, opcional, funciona como o **else** de um **switch**, sendo executado quando nenhum **case** corresponde ao valor testado.

24.2 Estruturas de repetição

Estruturas de repetição (ou laços) permitem executar um mesmo bloco de código múltiplas vezes, evitando repetição manual de instruções.

24.2.1 O laço for

O laço **for** clássico é composto por três partes, separadas por ponto e vírgula: a inicialização de uma variável de controle, a condição que mantém o laço em execução, e a atualização da variável de controle a cada iteração.

JavaScript

```
for (let i = 0; i < 5; i++) {  
  console.log("Iteração número " + i);  
}  
// imprime: 0, 1, 2, 3, 4
```

24.2.2 O laço while

O laço **while** executa um bloco de código enquanto uma condição permanecer verdadeira, sendo especialmente útil quando não se sabe, de antemão, quantas iterações serão necessárias:

JavaScript

```
let tentativas = 0;  
  
while (tentativas < 3) {  
  console.log("Tentativa " + tentativas);  
  tentativas++;  
}
```

Uma variação é o **do...while**, que garante que o bloco de código seja executado ao menos uma vez, mesmo que a condição já seja falsa desde o início, pois a verificação ocorre apenas ao final da primeira execução:

JavaScript

```
let numero = 10;

do {
  console.log("Executou ao menos uma vez: " + numero);
} while (numero < 5);
```

24.2.3 for...of e for...in

Além do `for` tradicional, o JavaScript oferece duas variações voltadas à iteração sobre coleções de dados. O `for...of` percorre os **valores** de uma coleção iterável, como um array ou uma string:

JavaScript

```
const frutas = ["maçã", "banana", "laranja"];

for (const fruta of frutas) {
  console.log(fruta);
}
// imprime: "maçã", "banana", "laranja"
```

Já o `for...in` percorre as **chaves** (ou índices) de um objeto:

JavaScript

```
const aluno = { nome: "Rodrigo", curso: "Ciência da Computação", ano: 2026 };

for (const chave in aluno) {
  console.log(chave + ": " + aluno[chave]);
}
// imprime: "nome: Rodrigo", "curso: Ciência da Computação", "ano: 2026"
```

Dica

Como regra prática: use `for...of` para percorrer os elementos de um array (ou de outra estrutura iterável), e reserve o `for...in` para percorrer as propriedades de um objeto. Usar `for...in` em arrays é possível, mas não é recomendado, pois percorre os índices como strings e pode incluir propriedades herdadas inesperadas.

24.3 Funções

Funções permitem agrupar um bloco de código sob um nome, para que ele possa ser reutilizado e chamado (executado) sempre que necessário, evitando repetição de código.

24.3.1 Declaração de função (*function declaration*)

A forma mais tradicional de definir uma função é a declaração de função, usando a palavra reservada `function`:

```
JavaScript
function saudacao(nome) {
  return "Olá, " + nome + "!";
}

console.log(saudacao("Ana")); // "Olá, Ana!"
```

Uma característica interessante das declarações de função é que elas sofrem *hoisting* completo: é possível chamar a função em uma linha anterior à sua definição no código-fonte, pois o JavaScript já "conhece" toda a função antes de começar a executar o programa.

24.3.2 Expressão de função (*function expression*)

Uma função também pode ser atribuída a uma variável, como qualquer outro valor — o que é chamado de expressão de função:

```
JavaScript
const saudacao = function (nome) {
  return "Olá, " + nome + "!";
};

console.log(saudacao("Bruno")); // "Olá, Bruno!"
```

Diferentemente das declarações de função, expressões de função não sofrem *hoisting* da mesma forma — a variável que armazena a função só pode ser chamada depois da linha em que ela foi definida.

24.3.3 Arrow functions

Introduzidas no ECMAScript 2015, as *arrow functions* (funções de seta) oferecem uma sintaxe mais compacta para escrever funções, muito utilizada atualmente, especialmente em callbacks passados para outras funções:

```
JavaScript
const saudacao = (nome) => {
  return "Olá, " + nome + "!";
};

console.log(saudacao("Carla")); // "Olá, Carla!"
```

Quando o corpo da função contém apenas uma expressão de retorno, é possível omitir as chaves e a palavra `return`, tornando a sintaxe ainda mais enxuta:

JavaScript

```
const dobro = (numero) => numero * 2;
console.log(dobro(5)); // 10

// com apenas um parâmetro, os parênteses também são opcionais:
const triplo = numero => numero * 3;
console.log(triplo(5)); // 15
```

Dica

Além da sintaxe mais curta, *arrow functions* têm uma diferença de comportamento importante em relação ao valor de **this** dentro delas (elas não criam seu próprio **this**, herdando-o do contexto onde foram definidas). Esse é um detalhe mais avançado, mas vale saber que ele existe: em código orientado a objetos mais sofisticado, essa diferença pode ser decisiva na escolha entre uma função tradicional e uma *arrow function*.

24.3.4 Parâmetros e retorno

Funções podem receber múltiplos parâmetros, valores padrão para parâmetros não informados, e devolver um resultado usando a palavra reservada **return**. Quando uma função não possui um **return** explícito, ela retorna **undefined**.

JavaScript

```
function calcularMedia(nota1, nota2, peso = 1) {
  return ((nota1 + nota2) / 2) * peso;
}

console.log(calcularMedia(7, 9)); // 8 (peso assume o valor padrão 1)
console.log(calcularMedia(7, 9, 2)); // 16
```

24.4 Arrays

Um array é uma estrutura de dados usada para armazenar uma coleção ordenada de valores, acessíveis por meio de um índice numérico que começa em zero.

JavaScript

```
const numeros = [10, 20, 30, 40];

console.log(numeros[0]); // 10 -- primeiro elemento
console.log(numeros[2]); // 30
console.log(numeros.length); // 4 -- quantidade de elementos
```

24.4.1 Métodos comuns de array

O JavaScript oferece uma quantidade rica de métodos embutidos para manipular arrays. Alguns dos mais usados no dia a dia são:

JavaScript

```
const frutas = ["maçã", "banana"];

frutas.push("laranja");    // adiciona ao final: ["maçã", "banana", "laranja"]
frutas.pop();              // remove o último: ["maçã", "banana"]
frutas.unshift("morango"); // adiciona ao início: ["morango", "maçã", "banana"]
frutas.shift();            // remove o primeiro: ["maçã", "banana"]
```

Três métodos merecem destaque especial por serem extremamente usados em código JavaScript moderno: `forEach`, `map` e `filter`.

JavaScript

```
const numeros = [1, 2, 3, 4, 5];

// forEach: executa uma função para cada elemento (não retorna array novo)
numeros.forEach(function (numero) {
  console.log(numero * 2);
});

// map: cria um NOVO array, transformando cada elemento
const dobrados = numeros.map(numero => numero * 2);
console.log(dobrados); // [2, 4, 6, 8, 10]

// filter: cria um NOVO array, mantendo apenas os elementos que satisfazem a condição
const pares = numeros.filter(numero => numero % 2 === 0);
console.log(pares); // [2, 4]
```

Dica

A diferença essencial entre `forEach` e `map` é o valor de retorno: `forEach` apenas executa uma ação para cada elemento e retorna `undefined`, sendo usado por seu efeito colateral (como imprimir algo no console ou atualizar a tela); já `map` constrói e retorna um novo array com os valores transformados, sem alterar o array original. Já `filter` sempre retorna um novo array, possivelmente menor, contendo apenas os elementos para os quais a função de teste retornou `true`.

24.5 Objetos literais: uma breve introdução

Além de arrays, o JavaScript utiliza extensivamente objetos literais para agrupar dados relacionados sob um único valor, na forma de pares de chave e valor:

JavaScript

```
const aluno = {
  nome: "Nicolas",
  curso: "Ciência da Computação",
  matriculado: true
};
```

```
console.log(aluno.nome);           // "Nicolas"
console.log(aluno["curso"]);       // "Ciência da Computação" (acesso alternativo)

aluno.ano = 2026;                  // adicionando uma nova propriedade
console.log(aluno.ano);            // 2026
```

Objetos podem ser acessados tanto pela notação de ponto (`aluno.nome`) quanto pela notação de colchetes (`aluno["nome"]`), sendo a segunda especialmente útil quando o nome da propriedade é dinâmico (armazenado em uma variável) ou contém caracteres que não são válidos em identificadores. Objetos são a base de praticamente toda estruturação de dados mais complexa em JavaScript, e serão retomados com mais profundidade quando o assunto de manipulação do DOM for aprofundado no próximo capítulo, já que cada elemento HTML manipulado via JavaScript é, na prática, representado internamente como um objeto.

Síntese do Capítulo

- Estruturas condicionais (`if/else if/else` e `switch`) permitem que um programa tome decisões com base em condições booleanas.
- Estruturas de repetição (`for`, `while`, `do...while`) evitam a repetição manual de código, executando um bloco múltiplas vezes.
- `for...of` percorre valores de coleções iteráveis (como arrays), enquanto `for...in` percorre chaves de objetos.
- Funções podem ser definidas como declarações, expressões ou *arrow functions*, cada uma com nuances próprias de *hoisting* e de comportamento.
- Parâmetros com valores padrão e o uso de `return` tornam funções flexíveis e reutilizáveis.
- Arrays armazenam coleções ordenadas de valores, manipuláveis por métodos como `push`, `pop`, `map`, `filter` e `forEach`.
- Objetos literais organizam dados relacionados em pares de chave e valor, servindo de base para representar entidades do mundo real e elementos do DOM.

Capítulo 25

JavaScript e o DOM: Manipulação de Elementos e Eventos

Chegado a este ponto, o leitor já domina os elementos internos da linguagem JavaScript: variáveis, tipos, operadores, estruturas de controle, funções e arrays. Este capítulo final da Parte IV conecta todo esse conhecimento com o que realmente torna o JavaScript especial no contexto da web: a capacidade de manipular o DOM (*Document Object Model*) e de reagir a eventos disparados pelo usuário. É exatamente essa combinação — selecionar elementos, alterá-los dinamicamente e responder a interações — que transforma páginas estáticas em aplicações front-end interativas.

25.1 Relembrando o DOM

Como visto anteriormente no livro, quando um navegador carrega um documento HTML, ele constrói em memória uma representação em forma de árvore desse documento, chamada de DOM. Cada elemento HTML (uma tag `div`, um `p`, um `button`) se torna um nó dessa árvore, com propriedades e métodos que podem ser acessados e manipulados via JavaScript. É essa ponte entre o código JavaScript e a árvore DOM que permite ler, criar, alterar e remover conteúdo da página em tempo real, sem depender de uma nova requisição ao servidor.

25.2 Selecionando elementos

O primeiro passo para manipular qualquer parte de uma página é obter uma referência ao elemento (ou elementos) desejado. O JavaScript oferece diversos métodos para isso, disponíveis no objeto global `document`.

25.2.1 `getElementById`

O método mais antigo e ainda muito utilizado é `getElementById`, que retorna o único elemento cujo atributo `id` corresponde ao valor informado:

HTML

```
<h1 id="titulo">Bem-vindo!</h1>
```

JavaScript

```
const titulo = document.getElementById("titulo");  
console.log(titulo.textContent); // "Bem-vindo!"
```

25.2.2 `querySelector` e `querySelectorAll`

Mais flexíveis, `querySelector` e `querySelectorAll` aceitam qualquer seletor CSS válido, o que torna a busca por elementos consistente com o que já se aprendeu em CSS.

JavaScript

```
// Retorna o PRIMEIRO elemento que corresponde ao seletor
const primeiroParagrafo = document.querySelector("p");
const botaoPrincipal = document.querySelector("#botaoPrincipal");
const primeiroDestaque = document.querySelector(".destaque");

// Retorna TODOS os elementos que correspondem ao seletor (uma NodeList)
const todosOsParagrafos = document.querySelectorAll("p");
const todosOsDestques = document.querySelectorAll(".destaque");
```

Dica

`querySelectorAll` retorna uma `NodeList`, uma coleção semelhante a um array (mas não exatamente um array). É possível percorrê-la com `forEach` diretamente, ou convertê-la para um array de fato com `Array.from()` caso seja necessário usar métodos como `map` ou `filter` sobre ela.

Como regra geral, `querySelector` e `querySelectorAll` são hoje a abordagem preferida na maior parte dos projetos, justamente por reaproveitarem os mesmos seletores já conhecidos do CSS, tornando o código mais consistente entre as duas linguagens.

25.3 Alterando conteúdo

Uma vez com a referência a um elemento em mãos, é possível ler e alterar seu conteúdo de diferentes formas, dependendo do que se deseja fazer.

25.3.1 `textContent`

A propriedade `textContent` lê ou define o conteúdo **textual puro** de um elemento, ignorando qualquer marcação HTML que porventura seja atribuída a ela (o texto é tratado literalmente, e não interpretado como HTML):

JavaScript

```
const mensagem = document.querySelector("#mensagem");

mensagem.textContent = "Operação concluída com sucesso!";
console.log(mensagem.textContent); // "Operação concluída com sucesso!"
```

25.3.2 innerHTML

Já a propriedade `innerHTML` permite ler ou definir o conteúdo de um elemento **interpretando-o como HTML**, o que possibilita inserir tags dentro de um elemento existente:

JavaScript

```
const caixa = document.querySelector("#caixa");

caixa.innerHTML = "<strong>Atenção:</strong> restam poucas vagas.";
```

Dica

Prefira sempre `textContent` quando o objetivo for apenas exibir texto simples, especialmente se esse texto vier de uma fonte externa ou de dados digitados pelo próprio usuário. Usar `innerHTML` com conteúdo não confiável (por exemplo, texto digitado livremente por um usuário e inserido sem tratamento) pode abrir brechas de segurança conhecidas como XSS (*Cross-Site Scripting*), nas quais um usuário mal-intencionado injeta código malicioso na página. Use `innerHTML` apenas quando realmente for necessário inserir marcação HTML, e com dados de origem confiável.

25.4 Alterando estilo

Assim como o conteúdo, o estilo visual de um elemento também pode ser manipulado diretamente via JavaScript.

25.4.1 A propriedade `style`

Todo elemento do DOM possui uma propriedade `style`, que dá acesso direto às propriedades CSS *inline* daquele elemento. Os nomes das propriedades CSS que contêm hífen (como `background-color`) são escritos em *camelCase* em JavaScript (`backgroundColor`):

JavaScript

```
const aviso = document.querySelector("#aviso");

aviso.style.backgroundColor = "red";
aviso.style.color = "white";
aviso.style.padding = "10px";
aviso.style.fontWeight = "bold";
```

25.4.2 `classList`

Embora alterar `style` diretamente funcione, a prática mais recomendada — e mais alinhada com uma boa separação entre estrutura, estilo e comportamento — é definir as regras visuais em uma classe CSS e, via JavaScript, apenas adicionar ou remover

essa classe do elemento. Isso é feito através da propriedade `classList`, que oferece métodos convenientes:

CSS

```
.destaque {  
  background-color: yellow;  
  font-weight: bold;  
}
```

JavaScript

```
const item = document.querySelector("#item1");  
  
item.classList.add("destaque");    // adiciona a classe  
item.classList.remove("destaque"); // remove a classe  
item.classList.toggle("destaque"); // adiciona se não tiver, remove se já tiver  
item.classList.contains("destaque"); // true ou false
```

O método `toggle` é particularmente útil para implementar comportamentos de "ligar/desligar", como mostrar ou esconder um menu, marcar um item como selecionado, ou alternar entre modo claro e escuro em uma interface.

25.5 Criando e removendo elementos

Além de alterar elementos existentes, o JavaScript permite criar novos elementos do zero e inseri-los no DOM, assim como remover elementos que já não são mais necessários.

JavaScript

```
// Criando um novo elemento  
const novoItem = document.createElement("li");  
novoItem.textContent = "Novo item da lista";  
  
// Inserindo esse elemento dentro de uma lista existente  
const lista = document.querySelector("#minhaLista");  
lista.appendChild(novoItem);
```

Para remover um elemento, é preciso acessar seu elemento pai (`parentNode`) e chamar `removeChild` a partir dele, já que, tradicionalmente, um elemento não pode remover a si mesmo diretamente:

JavaScript

```
const item = document.querySelector("#itemAntigo");  
item.parentNode.removeChild(item);  
  
// Alternativa mais moderna, que dispensa acessar o pai:  
item.remove();
```

Dica

O método `remove()`, chamado diretamente sobre o próprio elemento, é uma adição mais recente e mais direta do que o padrão `parentNode.removeChild()`. Ambos funcionam; `remove()` tende a deixar o código mais legível, mas é sempre bom reconhecer o padrão mais antigo, pois ele ainda aparece com frequência em código existente e em exemplos de documentação.

25.6 Reagindo a eventos com `addEventListener`

A peça final que conecta tudo isso é o tratamento de eventos: capturar ações do usuário (cliques, digitação, envio de formulários) e executar código em resposta a elas. O método padrão para isso é `addEventListener`, chamado sobre o elemento que se deseja observar.

JavaScript

```
elemento.addEventListener(tipoDoEvento, funcaoTratadora);
```

25.6.1 O evento `click`

HTML

```
<button id="botaoCurtir">Curtir</button>  
<p id="contador">0 curtidas</p>
```

JavaScript

```
const botao = document.querySelector("#botaoCurtir");  
const contador = document.querySelector("#contador");  
let curtidas = 0;  
  
botao.addEventListener("click", function () {  
  curtidas++;  
  contador.textContent = curtidas + " curtidas";  
});
```

25.6.2 O evento `input`

O evento `input` é disparado toda vez que o valor de um campo de texto é alterado, tecla a tecla — o que o torna ideal para validações em tempo real ou para atualizar a interface conforme o usuário digita:

HTML

```
<input type="text" id="campoNome">  
<p id="preview"></p>
```

JavaScript

```
const campoNome = document.querySelector("#campoNome");
const preview = document.querySelector("#preview");

campoNome.addEventListener("input", function () {
  preview.textContent = "Olá, " + campoNome.value + "!";
});
```

25.6.3 O evento submit

Ao trabalhar com formulários, o evento **submit** é disparado no próprio elemento **form** quando o usuário confirma o envio (seja clicando em um botão do tipo **submit**, seja pressionando Enter). É prática comum chamar **preventDefault()** sobre o objeto do evento para impedir o comportamento padrão do navegador, que seria recarregar a página:

HTML

```
<form id="formularioContato">
  <input type="text" id="nomeUsuario" placeholder="Seu nome">
  <button type="submit">Enviar</button>
</form>
<p id="resultado"></p>
```

JavaScript

```
const formulario = document.querySelector("#formularioContato");
const resultado = document.querySelector("#resultado");

formulario.addEventListener("submit", function (evento) {
  evento.preventDefault(); // impede o recarregamento da página

  const nome = document.querySelector("#nomeUsuario").value;
  resultado.textContent = "Obrigado por entrar em contato, " + nome + "!";
});
```

25.7 O fluxo evento, listener e callback

Vale consolidar, de forma visual, o fluxo conceitual por trás do tratamento de eventos em JavaScript: o usuário interage com a página (por exemplo, clicando em um botão), o navegador gera um evento correspondente, o *listener* registrado com **addEventListener** captura esse evento, e a função de *callback* associada é executada, realizando a lógica de negócio desejada.



Esse fluxo se repete, com pequenas variações, em praticamente toda interação front-end: um evento de `input` dispara a validação de um campo, um evento de `submit` dispara o envio de um formulário, um evento de `click` dispara a exibição de um menu. Entender esse padrão é entender a espinha dorsal de como aplicações web modernas respondem ao usuário.

25.8 Fechando a Parte IV: front-end interativo

Ao longo desta parte do livro, o leitor percorreu o caminho que vai desde a definição de uma variável até a construção de um jogo interativo completo, passando por depuração, tipos numéricos, operadores, estruturas de controle, funções, arrays e, por fim, a manipulação do DOM e o tratamento de eventos. É exatamente na junção dessas três linguagens — HTML fornecendo estrutura e semântica, CSS fornecendo estilo e layout, e JavaScript fornecendo comportamento e interatividade — que nasce o que se costuma chamar de **front-end interativo**: páginas que não apenas exibem informação, mas reagem, se atualizam e conversam com o usuário em tempo real, sem depender de recarregar o navegador a cada pequena mudança. Esse é o alicerce sobre o qual se constroem praticamente todas as aplicações web modernas, dos formulários mais simples aos painéis e sistemas mais sofisticados que o leitor encontrará adiante em sua trajetória como desenvolvedor.

Síntese do Capítulo

- `getElementById`, `querySelector` e `querySelectorAll` são as formas padrão de selecionar elementos do DOM a partir do JavaScript.
- `textContent` altera texto puro; `innerHTML` interpreta o conteúdo como HTML e deve ser usado com cautela por razões de segurança.
- A propriedade `style` altera CSS diretamente no elemento; `classList` (com `add`, `remove` e `toggle`) é a forma recomendada para alternar estilos definidos em folhas de estilo.
- `createElement` e `appendChild` criam e inserem elementos no DOM; `removeChild` (a partir do elemento pai) ou `remove()` os eliminam.
- `addEventListener` é o mecanismo padrão para reagir a eventos como `click`, `input` e `submit`.
- O fluxo evento – listener – callback é o padrão central por trás de toda interatividade front-end construída com JavaScript.
- A combinação de HTML, CSS e JavaScript forma o front-end interativo, o alicerce de praticamente toda aplicação web moderna.

Parte V

Desenvolvimento Back-End com Spring Boot

Capítulo 26

Frontend e Backend: a Arquitetura Cliente-Servidor

Até este ponto do livro, o leitor explorou a construção de páginas com HTML, a estilização com CSS e a interatividade proporcionada pelo JavaScript. Essas três tecnologias formam o que se convencionou chamar de *frontend*: a parte de uma aplicação web com a qual o usuário efetivamente interage. A partir deste capítulo, inicia-se a Parte V do livro, dedicada ao *backend* — a camada da aplicação que processa regras de negócio, acessa bancos de dados e sustenta tudo aquilo que o frontend exhibe. Compreender essa divisão e a forma como as duas partes colaboram é o primeiro passo para se tornar um desenvolvedor web completo.

26.1 Programação web: muito além do que se vê na tela

Quando alguém pensa em programação web, a associação mais imediata costuma ser com aquilo que aparece visualmente em um site: botões, formulários, imagens, animações. Essa é uma percepção natural, mas incompleta. Programação web é, de forma mais ampla, o processo de criar aplicações e sites que podem ser acessados pela internet, independentemente da plataforma utilizada para o acesso — desde que exista uma conexão com a rede, a aplicação pode ser consumida por um computador, um notebook, um smartphone, um relógio inteligente ou qualquer outro dispositivo conectado.

Para construir esse tipo de aplicação, o desenvolvedor lança mão de diversas linguagens de programação, linguagens de marcação e tecnologias variadas, que se dividem essencialmente em dois grandes grupos: aquelas voltadas para o que o usuário vê e manipula diretamente (o frontend) e aquelas que operam nos bastidores, de forma invisível ao usuário final (o backend).

Frontend e Backend

O **frontend** é a camada de uma aplicação web responsável pela estrutura, pelo design e pela interação direta com o usuário. O **backend** é a camada responsável pela lógica de negócio, pela integração com bancos de dados, pela autenticação de usuários e por todo o processamento que ocorre sem que o usuário o perceba diretamente.

26.2 Uma analogia do cotidiano: o restaurante

Antes de tratar do assunto em termos estritamente técnicos, é útil recorrer a uma situação do dia a dia que não envolve tecnologia alguma: a experiência de ir a um restaurante. Imagine o leitor chegando a um restaurante, sentando-se à mesa e recebendo o cardápio — que pode ser físico ou digital, acessado até mesmo por QR Code. Depois de escolher o prato, o cliente chama o garçom, faz o pedido, aguarda, recebe a refeição, come, conversa e, ao final, solicita a conta, paga e vai embora.

Nesse processo inteiro, nenhum site foi utilizado para realizar o pedido de comida. Ainda assim, esse cenário revela com clareza o paralelo entre frontend e backend: o frontend é a parte visível do restaurante, e o backend é a parte invisível — não porque seja irrelevante, mas porque o cliente, enquanto consumidor, normalmente não tem acesso a ela.

No frontend do restaurante estão a área de atendimento (mesas, decoração, iluminação, tudo que compõe a experiência visual e estética do salão), o menu (o cardápio, que deve ser claro e atraente) e os garçons, que funcionam como a interface entre o cliente e a cozinha. É o garçom quem recebe o pedido do cliente, leva-o até a cozinha e, quando o prato está pronto, retorna à mesa para entregá-lo. Repare que, em nenhum momento, o cliente fala diretamente com o chef, e em nenhum momento o chef entrega o prato diretamente ao cliente: existe sempre um intermediário. Esse detalhe será importante mais adiante, quando se discutir o papel das APIs na comunicação entre sistemas.

Já no backend do restaurante está a cozinha, onde os pratos são de fato preparados; o sistema de pedidos, que organiza e prioriza as refeições a serem feitas (por exemplo, distribuindo pedidos entre cozinhas especializadas em diferentes tipos de culinária); a gestão de estoque, que controla os ingredientes disponíveis e sinaliza quando é necessário reabastecer; e o faturamento e pagamento, responsável por processar o valor recebido e permitir que ele seja reinvestido no próprio negócio — seja para repor o estoque, melhorar a estrutura do restaurante ou remunerar os funcionários.

A integração entre essas partes segue uma sequência lógica: o cliente escolhe o prato no menu e faz o pedido ao garçom (frontend); o garçom leva o pedido à cozinha, onde o chef prepara a refeição (transição para o backend); o prato pronto é levado de volta ao cliente pelo garçom (interação entre backend e frontend); e, por fim, o pagamento é processado e uma confirmação é entregue ao cliente.

26.3 Frontend e backend em termos técnicos

Transportando essa analogia para o universo da programação, o frontend é a camada que interage diretamente com o usuário: é tudo aquilo que ele vê e com que ele interage, incluindo a estrutura, o design e a interatividade da aplicação. As tecnologias mais comuns associadas ao frontend são:

- **HTML** (*HyperText Markup Language*): responsável pela estrutura básica da página, por meio de tags como `main`, `header`, `article`, `p` e `div`;
- **CSS** (*Cascading Style Sheets*): responsável pela estilização — cores, fontes, layout, alinhamento e espaçamento dos elementos;

- **JavaScript:** linguagem de programação que confere interatividade à página, permitindo animações, validações de formulário e manipulação dinâmica de conteúdo.

Em um site de comércio eletrônico como os que o leitor certamente já utilizou, o frontend corresponde ao layout das páginas (página inicial, listagem de produtos, página de um produto específico, carrinho de compras, lista de desejos), aos botões de interação (adicionar ao carrinho, alterar quantidade, calcular frete), às animações (como o efeito de *scroll infinito*, comum em catálogos modernos) e aos formulários (cadastro, login, autenticação de dois fatores).

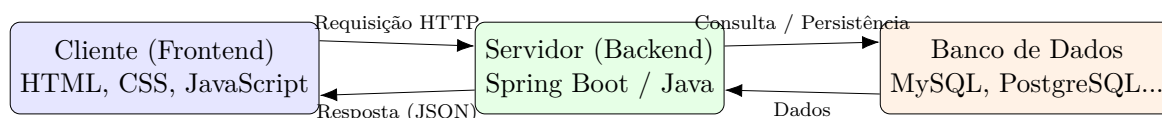
O backend, por sua vez, é a parte da aplicação que não é visível ao usuário. Ele lida com a lógica de negócio, a integração com bancos de dados, a autenticação de usuários e diversas outras responsabilidades críticas para o funcionamento correto do sistema. Um ponto importante — e que será retomado repetidamente ao longo desta parte do livro — é que o backend nunca deve confiar cegamente nos dados que chegam do frontend. Mesmo que o frontend realize validações (por exemplo, verificando se um campo obrigatório foi preenchido), o backend precisa revalidar essas informações, porque nem sempre é possível garantir de onde os dados efetivamente vieram, nem se algum agente malicioso os alterou no caminho.

Entre as tecnologias comuns de backend estão as linguagens de programação (Java, Python, Ruby, PHP, Node.js, Go, C#, entre outras), os sistemas de banco de dados (MySQL, PostgreSQL, MongoDB, SQL Server, MariaDB) e os servidores de aplicação (Apache HTTP Server, Nginx). Retomando o exemplo do e-commerce, cabe ao backend determinar como os produtos são exibidos com base nos dados armazenados (nome, descrição, quantidade em estoque), processar pagamentos via Pix, cartão de crédito, débito ou boleto, e gerenciar as contas de usuário, incluindo a validação de login e senha — algo que o frontend, por si só, jamais teria condições de fazer com segurança.

26.4 A arquitetura cliente-servidor

A relação entre frontend e backend é formalizada, do ponto de vista arquitetural, pelo modelo **cliente-servidor**. Nesse modelo, o *cliente* — tipicamente o navegador executando o frontend — envia requisições para o *servidor*, que representa o backend. O servidor processa essa requisição, eventualmente consultando ou alterando dados em um banco de dados, e devolve uma resposta ao cliente, que atualiza a interface de acordo com o resultado recebido.

A figura a seguir ilustra essa arquitetura de forma simplificada: o navegador do usuário se comunica com o servidor de backend, que por sua vez se comunica com o banco de dados para persistir ou recuperar informações.



Observe que o cliente nunca acessa o banco de dados diretamente: toda comunicação passa obrigatoriamente pelo servidor de backend, exatamente como, no restaurante, o cliente nunca fala diretamente com o chef, mas sempre por meio do garçom.

Essa intermediação é o que garante segurança, controle e consistência à aplicação — é o backend quem decide o que pode ou não ser feito com os dados armazenados.

Dica

Ao longo desta parte do livro, o leitor construirá aplicações backend em Java utilizando o framework Spring Boot. É fundamental ter em mente, desde já, que o backend nunca existe isoladamente: ele é sempre uma das pontas de uma conversa entre cliente e servidor, e essa conversa segue convenções bem estabelecidas, que serão detalhadas no próximo capítulo, dedicado às APIs.

26.5 A integração entre frontend e backend

A comunicação entre frontend e backend geralmente ocorre por meio de **APIs** (*Application Programming Interfaces*, ou interfaces de programação de aplicações). Quando o usuário clica em um botão para adicionar um produto ao carrinho, por exemplo, o frontend dispara uma requisição para o servidor (o backend), que processa essa ação e devolve uma resposta — que pode indicar sucesso (o produto foi adicionado) ou falha (o produto está esgotado, ou ocorreu um erro de conexão). Essa resposta é então utilizada pelo frontend para atualizar a interface e informar o usuário sobre o resultado da operação.

Esse mecanismo de comunicação — as APIs — será justamente o assunto do próximo capítulo. Sem ele, não haveria como um frontend construído em HTML, CSS e JavaScript conversar de forma organizada com um backend escrito em Java, Python ou qualquer outra linguagem. É a API que estabelece o "contrato" dessa conversa, permitindo que as duas partes evoluam de forma relativamente independente, desde que respeitem esse contrato compartilhado.

Síntese do Capítulo

- Programação web abrange tanto o frontend (o que o usuário vê e manipula) quanto o backend (a lógica que opera nos bastidores).
- A analogia do restaurante ilustra bem a divisão: o salão, o menu e o garçom representam o frontend; a cozinha, o estoque e o faturamento representam o backend.
- O frontend é construído com HTML, CSS e JavaScript; o backend utiliza linguagens como Java, Python, PHP ou Node.js, além de bancos de dados e servidores de aplicação.
- O backend nunca deve confiar cegamente em dados vindos do frontend — validações devem ser refeitas no servidor.
- A arquitetura cliente-servidor formaliza essa divisão: o cliente envia requisições, o servidor processa e consulta o banco de dados, e devolve uma resposta.

- O cliente nunca acessa o banco de dados diretamente; toda comunicação passa pelo backend, que atua como intermediário e guardião das regras de negócio.
- A integração entre frontend e backend é viabilizada por APIs, tema que será aprofundado no próximo capítulo.

Capítulo 27

APIs e o Estilo Arquitetural REST

No capítulo anterior, ficou estabelecido que a comunicação entre frontend e backend depende de um mecanismo padronizado: a API. Este capítulo aprofunda esse conceito, explicando por que as APIs surgiram, como funcionam os verbos HTTP que as sustentam, o que caracteriza o estilo arquitetural REST e quais são os códigos de status HTTP que indicam o resultado de uma requisição. Ao final deste capítulo, o leitor estará preparado para compreender — e futuramente construir — suas próprias APIs seguindo boas práticas de mercado.

27.1 A origem e a problemática das APIs

Antes da popularização da internet, era comum que aplicações funcionassem de forma inteiramente offline ou local. Com a evolução tecnológica e a expansão do acesso à rede — não apenas por computadores, mas também por notebooks, smartphones e dispositivos da chamada internet das coisas (relógios inteligentes, assistentes de voz como Alexa e Google Home, entre outros) — passou a ser cada vez mais comum a existência de aplicações que funcionam exclusivamente por meio da internet, consumidas por navegadores independentemente da plataforma.

Esse cenário gerou duas necessidades centrais: primeiro, a necessidade de disponibilizar software que pudesse ser acessado pela web, independentemente da plataforma do usuário; segundo, a necessidade de empresas alimentarem seus próprios sistemas, disponibilizando serviços que pudessem ser consumidos tanto por usuários finais quanto por outras aplicações. A partir dessas necessidades, surgiram diversas soluções de software voltadas a permitir a comunicação entre sistemas — sistema com sistema, e sistema com usuário — e esse conjunto de soluções ficou conhecido como API.

27.2 O que é uma API

API

API é a sigla para *Application Programming Interface* (Interface de Programação de Aplicações): um conjunto de regras e definições que facilita a interação entre diferentes softwares, permitindo que aplicativos se comuniquem entre si, requisitando e compartilhando dados ou funcionalidades de maneira padronizada.

Uma API é baseada em um contrato claro: quem consome a API sabe exatamente o que precisa enviar e o que receberá como resposta. Imagine, por exemplo, uma aplicação A escrita em Java, com um conjunto de rotinas documentadas e bem definidas. Uma aplicação B, escrita em C#, pode consumir os serviços de A sem conhecer ab-

solutamente nada sobre sua implementação interna — se ela usa Java puro, Spring Boot, Jakarta EE ou Quarkus é irrelevante para quem consome a API. Basta que a requisição siga o contrato estabelecido para que a resposta esperada seja recebida.

As APIs podem ser implementadas de diversas formas, incluindo o padrão **REST** (o foco deste capítulo), o protocolo **SOAP** (*Simple Object Access Protocol*) e o modelo **RPC** (*Remote Procedure Call*). Antes de detalhar o REST, no entanto, é indispensável compreender os verbos HTTP, que fornecem a base para qualquer comunicação via HTTP.

27.3 Os verbos (ou métodos) HTTP

Os verbos HTTP são comandos usados para indicar qual ação o cliente deseja realizar sobre um recurso específico de um servidor. Eles existem para viabilizar aplicações que seguem o padrão **CRUD** — as quatro operações básicas que podem ser realizadas sobre qualquer recurso: *Create* (criar), *Retrieve* (recuperar), *Update* (atualizar) e *Delete* (remover).

27.3.1 GET: recuperando informações

O verbo **GET** corresponde ao *Retrieve* do CRUD. Ele é utilizado para solicitar informações de um recurso sem alterar seu estado no servidor — é, portanto, uma operação de leitura pura. Todo navegador realiza requisições **GET** o tempo todo: ao digitar um endereço na barra de navegação ou realizar uma busca no Google, o navegador está executando uma requisição **GET**.

HTTP

```
GET /api/products HTTP/1.1
Host: api.bluevelvet.com
Accept: application/json
```

27.3.2 POST: criando novos recursos

O verbo **POST** envia dados ao servidor para a criação de um novo recurso, alterando, portanto, o estado do servidor. É comumente utilizado para enviar formulários, cadastrar novos usuários ou criar registros em um banco de dados. Diferentemente do **GET**, o **POST** não expõe seus dados na URL, mas sim no corpo (*body*) da requisição — o que é essencial quando informações sensíveis, como senhas, estão envolvidas.

HTTP

```
POST /api/products HTTP/1.1
Host: api.bluevelvet.com
Content-Type: application/json

{
  "name": "CD Player Retro",
  "shortDescription": "Toca-discos compacto",
  "brand": "Elgin",
```

```
"listPrice": 349.90
}
```

27.3.3 PUT: substituindo um recurso por completo

O verbo PUT atualiza um recurso existente, substituindo-o integralmente pelos dados fornecidos na requisição. Isso significa que, ao usar PUT, o cliente deve enviar todos os campos do recurso — mesmo aqueles que não mudaram — pois o servidor tratará a requisição como uma substituição completa, e não como uma alteração pontual.

```
HTTP
PUT /api/products/10 HTTP/1.1
Host: api.bluevelvet.com
Content-Type: application/json

{
  "name": "CD Player Retro Prata",
  "shortDescription": "Toca-discos compacto edicao especial",
  "brand": "Elgin",
  "listPrice": 379.90
}
```

27.3.4 PATCH: atualizações parciais

O verbo PATCH realiza alterações parciais em um recurso já existente, modificando apenas um campo ou um subconjunto de campos, sem exigir que o restante das informações seja reenviado. É comum, por exemplo, permitir que um usuário atualize apenas o apelido ou o endereço de e-mail de seu cadastro por meio de PATCH, mantendo inalterados campos sensíveis como CPF ou data de nascimento.

```
HTTP
PATCH /api/products/10 HTTP/1.1
Host: api.bluevelvet.com
Content-Type: application/json

{
  "listPrice": 399.90
}
```

27.3.5 DELETE: removendo um recurso

O verbo DELETE remove um recurso do servidor, sendo utilizado para excluir itens como produtos, comentários ou publicações.

```
HTTP
DELETE /api/products/10 HTTP/1.1
Host: api.bluevelvet.com
```

Vale destacar uma discussão relevante na literatura: em muitos sistemas reais, uma exclusão não é imediatamente definitiva. Um produto removido de uma loja pode simplesmente ser marcado como indisponível, e uma conta de usuário “deletada” pode permanecer inativa por um período antes da exclusão definitiva (como ocorre, por exemplo, com o Google Drive, que move itens para uma lixeira antes de excluí-los permanentemente). Alguns autores argumentam que, tecnicamente, essas operações deveriam ser tratadas como PATCH (já que apenas um campo, como uma *flag* de “inativo”, é alterado). Contudo, do ponto de vista de quem consome a API, essa é uma regra de negócio interna que não precisa ser exposta: o consumidor apenas precisa saber que solicitou a remoção do recurso, e o verbo DELETE continua sendo o mais adequado semanticamente para expressar essa intenção.

27.3.6 HEAD e OPTIONS

Além dos cinco verbos principais, existem dois métodos HTTP adicionais de uso mais específico. O HEAD recupera apenas os cabeçalhos de um recurso, sem o corpo da resposta, sendo útil para obter metadados sem transferir todo o conteúdo. O OPTIONS retorna quais métodos HTTP são suportados pelo servidor para um recurso específico, sendo frequentemente utilizado em situações de **CORS** (*Cross-Origin Resource Sharing*) — um mecanismo de segurança que restringe o acesso a recursos de uma página a requisições originadas do mesmo domínio (ou de domínios explicitamente autorizados), protegendo a aplicação contra interações indevidas vindas de outros domínios.

27.4 O estilo arquitetural REST

REST

REST (*Representational State Transfer*, ou Transferência de Estado Representacional) é um conjunto de convenções que permite a comunicação entre sistemas diferentes, geralmente via HTTP, utilizando os verbos GET, POST, PUT, PATCH e DELETE. Uma API que segue o padrão REST é chamada de **RESTful**.

O REST se apoia em cinco características principais:

- **Arquitetura stateless (sem estado):** cada requisição do cliente contém todas as informações necessárias para que o servidor a processe; o servidor não guarda nenhuma informação sobre requisições anteriores. Uma requisição é sempre independente da anterior.
- **Recursos identificáveis:** cada recurso (um usuário, um produto, um comentário) é identificado por uma URL única, geralmente no formato `/recursos/{id}`. Por convenção, o nome do recurso é sempre utilizado no plural: `/users`, `/products`, `/comments`.
- **Uso dos métodos HTTP:** as operações CRUD são mapeadas diretamente para os verbos HTTP correspondentes (GET, POST, PUT/PATCH, DELETE).

- **Formato de dados padronizado:** a troca de informações costuma ocorrer em JSON (*JavaScript Object Notation*) ou XML (*Extensible Markup Language*). O JSON é hoje amplamente preferido por ser mais leve e mais legível que o XML.
- **Independência de estado entre requisições:** reforçando a primeira característica, o servidor não mantém memória de interações passadas — cada chamada é autocontida.

Um exemplo clássico de API REST pública utilizada para fins didáticos é o *JSON-Placeholder*, que expõe recursos como `/posts`, `/users` e `/comments`. Uma requisição a `/posts` retorna a lista completa de publicações; uma requisição a `/posts/50` retorna especificamente o post de identificador 50; e uma requisição a `/posts/50/comments` retorna os comentários associados a esse post específico — tudo seguindo o mesmo padrão de URL previsível, sem que o consumidor precise conhecer detalhes internos da implementação.

Resposta JSON de uma API REST

Uma requisição GET `/api/products/10` poderia retornar a seguinte resposta, representando um único produto da loja:

JSON

```
{
  "id": 10,
  "name": "CD Player Retro",
  "shortDescription": "Toca-discos compacto",
  "fullDescription": "Toca-discos compacto com entrada USB e Bluetooth",
  "brand": "Elgin",
  "category": "Eletronicos",
  "listPrice": 349.90,
  "discount": 20.00,
  "isEnabled": true,
  "inStock": true,
  "creationTime": "2026-01-10",
  "updateTime": "2026-03-02"
}
```

27.4.1 Vantagens de se construir APIs REST

Três vantagens justificam a ampla adoção do REST no mercado. A primeira é a **simplicidade**: uma vez compreendido o padrão, torna-se fácil implementar, entender e utilizar qualquer API que o siga. A segunda é a **flexibilidade**: por ser um padrão simples e amplamente adotado, o REST permite a integração entre plataformas e serviços distintos — inclusive entre APIs que conversam com outras APIs, formando cadeias de comunicação. A terceira é a **escalabilidade**: como as operações CRUD já estão bem definidas desde o início, adicionar novos endpoints à medida que a aplicação cresce se torna uma tarefa simples, sem exigir mudanças estruturais profundas.

27.5 Códigos de status HTTP

Toda resposta HTTP inclui um código de status — um número de três dígitos que indica o resultado da requisição. O primeiro dígito indica a classe geral da resposta:

- **1xx — Informativo:** o servidor recebeu a requisição e está processando-a (por exemplo, 100 *Continue*).
- **2xx — Sucesso:** a requisição foi processada com êxito (por exemplo, 200 *OK*, 201 *Created*, 204 *No Content*).
- **3xx — Redirecionamento:** o cliente precisa realizar passos adicionais para completar a requisição (por exemplo, 301 *Moved Permanently*).
- **4xx — Erro do cliente:** houve um erro do lado de quem fez a requisição, como dados inválidos ou ausência de credenciais (400 *Bad Request*, 401 *Unauthorized*, 403 *Forbidden*, 404 *Not Found*, 422 *Unprocessable Entity*).
- **5xx — Erro do servidor:** a requisição estava correta, mas o servidor não conseguiu processá-la, geralmente por uma falha interna (500 *Internal Server Error*).

Dica

O código 404 *Not Found* é, provavelmente, o mais conhecido entre todos os status HTTP: ele indica que o recurso solicitado não foi encontrado — podendo nunca ter existido ou ter sido removido em algum momento. Já o 500 *Internal Server Error* costuma indicar uma falha não mapeada dentro da própria aplicação, e normalmente exige investigação de logs do servidor para ser diagnosticado.

Compreendidos os verbos HTTP, o padrão REST e os códigos de status, o leitor já possui a base conceitual necessária para avançar ao desenvolvimento prático de APIs. Os próximos capítulos apresentarão o ecossistema Spring e, em seguida, a implementação concreta de um controlador REST em Java.

Síntese do Capítulo

- APIs surgiram da necessidade de permitir comunicação padronizada entre sistemas distintos, independentemente de plataforma ou linguagem.
- Uma API é definida por um contrato claro: o que deve ser enviado e o que será recebido como resposta.
- Os verbos HTTP (GET, POST, PUT, PATCH, DELETE) mapeiam diretamente para as operações CRUD (Create, Retrieve, Update, Delete).
- REST é um estilo arquitetural baseado em cinco características: ausência de estado, recursos identificáveis por URL, uso dos métodos HTTP, formato de dados padronizado (geralmente JSON) e independência entre requisições.

-
- APIs que seguem o padrão REST são chamadas de RESTful, e trazem simplicidade, flexibilidade e escalabilidade ao desenvolvimento.
 - Códigos de status HTTP (1xx a 5xx) comunicam o resultado de uma requisição, distinguindo sucesso, erro do cliente e erro do servidor.
 - Esses conceitos formam a base necessária para a implementação de controladores REST, tema dos próximos capítulos.

Capítulo 28

Spring, Jakarta EE e Spring Boot

Compreendidos os conceitos de API e REST, o próximo passo é conhecer as ferramentas que permitirão construir um backend real em Java. Este capítulo apresenta o ecossistema Spring, compara-o com o Jakarta EE (antigo Java EE) e introduz o Spring Boot — o framework que será efetivamente utilizado na construção da aplicação prática deste livro.

28.1 O que é o Spring

O Spring não é apenas um framework de backend: é um verdadeiro ecossistema, composto por dezenas de projetos e soluções que resolvem diferentes problemas enfrentados por um desenvolvedor backend ao longo do ciclo de desenvolvimento. A proposta central do Spring é permitir que o desenvolvedor construa aplicações Java com simplicidade e flexibilidade, escolhendo, dentro desse ecossistema, apenas os módulos necessários para resolver o problema em questão.

Ecossistema Spring

O Spring é um conjunto modular de projetos — conhecido como *ecossistema Spring* — em que cada módulo resolve uma classe específica de problema, podendo ser incluído no projeto de forma independente dos demais.

Entre os principais projetos desse ecossistema, destacam-se:

- **Spring Core:** o núcleo do framework, contendo o contêiner de injeção de dependência que implementa o padrão de *Inversão de Controle* (IoC), além do gerenciamento do ciclo de vida dos *beans* e mecanismos de extensão como o Spring AOP (Programação Orientada a Aspectos).
- **Spring MVC / Spring Web:** fornece uma arquitetura *Model-View-Controller* para desenvolvimento web, com gerenciamento de requisições e respostas HTTP e suporte a *templates* e *views*.
- **Spring Data:** oferece uma camada de abstração para acesso a dados, simplificando operações CRUD e suportando tanto bancos relacionais (MySQL, PostgreSQL, MariaDB) quanto não relacionais (MongoDB), além de recursos como paginação de consultas.
- **Spring Security:** fornece autenticação e autorização de usuários, controle de acesso a recursos e funcionalidades, além de mecanismos de criptografia e proteção contra ataques.

- **Spring Cloud:** voltado ao desenvolvimento de aplicações escaláveis, com suporte a microsserviços, nuvem e contêineres.
- **Spring Integration:** facilita a integração entre sistemas usando protocolos de mensagens, sendo especialmente útil em fluxos de trabalho reativos — embora, na prática, seja um dos módulos mais complexos de se dominar.
- **Spring Batch:** voltado ao processamento de dados em lote, com suporte a *jobs* e *steps*, gerenciamento de execução e novas tentativas em caso de falha — útil, por exemplo, para rotinas noturnas que verificam boletos pendentes de pagamento.

A ideia central por trás de todo esse ecossistema é permitir que o desenvolvedor concentre seu tempo nas regras de negócio da aplicação, e não em códigos repetitivos de infraestrutura — como conexão com bancos de dados ou integração com sistemas de mensageria (Kafka, RabbitMQ). Por essa razão, o Spring é frequentemente descrito como um “canivete suíço” para desenvolvedores Java: ele resolve problemas de gerenciamento de objetos, injeção de dependência, acesso a diferentes bancos de dados e construção de APIs REST, entre muitos outros.

28.1.1 Por que utilizar o Spring

Seis razões justificam a ampla adoção do Spring no mercado:

- **Simplifica o desenvolvimento:** reduz a necessidade de configuração manual extensa, concentrando as configurações essenciais em arquivos como `application.properties` ou `application.yml`.
- **Maturidade:** por ser um projeto consolidado há mais de duas décadas, seus colaboradores já compreendem profundamente o que funciona e o que não funciona, incorporando esse aprendizado continuamente ao framework.
- **Modularidade:** cada módulo do Spring é independente; o desenvolvedor inclui apenas os módulos que precisa, sem sobrecarregar o projeto com dependências desnecessárias.
- **Evolução constante:** o Spring está em constante atualização, acompanhando as demandas do mercado — por exemplo, evoluindo da versão 2.x para a 3.x do Spring Boot em poucos anos.
- **Licença open source:** o código-fonte é aberto, permitindo inspeção e contribuição da comunidade.
- **Empregabilidade:** o Spring é utilizado por empresas como Netflix, TripAdvisor, Hotmart, PagSeguro, Santander, Itaú, JP Morgan, American Express, Amazon, Walmart e PayPal, entre muitas outras, tornando-o uma das tecnologias mais demandadas no mercado de trabalho de desenvolvimento Java.

Vale notar que, nos primórdios do Spring, a configuração era fortemente baseada em arquivos XML — prática comum na época na comunidade Java, mas considerada verbosa e pouco amigável por muitos desenvolvedores. Atualmente, o Spring favorece a

configuração baseada em anotações e em arquivos simples como `.properties` ou `.yaml`, tornando o processo de configuração muito mais enxuto (embora alguns módulos, como o Spring Integration, ainda dependam de XML em certos cenários).

28.2 Spring versus Jakarta EE

O Jakarta EE — anteriormente conhecido como Java EE — é outro framework amplamente utilizado para o desenvolvimento de aplicações empresariais em Java. Embora Spring e Jakarta EE compartilhem objetivos semelhantes, existem diferenças significativas entre eles, que vale a pena compreender antes de se decidir por um ou outro em um projeto real.

- **Origem:** o Spring foi desenvolvido pela SpringSource (hoje parte da VMware), surgindo como uma alternativa ao modelo de programação do Java EE, buscando resolver problemas de complexidade e excesso de configuração. O Jakarta EE, por sua vez, é um padrão originalmente mantido pela Oracle sob o nome Java EE, atualmente gerido pela Eclipse Foundation.
- **Arquitetura:** o Spring adota uma abordagem modular e leve, permitindo que o desenvolvedor escolha exatamente os módulos que deseja usar. O Jakarta EE se baseia em especificações mais rígidas e padronizadas, resultando em uma plataforma mais monolítica. O Jakarta EE inclui especificações como Servlets, Enterprise JavaBeans (EJB) e a Java Persistence API (JPA) — sendo esta última, inclusive, frequentemente utilizada mesmo em projetos que adotam o Spring como framework principal, o que mostra que as duas tecnologias não são mutuamente excludentes.
- **Injeção de dependência:** o Spring é conhecido por seu contêiner de Inversão de Controle (IoC) e por oferecer ampla flexibilidade na configuração de componentes — especialmente a injeção via construtor, combinada ao uso da biblioteca Lombok. O Jakarta EE historicamente dependia do EJB para injeção de dependência, mas evoluiu para adotar o CDI (*Contexts and Dependency Injection*) como sua principal tecnologia nesse aspecto.
- **Configuração:** o Spring favorece anotações como `@Autowired`, `@Entity`, `@Repository`, `@Service` e `@RestController`. O Jakarta EE, embora historicamente dependesse fortemente de XML, também tem avançado no sentido de adotar configuração baseada em anotações, principalmente com o CDI.
- **Licenciamento:** o Spring adota uma licença de código aberto mais permissiva, enquanto o Jakarta EE utiliza licenças que podem variar conforme a implementação específica adotada.

Na prática, a escolha entre Spring e Jakarta EE costuma depender das preferências da equipe, dos requisitos do projeto e do histórico tecnológico da organização — e algumas empresas até combinam elementos dos dois. De forma geral, o Spring tende a ser mais adequado para aplicações menores e mais ágeis, como microsserviços, enquanto o Jakarta EE, apesar de exigir uma configuração inicial mais robusta, pode

ser mais indicado para aplicações empresariais de grande porte. Para os fins deste livro — e para a maioria dos projetos de porte pequeno a médio — o Spring, e mais especificamente o Spring Boot, será a escolha adotada.

28.3 Spring Boot: simplicidade sobre configuração

Spring Boot

Spring Boot é uma extensão do Spring Framework que simplifica significativamente o processo de criação de aplicações Java, fornecendo uma maneira rápida e fácil de construir aplicativos autônomos e prontos para produção, com a mínima configuração possível.

O princípio central que sustenta o Spring Boot é conhecido como *convention over configuration* (convenção sobre configuração): em vez de exigir que o desenvolvedor configure manualmente cada detalhe de uma conexão de banco de dados, por exemplo, o Spring Boot já assume um conjunto de convenções padronizadas que funcionam para a grande maioria dos casos, exigindo apenas que o desenvolvedor forneça informações mínimas, como URL, porta, usuário e senha do banco.

28.3.1 Características principais do Spring Boot

- **Starter POMs:** artefatos Maven (ou Gradle) pré-configurados que simplificam a inclusão de dependências comuns, permitindo inicializar um novo projeto em poucos segundos.
- **Autoconfiguração:** o Spring Boot analisa o ambiente em que a aplicação está sendo executada e configura automaticamente os componentes necessários, reduzindo a quantidade de configuração manual e de código repetitivo (*boilerplate*) — entendido aqui como todo trecho de código que se repete de forma quase idêntica em diferentes contextos, como a estrutura básica de um documento HTML.
- **Aplicativos autônomos (*standalone*):** graças à inclusão de servidores embutidos, como o Tomcat, uma aplicação Spring Boot pode ser executada sem a necessidade de um servidor de aplicação externo instalado separadamente.
- **Monitoramento e gerenciamento:** o Spring Boot Actuator oferece endpoints HTTP prontos para verificação de saúde (*health*), métricas e informações do ambiente da aplicação — atualmente apoiado pela biblioteca Micrometer.
- **Integração com o ecossistema Spring:** o Spring Boot é totalmente compatível com Spring Data, Spring Security, Spring Cloud e demais módulos, permitindo que o desenvolvedor incorpore essas funcionalidades conforme a necessidade do projeto.
- **Integração com frontends modernos:** aplicações Spring Boot se integram facilmente a frameworks e bibliotecas de frontend como Angular, React e Vue.js, permitindo a construção de aplicações web completas.

28.4 Iniciando um projeto Spring Boot

Diferentemente de um projeto Java tradicional — no qual basta criar uma classe com um método `main` — um projeto Spring Boot é iniciado por meio de uma ferramenta chamada **Spring Initializr**. Trata-se de uma aplicação web (ou de uma funcionalidade integrada a determinadas IDEs, como o IntelliJ IDEA Ultimate) que gera o esqueleto inicial do projeto, já com as dependências e configurações necessárias para começar o desenvolvimento.

Existem duas formas principais de utilizar o Spring Initializr:

- Diretamente pela IDE, no caso de IDEs com suporte nativo (como o IntelliJ IDEA Ultimate), acessando o menu de criação de novo projeto e selecionando a opção Spring Initializr;
- Pelo site oficial do Spring Initializr, disponível em <https://start.spring.io>, que oferece uma interface gráfica equivalente, permitindo selecionar o gerenciador de dependências (Maven ou Gradle), a linguagem (Java, Kotlin ou Groovy), a versão do Spring Boot, informações do projeto (grupo, artefato, nome do pacote), a versão do Java e as dependências iniciais desejadas.

Ao final do processo, a ferramenta gera um arquivo compactado (ZIP) contendo toda a estrutura inicial do projeto, que pode então ser extraído e aberto na IDE de preferência do desenvolvedor.

Dica

No momento da escolha da versão do Java, é recomendável optar por uma versão *LTS* (*Long Term Support*), ou seja, uma versão com suporte de longo prazo garantido pela comunidade e pelo próprio Spring. No período em que este material foi produzido, o Java 21 era a versão LTS mais recente, ainda que versões mais novas já estivessem disponíveis.

Com a estrutura inicial do projeto criada e compreendida, o leitor está pronto para avançar à implementação prática. O próximo capítulo apresentará, passo a passo, a construção de um controlador REST real, no contexto de um estudo de caso: a loja fictícia BlueVelvet Music Store.

Síntese do Capítulo

- O Spring é um ecossistema modular de projetos — não apenas um framework — do qual o desenvolvedor seleciona somente os módulos necessários (Spring Core, Spring MVC, Spring Data, Spring Security, entre outros).
- O objetivo do Spring é permitir que o desenvolvedor foque nas regras de negócio, delegando ao framework grande parte da configuração de infraestrutura.
- O Spring se popularizou por sua simplicidade, maturidade, modularidade, evolução constante, licença open source e altíssima empregabilidade.

- O Jakarta EE (antigo Java EE) é a principal alternativa ao Spring, com arquitetura mais rígida e monolítica, mas ambos podem conviver no mesmo projeto (como no caso da JPA).
- O Spring Boot é uma extensão do Spring que aplica o princípio de "convenção sobre configuração", simplificando drasticamente a criação de aplicações prontas para produção.
- Recursos como Starter POMs, autoconfiguração, servidores embutidos (Tomcat) e o Spring Boot Actuator tornam o desenvolvimento mais ágil e com menos código repetitivo.
- Projetos Spring Boot são iniciados por meio do Spring Initializr, seja pela IDE ou pelo site oficial (start.spring.io).

Capítulo 29

Implementando um Controlador REST: Estudo de Caso BlueVelvet Music Store

Com os conceitos de API, REST e Spring Boot já apresentados, este capítulo parte para a prática: a implementação de um controlador REST real, em Java, utilizando o Spring Boot. O estudo de caso escolhido é a **BlueVelvet Music Store**, uma loja fictícia de produtos musicais que servirá de fio condutor para os exemplos de código apresentados ao longo desta parte do livro.

29.1 O projeto BlueVelvet Music Store

O projeto BlueVelvet Music Store é organizado em torno de um conjunto de *user stories* — descrições, sob a perspectiva do usuário, das funcionalidades que o sistema deve oferecer. Entre elas, destacam-se: login de usuários, registro de novos usuários, acesso a um painel (*dashboard*) de gerenciamento de produtos, criação de produtos, edição de informações de produtos, visualização de detalhes de produtos por administradores e exclusão de produtos.

Como as funcionalidades de login e registro exigem o uso do Spring Security — módulo que será tratado em um capítulo futuro — este capítulo concentra-se nas *user stories* relacionadas ao **gerenciamento de produtos**, implementando um controlador que segue o padrão REST e contempla as operações básicas do CRUD (criar, recuperar, atualizar e deletar produtos).

Dica

Nesta etapa, o objetivo é compreender a estrutura de um controlador REST isoladamente, sem ainda conectá-lo a um banco de dados real. Por isso, os dados retornados pelos endpoints serão simulados (*mockados*) diretamente no código — uma prática comum durante o desenvolvimento inicial de uma API, antes da camada de persistência estar pronta.

29.2 Estrutura inicial do projeto

O projeto foi criado por meio do Spring Initializr, com as seguintes configurações: gerenciador de dependências Gradle (syntaxe Groovy), linguagem Java, versão do Spring Boot 3.3.5, grupo `com.musicstore`, artefato `bluevelvet`, empacotamento JAR e Java 21. Foram selecionadas três dependências iniciais:

- **Lombok:** biblioteca que gera automaticamente código repetitivo como *getters*, *setters*, construtores e métodos utilitários de log, por meio de anotações;
- **Spring Web:** dependência que traz a infraestrutura necessária para criar controladores REST, incluindo o servidor embutido Tomcat;
- **SpringDoc OpenAPI:** responsável por gerar automaticamente uma documentação interativa da API, no formato conhecido popularmente pelo nome comercial *Swagger*.

A classe principal do projeto, gerada automaticamente, é anotada com `@SpringBootApplication` — anotação que identifica o ponto de entrada da aplicação e sinaliza ao Spring que ali deve ocorrer toda a inicialização automática do contêiner:

Java

```
package com.musicstore.bluevelvet;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BlueVelvetApplication {

    public static void main(String[] args) {
        SpringApplication.run(BlueVelvetApplication.class, args);
    }

}
```

Para simplificar a configuração da aplicação, o arquivo `application.properties` pode ser substituído por um arquivo `application.yml`, considerado mais legível e organizado:

YAML

```
spring:
  application:
    name: Blue Velvet Music Store
```

Ao executar a aplicação, o Spring Boot inicializa o servidor Tomcat embutido na porta padrão 8080. Acessando `http://localhost:8080` no navegador, é esperado observar a página padrão de erro do Spring (*Whitelabel Error Page*), o que indica que o servidor está no ar — apenas ainda não existe nenhum endpoint mapeado para a raiz da aplicação. Ao acessar `http://localhost:8080/swagger-ui/index.html`, é possível visualizar a interface do Swagger, inicialmente vazia, por não haver ainda nenhum controlador implementado.

29.3 Organizando o projeto: pacotes controller, request e response

Antes de escrever o primeiro controlador, é criada uma estrutura de pacotes que organiza o código de forma clara: um pacote `api`, contendo três subpacotes — `controller` (onde ficam os controladores REST), `request` (onde ficam as classes que representam os dados recebidos nas requisições) e `response` (onde ficam as classes que representam os dados devolvidos nas respostas).

29.3.1 A classe `ProductRequest`

A classe `ProductRequest` representa as informações necessárias para criar ou atualizar um produto. Ela inclui atributos como nome, descrições, marca, categoria, preços, disponibilidade, dimensões e detalhes adicionais:

Java

```
package com.musicstore.bluevelvet.api.request;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import lombok.ToString;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

@Getter
@Setter
@Builder
@ToString
@NoArgsConstructor
@AllArgsConstructor
public class ProductRequest {

    private String name;
    private String shortDescription;
    private String fullDescription;
    private String brand;
    private String category;
    private String mainImage;
    private String otherImages;
    private BigDecimal listPrice;
    private BigDecimal discount;
    private boolean isEnabled;
    private boolean inStock;
```

```
private LocalDate creationTime;
private LocalDate updateTime;
private ProductDimensionRequest dimension;
private BigDecimal cost;
private List<ProductDetailRequest> details;

}
```

As dimensões físicas do produto (comprimento, largura, altura e peso) são encapsuladas em uma classe auxiliar própria, já que representam um subconjunto coeso de informações:

```
Java
package com.musicstore.bluevelvet.api.request;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import lombok.ToString;

@Getter
@Setter
@Builder
@ToString
@NoArgsConstructor
@AllArgsConstructor
public class ProductDimensionRequest {

    private float length;
    private float width;
    private float height;
    private float weight;

}
```

Já os detalhes do produto — pares de nome e valor, como "Cor: Prata" ou "Material: Metal" — são representados por uma lista de objetos de uma classe auxiliar:

```
Java
package com.musicstore.bluevelvet.api.request;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import lombok.ToString;
```

```

@Getter
@Setter
@Builder
@ToString
@NoArgsConstructor
@AllArgsConstructor
public class ProductDetailRequest {

    private String name;
    private String value;

}

```

As anotações do Lombok utilizadas merecem destaque: `@Getter` e `@Setter` geram automaticamente os métodos de acesso e modificação de cada atributo; `@Builder` permite construir objetos informando apenas os atributos desejados, evitando a fragilidade de um construtor extenso (no qual a inserção de um novo parâmetro no meio da lista pode quebrar todo o código que já utiliza aquele construtor); `@ToString` gera uma representação textual do objeto, extremamente útil para fins de log; `@NoArgsConstructor` gera um construtor vazio; e `@AllArgsConstructor` gera um construtor com todos os atributos da classe.

29.3.2 A classe `ProductResponse`

A classe `ProductResponse` é praticamente idêntica à `ProductRequest`, mas inclui um atributo adicional: o identificador (`id`) do produto, atribuído após sua criação — geralmente pelo próprio banco de dados:

```

Java
package com.musicstore.bluevelvet.api.response;

import com.musicstore.bluevelvet.api.request.ProductDetailRequest;
import com.musicstore.bluevelvet.api.request.ProductDimensionRequest;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import lombok.ToString;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

@Getter
@Setter
@Builder
@ToString

```

```
@NoArgsConstructor
@AllArgsConstructor
public class ProductResponse {

    private Long id;
    private String name;
    private String shortDescription;
    private String fullDescription;
    private String brand;
    private String category;
    private BigDecimal listPrice;
    private BigDecimal discount;
    private boolean isEnabled;
    private boolean inStock;
    private LocalDate creationTime;
    private LocalDate updateTime;
    private ProductDimensionRequest dimension;
    private BigDecimal cost;
    private List<ProductDetailRequest> details;

}
```

29.4 O controlador REST: ProductController

Com as classes de requisição e resposta definidas, o controlador em si pode ser criado. O primeiro ponto de atenção é que, para o Spring reconhecer uma classe como um controlador REST, não basta nomeá-la como tal — é necessário utilizar a anotação `@RestController`. Existe também a anotação `@Controller`, mais adequada quando o objetivo é retornar páginas HTML (unindo frontend e backend no mesmo projeto), o que não é o caso aqui.

A anotação `@RequestMapping("/products")` define o caminho-base (*path*) para todos os endpoints do controlador. Seguindo a convenção REST, o nome do recurso é sempre utilizado no plural.

29.4.1 Buscando um produto por identificador (GET)

O primeiro endpoint implementado recupera um único produto a partir de seu identificador:

```
Java
package com.musicstore.bluevelvet.api.controller;

import com.musicstore.bluevelvet.api.request.ProductDetailRequest;
import com.musicstore.bluevelvet.api.request.ProductDimensionRequest;
import com.musicstore.bluevelvet.api.request.ProductRequest;
import com.musicstore.bluevelvet.api.response.ProductResponse;
import io.swagger.v3.oas.annotations.Operation;
```

```
import lombok.extern.log4j.Log4j2;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

@Log4j2
@RestController
@RequestMapping("/products")
public class ProductController {

    @Operation(
        summary = "Find product",
        description = "Retrieve a product from the Blue Velvet Music Store by its ID"
    )
    @GetMapping("/{id}")
    public ResponseEntity<ProductResponse> getProduct(@PathVariable Long id) {
        log.info("Request received to fetch a product by ID {}", id);

        // FIXME: remover mock apos implementarmos servico e repositorio
        return ResponseEntity.ok(assembleProduct(id));
    }

    private ProductResponse assembleProduct(Long id) {
        return ProductResponse.builder()
            .id(id)
            .name("CD Player")
            .brand("Elgin")
            .build();
    }
}
```

Alguns pontos merecem destaque. A anotação `@GetMapping("/{id}")` indica que esse método responde a requisições GET no caminho `/products/{id}` — a concatenação entre o `@RequestMapping` da classe e o `@GetMapping` do método. A anotação `@PathVariable`, aplicada ao parâmetro `id`, indica que esse valor deve ser extraído diretamente do caminho da URL (e não de um parâmetro de consulta na *query string*), sendo fundamental que o nome entre chaves no mapeamento (`{id}`) corresponda ao nome utilizado na anotação.

O retorno do método é sempre um `ResponseEntity`, um tipo do Spring que permite construir respostas HTTP completas de forma elegante, incluindo o código de status — neste caso, `ResponseEntity.ok(...)` corresponde a um status 200 OK. Como a camada de persistência ainda não foi implementada, o método auxiliar `assembleProduct` simula (*mocka*) um retorno, o que é sinalizado no código por um comentário `FIXME`, lembrando que esse trecho precisará ser substituído por uma consulta real ao banco de dados.

A anotação `@Log4j2`, do Lombok, disponibiliza o objeto `log` dentro da classe, permitindo o uso de `log.info`, `log.debug`, `log.warn`, `log.error` e `log.trace`. O uso de um framework de log é preferível ao tradicional `System.out.println` porque cada mensagem registrada já inclui automaticamente informações como *timestamp*, nível do log, identificador do processo e nome da aplicação — essenciais para depuração em ambientes de produção.

29.4.2 Buscando todos os produtos (GET)

Para recuperar a lista completa de produtos, é adicionado um segundo endpoint, mapeado diretamente para `/products`:

Java

```
@Operation(
    summary = "Find all products",
    description = "Retrieve all products from the Blue Velvet Music Store"
)
@GetMapping
public ResponseEntity<List<ProductResponse>> getProducts() {
    log.info("Request received to fetch all products");

    // FIXME: remover mock e adicionar Pageable do Spring Data futuramente
    return ResponseEntity.ok(
        List.of(assembleProduct(1L), assembleProduct(2L), assembleProduct(3L))
    );
}
```

Note que a busca paginada (utilizando um objeto `Pageable`) fica marcada como pendente: ela dependerá do módulo Spring Data, que será apresentado quando a aplicação for conectada a um banco de dados real.

29.4.3 Removendo um produto (DELETE)

A remoção de um produto segue o mesmo princípio de extração do identificador via `@PathVariable`, mas normalmente retorna uma resposta sem conteúdo (status 204 No Content):

Java

```
@Operation(
    summary = "Delete product",
    description = "Delete a product from the Blue Velvet Music Store"
)
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
    log.info("Request received to delete the product with ID {}", id);

    return ResponseEntity.ok(null);
}
```

29.4.4 Criando um novo produto (POST)

Para criar um novo produto, os dados não podem trafegar como parâmetros na URL (o que seria inadequado e, em muitos casos, inseguro — imagine, por exemplo, enviar nome de usuário e senha diretamente na barra de endereços). Em vez disso, utiliza-se a anotação `@RequestBody`, que instrui o Spring a extrair o objeto `ProductRequest` diretamente do corpo da requisição HTTP:

Java

```
@Operation(
    summary = "Create product",
    description = "Create a new product in the Blue Velvet Music Store"
)
@PostMapping
public ResponseEntity<ProductResponse> createProduct(@RequestBody ProductRequest request) {
    log.info("Request received to create a product with request {}", request);

    return ResponseEntity.ok(assembleProduct(7L));
}
```

Graças à anotação `@ToString` presente em `ProductRequest`, o log exibe de forma legível todos os campos recebidos na requisição, o que facilita a depuração durante o desenvolvimento.

29.4.5 Atualizando um produto (PUT)

Por fim, resta implementar a atualização de um produto existente. Como a especificação do projeto BlueVelvet determina que *todos* os campos do produto podem ser atualizados de uma só vez, o verbo apropriado é o PUT (e não o PATCH, reservado para atualizações parciais de um subconjunto de campos):

Java

```
@Operation(
    summary = "Update product",
    description = "Update a product from the Blue Velvet Music Store"
)
@PutMapping("/{id}")
public ResponseEntity<ProductResponse> updateProduct(
    @PathVariable Long id,
    @RequestBody ProductRequest request) {
    log.info("Request received to update the product with ID {} with request {}", id, request);

    return ResponseEntity.ok(assembleProduct(id));
}
```


29.5 O que realmente caracteriza um controlador REST

Um ponto conceitual merece destaque especial: o que determina se uma operação é de criação, recuperação, atualização ou remoção **não** é o nome do método Java, tampouco apenas a anotação `@RestController` presente na classe. O que efetivamente define essa distinção é a combinação entre o **caminho** (*path*) do endpoint e o **verbo HTTP** utilizado. No exemplo do `ProductController`, o endpoint de busca por identificador e o de remoção compartilham exatamente o mesmo caminho (`/products/{id}`), diferenciando-se unicamente pelo verbo (`GET` contra `DELETE`). Da mesma forma, o endpoint de listagem e o de criação compartilham o caminho `/products`, diferenciando-se entre `GET` e `POST`.

Dica

O controlador é apenas a porta de entrada da aplicação: ele não deve conter regras de negócio. Sua responsabilidade é receber a requisição e repassá-la à camada de **serviço**, que concentra as regras de negócio e, quando necessário, aciona a camada de **repositório**, responsável pelo acesso efetivo ao banco de dados. Depois de processada a informação, o serviço a devolve ao controlador, que apenas monta a resposta para o cliente. Essa separação de responsabilidades — controlador, serviço e repositório — será aprofundada nos próximos capítulos, à medida que a aplicação evoluir para incluir persistência real de dados.

A documentação gerada automaticamente pelo SpringDoc (visível na interface do Swagger, em `/swagger-ui/index.html`) permite testar interativamente cada um desses endpoints, sem a necessidade de ferramentas externas como o Postman — embora, para os verbos diferentes de `GET`, o navegador sozinho não seja suficiente, já que ele só realiza requisições `GET` a partir da barra de endereços.

Síntese do Capítulo

- O estudo de caso BlueVelvet Music Store organiza o desenvolvimento em torno de user stories, começando pelo gerenciamento de produtos (CRUD), deixando login e registro para quando o Spring Security for abordado.
- O projeto é estruturado em pacotes `controller`, `request` e `response`, separando claramente entrada, saída e lógica de roteamento.
- Classes de requisição e resposta usam anotações do Lombok (`@Getter`, `@Setter`, `@Builder`, `@ToString`, `@NoArgsConstructor`, `@AllArgsConstructor`) para eliminar código repetitivo.
- `@RestController` e `@RequestMapping` definem, respectivamente, que a classe é um controlador REST e qual o caminho-base de seus endpoints.
- `@GetMapping`, `@PostMapping`, `@PutMapping` e `@DeleteMapping` associam métodos Java a verbos HTTP específicos; `@PathVariable` extrai valores do caminho da URL, e `@RequestBody` extrai objetos do corpo da requisição.

- O que caracteriza uma API REST é a combinação entre caminho e verbo HTTP — nunca apenas o nome do método ou da classe.
- O controlador deve se limitar a rotear requisições; regras de negócio pertencem à camada de serviço, e o acesso a dados, à camada de repositório.

Capítulo 30

Docker: Contêineres para Aplicações Web

Depois de implementar um primeiro controlador REST, é natural que o desenvolvedor comece a se perguntar como essa aplicação será executada em diferentes ambientes — na máquina de outro desenvolvedor, em um servidor de testes, em produção. Este capítulo apresenta o Docker, a ferramenta que se tornou padrão de mercado para resolver justamente esse problema, por meio do conceito de contêineres.

30.1 O problema da divergência de ambientes

No desenvolvimento de software, é comum a existência de diferentes tipos de ambiente ao longo do ciclo de vida de uma aplicação: o ambiente de **desenvolvimento** (a máquina local do desenvolvedor, onde ele testa e aperfeiçoa seu código livremente); o ambiente de **homologação**, no qual uma equipe de qualidade (QA) verifica se tudo está de acordo com o especificado; o ambiente de **staging**, no qual pessoas interessadas no produto — como *product owners* e outros *stakeholders* — podem validar o que foi desenvolvido; e, por fim, o ambiente de **produção**, onde o sistema efetivamente opera para os usuários finais.

Um problema recorrente nesse cenário é a **divergência entre ambientes**: a máquina de desenvolvimento pode ter uma arquitetura diferente, uma quantidade diferente de recursos de processamento e, principalmente, versões diferentes de softwares instalados em comparação com os ambientes de homologação, staging e produção. Esse tipo de divergência é responsável por uma das frases mais repetidas — e mais frustrantes — no universo do desenvolvimento de software: *"na minha máquina funciona"*.

O problema resolvido pelo Docker

O Docker resolve o problema da divergência de ambientes ao empacotar uma aplicação junto de todas as suas dependências (bibliotecas, variáveis de ambiente, configurações) em uma unidade isolada e portátil, chamada **contêiner**, garantindo que ela se comporte da mesma forma independentemente de onde for executada.

Antes do Docker, a solução mais comum para esse problema era a virtualização convencional, por meio de máquinas virtuais (VirtualBox, VMware, Parallels). O problema dessa abordagem é que, para instalar qualquer software dentro de uma máquina virtual, é necessário primeiro instalar um sistema operacional completo — e, caso uma mesma biblioteca precise estar disponível em várias máquinas virtuais, ela precisa ser instalada manualmente em cada uma delas, junto com todo o hardware virtualizado

exigido por aquele sistema operacional.

30.2 O que é um contêiner

O Docker propõe uma abordagem de virtualização não convencional: ele não depende de um sistema operacional completo por trás de cada aplicação, sendo autossuficiente por meio do conceito de **contêineres**.

Uma boa analogia para entender o conceito vem do transporte de cargas. Antes da invenção do contêiner de transporte marítimo, carregar e descarregar um navio era um processo lento, manual e sujeito a perdas — mercadorias podiam quebrar, deteriorar ou ser desviadas. Com o contêiner físico, tornou-se possível transportar mercadorias de forma segura, padronizada e com o mínimo de trabalho manual, já que máquinas especializadas passaram a lidar com a movimentação. O Docker aplica essa mesma lógica ao software: em vez de instalar manualmente cada dependência em cada ambiente, empacota-se a aplicação inteira, com tudo que ela precisa para funcionar, dentro de uma unidade padronizada e facilmente movimentável — o contêiner de software.

Contêiner

Um contêiner é um pacote de software que reúne uma aplicação e todas as suas dependências necessárias para execução. As instruções para iniciar ou parar um contêiner são determinadas por uma **imagem** do Docker; toda vez que uma imagem é executada, um novo contêiner é criado a partir dela.

O gerenciamento de contêineres é feito por meio da API do Docker ou de sua interface de linha de comando (ILC, ou CLI em inglês). Quando é necessário orquestrar vários contêineres simultaneamente, entra em cena o **Docker Compose**, ferramenta que será apresentada mais adiante neste capítulo. Historicamente, o Docker se apoiava em um projeto chamado LXC (*Linux Container*), que por sua vez utiliza recursos do kernel Linux como **chroot**, *control groups* (cgroups) e *namespaces* do kernel.

30.3 Trabalhando com imagens e contêineres

O Docker está disponível em duas edições: **Community Edition** (gratuita, mais do que suficiente para o uso de um desenvolvedor individual) e **Enterprise Edition** (voltada a ambientes corporativos, com infraestrutura homologada e certificada). Para os propósitos deste livro, a Community Edition é adequada.

30.3.1 Imagens: o ponto de partida de um contêiner

Para colocar um contêiner em funcionamento, o Docker precisa antes ter, em sua máquina hospedeira (*host*), uma **imagem** correspondente ao software desejado — por exemplo, uma imagem do MongoDB, do MySQL ou do Nginx. Essas imagens podem ser baixadas de um repositório remoto (chamado *registry*), sendo o **Docker Hub** o mais conhecido e utilizado deles, ou podem ser criadas localmente pelo próprio desenvolvedor.

O comando `docker pull` realiza o download de uma imagem a partir de um registry:

Docker

```
# Baixa a versao mais recente (latest) da imagem do Ubuntu
docker pull ubuntu

# Baixa uma versao especifica de uma imagem
docker pull mysql:5.7
```

Uma vez baixadas, as imagens disponíveis localmente podem ser listadas com o comando `docker images`, que exibe o repositório, a versão (*tag*), o identificador da imagem, a data de criação e o tamanho ocupado em disco.

30.3.2 Executando e gerenciando contêineres

A partir de uma imagem já baixada, é possível iniciar quantos contêineres forem necessários utilizando o comando `docker run`. Para acessar interativamente o terminal de um contêiner — por exemplo, um contêiner baseado na imagem do Ubuntu — utiliza-se:

Docker

```
# -i mantém o contêiner interativo
# -t anexa um terminal virtual ao host
docker run -it ubuntu
```

É importante ter em mente que o Docker não fornece interface gráfica: ao "instalar" o Ubuntu por meio de uma imagem Docker, o que se obtém é um ambiente de linha de comando, e não uma área de trabalho gráfica como a fornecida pelo GNOME em uma instalação convencional.

Para listar os contêineres em execução no momento, utiliza-se `docker ps`; para listar também os contêineres que já foram encerrados (mas que ainda existem, apenas não estão rodando), utiliza-se `docker ps -a`:

Docker

```
# Lista apenas os contêineres em execução
docker ps

# Lista todos os contêineres, inclusive os parados
docker ps -a
```

30.4 Criando imagens próprias: o Dockerfile

Até aqui, foram utilizadas apenas imagens já publicadas por terceiros no Docker Hub. Mas como criar uma imagem própria — por exemplo, uma imagem customizada do MySQL, já configurada com scripts específicos de inicialização de um banco de dados? A resposta está no **Dockerfile**, um arquivo de definição no qual se declaram, por meio de diretivas específicas, as instruções necessárias para montar a imagem.

Docker

```
# Dockerfile

# Toda imagem geralmente parte de uma imagem-base já existente
FROM mysql:latest

# Copia scripts locais de inicializacao do banco para dentro do contêiner
# Esses scripts serao executados automaticamente na primeira inicializacao
COPY ./db/ /docker-entrypoint-initdb.d/
```

Um Dockerfile é compilado (ou “construído”) por meio do comando `docker build`, que gera uma imagem a partir das instruções contidas no arquivo:

Docker

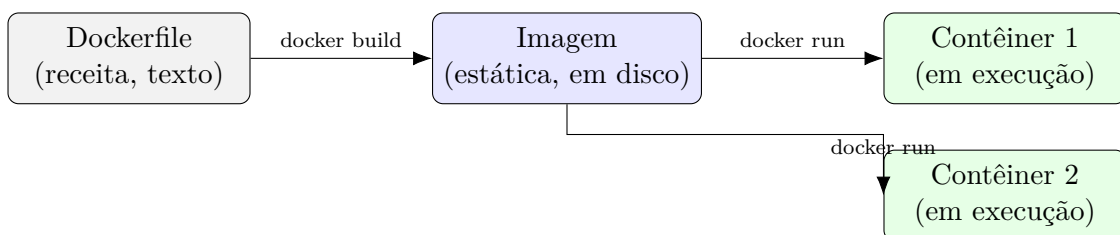
```
# Constroi uma imagem chamada bluevelvet-db a partir do Dockerfile
# no diretorio atual
docker build -t bluevelvet-db .

# Executa um contêiner a partir da imagem recém-criada
docker run -d --name meu-banco bluevelvet-db
```

Dica

É útil pensar na relação entre Dockerfile, imagem e contêiner como uma cadeia de transformações: o Dockerfile é a *receita*; a imagem, gerada a partir dessa receita, é algo estático — comparável a um programa gravado em disco; e o contêiner, criado a partir da execução da imagem, é a instância dinâmica em execução — comparável a um processo rodando na memória.

A figura a seguir resume essa cadeia de transformações:



Note que uma mesma imagem pode dar origem a múltiplos contêineres independentes entre si, exatamente como uma mesma classe em Java pode dar origem a múltiplos objetos independentes.

30.5 Orquestrando múltiplos contêineres com o Docker Compose

Quando uma aplicação depende de mais de um serviço — por exemplo, um banco de dados relacional e um sistema de mensageria — gerenciar cada contêiner manualmente se torna trabalhoso. O **Docker Compose** resolve esse problema por meio de

um arquivo de definição no formato **YAML** (*YAML Ain't Markup Language*), uma linguagem de serialização de dados criada para ser legível e de fácil escrita, também utilizada, por exemplo, nos arquivos `application.yml` do Spring Boot.

```
Docker
# docker-compose.yml

services:
  db:
    image: mysql:5.7
    restart: unless-stopped
    environment:
      MYSQL_DATABASE: DB
      MYSQL_USER: user
      MYSQL_PASSWORD: password
      MYSQL_ROOT_PASSWORD: password
    ports:
      - "3306:3306"
    expose:
      - "3306"
    volumes:
      - mydb:/var/lib/mysql

volumes:
  mydb:
```

Alguns pontos merecem atenção nessa estrutura. Em YAML, a indentação define hierarquia: tudo que está indentado sob `services` pertence a `services`, e tudo que está indentado sob `db` pertence especificamente a esse serviço. O nome `db` funciona como um simples identificador (*alias*) — poderia ser qualquer outro nome — utilizado para referenciar esse serviço dentro do arquivo.

A chave `image` define qual imagem e qual versão serão utilizadas; nesse exemplo, a versão 5.7 do MySQL foi fixada propositalmente, eliminando qualquer ambiguidade sobre qual versão está rodando em cada ambiente — justamente o problema de divergência discutido no início deste capítulo. A chave `restart` define a política de reinicialização do contêiner em caso de falha (`no`, `on-failure`, `always` ou `unless-stopped`). A seção `environment` define variáveis de ambiente específicas da imagem — no caso do MySQL, o nome do banco, o usuário, a senha e a senha do usuário administrador (*root*).

A seção `ports` expõe uma porta do contêiner para a máquina hospedeira: no formato `"porta-externa:porta-interna"`, o exemplo expõe a porta 3306 do contêiner (padrão do MySQL) também como porta 3306 na máquina local — mas nada impede que se exponha, por exemplo, como porta 3307 externamente, caso a 3306 já esteja em uso por outro serviço.

Por fim, a seção `volumes` resolve um detalhe importante: por padrão, tudo o que ocorre dentro de um contêiner é volátil, ou seja, é perdido quando o contêiner é encerrado. Ao mapear um volume — nesse caso, associando o diretório `/var/lib/mysql` do contêiner a um volume nomeado `mydb` na máquina hospedeira — os dados persistem mesmo após o contêiner ser encerrado e reiniciado.

Dica

Em versões mais antigas do Docker Compose, era comum declarar, no topo do arquivo, uma chave `version` (por exemplo, `version: "3.8"`), indicando a versão da especificação do Compose sendo utilizada. Essa chave tornou-se obsoleta nas versões mais recentes do Docker e pode ser omitida — mas, como ferramentas de inteligência artificial treinadas com dados mais antigos ainda costumam sugeri-la, vale saber que ela não é mais necessária.

Para subir todos os contêineres definidos no arquivo, basta executar, no mesmo diretório do arquivo `docker-compose.yml`:

Docker

```
# Sobe todos os contêineres definidos no docker-compose.yml
docker compose up

# Encerra e remove os contêineres criados pelo compose
docker compose down
```

30.6 Visão geral da arquitetura do Docker

A arquitetura do Docker é composta por quatro elementos principais: o **cliente Docker**, utilizado para emitir comandos como `build`, `pull` e `run`; o **servidor Docker** (ou *daemon*), que aguarda solicitações via API REST do cliente para gerenciar imagens e contêineres; as **imagens**, que instruem o servidor sobre como criar um contêiner; e o **registry**, aplicação responsável por hospedar e distribuir imagens — podendo ser o Docker Hub público ou um registry privado mantido pela própria organização.

Vale destacar que o Docker não realiza limpeza automática de imagens não utilizadas: cabe ao próprio usuário remover imagens que não sejam mais necessárias, seja pela linha de comando, seja pelo Docker Desktop — interface gráfica que facilita bastante o dia a dia de quem prefere não memorizar todos os comandos apresentados neste capítulo.

30.7 Vantagens e desvantagens do Docker

Entre as principais vantagens do Docker, destacam-se:

- **Portabilidade:** um Dockerfile, um arquivo Docker Compose ou mesmo uma imagem isolada funcionam de forma idêntica independentemente do sistema operacional da máquina hospedeira, eliminando o problema de "na minha máquina funciona".
- **Automação:** combinado a ferramentas de automação (como *cron jobs*), o Docker permite replicar ambientes de desenvolvimento, homologação, staging e produção de forma consistente, sem configuração manual repetida.

- **Comunidade:** o Docker conta com uma comunidade extensa e madura, com canais dedicados, fóruns próprios e milhões de imagens disponíveis publicamente no Docker Hub.

Por outro lado, algumas desvantagens merecem consideração:

- **Desempenho:** embora um contêiner seja mais rápido que uma máquina virtual tradicional, ele ainda é mais lento do que executar uma aplicação nativamente em um servidor físico.
- **Curva de aprendizado:** o Docker não é trivial para quem não tem familiaridade com linha de comando, e cenários com múltiplos serviços dependentes entre si (como Kafka e Zookeeper) exigem uma orquestração cuidadosa, adicionando complexidade.
- **Segurança:** como o Docker é executado sobre o sistema operacional da máquina hospedeira, qualquer software malicioso escondido em uma imagem não oficial pode, em tese, comprometer a máquina host — reforçando a importância de utilizar apenas imagens confiáveis e, preferencialmente, oficiais.

Compreendidos esses conceitos, o leitor está preparado para utilizar o Docker como ferramenta de suporte ao desenvolvimento das próximas etapas deste livro — em especial, para executar bancos de dados de forma isolada e reproduzível, sem a necessidade de instalá-los diretamente na máquina local.

Síntese do Capítulo

- O Docker resolve o problema da divergência entre ambientes de desenvolvimento, homologação, staging e produção, empacotando aplicações e suas dependências em contêineres.
- Um contêiner é uma instância em execução de uma imagem; a imagem, por sua vez, é estática e definida por um Dockerfile.
- Comandos essenciais incluem `docker pull` (baixar imagem), `docker run` (executar contêiner), `docker images` e `docker ps` (listar imagens e contêineres) e `docker build` (gerar imagem a partir de um Dockerfile).
- O Docker Compose, definido em um arquivo YAML, permite orquestrar múltiplos contêineres simultaneamente, com configuração de portas, variáveis de ambiente e volumes persistentes.
- A arquitetura do Docker é composta por cliente, servidor (daemon), imagens e registry (como o Docker Hub).
- As principais vantagens do Docker são portabilidade, automação e uma comunidade madura; as desvantagens incluem desempenho inferior ao nativo, curva de aprendizado acentuada e riscos de segurança ao usar imagens não confiáveis.
- O domínio básico do Docker é um pré-requisito prático comum para trabalhar com bancos de dados e outras dependências de infraestrutura em projetos Spring Boot modernos.

Capítulo 31

Docker, Spring Data JPA: Entidades e Repositórios

Até aqui, a aplicação *Blue Velvet Music Store* sabia receber requisições HTTP, validar dados de entrada e devolver respostas — mas não sabia conversar com um banco de dados. Todo produto criado ou consultado existia apenas durante o tempo de vida do processo Java, na memória. Este capítulo fecha essa lacuna: o leitor vai subir um banco de dados MySQL em um contêiner Docker, conectar a aplicação Spring Boot a ele e criar o mapeamento entre tabelas relacionais e classes Java por meio do Spring Data JPA.

31.1 Subindo o banco de dados com Docker Compose

Em disciplinas de Banco de Dados é comum ter o MySQL instalado localmente, seja via XAMPP, seja via um pacote nativo do sistema operacional. Neste curso, entretanto, optou-se por não depender dessa instalação: o banco de dados é executado dentro de um contêiner Docker, descrito por um arquivo `docker-compose.yml` na raiz do projeto. Essa escolha tem uma vantagem prática importante: qualquer pessoa que clone o repositório consegue subir um ambiente de banco de dados idêntico ao do professor, sem precisar instalar nada além do Docker.

Docker Compose

O Docker Compose é uma ferramenta que permite descrever, em um único arquivo YAML, um ou mais serviços (contêineres) que devem ser executados em conjunto — por exemplo, um banco de dados, uma fila de mensagens e a própria aplicação. Em vez de executar comandos `docker run` longos e repetitivos, basta executar `docker compose up` e todos os serviços descritos sobem de forma coordenada.

O arquivo de composição para o banco de dados do projeto define um serviço chamado `database`, utilizando a imagem `mysql:latest`. Nesse serviço são configuradas quatro variáveis de ambiente essenciais: o nome do banco de dados a ser criado (`MYSQL_DATABASE`), o usuário de acesso (`MYSQL_USER`), a senha desse usuário (`MYSQL_PASSWORD`) e a senha do usuário administrador `root` (`MYSQL_ROOT_PASSWORD`). Além disso, a porta padrão do MySQL, 3306, é exposta para que a aplicação Java, rodando fora do contêiner, consiga se conectar a ela — e, caso essa porta já esteja em uso na máquina do leitor (por exemplo, por uma instalação local de MySQL), basta redirecioná-la para outra porta livre, como 3307.

Dois volumes são mapeados no serviço. O primeiro garante a **persistência** dos

dados: todo o conteúdo interno do contêiner é, por padrão, volátil — se o contêiner for removido, os dados são perdidos junto com ele, a menos que sejam gravados em um volume mapeado para fora do contêiner. O segundo volume aponta para o diretório especial `/docker-entrypoint-initdb.d/` dentro da imagem oficial do MySQL: qualquer script SQL colocado nesse diretório é executado automaticamente na primeira inicialização do banco.

Dica

Se você dividir a inicialização em múltiplos arquivos SQL dentro dessa pasta, lembre-se de que a execução respeita a ordem alfabética dos nomes de arquivo, não a ordem em que foram criados. Nomeie os scripts de criação de tabelas de forma que precedam, alfabeticamente, os scripts de inserção de dados — caso contrário, o banco tentará inserir registros em tabelas que ainda não existem.

31.2 O script de inicialização e o modelo de dados

O script `initial.sql`, colocado dentro de `src/main/resources/static`, contém os comandos `CREATE TABLE` e `INSERT` que estruturam o banco de dados do projeto e populam-no com alguns produtos de exemplo. A escolha de mantê-lo em `static` — e não em uma pasta específica de migrações, como faria uma ferramenta como o Flyway — é deliberadamente simples: o foco da disciplina é o desenvolvimento back-end com Spring, não a gestão avançada de versionamento de esquema de banco de dados.

O modelo relacional gira em torno de três tabelas. A tabela `product` concentra os atributos principais: identificador, nome, descrições curta e completa, marca, categoria, preço, desconto, disponibilidade em estoque, datas de criação e atualização, e custo. Seguindo os princípios de normalização estudados em disciplinas de Banco de Dados, duas tabelas dependentes foram extraídas: `box_dimension`, que guarda as dimensões físicas (largura, altura, comprimento, peso) da caixa usada para embalar o produto, e `product_detail`, que guarda pares de nome e valor associados ao produto — por exemplo, cada faixa de um CD, identificada por um nome e uma posição.

A relação entre `product` e `box_dimension` é de um-para-um: cada produto tem exatamente uma dimensão de caixa associada. Já a relação entre `product` e `product_detail` é de um-para-muitos: um único produto pode ter diversos detalhes (as várias faixas de um CD, por exemplo), mas cada detalhe pertence a um único produto.

31.3 Conectando o Spring Boot ao banco de dados

Ter um banco de dados em execução não conecta magicamente a aplicação a ele. São necessários dois passos adicionais: adicionar as dependências corretas e configurar as credenciais de acesso.

No arquivo de build (`build.gradle` ou `pom.xml`, dependendo da ferramenta escolhida), duas dependências precisam ser adicionadas. A primeira é o `starter` do Spring Data JPA, responsável por toda a infraestrutura de mapeamento objeto-relacional:

Java

```
implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
```

A segunda é o *driver* JDBC específico do banco de dados utilizado — no caso do MySQL, o conector oficial:

Java

```
implementation 'com.mysql:mysql-connector-j:9.1.0'
```

Com as dependências baixadas, a aplicação ainda falhará ao iniciar, pois o Spring Boot não sabe qual banco de dados acessar nem com qual credencial. Essas informações são fornecidas no arquivo `application.properties` (ou `application.yml`):

Java

```
spring.datasource.username=rei
spring.datasource.password=rei-pass
spring.datasource.url=jdbc:mysql://localhost:3306/db
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

A estrutura da URL — `jdbc:mysql://host:porta/nome_do_banco` — é o padrão de qualquer conexão JDBC ao MySQL; ao usar outro SGBD, tanto o prefixo quanto o nome da classe do driver mudam (por exemplo, PostgreSQL e MariaDB possuem seus próprios drivers e prefixos). Com apenas essas cinco linhas de configuração e as duas dependências declaradas, o Spring Boot passa a abrir e fechar sozinho as conexões com o banco — o desenvolvedor não precisa escrever nenhum código de baixo nível para isso.

31.4 Mapeando tabelas como entidades JPA

Uma **entidade** JPA é a representação, em uma classe Java, de uma tabela do banco de dados. Cada atributo da classe corresponde a uma coluna da tabela, e o Spring Data JPA se encarrega de converter tipos — um `INT` do banco vira um tipo numérico Java (geralmente `Long`, mesmo quando a coluna é um inteiro simples), um `VARCHAR` vira `String`, um `DECIMAL` vira `BigDecimal`, e assim por diante.

Entidade JPA

Uma entidade é uma classe Java anotada com `@Entity`, que informa ao Spring que aquela classe deve ser tratada como uma tabela do banco de dados. Toda entidade precisa declarar um identificador único, marcado com `@Id`, que corresponde à chave primária da tabela.

O exemplo a seguir mostra a entidade `Product`, com as anotações principais explicadas em seguida:

Java

```
package com.bluevelvet.api.infrastructure.persistence.entity;

import jakarta.persistence.*;
import lombok.*;

import java.io.Serializable;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
@Table(schema = "db", name = "product")
public class Product implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    private String name;

    @Column(name = "short_description")
    private String shortDescription;

    @Column(name = "full_description")
    private String fullDescription;

    private String brand;

    private String category;

    @Column(name = "list_price")
    private BigDecimal listPrice;

    private BigDecimal discount;

    @Column(name = "in_stock")
    private Boolean inStock;

    @Column(name = "creation_time")
    private LocalDateTime creationTime;

    @Column(name = "update_time")
    private LocalDateTime updateTime;
```

```
private BigDecimal cost;
}
```

Repare no papel de cada anotação. `@Table` indica o esquema (`db`) e o nome da tabela (`product`) a que a classe corresponde — útil sobretudo quando o nome da classe Java difere do nome da tabela. `@Id` marca o atributo que representa a chave primária; sem essa anotação, o Spring recusa a classe como entidade válida. `@GeneratedValue` define como o valor do identificador é gerado: a estratégia `AUTO` delega a geração ao próprio banco de dados, apropriada quando a coluna foi criada como `AUTO_INCREMENT` no MySQL (outros bancos, como o PostgreSQL, costumam usar a estratégia `IDENTITY` ou `SEQUENCE`).

Por fim, `@Column` resolve um problema de convenção: o banco de dados usa *snake_case* (`short_description`), enquanto o Java usa *camelCase* (`shortDescription`). Sempre que o nome do atributo Java, convertido para *snake_case*, coincidir exatamente com o nome da coluna, a anotação `@Column` é dispensável — é o caso de `brand` e `category` no exemplo acima.

31.5 Relacionamentos entre entidades

As tabelas `box_dimension` e `product_detail` possuem uma chave estrangeira apontando para `product`. Esse relacionamento pode ser expresso em Java por meio das anotações `@OneToOne`, `@OneToMany`, `@ManyToOne` e `@ManyToMany`, que instruem o JPA a realizar os *joins* necessários automaticamente, sem que o desenvolvedor escreva SQL manualmente.

Do lado de `BoxDimension`, o relacionamento com `Product` é declarado como muitos-para-um (na prática, um-para-um) usando `@JoinColumn` para indicar qual coluna local guarda a chave estrangeira:

```
Java
@Entity
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
@Table(schema = "db", name = "box_dimension")
public class BoxDimension implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    private Float width;
    private Float length;
    private Float height;
    private Float weight;
```

```
@OneToOne
@JoinColumn(name = "product", referencedColumnName = "id")
private Product product;
}
```

Do lado de `Product`, em vez de repetir a chave estrangeira, usa-se o atributo `mappedBy`, que indica qual atributo da outra entidade já é responsável pelo relacionamento:

```
Java

@OneToOne(mappedBy = "product")
private BoxDimension boxDimension;

@OneToMany(mappedBy = "product")
private List<ProductDetail> productDetails;
```

Assim, ao buscar um produto, o Spring Data JPA consegue trazer junto, de forma automática, tanto a sua dimensão de caixa quanto a lista de detalhes associados — sem que o desenvolvedor precise escrever nenhuma consulta SQL com JOIN manualmente.

31.6 O repositório: acesso a dados sem SQL manual

Com as entidades definidas, o próximo passo é criar um **repositório**: uma interface responsável por mediar o acesso da aplicação ao banco de dados.

Repositório JPA

Um repositório é uma interface anotada com `@Repository` que estende `JpaRepository<T, ID>`, onde `T` é o tipo da entidade gerenciada e `ID` é o tipo do seu identificador. Ao estendê-la, a interface ganha automaticamente métodos prontos como `findAll()`, `findById(ID)`, `save(T)` e `deleteById(ID)`, sem que uma única linha de implementação precise ser escrita.

```
Java

package com.bluevelvet.api.infrastructure.persistence.repository;

import com.bluevelvet.api.infrastructure.persistence.entity.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {
}
```

Apenas com essa declaração, `ProductRepository` já oferece consultas básicas de CRUD (*Create, Read, Update, Delete*). Um detalhe merece atenção: o método `findById` não retorna um `Product` diretamente, mas um `Optional<Product>`.

Optional

`Optional<T>` é um tipo genérico que representa um valor que pode existir ou não. Uma busca por identificador pode ou não encontrar um registro correspondente no banco; em vez de retornar `null` — fonte clássica de `NullPointerException` — o JPA retorna um `Optional`, obrigando o código chamador a tratar explicitamente os dois casos: valor presente ou ausente.

Vale reforçar uma boa prática de arquitetura, mesmo que ela ainda não tenha sido implementada neste ponto do curso: o controlador **não deve** injetar o repositório diretamente. Do ponto de vista do controlador, é irrelevante se os dados vêm de um MySQL, um PostgreSQL, um MongoDB ou um arquivo texto — essa responsabilidade pertence à camada de serviço, que será apresentada no próximo capítulo. Usar o repositório dentro do controlador, como foi feito apenas para fins de teste e depuração nesta etapa, é uma prática que deve ser evitada em código de produção.

Síntese do Capítulo

- O Docker Compose permite descrever e subir um banco de dados MySQL em contêiner, com variáveis de ambiente para nome do banco, usuário e senhas.
- Volumes Docker garantem a persistência dos dados e permitem a execução automática de scripts SQL de inicialização, respeitando ordem alfabética.
- Conectar o Spring Boot ao banco exige duas dependências (`spring-boot-starter-data-jpa` e o driver JDBC) e cinco propriedades de configuração (usuário, senha, URL e driver).
- Uma entidade JPA é uma classe anotada com `@Entity` e `@Table`, com um atributo marcado como `@Id` e, opcionalmente, `@GeneratedValue` para geração automática de chave primária.
- A anotação `@Column` resolve divergências de nome entre o *snake_case* do banco e o *camelCase* do Java.
- Relacionamentos entre entidades (`@OneToOne`, `@OneToMany`) evitam a necessidade de escrever *joins* SQL manualmente.
- Um repositório que estende `JpaRepository<T, ID>` já oferece métodos prontos de CRUD, e o controlador nunca deve acessá-lo diretamente em uma arquitetura bem estruturada.

Capítulo 32

Camada de Serviço e Tratamento de Exceções

O capítulo anterior deixou uma pergunta em aberto: se o controlador não deve conhecer o repositório diretamente, quem faz a ponte entre a requisição HTTP e o banco de dados? A resposta é a **camada de serviço**. Este capítulo apresenta seu papel na arquitetura de uma aplicação Spring Boot e, em seguida, discute como tratar situações excepcionais — um produto que não existe, uma requisição malformada — de forma organizada e consistente, devolvendo ao cliente da API respostas HTTP claras em vez de erros genéricos ou, pior, telas de erro internas do servidor.

32.1 Por que separar regras de negócio do controlador

Até este ponto, o `ProductController` foi responsável por receber requisições, e, apenas para fins de teste, chegou a acessar diretamente o `ProductRepository`. Essa mistura de responsabilidades é problemática por diversos motivos, e vale a pena enumerá-los com cuidado.

Em primeiro lugar, o controlador é a camada de entrada da aplicação: ele lida com o protocolo HTTP — verbos, códigos de status, cabeçalhos, corpo da requisição. Ele não deveria precisar saber se os dados persistem em um MySQL, em um PostgreSQL ou em um serviço remoto. Se o repositório aparece diretamente no controlador, qualquer mudança na forma de acessar os dados obriga a alterar também a camada exposta publicamente pela API — um acoplamento desnecessário.

Em segundo lugar, regras de negócio — validações, cálculos, decisões sobre o que fazer quando um dado não é encontrado — não são responsabilidade do controlador nem do repositório. O controlador apenas encaminha a requisição e devolve a resposta; o repositório apenas executa consultas e comandos no banco. É a camada de serviço que concentra a lógica que dá sentido de negócio à aplicação: por exemplo, decidir que, ao criar um produto, o nome deve ser único, ou que um desconto não pode ser maior do que o preço de lista.

Camada de Serviço

A camada de serviço é o conjunto de classes — geralmente anotadas com `@Service` — responsáveis por implementar as regras de negócio de uma aplicação. Ela fica posicionada entre o controlador (que lida com HTTP) e o repositório (que lida com persistência), orquestrando chamadas a um ou mais repositórios, aplicando validações e lançando exceções quando uma regra de negócio é violada.

Essa separação em camadas — controlador, serviço, repositório — é uma das formas mais comuns de organizar uma aplicação back-end e aparece, com pequenas variações de nome, em praticamente qualquer material sobre arquitetura de software corporativo. Ela traz benefícios concretos: cada camada pode ser testada isoladamente (é possível testar o serviço com um repositório falso, sem subir um banco de dados de verdade), cada camada pode ser substituída sem afetar as demais (trocar o banco de dados relacional por um NoSQL não deveria exigir reescrever o controlador), e o código fica mais fácil de entender, pois cada classe tem uma responsabilidade clara e única.

32.2 Criando a camada de serviço

Uma classe de serviço, no ecossistema Spring, é uma classe comum anotada com `@Service` — uma especialização de `@Component` que sinaliza ao Spring que aquela classe deve ser gerenciada pelo contêiner de injeção de dependência e disponibilizada para ser injetada em outras classes, como os controladores.

Java

```
package com.bluevelvet.api.service;

import com.bluevelvet.api.dto.ProductRequest;
import com.bluevelvet.api.dto.ProductResponse;
import com.bluevelvet.api.exception.ProductNotFoundException;
import com.bluevelvet.api.infrastructure.persistence.entity.Product;
import com.bluevelvet.api.infrastructure.persistence.repository.ProductRepository;
import org.springframework.stereotype.Service;

@Service
public class ProductService {

    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    public ProductResponse findById(Long id) {
        Product product = productRepository.findById(id)
            .orElseThrow(() -> new ProductNotFoundException(id));
        return ProductResponse.fromEntity(product);
    }

    public ProductResponse createProduct(ProductRequest request) {
        Product product = request.toEntity();
        Product saved = productRepository.save(product);
        return ProductResponse.fromEntity(saved);
    }
}
```

Note que é o serviço — e não o controlador — quem recebe o `ProductRepository` injetado, quem resolve o `Optional` retornado por `findById`, e quem decide o que fazer quando o produto não é encontrado: lançar uma exceção específica, em vez de devolver `null` ou deixar o `Optional` vaziar para fora da camada de serviço.

32.2.1 Injeção de dependência: por que preferir o construtor

O Spring oferece três formas de injetar uma dependência em uma classe: por campo (usando `@Autowired` diretamente sobre o atributo), por *setter*, ou por construtor. A injeção por campo é a mais compacta de escrever, mas é desaconselhada — inclusive sinalizada como prática não recomendada por IDEs modernas — porque dificulta a criação de testes unitários (não há como fornecer um repositório falso sem usar reflexão) e porque esconde dependências obrigatórias que deveriam ser explícitas na assinatura da classe.

Java

```
// Evitar: injeção por campo
@Service
public class ProductService {
    @Autowired
    private ProductRepository productRepository;
}

// Preferir: injeção por construtor
@Service
public class ProductService {
    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }
}
```

A injeção por construtor tem duas vantagens práticas importantes. A primeira é permitir que o atributo seja declarado `final`, deixando explícito que aquela dependência é obrigatória e não pode ser reatribuída depois da construção do objeto. A segunda é facilitar testes: em um teste unitário, basta instanciar o serviço passando um repositório de teste (ou um *mock*) diretamente pelo construtor, sem depender do contêiner do Spring. Quando uma classe possui múltiplos atributos `final`, o Lombok oferece a anotação `@RequiredArgsConstructor`, que gera automaticamente um construtor recebendo exatamente os atributos marcados como `final` — economizando a escrita manual do construtor mostrado acima.

32.3 Tratamento de exceções: o problema

Regras de negócio falham. Um cliente pode pedir o produto de identificador 9999, que não existe; pode enviar um preço negativo; pode tentar cadastrar um nome de produto que já existe no catálogo. Sem tratamento adequado, esses cenários resultam

em exceções não capturadas que sobem até o servidor, e o cliente da API recebe uma resposta HTTP 500 (*Internal Server Error*) — o pior tipo de resposta possível, pois não indica o que realmente deu errado nem se o erro foi causado pelo próprio cliente.

O ideal é que cada tipo de falha resulte em um código de status HTTP apropriado: 404 **Not Found** quando o recurso solicitado não existe, 400 **Bad Request** quando os dados enviados pelo cliente são inválidos, 409 **Conflict** quando há uma violação de regra de unicidade, e assim por diante.

32.3.1 Exceções customizadas

O primeiro passo é criar exceções específicas para o domínio da aplicação, em vez de lançar exceções genéricas do Java como `RuntimeException`. Uma exceção customizada carrega, no próprio nome e nos dados que armazena, informação suficiente para que quem a captura saiba exatamente o que aconteceu.

Java

```
package com.bluevelvet.api.exception;

public class ProductNotFoundException extends RuntimeException {

    public ProductNotFoundException(Long id) {
        super("Produto com id " + id + " nao foi encontrado.");
    }
}
```

Estender `RuntimeException` (em vez de `Exception`) é uma escolha deliberada: exceções que estendem `RuntimeException` são *unchecked*, ou seja, não obrigam todo método da cadeia de chamadas a declarar `throws` ou a envolver a chamada em um bloco `try/catch`. Isso mantém o código do serviço e do controlador mais limpo, deixando o tratamento centralizado em um único lugar — que veremos a seguir.

32.4 Tratamento centralizado com `@ControllerAdvice`

Uma abordagem possível é usar `try/catch` diretamente dentro de cada método do controlador:

Java

```
@GetMapping("/{id}")
public ResponseEntity<ProductResponse> getById(@PathVariable Long id) {
    try {
        ProductResponse response = productService.findById(id);
        return ResponseEntity.ok(response);
    } catch (ProductNotFoundException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
    }
}
```

Essa solução funciona, mas não escala: se a API tiver dezenas de *endpoints*, o mesmo bloco `try/catch` — capturando os mesmos tipos de exceção e traduzindo-os para os mesmos códigos de status — se repetiria em cada método, violando o princípio de não repetir código (*DRY, Don't Repeat Yourself*). O Spring resolve esse problema com a anotação `@ControllerAdvice` combinada a `@ExceptionHandler`, que permite centralizar o tratamento de exceções de toda a aplicação em uma única classe.

@ControllerAdvice e @ExceptionHandler

`@ControllerAdvice` marca uma classe como um componente global de apoio a controladores, capaz de interceptar exceções lançadas por qualquer controlador da aplicação. Dentro dela, cada método anotado com `@ExceptionHandler(TipoDaExcecao.class)` define como uma exceção específica deve ser convertida em uma resposta HTTP.

Java

```
package com.bluevelvet.api.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ProductNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ProductNotFoundException ex) {
        ErrorResponse body = new ErrorResponse(
            HttpStatus.NOT_FOUND.value(),
            ex.getMessage(),
            LocalDateTime.now()
        );
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(body);
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<ErrorResponse> handleBadRequest(IllegalArgumentException ex) {
        ErrorResponse body = new ErrorResponse(
            HttpStatus.BAD_REQUEST.value(),
            ex.getMessage(),
            LocalDateTime.now()
        );
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(body);
    }

    @ExceptionHandler(Exception.class)
```

```
public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
    ErrorResponse body = new ErrorResponse(
        HttpStatus.INTERNAL_SERVER_ERROR.value(),
        "Erro interno inesperado.",
        LocalDateTime.now()
    );
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(body);
}
```

A anotação `@RestControllerAdvice` é equivalente a `@ControllerAdvice` combinada com `@ResponseBody`, garantindo que o objeto retornado por cada método seja automaticamente serializado como JSON no corpo da resposta. A classe `ErrorResponse`, usada acima, é um simples objeto de transferência de dados (DTO) contendo o código de status, uma mensagem descritiva e o instante em que o erro ocorreu — útil tanto para quem consome a API quanto para depuração e registro de logs.

Com essa estrutura, o controlador volta a ficar limpo, livre de blocos `try/catch`: ele apenas delega ao serviço, e, caso uma exceção seja lançada, o `@ControllerAdvice` intercepta-a automaticamente e devolve a resposta HTTP apropriada.

Java

```
@RestController
@RequestMapping("/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<ProductResponse> getById(@PathVariable Long id) {
        return ResponseEntity.ok(productService.findById(id));
    }
}
```

Dica

Ao definir suas próprias exceções, considere criar uma hierarquia — por exemplo, uma `BusinessException` genérica da qual `ProductNotFoundException` e outras exceções de domínio herdam. Isso permite, quando fizer sentido, capturar toda uma família de erros de negócio com um único `@ExceptionHandler`, sem perder a granularidade de mensagens específicas para cada caso.

32.5 Validação de entrada

Além de tratar exceções depois que elas ocorrem, é possível *prevenir* boa parte delas validando os dados de entrada antes que cheguem à camada de serviço. O Spring integra-se facilmente à especificação *Bean Validation*, permitindo anotar os campos de um DTO de requisição com restrições como `@NotBlank`, `@NotNull`, `@Positive` ou `@Size`, e então acionar essa validação automaticamente com `@Valid` no parâmetro do controlador. Quando uma validação falha, o Spring lança uma `MethodArgumentNotValidException`, que pode ser capturada no mesmo `@ControllerAdvice` e traduzida para uma resposta 400 Bad Request detalhando quais campos são inválidos e por quê.

Síntese do Capítulo

- A camada de serviço, marcada com `@Service`, concentra as regras de negócio e fica posicionada entre o controlador e o repositório.
- Separar controlador, serviço e repositório reduz acoplamento, facilita testes isolados e torna cada camada substituível sem afetar as demais.
- Injeção de dependência por construtor é preferível à injeção por campo: permite atributos `final` e facilita a criação de testes unitários.
- Exceções customizadas, estendendo `RuntimeException`, comunicam de forma explícita o que deu errado em um ponto específico do domínio.
- `@ControllerAdvice` (ou `@RestControllerAdvice`) combinado a `@ExceptionHandler` centraliza o tratamento de exceções de toda a aplicação em um único lugar, evitando repetição de `try/catch` em cada *endpoint*.
- Cada tipo de erro deve ser traduzido para o código de status HTTP correto — 404 para recurso inexistente, 400 para requisição inválida, entre outros.
- A validação de entrada com anotações de *Bean Validation* (`@NotBlank`, `@Positive` etc.) previne erros antes mesmo de chegarem à camada de serviço.

Capítulo 33

Introdução ao Thymeleaf

Até este ponto do curso, a aplicação *Blue Velvet Music Store* expõe apenas uma API REST: os dados trafegam em JSON, e é responsabilidade de outro sistema — um front-end separado, escrito em JavaScript, ou até uma ferramenta como o Postman — consumir essas respostas e apresentá-las visualmente. Este capítulo introduz uma alternativa diferente: gerar páginas HTML diretamente no servidor, com dados dinâmicos já embutidos, usando o motor de templates Thymeleaf.

33.1 O que é o Thymeleaf

Thymeleaf

Thymeleaf é um motor de templates Java para aplicações web, que permite criar páginas HTML dinâmicas processadas no servidor. Diferentemente de outros motores de template, o Thymeleaf trabalha sobre HTML puro: um arquivo de template Thymeleaf é um HTML válido, que pode inclusive ser aberto diretamente em um navegador (exibindo valores estáticos ou vazios), mas que ganha atributos especiais — prefixados por `th:` — reconhecidos e processados pelo servidor no momento em que a página é renderizada.

Essa característica é chamada de *natural templating*: como o Thymeleaf usa atributos HTML válidos (por exemplo, `th:text` em vez de uma sintaxe de chaves não reconhecida pelo navegador), designers e desenvolvedores conseguem visualizar o layout do template mesmo sem executar o servidor Spring. No momento em que a aplicação processa o template, o Spring substitui esses atributos por conteúdo real, vindo de objetos Java disponibilizados pelo controlador.

O Thymeleaf é um projeto extenso, com dezenas de recursos — fragmentos reutilizáveis, internacionalização, integração com formulários complexos, entre outros. Este capítulo cobre o essencial para que o leitor entenda como ele funciona e consiga, a partir daqui, aprofundar-se de forma autônoma sempre que precisar de um recurso específico.

33.2 Configurando o projeto para usar Thymeleaf

O primeiro passo é adicionar a dependência do Thymeleaf ao arquivo de build do projeto:

Java

```
implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'
```


Com a dependência baixada, o Spring Boot já reconhece automaticamente dois diretórios especiais dentro de `src/main/resources`: `templates`, onde ficam os arquivos HTML processados pelo Thymeleaf, e `static`, onde ficam arquivos que não mudam em tempo de execução — folhas de estilo CSS, imagens, scripts JavaScript client-side. Essa separação é proposital: o CSS de uma página não é gerado dinamicamente a cada requisição, então ele pertence a `static`, e não a `templates`.

Para verificar que a configuração está correta, cria-se um arquivo simples, `index.html`, dentro de `templates`, e um controlador dedicado a servir páginas HTML — diferente do controlador REST já existente, que continua respondendo em JSON:

HTML + Thymeleaf

```
<!DOCTYPE html>
<html>
<head>
  <title>Blue Velvet Music Store</title>
</head>
<body>
  <h1>Blue Velvet Music Store</h1>
</body>
</html>
```

Java

```
package com.bluevelvet.web.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class ThymeleafController {

    @GetMapping("/")
    public String getHomePage() {
        return "index";
    }
}
```

Dois detalhes merecem atenção aqui. Primeiro, a anotação usada é `@Controller`, e não `@RestController`: enquanto `@RestController` serializa automaticamente o retorno do método como corpo da resposta (por exemplo, JSON), `@Controller` espera que o valor retornado seja o **nome de uma view** — neste caso, `"index"`, que o Thymeleaf resolve para o arquivo `templates/index.html`. Segundo, apenas colocar um arquivo HTML dentro de `templates` não o torna acessível: é preciso criar uma rota no controlador que devolva o nome desse template como resposta a uma requisição GET.

33.3 Expressões do Thymeleaf

O Thymeleaf define diferentes tipos de expressão, cada uma identificada por um símbolo específico antes dos colchetes. As três mais usadas no dia a dia são:

- **Expressões de variável**, escritas como `${...}`: acessam um objeto disponibilizado pelo controlador, como um produto vindo do banco de dados.
- **Expressões de mensagem**, escritas como `#{...}`: exibem um texto pronto, tipicamente vindo de um arquivo de internacionalização.
- **Expressões de link**, escritas como `@{...}`: constroem URLs relativas ao contexto da aplicação, usadas por exemplo para referenciar arquivos estáticos ou outras rotas.

33.3.1 Exibindo texto com `th:text`

O atributo `th:text` substitui o conteúdo textual de uma tag HTML pelo valor de uma expressão Thymeleaf. Considere que o controlador disponibiliza um objeto chamado `product` para a view — assunto detalhado na próxima seção — contendo os atributos `name`, `brand` e `listPrice`:

HTML + Thymeleaf

```
<div th:object="${product}">
  <h2 th:text="*{name}">Nome do Produto</h2>
  <p th:text="*{brand}">Marca</p>
  <p th:text="| $ ${product.listPrice} |">Preço</p>
</div>
```

Repare em dois recursos combinados nesse trecho. O atributo `th:object`, aplicado à `div` externa, declara que, dentro dela, o objeto de referência é o `product` vindo do modelo — isso permite usar o asterisco (`*{...}`) como atalho para `${product...}` nos elementos internos: `*{name}` é equivalente a `${product.name}`. Já o último parágrafo mostra como concatenar texto literal (o símbolo de cifrão) com uma expressão de variável: como o cifrão é um caractere reservado do Thymeleaf para expressões de variável, ele não pode ser escrito diretamente antes de uma expressão — é necessário envolver todo o conteúdo entre barras verticais (`|...|`), que sinalizam uma expressão de concatenação de texto.

33.3.2 Iterando sobre coleções com `th:each`

Quando um atributo do modelo é uma coleção — por exemplo, a lista de faixas de um CD — usa-se o atributo `th:each`, equivalente a um laço `for-each` do Java:

HTML + Thymeleaf

```
<ul>
  <li th:each="detail : ${product.productDetails}">
    <span th:text="${detail.name}">Faixa</span>
    -
  </li>
</ul>
```

```
<span th:text="${detail.value}">01</span>
</li>
</ul>
```

Aqui, `detail` é uma variável criada pelo próprio `th:each`, representando, a cada iteração, um elemento da lista `product.productDetails`. Se a lista tiver quinze elementos — como as quinze faixas de um álbum —, o `<li>` inteiro é repetido quinze vezes, uma para cada item; se a lista estiver vazia, nada é exibido, sem necessidade de verificação manual.

33.3.3 Condicionais com `th:if`

O atributo `th:if` exibe um elemento apenas quando a expressão associada é avaliada como verdadeira; caso contrário, o elemento é omitido inteiramente do HTML final enviado ao navegador:

HTML + Thymeleaf

```
<span th:if="${product.inStock}">Disponível em estoque</span>
<span th:unless="${product.inStock}">Fora de estoque</span>
```

O Thymeleaf também oferece o atributo complementar `th:unless`, que exibe o elemento exatamente quando a condição é falsa — útil para pares de mensagens mutuamente exclusivas, como no exemplo acima. Para verificações que evitem uma `NullPointerException` ao acessar um atributo de um objeto que pode ser nulo, o Thymeleaf aceita o operador de segurança `?:` uma expressão como `${product.productDetail?.name}` simplesmente não avalia `.name` caso `productDetail` seja nulo, em vez de lançar uma exceção.

33.4 Referenciando arquivos estáticos com expressões de link

Como vimos, arquivos CSS pertencem ao diretório `static`, e não a `templates`. Para referenciar um arquivo estático dentro de um template, usa-se uma expressão de link (`@{...}`) no atributo `th:href`, em vez do atributo comum `href`:

HTML + Thymeleaf

```
<head>
  <link rel="stylesheet" th:href="@{/css/styles.css}">
</head>
```

O caminho `/css/styles.css` é resolvido a partir da raiz de `static`: um arquivo salvo em `src/main/resources/static/css/styles.css` é acessado, em tempo de execução, por esse caminho relativo. Usar `th:href` em vez de um `href` comum garante que o Thymeleaf resolva corretamente o caminho considerando o contexto da aplicação (por exemplo, se ela estiver publicada sob um subdiretório específico do servidor).

33.5 Passando dados do controlador para a view com Model

Os exemplos anteriores pressupõem a existência de um objeto `product` disponível para o template. Essa disponibilização é feita pelo controlador, por meio da classe `Model` do Spring, injetada como parâmetro do método:

Java

```
@Controller
@RequestMapping("/product")
public class ThymeleafController {

    private final ProductService productService;

    public ThymeleafController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping("/{id}")
    public String viewProduct(@PathVariable Long id, Model model) {
        ProductResponse response = productService.findById(id);
        model.addAttribute("product", response);
        return "view-product";
    }
}
```

O método `model.addAttribute("product", response)` associa, ao modelo da view, uma variável chamada `product` cujo conteúdo é o objeto `response`. É exatamente esse nome — `product` — que será usado dentro do template, tanto em `th:object="${product}"` quanto em qualquer outra expressão de variável que o referencie. Caso o produto não seja encontrado, a camada de serviço lança a exceção customizada apresentada no capítulo anterior, que pode ser tratada pelo mesmo mecanismo de `@ControllerAdvice` já estudado.

Um detalhe importante de arquitetura aparece aqui: o `ThymeleafController` não pode reaproveitar diretamente o `@PostMapping` já existente no controlador REST para criar produtos, porque aquele *endpoint* espera um corpo de requisição em JSON (via `@RequestBody`), e o Thymeleaf, ao submeter um formulário HTML, não converte automaticamente os campos preenchidos em JSON. Por essa razão, é necessário criar uma rota específica para o formulário, usando `@ModelAttribute` para vincular os campos do formulário a um objeto de requisição:

HTML + Thymeleaf

```
<form th:action="@{/product}" th:object="${product}" method="post">
    <input type="text" th:field="*{name}">
    <input type="text" th:field="*{brand}">
    <button type="submit">Adicionar produto</button>
</form>
```

Java

```
@PostMapping("/product")
public String createProductForm(@ModelAttribute ProductRequest request) {
    ProductResponse response = productService.createProduct(request);
    return "redirect:/product/" + response.getId();
}
```

O atributo `th:field` substitui o `name` e o `value` tradicionais de um campo de formulário HTML, vinculando-o automaticamente ao atributo correspondente do objeto declarado em `th:object`. Após criar o produto com sucesso, o controlador devolve uma string prefixada com `redirect:`, instruindo o navegador a realizar uma nova requisição GET para a página do produto recém-criado — em vez de simplesmente re-exibir o formulário, seguindo o padrão conhecido como *Post/Redirect/Get*, que evita o reenvio acidental do formulário caso o usuário atualize a página.

Dica

O Thymeleaf é um projeto extenso e este capítulo cobriu apenas o essencial para o dia a dia — `th:text`, `th:each`, `th:if/th:unless`, expressões de link e vínculo com formulários. Recursos mais avançados, como fragmentos reutilizáveis (`th:fragment` e `th:insert`) e internacionalização de mensagens, valem a pena ser explorados na documentação oficial sempre que o projeto exigir.

Síntese do Capítulo

- O Thymeleaf é um motor de templates que processa HTML puro no servidor, usando atributos `th:*` reconhecidos apenas em tempo de execução Spring.
- Controladores que retornam páginas HTML usam `@Controller` (não `@RestController`) e devolvem o nome do template como string.
- Templates ficam em `src/main/resources/templates`; arquivos estáticos como CSS ficam em `src/main/resources/static`.
- Existem três tipos principais de expressão: variável (`${...}`), mensagem (`#{...}`) e link (`@{...}`), cada uma usada em um contexto diferente.
- `th:text` exibe valores dinâmicos, `th:each` itera sobre coleções e `th:if/th:unless` controlam a exibição condicional de elementos.
- A classe `Model`, injetada no controlador, é o mecanismo usado para disponibilizar objetos Java (como um produto vindo do serviço) para o template.
- Formulários HTML processados por Thymeleaf exigem um *endpoint* próprio, distinto da API REST em JSON, usando `@ModelAttribute` e o padrão *Post/Redirect/Get*.

Capítulo 34

Spring Security: Fundamentos de Autenticação e Autorização

Ao longo dos capítulos anteriores, a aplicação *Blue Velvet Music Store* ganhou persistência de dados, uma camada de serviço organizada e páginas web renderizadas com Thymeleaf. Um aspecto, no entanto, ainda não foi tratado: qualquer pessoa que conheça o endereço da aplicação pode acessar qualquer *endpoint*, inclusive aqueles que criam ou apagam produtos. Este capítulo, o último antes das considerações finais, introduz o Spring Security — o módulo do ecossistema Spring responsável por proteger aplicações web contra acesso não autorizado — e fecha o percurso de desenvolvimento back-end com uma reflexão sobre por que segurança nunca deveria ser um item opcional em um projeto real.

34.1 Autenticação e autorização: duas perguntas diferentes

É comum tratar autenticação e autorização como sinônimos, mas são conceitos distintos, e a diferença entre eles orienta boa parte do desenho de segurança de qualquer aplicação.

Autenticação

Autenticação é o processo de verificar **quem é** o usuário que está fazendo uma requisição. Tipicamente envolve a apresentação de credenciais — um par usuário/senha, um token, um certificado — que o sistema confere contra uma fonte confiável de identidade (um banco de dados de usuários, um provedor externo, etc.).

Autorização

Autorização é o processo de verificar **o que** um usuário já autenticado tem permissão para fazer. Mesmo depois de confirmada a identidade de alguém, ainda é preciso decidir se essa pessoa pode, por exemplo, apenas visualizar produtos, ou também criar, editar e remover produtos do catálogo.

Um exemplo concreto ajuda a fixar a diferença. Imagine dois usuários da Blue Velvet Music Store: um cliente comum e um administrador do catálogo. Ambos passam pelo mesmo processo de autenticação — inserem usuário e senha, e o sistema confirma que ambos são, de fato, quem dizem ser. A partir daí, porém, a autorização diverge: o cliente comum pode ter permissão apenas para consultar produtos (GET

/products), enquanto o administrador tem permissão adicional para criar, atualizar e remover produtos (POST, PUT, DELETE). Autenticar sem autorizar corretamente é como verificar a identidade de alguém na portaria de um prédio e, em seguida, deixá-lo entrar em qualquer sala, incluindo o cofre.

34.2 O filtro de segurança: onde a proteção acontece

O Spring Security se integra à aplicação por meio de um mecanismo chamado **filtro de segurança** (*security filter chain*). Antes que uma requisição HTTP alcance o controlador que ela pretende acessar, ela atravessa uma cadeia de filtros responsáveis por diferentes verificações de segurança: extrair e validar credenciais, checar se o usuário autenticado tem a permissão necessária para aquele recurso, e, caso alguma dessas checagens falhe, interromper a requisição antes mesmo que ela chegue ao código de negócio da aplicação.

O diagrama a seguir ilustra esse fluxo básico: uma requisição chega à aplicação, passa pelo filtro de segurança, que decide se ela é autenticada e autorizada; em caso positivo, a requisição segue até o controlador; em caso negativo, o filtro já responde com um código de status apropriado (401 Unauthorized ou 403 Forbidden), sem que o controlador chegue a ser executado.

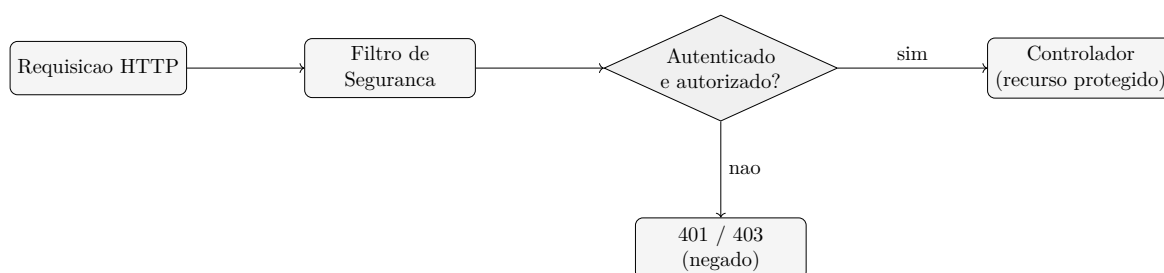


Figura 34.1: Fluxo simplificado de uma requisição através do filtro de segurança do Spring Security.

Esse desenho tem uma implicação importante: a lógica de segurança fica desacoplada da lógica de negócio. O controlador e o serviço de produtos não precisam saber nada sobre como a autenticação é feita — essa responsabilidade é inteiramente do filtro de segurança, configurado de forma centralizada.

34.3 Configurando o Spring Security

Ao adicionar a dependência `spring-boot-starter-security` ao projeto, o Spring Security já entra em ação com uma configuração padrão extremamente restritiva: todos os *endpoints* passam a exigir autenticação, e uma senha aleatória é gerada e impressa no console a cada inicialização da aplicação, associada a um usuário padrão chamado **user**. Esse comportamento é proposital — o framework prefere começar "fechado" e obrigar o desenvolvedor a abrir explicitamente o que deve ser público, em vez do contrário.

Java

```
implementation 'org.springframework.boot:spring-boot-starter-security'
```

A configuração customizada é feita por meio de uma classe anotada com `@Configuration`, que declara um *bean* do tipo `SecurityFilterChain`. É nessa classe que se define, entre outras coisas, quais rotas exigem autenticação, quais são públicas, e qual mecanismo de autenticação é usado.

Java

```
package com.bluevelvet.api.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/products/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .httpBasic(basic -> {})
            .csrf(csrf -> csrf.disable());

        return http.build();
    }
}
```

Nessa configuração, a primeira regra libera o acesso público de leitura ao catálogo de produtos (rotas `/products/**`), a segunda exige que o usuário autenticado possua o papel `ADMIN` para acessar rotas administrativas (`/admin/**`), e a terceira, `anyRequest().authenticated()`, exige no mínimo autenticação para qualquer outra rota não listada explicitamente. A ordem das regras importa: o Spring Security avalia os `requestMatchers` na sequência em que aparecem, aplicando a primeira regra compatível com a rota da requisição.

34.4 Autenticação básica versus autenticação com token

O exemplo anterior usa `httpBasic()`, que ativa a **autenticação básica** HTTP: o cliente envia usuário e senha em todas as requisições, codificados (não criptografados) em um cabeçalho `Authorization`. É simples de configurar e adequada para cenários

internos ou de aprendizado, mas tem limitações práticas importantes: como a senha viaja em todas as requisições, ela depende inteiramente de HTTPS para não ser exposta; além disso, não há um conceito nativo de "sessão expirada" ou "logout"— a única forma de revogar o acesso é trocar a senha.

Autenticação Básica HTTP

Mecanismo definido pelo próprio protocolo HTTP em que o cliente envia, a cada requisição, um cabeçalho **Authorization: Basic** <usuario:senha em Base64>. O servidor decodifica esse cabeçalho e verifica as credenciais a cada chamada, sem manter estado entre requisições.

Para aplicações modernas, especialmente aquelas que separam front-end e back-end (como discutido em capítulos anteriores sobre arquitetura REST), é mais comum usar autenticação baseada em **token**, e o padrão mais difundido para isso é o JWT (*JSON Web Token*).

JWT — JSON Web Token

Um JWT é um token compacto, assinado digitalmente, que carrega um conjunto de informações (chamadas *claims*) sobre o usuário autenticado — por exemplo, seu identificador e seus papéis — e uma data de expiração. O fluxo típico é: o cliente autentica-se uma única vez, enviando usuário e senha a um *endpoint* de login; o servidor valida essas credenciais e devolve um JWT; a partir daí, o cliente anexa esse token (tipicamente no cabeçalho **Authorization: Bearer** <token>) em cada requisição subsequente, e o servidor apenas valida a assinatura e a expiração do token, sem precisar consultar novamente usuário e senha.

A vantagem central do JWT é que o servidor não precisa manter estado de sessão: toda a informação necessária para autorizar a requisição já está — de forma assinada e verificável — dentro do próprio token. Isso facilita a escalabilidade horizontal da aplicação (qualquer instância do servidor consegue validar o token de forma independente) e se encaixa naturalmente em arquiteturas de APIs REST consumidas por aplicações front-end desacopladas, como as construídas com frameworks JavaScript modernos. A implementação completa de emissão e validação de JWT no Spring Security envolve componentes adicionais — um filtro customizado que intercepta o cabeçalho **Authorization**, uma biblioteca de assinatura de tokens e um serviço de geração de tokens no momento do login — que fogem do escopo introdutório deste capítulo, mas que valem a pena ser estudados em profundidade antes de colocar uma API em produção.

34.5 Protegendo endpoints por papel

Além de configurar regras de acesso de forma centralizada no `SecurityFilterChain`, o Spring Security permite proteger métodos individuais do controlador ou do serviço usando a anotação `@PreAuthorize`, que avalia uma expressão de segurança antes mesmo de o método ser executado.

Java

```
import org.springframework.security.access.prepost.PreAuthorize;

@RestController
@RequestMapping("/products")
public class ProductController {

    @PostMapping
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<ProductResponse> create(@RequestBody ProductRequest request) {
        return ResponseEntity.ok(productService.createProduct(request));
    }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        productService.deleteById(id);
        return ResponseEntity.noContent().build();
    }

    @GetMapping("/{id}")
    public ResponseEntity<ProductResponse> getById(@PathVariable Long id) {
        return ResponseEntity.ok(productService.findById(id));
    }
}
```

Nesse exemplo, criar e remover produtos exige o papel ADMIN, enquanto a consulta por identificador permanece acessível de acordo com a regra geral definida no filtro de segurança. Para que `@PreAuthorize` funcione, é necessário habilitar explicitamente esse recurso, anotando a classe de configuração com `@EnableMethodSecurity`. Essa abordagem — proteger métodos individualmente, além (ou no lugar) de proteger rotas inteiras — é particularmente útil quando a regra de autorização depende de lógica mais fina do que apenas o caminho da URL, como verificar se o próprio usuário é o dono de um recurso específico.

Dica

Prefira concentrar as regras de acesso mais gerais (rotas públicas versus rotas que exigem autenticação) no `SecurityFilterChain`, e reserve `@PreAuthorize` para regras mais específicas, ligadas a um único método ou a uma condição de negócio. Misturar as duas abordagens sem critério tende a espalhar a lógica de segurança pelo código e dificultar a auditoria de quem pode acessar o quê.

34.6 HTTPS: a base sobre a qual tudo isso se sustenta

Toda a discussão anterior — autenticação básica, tokens JWT, papéis e permissões — perde grande parte do seu valor se a aplicação não for servida sobre HTTPS. Sem

criptografia de transporte, qualquer credencial, token ou dado sensível trafega em texto claro pela rede, podendo ser interceptado por um terceiro na mesma rede local, em um Wi-Fi público, ou em qualquer ponto intermediário entre cliente e servidor.

HTTPS

HTTPS é o protocolo HTTP operado sobre uma camada de criptografia (TLS — *Transport Layer Security*), que garante três propriedades: confidencialidade (o conteúdo da comunicação não pode ser lido por terceiros), integridade (o conteúdo não pode ser alterado em trânsito sem detecção) e autenticidade do servidor (o cliente pode verificar que está de fato se comunicando com o servidor legítimo, por meio de um certificado digital).

Em ambiente de desenvolvimento, é comum testar aplicações Spring Boot sobre HTTP simples, sem TLS configurado. Em produção, no entanto, HTTPS deixou de ser opcional: navegadores modernos sinalizam sites sem HTTPS como "não seguros", muitos recursos de navegador (geolocalização, notificações push, entre outros) simplesmente não funcionam fora de um contexto seguro, e qualquer mecanismo de autenticação — básico ou por token — fica vulnerável à interceptação sem essa camada de proteção. Na prática, a terminação de TLS costuma ser delegada a um componente de infraestrutura à frente da aplicação Spring Boot — um balanceador de carga, um *reverse proxy* como o Nginx, ou o próprio provedor de nuvem — mas cabe ao desenvolvedor garantir que essa camada exista antes de expor qualquer aplicação com dados de usuários reais.

Segurança em uma aplicação web nunca é um recurso isolado que se adiciona ao final de um projeto: ela atravessa cada decisão de arquitetura discutida ao longo deste livro — como os dados trafegam entre cliente e servidor, como são validados, como são persistidos, e, por fim, quem tem permissão para acessá-los. Autenticação, autorização e criptografia de transporte formam, juntas, a base mínima sem a qual nenhuma aplicação deveria ser considerada pronta para uso real.

Síntese do Capítulo

- Autenticação responde "quem é o usuário"; autorização responde "o que esse usuário pode fazer" — são etapas distintas e sequenciais.
- O filtro de segurança do Spring Security intercepta requisições antes que cheguem ao controlador, decidindo se elas prosseguem ou são recusadas com 401/403.
- A configuração central de segurança é feita por um **SecurityFilterChain**, declarando quais rotas são públicas, quais exigem autenticação e quais exigem papéis específicos.
- Autenticação básica HTTP envia credenciais a cada requisição e depende inteiramente de HTTPS; tokens JWT permitem autenticação sem estado, validando um token assinado em vez de repetir usuário e senha.
- **@PreAuthorize** protege métodos individuais com expressões de segurança, complementando (não substituindo) as regras gerais do filtro de segurança.

- HTTPS garante confidencialidade, integridade e autenticidade na comunicação, e é pré-requisito prático para qualquer mecanismo de autenticação em produção.
- Segurança não é um item adicionado ao final de um projeto: é uma dimensão que atravessa toda decisão de arquitetura de uma aplicação web.

Considerações Finais

Ao longo deste livro, percorremos o caminho da programação web desde os fundamentos da profissão e das redes de computadores até a construção de interfaces com HTML e CSS, a programação de comportamento com JavaScript e, por fim, o desenvolvimento de um back-end completo com Java, Spring Boot, Docker, JPA e Spring Security.

Mais do que decorar tags, propriedades ou APIs, o mais importante é entender a lógica por trás de cada camada: o HTML estrutura o conteúdo, o CSS cuida da apresentação, o JavaScript adiciona comportamento e interação, e o back-end fornece os dados e as regras de negócio que sustentam a aplicação. Compreendida essa lógica, qualquer framework ou ferramenta nova se torna consideravelmente mais fácil de aprender.

Que este material sirva como referência de consulta sempre que for necessário relembrar, aprofundar ou aplicar esses conceitos ao longo da disciplina e da trajetória profissional.